

全国计算机等级考试



二级教程

教育部考试中心
(2021年版)

公共基础知识

高等教育出版社



全国计算机等级考试二级教程

——公共基础知识

(2021 年版)

教育部考试中心

主编 徐士良

编者 陈 英 刘晓鸿

高等教育出版社·北京

内容简介

本书是根据教育部考试中心颁布的《全国计算机等级考试二级公共基础知识考试大纲(2020年版)》的要求,在2020年版的基础上修订而成。主要内容包括:计算机系统、数据结构与算法、程序设计基础、软件工程基础、数据库设计基础。

本书不仅是应试者必备的自学和辅导材料,也可以作为一般院校相应课程的教材或自学参考书。

图书在版编目(CIP)数据

全国计算机等级考试二级教程.公共基础知识:
2021年版/教育部考试中心编.——北京:高等教育出版社,2020.11

ISBN 978-7-04-054533-3

I. ①全… II. ①教… III. ①电子计算机-水平考试-教材 IV. ①TP3

中国版本图书馆CIP数据核字(2020)第119933号

策划编辑 何新权 责任编辑 何新权 封面设计 李树龙 版式设计 马云
责任校对 王雨 责任印制 朱琦

Quanguo Jisuanji Dengji Kaoshi Erji Jiaocheng——Gonggong Jichu Zhiishi (2021 Nian Ban)

出版发行	高等教育出版社	网 址	http://www.hep.edu.cn
社 址	北京市西城区德外大街4号		http://www.hep.com.cn
邮政编码	100120	网上订购	http://www.hepmall.com.cn
印 刷	保定市中国美联印刷有限公司		http://www.hepmall.com
开 本	787mm×1092mm 1/16		http://www.hepmall.cn
印 张	11.75	版 次	2020年11月第1版
字 数	280千字	印 次	2020年11月第1次印刷
购书热线	010-58581118	定 价	30.00元
咨询电话	400-810-0598		

本书如有缺页、倒页、脱页等质量问题,请到所购图书销售部门联系调换

版权所有 侵权必究

物 料 号 54533-00

前 言

本书是根据教育部考试中心颁布的《全国计算机等级考试二级公共基础知识考试大纲(2020年版)》的要求组织编写的。

全书共分五章。

第1章主要介绍计算机系统的基础知识,包括计算机的发展史、计算机体系结构、计算机系统的基本组成、计算机硬件系统与操作系统,其中还讲述了数在计算机中的表示。

第2章主要介绍算法的基本概念,数据结构的基本概念及其定义,线性表及其基本运算,栈和队列及其基本运算,线性链表及其基本运算,二叉树的基本概念、存储结构及其遍历,最后还介绍了几种常用的查找与排序算法。

第3章主要介绍程序设计方法与风格,结构化程序设计,面向对象的程序设计方法,对象,方法,属性及继承与多态性。

第4章主要介绍软件工程基本概念,结构化分析方法,结构化设计方法,软件测试的基本方法,程序的调试方法。

第5章主要介绍数据库,数据库管理系统,数据库系统的基本概念,数据模型,实体联系模型及E-R图等基本概念,关系代数理论中的基本运算,数据库设计的基本方法和步骤。

每章后面都附有一定数量的习题。

本书的第1、5章由刘晓鸿老师编写,第2章由徐士良老师编写,第3、4章由陈英老师编写,全书由徐士良、黄啸波统稿,徐士良审定,陈向群老师对全书进行了最后审阅。

由于时间紧迫,以及作者水平有限,书中难免有错误或不妥之处,恳请读者批评指正。

编者

目 录

第1章 计算机系统	1	2.5 线性链表	63
1.1 概述	1	2.5.1 线性链表的基本概念	63
1.1.1 计算机的发展历程	1	2.5.2 线性链表的基本运算	67
1.1.2 计算机体系结构	1	2.5.3 循环链表	69
1.1.3 计算机系统基本组成	2	2.6 树与二叉树	70
1.2 计算机硬件系统	3	2.6.1 树的基本概念	70
1.2.1 中央处理器	4	2.6.2 二叉树及其基本性质	73
1.2.2 计算机的基本工作原理	5	2.6.3 二叉树的存储结构	76
1.2.3 存储器	7	2.6.4 二叉树的遍历	77
1.2.4 数据的内部表示	9	2.7 查找技术	79
1.2.5 总线和外设	11	2.7.1 顺序查找	79
1.3 操作系统	15	2.7.2 二分法查找	80
1.3.1 操作系统概述	16	2.8 排序技术	80
1.3.2 进程管理	19	2.8.1 交换类排序法	80
1.3.3 存储管理	26	2.8.2 插入类排序法	82
1.3.4 文件管理	31	2.8.3 选择类排序法	84
1.3.5 I/O设备管理	35	习题2	86
习题1	38	第3章 程序设计基础	88
第2章 数据结构与算法	39	3.1 程序设计方法与风格	88
2.1 算法	39	3.2 结构化程序设计	90
2.1.1 算法的基本概念	39	3.2.1 结构化程序设计的原则	90
2.1.2 算法设计基本方法	40	3.2.2 结构化程序的基本结构与特点	90
2.1.3 算法复杂度	44	3.2.3 结构化程序设计原则和方法的 应用	91
2.2 数据结构的基本概念	47	3.3 面向对象的程序设计	92
2.2.1 什么是数据结构	47	3.3.1 关于面向对象方法	92
2.2.2 数据结构的图形表示	50	3.3.2 面向对象方法的基本概念	95
2.2.3 线性结构与非线性结构	51	习题3	98
2.3 线性表及其顺序存储结构	52	第4章 软件工程基础	99
2.3.1 线性表的基本概念	52	4.1 软件工程基本概念	99
2.3.2 线性表的顺序存储结构	53	4.1.1 软件定义与软件特点	99
2.3.3 顺序表的插入运算	54	4.1.2 软件危机与软件工程	100
2.3.4 顺序表的删除运算	56	4.1.3 软件过程与软件生命周期	102
2.4 栈和队列	57	4.1.4 软件工程的目标与原则	103
2.4.1 栈及其基本运算	57	4.1.5 软件开发工具与软件开发环境	104
2.4.2 队列及其基本运算	60		

4.2 结构化分析方法	105	5.2 数据模型	147
4.2.1 需求分析与需求分析方法	105	5.2.1 数据模型的基本概念	147
4.2.2 结构化分析方法	106	5.2.2 E-R 模型	148
4.2.3 软件需求规格说明书	110	5.2.3 层次模型	152
4.3 结构化设计方法	111	5.2.4 网状模型	153
4.3.1 软件设计的基本概念	112	5.2.5 关系模型	154
4.3.2 概要设计	113	5.3 关系代数	156
4.3.3 详细设计	118	5.4 数据库设计与管理	163
4.4 软件测试	123	5.4.1 数据库设计概述	163
4.4.1 软件测试的目的和定义	123	5.4.2 数据库设计的需求分析	164
4.4.2 软件测试的准则	123	5.4.3 数据库概念设计	165
4.4.3 软件测试方法与技术综述	124	5.4.4 数据库的逻辑设计	168
4.4.4 软件测试的策略	131	5.4.5 数据库的物理设计	171
4.5 程序的调试	134	5.4.6 数据库管理	171
4.5.1 基本概念	134	习题 5	172
4.5.2 软件调试方法	136	附录 1 全国计算机等级考试二级 公共基础知识考试大纲 (2020 年版)	174
习题 4	137	附录 2 全国计算机等级考试二级 公共基础知识样题及参考 答案	176
第 5 章 数据库设计基础	139	附录 3 习题参考答案	178
5.1 数据库系统的基本概念	139		
5.1.1 数据、数据库、数据库管理系统	139		
5.1.2 数据库系统的发展	143		
5.1.3 数据库系统的基本特点	145		
5.1.4 数据库系统的内部结构体系	146		

1.1 概述

计算机作为一种现代化的信息处理工具,它的发明是 20 世纪重要的科学成就,作为第三次科技革命中信息控制技术革命的核心内容,它极大地改变了人类历史。

1.1.1 计算机的发展历程

公认的第一台电子数字计算机 ENIAC (electronic numerical integrator and computer) 于 1946 年诞生在美国宾夕法尼亚大学。ENIAC 使用了 18 000 个电子管、1 500 个继电器,重达 30 吨,占地 170 m^2 ,运行时耗电达 140 kW。该计算机每秒可计算 5 000 次加法。自此以后,计算机以极高的速度发展。

计算机的发展一般分为四个阶段:第一阶段(1946 年—20 世纪 50 年代后期)为电子管计算机时代;第二阶段(20 世纪 50 年代后期—20 世纪 60 年代中期)为晶体管计算机时代;第三阶段(20 世纪 60 年代中期—20 世纪 70 年代初期)为集成电路计算机时代;第四阶段(20 世纪 70 年代初至今)为大规模集成电路计算机时代。

1.1.2 计算机体系结构

在研制 ENIAC 的同时,以美籍匈牙利数学家冯·诺依曼(John von Neumann, 1903—1957 年)为首的研制小组提出了“存储程序控制”的计算机结构,并开始了存储程序控制的计算机 EDVAC 的研制。由于种种原因,EDVAC 直到 1951 年才问世。而吸收了冯·诺依曼的设计思想,由英国剑桥大学研制的 EDSAC 则先于 EDVAC 两年诞生,成为事实上的第一台存储程序的计算机(stored program computer)。

“存储程序”思想是冯·诺依曼等人在 1946 年 6 月首先提出来的,它可以简要地概括为以下几点。

- ① 计算机(指硬件)由运算器、存储器、控制器、输入设备和输出设备五大基本部件组成;
- ② 计算机内部采用二进制来表示指令和数据;



- ③ 需将编好的程序和原始数据事先存入存储器中,然后再启动计算机工作。
这就是存储程序思想的基本含义。冯·诺依曼体系结构的计算机如图 1.1 所示。



图 1.1 冯·诺依曼体系结构

虽然计算机的发展速度是惊人的,但就其结构原理来说,目前绝大多数计算机仍建立在存储程序概念的基础上。随着计算机技术的不断发展,也暴露出了冯·诺依曼型计算机的一些缺点。目前已出现了一些突破冯·诺依曼结构的计算机,统称为非冯·诺依曼结构计算机,如数据驱动的数据流计算机、需求驱动的归约计算机和模式匹配驱动的智能计算机等。

1.1.3 计算机系统基本组成

一个完整的计算机系统应包括硬件系统和软件系统两大部分。

计算机硬件是指组成一台计算机的各种物理装置,它们是由借助电、磁、光、机械等原理构成的各种物理部件所组成。直观地看,计算机硬件是一大堆设备,它是计算机进行工作的物质基础。

计算机软件是指在硬件设备上运行的各种程序、数据以及有关的资料。所谓程序实际上是用于指挥计算机执行各种动作以便完成指定任务的指令集合。人们要让计算机做的工作可能是很复杂的,因而指挥计算机工作的程序也就可能是庞大而复杂的,而且可能要经常对程序进行修改与完善。此外,为了便于阅读和修改,还必须对程序做必要的说明,并整理出有关的资料。这些说明和资料(称之为文档)在计算机执行过程中可能是不需要的,但对于人们阅读、修改、维护、交流这些程序却是必不可少的。

一般计算机系统的组成框图如图 1.2 所示。

硬件系统也被称为裸机,裸机只能识别由 0 和 1 组成的机器代码。没有软件系统的计算机是无法工作的。计算机的功能不仅取决于硬件系统,而且在更大程度上是由所安装的软件系统所决定的。计算机系统中硬件系统和软件系统互相依赖,不可分割。计算机硬件系统将在 1.2 节中专门介绍。

软件系统是为运行、管理和维护计算机而编制的各种程序、数据和文档的总称。计算机软件按照面向应用对象的不同主要分为系统软件和应用软件。系统软件是指控制和协调计算机及外部设备,支持应用软件开发和运行的软件。系统软件的主要功能是调度、监控和维护计算机系统,合理分配系统资源,管理计算机系统中各独立硬件,使它们协调地工作,确保计算机正常高效地运行。系统软件主要包括操作系统(operating system, OS)、语言处理系统、数据库管理系统和



图 1.2 计算机系统的组成框图

系统辅助处理程序等。其中最主要的是操作系统,它提供了一个软件运行的环境。在 1.3 节将主要介绍操作系统。

1.2 计算机硬件系统

原始的冯·诺依曼计算机在结构上是以运算器为中心的,而发展到现在,已转向以存储器为中心。计算机硬件系统主要包含中央处理器、内存、各种输入/输出设备,它们相互之间通过总线连接到一起。计算机基本的硬件组成框图如图 1.3 所示。

(1) 中央处理器

中央处理器主要包括运算器和控制器两个部件。运算器负责对数据进行算术运算和逻辑运算(即对数据进行加工处理);控制器负责对程序所规定的指令进行分析,控制并协调输入、输出操作或对内存的访问。

(2) 存储器

负责存储程序和数据,并根据控制命令提供这些程序和数据。存储器又分为内存和外存储器(简称外存)。

(3) 输入设备

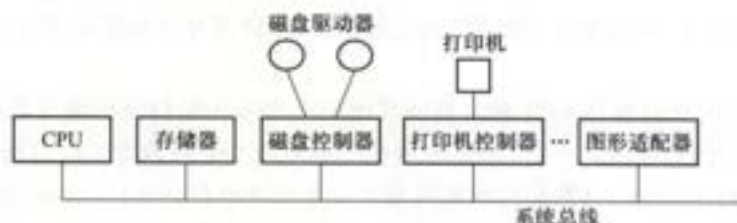


图 1.3 计算机硬件组成框图



负责把用户的信息(包括程序和数据)输入到计算机中。

(4) 输出设备

负责将计算机中的信息(包括程序和数据)传送到外部媒介供用户查看或保存。

由此可以看出,计算机硬件的基本功能是接受计算机程序的控制来实现数据输入、运算、数据输出等一系列根本性的操作。

1.2.1 中央处理器

中央处理器简称处理器,也叫 CPU(central processing unit),它是计算机系统的核心。

CPU 能与其他电路联合工作,比如与内存一起执行处理过程。CPU 是计算机的“大脑”,它执行软件(程序)指令将数据加工成信息。CPU 包括两部分:控制器与算术逻辑单元(即运算器),它们都包含有寄存器或高速存储区域,并用一种叫作总线的电子线路连接起来。

通常,运算器和控制器被合成在一块集成电路的芯片上,这就是人们常说的 CPU 芯片。

(1) 控制器

控制器是整个计算机的控制中心和指挥中心。控制器(control unit, CU)解译存储在 CPU 中的指令,然后执行指令。它能指挥内存和运算器之间电信号的运转,它也能指挥内存和输入输出设备间的信号的运转。

对于每个指令,控制单元都要执行 4 个基本操作:首先获取指令,然后分析指令,再执行指令,最后存储结果。

(2) 运算器

运算器(arithmetic logic unit, ALU)可以执行算术运算和逻辑运算,并能控制这些操作的速度。算术运算指的是基本的数学运算:加、减、乘、除。逻辑运算是指比较,就是说 ALU 可以比较两个数据间的关系,如等于(=)、大于(>)、大于或等于($\geq$)、小于(<)、小于或等于($\leq$)、不等于($\neq$)。

(3) 寄存器

控制器和运算器中都使用寄存器(registers),它是特殊的 CPU 区域,能提高计算机性能。寄存器是高速存储区域,可以在处理过程中临时存储数据。它们可以在分析指令的时候存储程序指令,可以在运算器处理数据的时候存储数据,或者存储计算结果。所有的数据在处理之前都存在寄存器中,比如要计算两个数的乘积,则将两个数全都放在寄存器中,计算结果也要放在一个寄存器中。(寄存器中也可以存放存储数据的内存地址,而不是数据本身)。

CPU 中寄存器的数量和每个寄存器的大小(多少位)可以确定 CPU 的性能和速度。比如,一个 32 位的 CPU 是指 CPU 中的寄存器是 32 位的。所以,每个 CPU 指令可以处理 32 位的数据。寄存器的类型很多,包括指令寄存器、地址寄存器、存储寄存器和累加寄存器。

(4) 总线

总线是在 CPU 内部以及在 CPU 和主板的其他部件之间传输信息的电子数据线路。总线就像是多车道的高速公路,通道越多,位的传输越快。早期微型处理器为 8 位总线,只有 8 条通道;而有 32 条通道的 32 位总线,其数据传输宽度是 8 位总线计算机的 4 倍。Intel 的 Pentium(奔腾)芯片同 Macintosh(麦金托什机)G5 处理器一样都是 64 位处理器(一些型号的计算机中有两个处理器)。一些高性能计算机的处理器是 128 位的。对总线进一步的说明见 1.2.5 节。

CPU 品质的高低直接决定了计算机系统的性能。反映 CPU 品质的最重要的指标是主频与字长。CPU 工作节拍受主时钟控制,主时钟不断产生固定频率的时钟,主时钟的频率即(CPU 的)主频。主频说明了 CPU 的工作速度;主频越高,CPU 的运算速度就越快。字长是指 CPU 可以同时处理的二进制数据的位数。

在执行指令过程中,CPU 除访问内存之外,还可以通过总线访问各种输入输出设备。

1.2.2 计算机的基本工作原理

1. 计算机指令格式

计算机指令是能够被计算机识别并执行的二进制代码,它规定了计算机能完成的某种操作。一条计算机指令通常由两部分组成:操作码和操作数(地址码)。即一条计算机指令的结构如下:

操作码	操作数(地址码)
-----	----------

指令中的操作码指出该指令需要完成操作的类型或性质。例如,取数、加法、减法、输出等不同的操作具有不同的操作码。计算机就是根据一条指令的操作码来决定做什么样的操作。由于一条指令是二进制代码,因此,其中的操作码也是二进制代码。对于一种类型的计算机来说,各种指令的操作码是互不相同的,它们分别表示不同的操作,因此,指令中操作码的二进制位数决定了该种计算机最多能具有的指令条数(即操作种类)。一般来说,如果某种类型计算机的指令操作码所占的二进制位数为 k ^①,则该种计算机最多可以有 2^k 条指令(即 2^k 种操作);反过来,为了使计算机具有 n 条指令(即 n 种操作),则该计算机指令中的操作码至少应有 $\lceil \log_2(n+1) \rceil$ 个二进制位(此处的一对方括号 $\lceil \cdot \rceil$ 表示取整)。

指令中的地址码用来描述该指令的操作对象,或者直接给出操作数,或者指出操作数的存储器地址或寄存器地址(即寄存器名)。根据指令中操作数的性质,操作数又可以分为源操作数和目的操作数两类。例如,在一般的加法指令中有 3 个操作数,其中加数和被加数为源操作数,计算结果(即它们的和)为目的操作数。在大多数情况下(即在大多数指令中),指令中给出的操作数一般是存放数据的地址,而并不是具体数据本身,甚至在有些指令中实际上给出的只能是地址而不是数据,例如,在转移指令中,除了操作码(指出需要转移)以外,还需要指出转移到什么地方,在这种情况下,实际给出的是地址。因此,指令中的操作数一般又称为地址码。每条指令的地址码个数是不一样的,要视具体的操作需要。当然,在有的指令中只有操作码而没有地址码,这种指令往往只需要指出做什么操作,而不需要具体的操作数,例如暂停指令、停机指令等。

计算机指令所占的字节数是各不相同的。只占一个字节的指令称为单字节指令,占两个字节的指令称为双字节指令,一般来说,如果指令中的操作码和操作数(即地址码)共占 n 个字节,则称该指令为 n 字节指令。

2. 计算机指令的寻址方式

指令中操作数的真实地址称为有效地址(effective address, EA),它是由寻址方式和形式地址共同决定的。寻址方式是指确定本条指令的数据地址以及下一条将要执行的指令的地址,与

^① 为与后面的程序中变量保持一致,本书中的变量全部使用正体。



硬件结构密切相关。寻址方式分为指令寻址和数据寻址两大类。指令寻址分为顺序寻址和跳跃寻址两种。常见的数据寻址方式包括立即寻址(所需的操作数由指令的地址码部分直接给出)、直接寻址(指令的地址码部分给出操作数在存储器中的地址)、隐含寻址(操作数的地址隐含在操作码或者某个寄存器中)及更为复杂的寻址方式,如间接寻址、寄存器寻址、寄存器间接寻址和堆栈寻址等方式。

3. 计算机指令系统

某种计算机的所有指令的集合,称为该计算机的指令系统。不同类型的计算机,其指令系统(包括指令的条数以及每一条指令中的操作码和地址码)是不同的。但不管是哪种类型的计算机,在它的指令系统中一般都具有以下功能的指令(其操作码对于不同类型的计算机可能是不同的)。

① 数据传送指令。将数据在内存之间,或在 CPU 的各寄存器之间,或在 CPU 的寄存器与内存之间进行传送。

② 数据处理指令。对数据进行处理,主要包括算术运算、逻辑运算与关系运算。

③ 程序控制指令。控制程序中指令的执行顺序,如条件转移、无条件转移、调用子程序、返回、停机等。

④ 输入/输出指令。实现外部设备与主机之间的数据传输。

⑤ 其他指令。对计算机的硬件进行管理、堆栈操作等。

4. 计算机执行指令的基本过程

计算机的工作就是自动快速地执行程序。所谓程序是解决实际问题的计算机指令的集合。

在计算机中,用程序计数器(PC)来决定程序中各条指令的执行顺序。在计算机开始执行程序时,程序计数器的值为该被执行程序的第一条指令所在的内存单元地址,此后按如下步骤依次执行程序中的各指令。

(1) 取指令

按照程序计数器中的地址,从内存储器中取出当前要执行的指令送到指令寄存器。

(2) 分析指令

对指令寄存器中的指令进行分析。由译码器对指令中的操作码进行译码,将指令中的操作码转换成相应的控制信息。由指令中的地址码确定操作数存放的地址。

(3) 执行指令

由操作控制电路发出完成该操作所需要的一系列控制信息,对由源地址码所指出的源操作数做该指令所要求的操作,并将操作结果存放到由目的地址码所指出的地方。

(4) 修改程序计数器

一条指令执行完后,根据程序的要求修改程序计数器的值。如果当前执行完的指令中不产生转移地址,则将程序计数器加 n (当前执行完的指令是 n 字节指令);如果当前执行完的指令是转移指令,则将转移地址送入程序计数器。最后转(1)继续执行。

一般把计算机完成一条指令所花费的时间称为一个指令周期。指令周期越短,指令执行就越快。

CPU 从内存取出一条指令分析并执行,一条指令执行完后,再从内存取出下一条指令分析并执行。CPU 不断地取指令、分析指令、执行指令,这就是程序的执行过程。

5. 指令执行的时序

计算机工作的过程就是取指令、分析指令、执行指令 3 个基本动作的重复。考虑到所有的器件中(寄存器、存储器)存储器的速度最慢,CPU 访问一次内存所花的时间较长,因此,通常用内存中读取一个指令字的最短时间来规定 CPU 周期,也称为机器周期。每个机器周期至少完成一个基本操作。一般最长的操作是访问存储器(读/写),这个时间也用于访问外设接口(寄存器)。将每条指令占用的时间称为指令周期。不同指令执行的时间可能不同,因此指令周期长短也可能不同。

由于指令执行时取指令必须访问存储器,所以占用一个机器周期。分析指令是由指令译码电路完成的,所占用的时间极短,无须分配一个完整的机器周期,一般是在取指周期后期(取指结束之前的很短时间内)就可以完成。指令的执行和指令中的操作数有关,比较复杂:可能不访问存储器(无操作数);访问一次存储器(单地址直接寻址等);访问两次或多次存储器等。因此,指令执行可能会是一个机器周期到几个机器周期。这样,每条指令的执行过程大略如下(执行周期取决于具体指令):

取指周期	执行周期 1	执行周期 2	执行周期 3	执行周期 4
------	--------	--------	--------	--------

CPU 在执行指令时,不同的指令所占用的机器周期不同。根据每个机器周期完成的任务不同,会将每个机器周期按照任务命名,类似于用取指周期命名第一个机器周期一样。

1.2.3 存储器

计算机系统中有大量的存储器,在现代以存储器为中心的计算机系统中,存储器的特性已经成为影响整个系统最大吞吐量的决定性因素。存储器从不同角度有多种分类方式,按存储介质分类,有半导体存储器、磁表面存储器、磁芯存储器和光盘存储器等;按存取方式分类,包括随机存储器(random access memory, RAM)、只读存储器(read only memory, ROM)和串行访问存储器;按在计算机中的作用分类,包括主存、辅存、缓存和闪存等,存储器分类如图 1.4 所示。存储器中最重要的是主存储器。主存储器(main memory)一般采用半导体存储器,与辅助存储器相比有容量小、读写速度快、价格高等特点。



图 1.4 存储器分类图

1. RAM 存储器

(1) 静态存储单元

静态存储单元(static RAM, SRAM)的每个存储位由 4~6 个晶体管组成。静态存储单元保存的信息比较稳定,并且这些信息为非破坏性读出,故不需要重写或者刷新操作。静态存储单元具有结构简单、可靠性高、速度较快的特点,但其所用元件较多、占硅片面积大而且功耗大,所以集成度不高。

(2) 动态存储单元

常见的动态存储单元(dynamic RAM, DRAM)有三管式和单管式两种,它们的共同特点是靠电容存储电荷的原理来寄存信息。电容上的电荷一般只能维持 1~2 ms,因此即使电源不掉电,



电容上的电荷也会自动消失。因此,为保证不丢失信息,就必须在 2 ms 之内对存储单元进行一次恢复操作,这个过程称为再生(regenerate)或者刷新(refresh)。与静态 RAM 相比,动态 RAM 具有集成度更高、功耗更低等特点,目前被各类计算机广泛使用。

2. 只读存储器

DRAM 和 SRAM 均为可任意读/写的随机存储器,当它们掉电时,所存储的内容就会消失,所以 RAM 是易失性(volatile)存储器。而只读存储器即使停电所存储的内容也不会丢失。根据半导体制造工艺的不同,可分为只读存储器 ROM、可编程的只读存储器 PROM(programmable ROM)(比如熔丝型,根据用户要写入的内容用大电流将某些熔丝烧断,从而写入了需要的内容)、掩膜式 MROM(masked ROM)、可擦可编程的只读存储器 EPROM(erasable programmable ROM)、可电擦可编程只读存储器 EEPROM(electronically EPROM,也有的书籍表示成 E²PROM 或 E²PROM)等。

3. 闪存存储器

闪存存储器(flash memory)是在 EEPROM 基础上发展而来的,是一种新的电可擦除的非易失器件,兼有 EPROM 的价格便宜、集成密度高和 EEPROM 的电可擦除和可重写特性,且速度比 EEPROM 要快得多。与磁盘相比,闪存存储器具有抗震、节能、体积小、容量大和价格便宜等特点,作为便携式存储设备而得到广泛使用。闪存读写操作一般是以块为单位进行的。

4. 高速缓冲存储器

从大量计算机运行时的统计数据中得到一个规律:程序中对于存储空间 90% 的访问局限于存储空间 10% 的区域中,而另外 10% 的访问则分布在存储空间其余 90% 的区域中。这就是通常说的程序局部性原理。程序的局部性规律包括时间局部性和空间局部性两个方面:时间局部性是指如果一个存储项被访问,则该项可能很快被再次访问;空间局部性是指如果一个存储项被访问,则该项及其邻近的项也可能很快被访问。程序的局部性使得可以通过少量高速存储器的使用而大大提升系统的性能。

高速缓冲存储器(Cache)一般用速度高的 SRAM 元件组成,其速度与 CPU 相当,但价格较贵。Cache 通常由两部分组成:块表和快速存储器。其工作原理是:处理机按主存地址访问存储器时,取出存储器地址的高段,通过地址映象机构查主存-Cache 表,判定该地址的存储单元是否在 Cache 中;如果在,则 Cache 命中,按所给的 Cache 地址访问 Cache;否则,Cache 不命中,则需要访问主存,并从主存中调入相应数据块到 Cache 中。调入时若 Cache 中已写满,则还需要按某种算法将 Cache 中的某一块替换出去,并修改有关的主存-Cache 地址映象关系。从 Cache 的工作原理可以看出,Cache 管理需要解决好两个问题,即内存内容的定位和 Cache 内容的替换。

Cache 的存在对程序员是透明的。Cache 地址变换和数据块的替换算法均由硬件实现,通常被集成到 CPU 内以提高访问速度。

5. 存储器的层次化结构

存储器有 3 个重要的指标:速度、容量和每位(bit)价格。一般来说,速度越快,位价格越高;容量越大,位价格越低;容量越大,速度越低。层次化的存储将上面诸多存储器结合起来,在相对合理的价格下给出最好的性能,如图 1.5 所示。在计算机系统中,主要体现在将缓存-主存-辅存这几个存储层次组合在一起给出相对最优的性能。

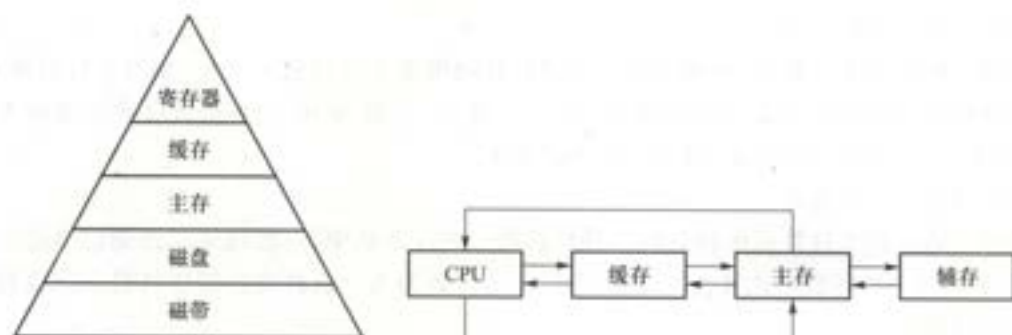


图 1.5 存储器的层次化结构和在计算机系统体现

1.2.4 数据的内部表示

1. 进位计数制及其相互转换

进位计数制是指按照进位制的方法表示数,涉及基数(radix)和权(weight)两个概念。基数是进位计数制中所拥有数字的个数,权是每位上数字的值。某一位上所表示的数等于该位上的数字乘以该位的权,整个数字的值就是各位上表示数的和。

一个数 X 记成为 r 进制的形式 $(a_{n-1}a_{n-2}\cdots a_0 \cdot a_{-1}a_{-2}\cdots a_{-m})$, 则其整数部分为 n 位, 小数部分为 m 位, 其值为:

$$X = a_{n-1}r^{n-1} + a_{n-2}r^{n-2} + \cdots + a_0r^0 + a_{-1}r^{-1} + a_{-2}r^{-2} + \cdots + a_{-m}r^{-m}$$

或

$$(X)_r = \sum_{i=-m}^{n-1} a_i r^i$$

比如日常用到的十进制数 123.567, 代表的值为

$$(123.567)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 5 \times 10^{-1} + 6 \times 10^{-2} + 7 \times 10^{-3}$$

计算机中常以 2、8 和 16 为数制的基数, 分别得到了二进制、八进制和十六进制数。转换成十进制数可利用上面的公式进行计算。比如,

$$\begin{aligned} (10101.011)_2 &= (1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3})_{10} \\ &= (21.375)_{10} \end{aligned}$$

将十进制数转换成二进制数, 通常要对一个数的整数部分和小数部分分别进行处理, 各自得出结果后再合并。对整数部分, 一般采用除以 2 取余数法得到各位, 对得到的商循环进行, 直到商为 1; 对小数部分, 一般用乘以 2 取整数法, 再对所余数的小数部分乘以 2, 循环进行直到乘积的小数部分为 0, 或结果已满足所需精度要求为止。

将二进制数转换为八进制数或十六进制数, 可很方便地得到: 由 3 位二进制数组成 1 位即得八进制数; 由 4 位二进制数组成 1 位即得十六进制数。对于一个兼有整数和小数部分的数以小数点为界, 小数点前后的数分别分组进行处理, 不足的位数用 0 补足。对整数部分将 0 补在数的左侧, 对小数部分将 0 补在数的右侧, 这样数值不会发生差错。

2. 定点数的表示和运算

计算机中的定点数分成无符号数和带符号的数, 其表示范围与机器的位数相关。



(1) 无符号数的表示

无符号数是指非负整数,机器字长的全部位数均用来表示数值的大小,相当于数的绝对值。字长为 n 位的无符号数的表示范围为 $0 \sim 2^n - 1$ 。比如,机器为 16 位字长时可表示的最大值为 $2^{16} - 1 = 65\,535$,而 32 位时为 $2^{32} - 1 = 4\,294\,967\,295$ 。

(2) 带符号数的表示

带符号数是指在计算机中将数的符号数码化。在计算机中,一般规定二进制的最高位为符号位,最高位为“0”时表示该数为正,为“1”时表示该数为负。这种在机器中将符号位也数码化的数称为机器数。

根据符号位和数值位的编码方法不同,机器数有原码、补码和反码 3 种表示。在数的原码表示中,机器数的最高位为符号位,0 表示正数,1 表示负数,数值跟随其后,并以绝对值形式给出。这是与真值最接近的一种表示形式。对原码表示的两个操作数进行加减运算时,计算机的实际操作无论是加还是减,不仅取决于指令中的操作码,还取决于两个操作数的符号,而且运算结果的符号判断也较复杂。由于原码加减法比较烦琐,需要复杂的硬件逻辑才能实现,因此在计算机中很少被采用。

数的反码表示法规定:正数的反码和原码相同;负数的反码是对该数的原码除符号位外各位取反(即将“0”变为“1”,“1”变为“0”)。容易验证,一个数的反码的反码还是原码本身。

数的补码表示法规定:正数的补码和原码相同;负数的补码是在该数的反码的最后(即最右边)一位上加 1。容易验证,一个数的补码的补码还是原码本身。

引入补码以后,计算机中带符号数的加减运算都可以用加法来实现,并且,两数的补码之“和”等于两数“和”的补码。在采用补码运算时,符号位也当作一位二进制数一起参与运算,符号位产生的进位会自然丢掉,较为简单地实现了两个定点数的加法和减法。在类似 C 语言的高级语言中,整型数在机器中就是用补码表示的。

此外,定点数还有偏移码表示:不管是正数还是负数,其补码的符号位取反即是偏移码。在 C 语言中,实型数在计算机中的表示要用到偏移码。

除去加减运算,定点数还有算术移位运算和逻辑移位运算等。两种移位之间的区别是:算术移位是对带符号数移位,移位中保持操作数的符号不变;而逻辑移位是对无符号数移位。

3. 浮点数的表示和运算

(1) 浮点数的表示范围

浮点数是指小数点位置可浮动的数据,通常以下式表示:

$$N = M \cdot R^E$$

其中, N 为浮点数, M 为其尾数, E 为其阶码, R 称为“阶的基数(底)”,而且 R 为一常数,一般为 2、8 或 16。在一台计算机中,所有数据的 R 都是相同的,于是不需要在每个数据中表示出来。因此,浮点数的机内表示一般采用以下形式:

M_s	E	M
1 位	$n+1$ 位	m 位

其中, M_s 是尾数的符号位,设置在最高位上; E 为阶码,有 $n+1$ 位,一般为整数,其中有一位符号位,设置在 E 的最高位上,用来表正阶或负阶; M 为尾数,有 m 位,由 M_s 和 M 组成一个定点



信息的传送方式分为串行传输和并行传输。计算机中 CPU 通过总线与内存、外设等连接。总线依据功能和实现方式的不同可分为片内总线、系统总线和通信总线等。

① 片内总线:芯片内部的总线。

② 系统总线:计算机各部件之间的信息传输线,包括数据总线(data bus)、地址总线(address bus)和控制总线(control bus)。其中数据总线为双向总线,宽度与机器字长、存储字长有关;地址总线为单向总线,与存储地址、I/O 地址有关;而控制总线为部分出、部分入方式,控制器通过该总线控制所有部件。

③ 通信总线:用于计算机系统之间或计算机系统与其他系统(如控制仪表、移动通信等)之间的通信,依据总线的不同传输方式又分为串行通信总线和并行通信总线。

(2) 总线的组成及性能指标

总线的结构通常分为单总线结构和多总线结构。其中,单总线结构是将 CPU、主存、I/O 设备(通过 I/O 接口与总线连接)都挂在一组总线上;而多总线结构的特点是将速度较低的 I/O 设备从单总线上分离出来,形成主总线与 I/O 设备总线分开的结构。

总线的性能指标包括总线宽度(数据总线的根数)、总线带宽(数据传输率)及时钟同步/异步(总线上的数据与时钟同步的称为同步总线,与时钟不同步的称为异步总线)等。

(3) 总线仲裁

由于总线上连接着多个部件,因此诸如某一时刻应当由哪个部件发送信息、传输的定时、传输中防止信息丢失、避免多个设备同时发送信息以及接收部件的确认等一系列问题都需要总线控制器统一管理。总线控制器的工作主要包括总线的判优控制(仲裁逻辑)和通信控制。

总线仲裁逻辑可分为集中式和分布式两种,前者将控制逻辑集中在一处(如在 CPU 中),后者将控制逻辑分散在总线的各个部件之上。

(4) 总线操作

在总线上的操作主要有读和写、块传送、写后读或读后写、广播和广集等。其中,读和写操作的读是将从设备(如存储器)中的数据读出并经总线传输到主设备(如 CPU);而写是由主设备到从设备的数据传输过程。块传送操作是主设备给出要传输的数据块的起始地址后,就可以利用总线对固定长度的数据一个接一个地读出或写入。主设备给出地址一次,可以进行先写后读或者先读后写操作:先读后写往往用于校验数据的正确性,而先写后读往往用于多道程序的对共享存储资源的保护。对于一个主设备和多个从设备间的数据传输,主设备同时向多个从设备传输数据的操作模式称为广播;与广播操作正好相反,广集操作将多个从设备的数据在总线上完成 AND 或 OR 操作,常用于检测多个中断源。

(5) 总线标准

总线标准就是系统与各模块、模块与模块之间的一个互连的标准界面。目前流行的总线标准有以下几种:常用的系统总线包括 ISA、EISA、VESA、PCI 和 AGP 等。PCI(peripheral component interconnection,外围设备互联)局部总线是高性能的 32 位或 64 位总线,它是专门为高集成度的外围部件、扩充插板和处理器/存储器系统而设计的互连机制。AGP(accelerated graphics port)是一种视频接口的技术标准,专用于连接主存和图形存储器。AGP 总线宽 32 位,时钟频率 66 MHz,能以 133 MHz 工作,最高的传输速率可达 533Mbps。

常用的设备总线标准包括 IDE、SCSI、RS-232 和 USB 等。IDE(integrated drive electronics,集



成驱动电子设备)总线,早期大部分 PC 的硬盘和相当数量的 CD-ROM 驱动器都是通过这种接口和主机连接的;SCSI(small computer system interface,小型计算机系统接口)现已成为各种计算机(包括工作站、小型机甚至大型机)的系统接口。RS-232(recommended standard-232C)是一种串行通信总线标准。USB(universal serial bus)接口基于通用的连接技术,可实现外设的简单快速连接,USB1.1 的最高传输率可达 12 Mbps(比串口快 100 倍),而 2013 年 12 月推出的 USB3.1 Gen2 达到了 10 Gbps。USB 总线接口和设备得到了广泛使用。

2. 输入/输出系统

(1) 外部设备的分类

中央处理器和主存储器构成计算机的主机。除主机以外,围绕着主机设置的各种硬件装置称为外部设备或外围设备(peripheral devices)。它们主要用来完成数据的输入、输出、成批存储以及对信息加工处理的任务。外部设备的种类很多,从它们的功能及其在计算机系统中的作用来看,可以分为五类:输入/输出(input/output, I/O)设备、辅助存储器、终端设备、过程控制设备和脱机设备等,其中前面 3 类设备应用较广,下面主要介绍这 3 类设备。

① 输入/输出设备。从计算机的角度出发,向计算机输入信息的外部设备称为输入设备;接收计算机输出信息的外部设备称为输出设备。输入设备有键盘、鼠标、扫描仪、数字化仪、磁卡输入设备、语音输入设备等。输出设备有显示设备、绘图机、打印输出设备等。另外,还有一些兼有输入和输出功能的复合型输入/输出设备,如电传打字机、控制台打字机以及键盘和显示器相结合的终端设备等。

② 辅助存储器。辅助存储器(secondary storage / external memory / auxiliary storage)是指主机以外的存储装置,又称为后备存储器或外存。目前,常见的辅助存储器有软磁盘存储器、硬磁盘存储器、磁带存储器及光盘存储器等。

③ 终端设备。终端设备由输入设备、输出设备和终端控制器组成,终端设备一般通过通信线路与主机相连。终端设备具有向计算机输入信息和接收计算机输出信息的能力,具有与通信线路连接的通信控制能力,有些还具有一定的数据处理能力。终端设备可进一步分为通用终端设备和专用终端设备两大类。

(2) 硬盘存储器

硬盘是最重要的大容量外存设备,使用极其广泛,其性能对计算机系统有重要的影响。硬盘可以分为固定磁头磁盘存储器和移动磁头磁盘存储器两种。磁头和磁臂是驱动器的重要组成部分,磁头安装在磁臂上,负责读写各磁道上的数据。固定头(臂)磁盘存储器又称每道一头型磁盘存储器,该种硬盘不需要头臂定位机构,从而减少数据读取时的平均等待时间,盘片的转速一般较高,但造价昂贵,因而应用范围很小。移动磁头的磁盘存储器在存取数据时,磁头在盘面上做径向运动,这类存储器可以由一个盘片组成,也可以由多个盘片装在一个同心的主轴上,每个记录盘面各有一个磁头。

一个具有 n 个盘片的磁盘组,可将 n 个面上的同一半径的磁道看成一个圆柱面。这些磁道存储的信息称为柱面信息。盘面又分为若干个扇区,每条磁道又被分割为若干个扇段,数据在盘片上的布局如图 1.6 所示。因此,硬盘寻址用的磁盘地址应该由磁头号、磁道号、盘面号、扇段号等字段组成,也可将扇段号用扇区号代替。

磁盘存储器的主要性能指标包括存储密度、存储容量、存取时间和数据传输率等。其中,存



存储密度常用道密度和位密度表示,道密度是沿磁盘半径方向单位长度上的磁道数,单位为道/英寸;位密度是磁道单位长度上能记录的二进制代码位数,单位为位/英寸;存储容量是指一个磁盘存储器所能存储的字节总数;存取时间是指从发出读写命令后,磁头从某一起始位置移动至新的记录位置,到完成从盘片表面读出或写入信息所需要的时间。存取时间又进一步划分为3个部分:将磁头定位至所要求的磁道上所需的时间(定位时间或寻道时间);寻道完成后至磁道上需要访问的信息到达磁头下的时间(等待时间);读写相应块的时间。实际中使用存取时间的平均值来表示。数据传输率是指磁盘存储器在单位时间内向主机传送数据的字节数。

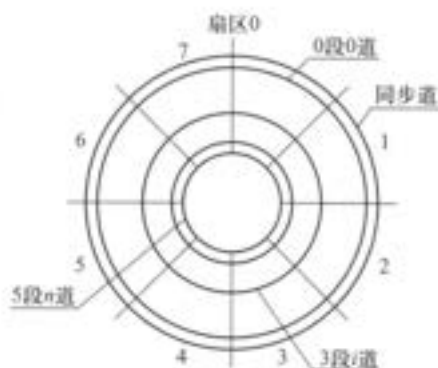


图 1.6 盘片结构

(3) I/O 接口

I/O 设备种类繁多,速度各异,操作的方式和信号差异很大,不可能简单地连到总线上。这样,就需要一种设备一边与总线相连,另一边和设备相连,这种设备就是 I/O 接口(I/O interface)或 I/O 控制器(I/O controller)。I/O 接口的基本功能是:

- ① 实现设备的选择。
- ② 实现数据缓冲以达到速度匹配。
- ③ 实现数据串并格式转换。
- ④ 实现电平转换。
- ⑤ 传送控制命令。
- ⑥ 反映设备的状态(“忙”“就绪”“中断请求”等)。

一个简单的 I/O 接口模型如图 1.7 所示:I/O 接口的系统端和系统总线进行通信,设备端控制设备驱动电路进行数据的传输。



图 1.7 I/O 接口的简单模型基本组成

(4) I/O 方式

I/O 方式包括程序查询、程序中断、DMA 和通道等。

① 程序查询方式。程序主动查询输入/输出设备是否准备好;如果准备好,CPU 执行 I/O 操作;否则,CPU 会一直查询并等待设备准备好后执行 I/O 操作。这就会使 CPU 大部分时间处于等待状态,系统效率不高。

② 程序中断方式。计算机在执行程序的过程中,当出现异常情况或者特殊情况时,CPU 停

止当前程序的运行,转而执行对这些异常情况或者特殊情况进行处理的程序(称为中断服务处理程序, interrupt service routine, ISR),处理结束之后再返回到现行程序的断点处继续运行,该过程就是中断(interrupt)。

当多个中断源同时请求时,CPU 必须对各个中断源的请求进行排队,且只能接受级别最高的中断源的请求,不允许低级别的中断源中断正在运行的中断服务程序。CPU 对这些中断源的请求进行排队的过程,称为中断判优。

在单 CPU 的机器中,典型的中断处理过程是:保存当前进程的现场、关中断,执行中断服务程序,恢复现场、恢复断点,开中断,返回原程序执行。

在处理某一个中断的过程中又发生了新的中断请求,从而中断当前服务程序的执行,又转去进行新的中断处理,这就是多重中断(multiple interrupts)或中断嵌套(nested interrupts)。当产生中断请求后,用程序方式有选择地封锁部分中断,而允许其余部分中断仍得到响应,称为中断屏蔽(interrupt masking)。

③ DMA 方式。直接内存存取(direct memory access,DMA)是 I/O 设备与主存储器之间由硬件组成的直接数据通路,用于高速 I/O 设备与主存之间的成组数据传送。DMA 方式下由 CPU 初始化传输,传输动作本身是由 DMA 控制器来执行和完成的,CPU 可以继续执行程序。

传输数据时,由 DMA 控制器给出当前正在传送的数据字的主存地址,并统计传送数据的个数以确定一组数据的传送是否已结束。主存中要开辟连续地址的专用缓冲区,来提供或接收传送的数据。在数据传送前和结束后要通过程序查询或中断方式对缓冲区和 DMA 控制器进行预处理和后处理。

④ 通道方式。通道(channel)是一个独立于 CPU 的专门管理 I/O 的处理机,它控制设备与内存直接进行数据交换。通道有自己的通道指令,通道指令由 CPU 启动,并在操作结束时向 CPU 发出中断信号。但与 DMA 方式不同,在通道方式中,数据的传送方向、存放数据的内存起始地址以及传送的数据块长度等都由通道来进行控制。通道控制方式可以做到一个通道控制多台设备与内存进行数据交换,因而,与 DMA 方式相比,通道方式进一步减轻了 CPU 的工作负担,增加了计算机系统的并行工作程度。

1.3 操作系统

计算机系统中所有的硬件和软件,统称为计算机资源。人们总是希望能够充分利用计算机系统中的所有资源,尽可能增强计算机系统的功能,而且能够为使用者提供良好的工作环境,操作系统正是为此而发展起来的。

由于操作系统的结构庞大、模块众多、各模块之间的联系也很复杂,因而对操作系统的研究带来一定的难度。为此,人们提出了各种观点,试图给出一种系统的方法,以便研究、分析和设计操作系统的功能、组成部分、工作过程以及体系结构,最典型的如资源管理观点、进程观点、层次观点等。这些观点从不同的角度来分析和研究操作系统,互相并不矛盾。

本节将从资源管理的观点来讨论操作系统的有关技术。



1.3.1 操作系统概述

1. 操作系统的功能与任务

操作系统是最基本的和最核心的系统软件,也是当今计算机系统中不可缺少的组成部分,所有其他的软件都依赖于操作系统的支持。

从计算机系统的组成层次出发,操作系统是直接和硬件层相邻的第一层软件,它对硬件进行首次扩充,是其他软件运行的基础。其主要作用有以下几个方面:

① 管理系统资源。包括对 CPU、内存储器、输入/输出设备、数据文件和其他软件资源的管理。

② 为用户提供资源共享的条件和环境,并对资源的使用进行合理调度。

③ 提供输入/输出的方便环境,简化用户的输入/输出工作,提供良好的用户界面。

④ 规定用户的接口,发现、处理或报告计算机操作过程中所发生的各种错误。

由此可以看出,操作系统既是计算机系统资源的控制和管理者,又是用户和计算机系统之间的接口,当然它本身也是计算机系统的一部分。因此,概略地说,操作系统是用以控制和管理系统资源、方便用户使用计算机的程序的集合。

如果把操作系统看成是计算机系统资源的管理者,则操作系统的功能和任务主要有以下 5 个方面。

① 处理机管理。处理机(即 CPU)是整个计算机硬件的核心。处理机管理的主要任务是:充分发挥处理机的作用,提高它的使用效率。

② 存储器管理。计算机的内存储器是计算机硬件系统中的重要资源,它的容量总是有限的。存储器管理的主要任务是:对有限的内存储器进行合理的分配,以满足多个用户程序运行的需要。

③ 设备管理。通常,用户在使用计算机时或多或少地用到输入/输出操作,而这些操作都要涉及各种外部设备。设备管理的主要任务是:有效地管理各种外部设备,使这些设备充分发挥效率;并且还要给用户提供一个简单而易于使用的接口,以便在用户不了解设备性能的情况下,也能很方便地使用它们。

④ 文件管理。由于内存储器是有限的,因此,大部分的用户程序和数据,甚至是操作系统本身的部分以及其他系统程序的大部分,都要存放在外存储器上。文件管理的主要任务是:实现唯一地标识计算机系统中的每一组信息,以便能够对它们进行合理的访问和控制;以及有条理地组织这些信息,使用户能够方便且安全地使用它们。

⑤ 用户接口。为了使用户能灵活、方便地使用计算机和操作系统,有效地组织自己的工作流,操作系统还提供了一组友好的用户接口,使整个系统能高效地运行。这也是操作系统的一个重要功能。

对于实际的操作系统,可能由于其性能、使用方式各不相同,使系统功能、基本结构、支持硬件和应用环境等方面也有所不同,因此,操作系统所要完成的任务也各不相同。

2. 操作系统的发展过程

(1) 手工操作(无操作系统)

计算机发明之初,用户直接面对硬件,程序设计全部采用机器语言。人们先把程序纸带(或

卡片)装上计算机,然后启动输入机把程序和数据送入计算机,接着通过控制台(console)开关启动程序运行。计算完毕,打印机输出计算结果,用户卸下并取走纸带(或卡片)。其特点是:用户独占机器,CPU 等待手工操作,CPU 利用不充分。

(2) 批处理系统

加载一个监督软件到计算机上,在监督程序的控制下,计算机能够自动地、成批地处理一个或多个用户的作业(包括程序、数据、命令)。批处理系统又分为联机批处理系统和脱机批处理系统。联机批处理系统中作业的输入/输出由 CPU 来处理,在主机和输入机之间增加一个存储设备——磁带机。联机批处理系统在输入作业和输出结果时,CPU 还是会处于空闲状态。

在脱机批处理系统中,输入/输出设备脱离主机控制。增加一台不与主机直接相连,而专门用来与输入/输出设备打交道的卫星机。这样,主机不再与慢速的输入/输出设备连接,并且主机与卫星机两者并行工作,主机的计算能力得以充分发挥。

在 1950 年代末至 1960 年代初,计算机硬件中出现中断机构和通道技术,硬件具有了较强的并行工作能力,驻留内存的管理程序(resident monitor)被研制出来,提高了 CPU 和 I/O 设备的利用率,缩短作业的准备和建立时间,减少了人工干预和操作失误。这些系统都配备专门的计算机操作员。

(3) 多道程序系统

多道程序(multiprogramming)设计技术是指允许多个程序同时进入内存并运行。即同时把多个程序放入内存,并允许它们交替在 CPU 中运行,多个程序可共享系统中的各种硬、软件资源。当一个程序因 I/O 请求而暂停运行时,CPU 便立即转去运行另一个程序。

多道程序设计技术不仅使 CPU 得到充分利用,同时也改善了输入/输出设备和内存的利用率,从而提高了整个系统的资源利用率和系统吞吐量(单位时间内处理作业/程序的个数)。

(4) 分时系统

操作系统进一步发展就出现了分时操作系统。分时操作系统中对后来计算机科学与技术发展有巨大影响的是 UNIX 系统,它被称为计算机/互联网行业的基石。1969 年, Ken Thompson 在 PDP-7 上写了一个作业系统和一些常用的工具程序,称之为 Unics。之后他把这一系统移植到更多的平台上,并更名为 UNIX。他和 Ritchie 共同对其进行了改造,并于 1971 年共同发明了影响深远的 C 语言。1973 年他们用 C 语言重写了 UNIX,UNIX 的正式版本发行了。这之后出现了大量不同版本的 UNIX。

UNIX 在理论上和实践上都对整个操作系统的发展产生了广泛而深远的影响,其许多设计思想被后来的操作系统所采纳。

(5) 个人计算机操作系统

20 世纪 70 年代末期出现了个人计算机(personal computer, PC),同时出现了个人计算机操作系统,其中微软公司的 MS-DOS 和苹果公司的 CPM 是最有影响的。1984 年苹果公司的图形操作系统诞生,1992 年微软的 Windows 3.1 也面世,之后这两大系统一直在演进,伴随硬件的飞速发展发布了很多市场占有率极高的版本。1991 年 Linus 在 Internet 上发布了 Linux 操作系统,在众多爱好者的参与下,系统不断丰富完善,支持众多 CPU 和不同架构,作为开源软件的 Linux 在很多方面已经接近商用操作系统的品质和性能,极其健壮。

2007 年苹果公司在 Macworld 大会上发布了 iOS 操作系统,应用于其移动设备 iPad 和 iPhone



等。2007年11月,Google与84家硬件制造商、软件开发商及电信运营商组建“开放手机联盟”,共同研发改良Android系统。Android是一种基于Linux的自由及开放源代码的操作系统,主要应用于移动设备,如智能手机和平板电脑,并逐渐扩展到其他领域中,如数字电视、数码相机、游戏机、智能手表等。

3. 操作系统的分类

对操作系统进行分类的方法有很多。例如,按照计算机硬件规模的大小,可以分为大型机操作系统、小型机操作系统和微型机操作系统;按照操作系统在用户面前的使用环境以及访问方式,可以将操作系统分为多道批处理操作系统、分时操作系统和实时操作系统等。

(1) 多道批处理操作系统

多道批处理操作系统包含“多道”和“批处理”两层意思。所谓“多道”是指在计算机内存中存入多个用户作业。所谓“批处理”是指这样一种操作方式,在外存中存入大量的后备作业,作业的运行完全由系统控制,用户与其作业之间没有交互作用,用户不能直接控制其作业的运行,通常称这种方式为批操作或脱机操作。

在多道批处理操作系统中,系统资源利用率高,作业的吞吐量大,但用户不能干预自己程序的运行,对程序的调试和排错不利。

(2) 分时操作系统

允许多个联机用户同时使用一台计算机系统进行计算的操作系统称为分时操作系统(time sharing operating system)。分时系统把中央处理器的时间划分成时间片,轮流分配给每个联机终端用户,每个用户只能在极短时间内执行,若程序未执行完,则等待分到下次时间片时再执行。这样,系统的每个用户的每次要求都能得到快速响应,且用户感觉好像自己独占计算机。

分时操作系统具有以下特点:多路性(又称同时性,终端用户感觉上好像独占计算机)、交互性、独立性(终端用户彼此独立,互不干扰)和及时性(快速得到响应)。

(3) 实时操作系统

实时操作系统(real time operating system, RTOS)是指当外界事件或数据产生时,系统能够接收并以足够快的速度予以处理和响应,能够控制所有任务协调一致运行。目前有3种典型的实时系统:过程控制系统(如工业生产自动控制、航空器飞行控制和航天器发射控制)、信息查询系统(如仓库管理系统、图书资料查询系统)和事务处理系统(如飞机或铁路订票系统、银行管理系统)。

实时操作系统分为硬实时(hard real-time)操作系统及软实时(soft real-time)操作系统。硬实时操作系统必须使任务在确定的时间内完成,而软实时操作系统能让绝大多数任务在确定时间内完成。实时操作系统一般会采用基于优先级的抢占调度方式。

(4) 网络操作系统

为了使计算机能方便地传送信息和共享网络资源,将计算机加入网络中,这样的计算机上的操作系统称网络操作系统(network operating system)。网络操作系统应该具备以下几项功能:网络通信(源和目标之间无差错数据传输)、资源管理、网络管理(包括安全控制、性能监视、故障管理等)、网络服务和通信透明性(提供对多种通信协议支持等)。

(5) 分布式操作系统

分布式计算机系统是指由多台分散的计算机经网络互连而成的系统。联网的每台计算机高

度自治又相互协同,能在分布式系统范围内实现资源管理、任务分配及并行地运行分布式程序。系统中的资源为所有用户共享,多台机器可以互相协作来完成同一个任务(一个程序可以分布于多台计算机上并行运行)。分布式系统中的一个节点出错不影响其他节点运行。分布式计算机系统的主要优点是健壮性强、扩充容易、可靠性高、维护方便和效率较高。

用于管理分布式计算机系统的操作系统称为分布式操作系统(distributed operating system)。它与单机的集中式操作系统的主要区别在于资源管理、进程通信和系统结构3个方面。

(6) 嵌入式操作系统

运行于嵌入式系统之上的操作系统称为嵌入式操作系统(embedded operating system)。由于资源受限,微型化是嵌入式系统的重要特点;而多种多样的硬件平台使其表现出专业化的特点;由于嵌入式系统广泛应用于过程控制、数据采集、通信、信息家电等要求迅速响应的场合,实时性也是其重要特点。

嵌入式操作系统一般要求占用内存小,需要根据系统的实际硬件资源对操作系统进行裁减定制。

1.3.2 进程管理

1. 并发程序设计

操作系统的主要目标是提高计算机系统的处理效率,增强系统中各种硬件的并行操作能力。为了达到这个目标,必须要求程序结构适应并发处理的需要,使计算机系统中能够同时存在两个或两个以上正在执行的程序。为此引出了多道程序设计的问题。

而在多道程序系统中,因为在系统中同时有多个程序在执行,这些程序具有并发性、共享性、异步性、制约性和动态性等特点,因此,传统的程序设计方法中所用的程序概念已难以刻画、反映系统中的各种复杂情况,顺序程序的结构已不能适应系统的需要。

许多问题的处理过程有着特定的顺序,用以处理这些问题的相应程序,其执行也必然有一定的先后次序。也就是说,在这种情况下,程序中与各个操作相对应的程序段的执行一定是顺序的。这样的程序称为顺序程序。具体地说,所谓顺序程序设计,是指所设计的程序具有以下3个特点。

① 程序所规定的动作严格地按顺序执行,即每个动作都必须在上一个动作执行完成以后才开始。或者说,每个动作都必须在下一个操作开始执行之前结束。这就是程序的顺序性。

② 程序一旦开始执行,其计算结果不受外界因素的影响。也就是说,顺序程序的静态文本与其计算过程有着——对应的关系。这就是顺序程序的封闭性。

③ 在程序运行过程中,任何两个动作之间的停顿,对程序的计算结果不发生任何影响,即程序的计算结果与它的运行速度无关。只要给定相同的初始条件,并给以同样的输入,重复执行同一个程序一定会得到相同的结果。这就是顺序程序的可再现性。

即顺序程序所具有的顺序性、封闭性和可再现性的特点,使得程序设计者能够控制程序执行的过程(包括执行顺序、执行时间),对程序执行的中间结果和状态可以预先估计,这样就可以方便地进行程序的测试和调试。

为了充分利用系统资源,提高系统的效率,除了在硬件上采用通道技术、中断技术等措施,使设备间实现并行操作外,还要从软件上采取相应的措施,使多个程序能并发执行。

所谓多个程序并发执行(concurrent execution),是指一组在逻辑上互相独立的程序或程序段



在执行过程中,其执行时间在客观上互相重叠,即一个程序段的执行尚未结束,另一个程序段的执行已经开始的这种执行方式。只有在多处理机系统中才可能做到物理上的真正并行,而在单处理机的系统中,这些程序只能在逻辑上或在宏观上做到并行执行。也就是说,在单处理机的情况下,并发执行是指多个程序的运行在时间上是重叠的,而并不是说这些程序在某一时刻同时占用处理机在运行。

多个程序并发执行是多道程序系统的特点。显然,并发程序的执行情况与顺序程序的执行情况是很不相同的,它比顺序程序的执行情况要复杂得多,程序设计时要考虑的因素也得多。考虑各种并行性的程序设计方法称为并发程序设计。

并发程序在执行过程中有以下几个特点。

(1) 并发程序没有封闭性

顺序程序具有封闭性,程序执行后的输出结果与时间无关。但是,在并发程序的执行过程中,某个程序中的变量可能被另外程序的执行所改变;或者某个程序中的变量在用另外一个程序输出时,不同时刻其输出值可能是不一样的。即并发程序的输出结果与各程序执行的相对速度有关,失去了程序的封闭性这个特点。

例 1.1 设有两个并发程序 A 和 B 互相独立地运行,如图 1.8 所示。当程序 A 执行到 I_1 时,由于某个原因,将控制转到执行程序 B,由程序 B 打印 n 的值为 0,而后当程序 B 运行到 I_2 时,又将控制转到执行程序 A,接着 I_1 之后继续执行。

现假设程序 B 运行的速度慢一些,如果在程序 A 运行到 I_2 时,才将控制转到执行程序 B(如图中虚线所示),此时由程序 B 打印的 n 值为 1,而不是 0。

从这个例子可以看出,程序 B 的运行结果与它们的相对运行速度有关,这就说明程序没有封闭性。因为程序的执行结果与时间有关,因此,结果是不可再现的,即使输入相同的初始条件,也可能得到不同的结果,这称为“结果的不确定性”。

(2) 程序与其执行过程不是一一对应的关系

在顺序程序设计中,程序的封闭性决定了程序与其执行过程是完全对应的,程序的执行路径、执行时间和所执行的操作都可以从程序中反映出来。但在多道程序并发执行的情况下,程序的执行过程由当时的系统环境与条件所决定,程序与其执行过程就不再有一一对应的关系。当多个执行过程共享某个程序时,它们都可以调用这个程序,调用一次即对应一个执行过程,也就是说,这个共享的程序对应多个执行过程。

例 1.2 设有两个并发程序 A 和 B,它们在执行过程中都要调程序 C,如图 1.9 所示。

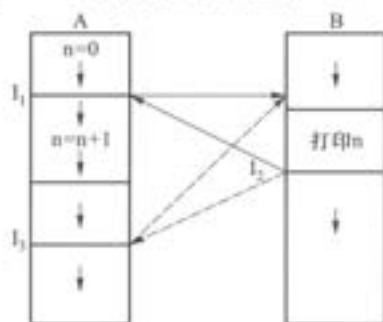


图 1.8 两个并发程序的执行

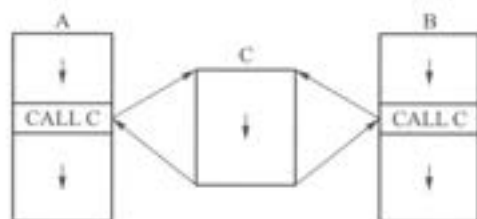


图 1.9 并发程序的共享

在这种情况下,程序 C 包含在来自程序 A 和程序 B 的两个不同执行过程中。显然,程序 C 与它的执行过程并不是一一对应的。在此特别要指出,这里的程序 C 要能够为不同的程序所共享,因此,在其执行过程中本身不能有任何修改。这种可共享的过程称为纯过程或可重入过程(reentrant procedures)。例如,在多道程序环境下,两个用户作业都需要用 C 编译程序,为了减少编译程序副本,它们共享一个编译程序,这样,一个编译程序能同时为两个作业服务,即这个编译程序对应两个执行过程。在这种情况下,C 编译程序必须是可重入程序。

(3) 程序并发执行可以互相制约

并发程序的执行过程是复杂的,这是因为它们之间不但可能有互为因果的直接制约关系,而且还可能由于共享某些资源或过程而具有间接的互相制约关系。

例 1.3 设有两个并发程序 A 和 B,它们不仅共享程序 S,并且它们在分别执行 S 时还要发生相互作用,如图 1.10 所示。

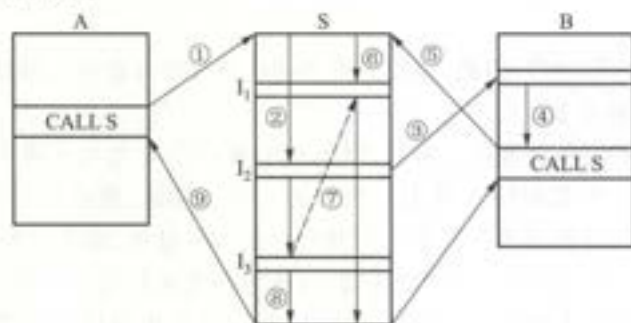


图 1.10 有制约关系的并发程序

在程序 S 中,如果规定 I_1 到 I_3 这段代码只能属于一个计算过程,则当某个执行过程到达 I_1 处,都要检查一下是否有其他计算在这段代码中运行,如果已有一个计算正在这段代码中运行,则要在此等待,直到已有的计算退出 I_3 时才唤醒在 I_1 处等待的计算。

现假定程序 A 先调用了程序 S (图 1.10 中①),并且在穿过 I_1 之后到达 I_2 时 (图 1.10 中②),由于某种原因将控制转到执行程序 B (图 1.10 中③与④),而程序 B 又调用程序 S (图 1.10 中⑤),并在 I_1 处等待 (图 1.10 中⑥),这是因为已有程序 A 调用程序 S 的计算在 I_1 到 I_3 这段代码中。此时又将控制转到执行程序 A 调用程序 S 的过程 (图 1.10 中⑦),当运行到 I_3 的计算时唤醒程序 B 调用程序 S 的过程,然后继续运行 (图 1.10 中⑧),直到退出程序 S 返回到程序 A (图 1.10 中⑨),或者将控制转到执行程序 B 调用程序 S 的过程。

由此可以看出,并发程序 A 和 B 在执行过程中可以互相制约。在这个例子中,其相互制约点发生在程序 S 中。

并发程序的这些特点说明它与顺序程序之间有着本质的区别,并发程序具有并行性和共享性,而顺序程序则以顺序性和封闭性为基本特征。

2. 进程的基本概念

在多道程序系统的环境下,程序是并发执行的,在执行过程中它们互相制约,系统与其中各程序的状态在不断地变化,因此,系统的状态和各程序在其中活动的描述一定是动态的。程序是一个静态的概念。因此,程序本身不能刻画多道程序并发执行时的动态特性和并行特性,也就不



能深刻地反映并发程序的活动规律和状态变化。为此,需要引进一个能够从变化的角度,动态地反映并发程序活动的新概念,这就是进程。

所谓进程(process),是指一个具有一定独立功能的程序关于某个数据集合的一次运行活动。简单地说,进程是可以并发执行的程序的执行过程,它是控制程序管理下的基本的多道程序单位。

由此可以看出,进程与程序有关,但它与程序又有本质的区别。主要反映在以下几个方面。

① 进程是程序在处理机上的一次执行过程,它是动态的概念。而程序只是一组指令的有序集合,其本身没有任何运行的含义,它是一个静态的概念。

② 进程是程序的执行过程,是一次运行活动。因此,进程具有一定的生命期,它能够动态地产生和消亡。即进程可以由创建而产生,由调度而执行,因得不到资源而暂停,以致最后由撤销而消亡。也就是说,进程的存在是暂时的。而程序是可以作为一种软件资源长期保存的,它的存在是永久的。

③ 进程是程序的执行过程,因此,进程的组成应包括程序和数据。除此之外,进程还包括记录进程相关信息的“进程控制块”。

④ 一个程序可能对应多个进程。例如,图1.9中的程序C对应了两个进程,一个是程序A调用它执行的进程,另一个是程序B调用它执行的进程。又如,当有多个C源程序共享一个编译程序同时进行编译时,该编译程序对应每个源程序的编译过程,都可以看作是编译程序在不同数据上的运行,因此,它对应了多个不同的进程,这些进程都运行同一个编译程序。

⑤ 一个进程可以包含多个程序。例如,主程序执行过程中可以调用其他程序,共同组成一个运行活动。

3. 进程的状态及其转化

进程是程序的执行过程,它具有一定的生命期,并且,在进程的存在过程中,由于系统中各进程并发执行以及相互制约的关系,使各进程的状态会不断发生变化。一般情况下,一个进程并不是自始至终都处于运行状态,而是时而处在运行状态,时而又由于某种原因暂停运行而处于等待状态,当使它暂停的原因消失后,它又处于准备运行的状态。这就是说,进程有着“走走停停”的活动规律,而它从“走”到“停”和从“停”到“走”的变化是由系统中不同事件引起的。一般来说,一个进程的活动情况至少可以划分为以下5种基本状态。

(1) 运行状态

处于运行状态下的进程实际上正占据着CPU。显然,处于这种状态的进程数目不能多于CPU的数目。在单CPU的情况下,处于运行状态的进程只能有一个。

(2) 就绪状态

这种状态下的进程已获得了除CPU以外的一切所需的资源,只是因为缺少CPU而不能运行,一旦获得CPU,它就立即投入运行。这种状态下的进程也称为逻辑上是可以运行的进程。在一个系统中,处于就绪状态的进程可以有多个,通过称为就绪队列的结构进行管理。在多处理器系统中,每个CPU都有自己的就绪队列,而每个活动进程某一时刻只会在一个就绪队列中。

(3) 等待状态

一个进程正在等待某一事件(如等待输入输出操作的完成、等待某系统资源、等待其他进程来的信息等)的发生而暂时停止执行。在这种状态下,即使把CPU分配给它,该进程也不能运

行,即处于等待状态,又称为阻塞状态或封锁状态。处于等待状态的进程有时也称为逻辑上是不可执行的。

(4) 创建状态

进程正在创建过程中,尚不能运行。

(5) 终止状态

进程运行结束。

一个作业(程序)一旦被调入内存,系统就为它建立一个或若干个进程。而每个进程的创建是以建立进程控制块为标志。一个进程被创建后,并不是固定地、静止地处于某个状态,而是随着进程自身的推进和外界条件的变化其状态在不断地变化。也就是说,进程的3种基本状态之间在一定的条件下是可以互相转化的。图1.11表示了进程的3种基本状态之间在一定条件下的转化。进程3种状态之间的转换条件如下。

① 处于就绪状态的进程,一旦分配到CPU,就转为运行状态。

② 处于运行状态的进程,当需要等待某个事件发生才能继续运行时,则转为等待状态;或者由于分配给它的时间片用完,就让出CPU而转为就绪状态。

③ 处于等待状态的进程,如果它等待的事件已经发生,即条件得到满足,就转为就绪状态。

通常,进程创建完成后会进入就绪状态,在运行、阻塞和就绪间迁移,进行相关的任务执行;完成相关任务后,由运行状态进入终止状态,结束进程并释放相关资源。



图 1.11 进程状态的转换

4. 进程控制块及其组织

一个进程的存在,除了要有程序和操作的数据这个实体外,更重要的是,在创建一个进程时,还要建立一个能够描述该进程执行情况,能够反映该进程和其他进程以及系统资源的关系,能够刻画该进程在各个不同时期所处的状态的数据块,即进程控制块PCB(process control block)。

(1) 进程控制块PCB

进程控制块是由系统为每个进程分别建立的,用以记录对应进程的程序和数据的存储情况,记录进程的动态信息。系统根据PCB而感知进程的存在,根据PCB中的信息对进程实施控制管理。当进程结束时,系统即收回它的PCB,进程也随之消亡。因此可以说,PCB是一个进程存在的标志。

为了能够充分地描述一个进程,PCB中通常应包括以下一些基本内容。

① 进程名。它是唯一标识对应进程的一个标识符或数字,系统根据该标识符来识别一个进程。

② 特征信息。它反映了该进程是不是系统进程等信息。

③ 执行状态信息。说明对应进程当前的状态。



④ 通信信息。反映该进程与其他进程之间的通信关系。例如,当进程处于等待状态时,指明等待的理由等。

⑤ 调度优先数。用于分配 CPU 时参考的一种信息,它决定在所有就绪的进程中,究竟哪一个进程先得到 CPU。

⑥ 现场信息。在对应进程放弃 CPU(由运行状态转换为就绪或等待状态)时,将处理机的一些现场信息(如指令计数器值、各寄存器值等)保留在该进程的 PCB 中,当下次再恢复运行时,只要按保存值重新装配即可继续运行。

⑦ 系统栈。这是在对应进程进入操作系统时,实现子程序嵌套调用时用的栈,主要用于保留每次调用时的程序现场。系统栈的内容主要反映了对应进程在执行时的一条嵌套调用路径上的历史。

⑧ 进程映像信息。用以说明该进程的程序和数据存储情况。

⑨ 资源占有信息。指明对应进程所占有的外设种类、设备号等。

⑩ 族关系。反映该进程与其他进程间的隶属关系。例如,该进程是由哪个进程建立的,它的子进程是谁等。

除此之外,进程控制块中还包含有文件信息、工作单元等内容。总之,进程控制块中的这些信息为系统对进程的管理和控制提供了依据。

(2) 进程的组织

系统中有许多进程,它们所处的状态各不相同,有的处于就绪状态,有的处于等待状态(等待的原因又各不相同),还有正在运行的进程。对这些处于不同状态的进程的物理组织形式将直接影响系统的效率。对进程的物理组织方式通常有线性表和链接表两种。

在线性表组织方式中,一种方法是将所有不同状态进程的 PCB 组织在一个表中。这种方法最为简单,适用于系统中进程数目不多的情况。其缺点是管理不方便,经常要扫描整个表,影响了整个系统的效率。

线性表形式的另一种组织方法是,分别把具有相同状态进程的 PCB 组织在同一个表中。这样就分别构成就绪进程表、各种等待事件的等待进程表以及运行进程表(多处理机系统中)。

链接表组织形式是按照进程的不同状态将相应的 PCB 放入不同的带链队列中。特别要指出的是,等待队列有多个,它们分别等待在不同的事件上,当某个事件发生后,则要把相应的等待队列中的所有 PCB 送到就绪队列中。采用链接表的优点在于使系统的进程数目不受限制,可以动态地申请;又由于各种等待状态的队列是分开的,管理起来比较方便。

5. 进程调度

进程调度(process scheduling)就是按一定策略动态地把 CPU 分配给处于就绪队列中的某一进程并使之执行的过程。进程调度亦可称为处理器调度或低级调度,相应的进程调度程序可称为分配程序或低级调度程序。高级调度是作业调度,作业调度负责对 CPU 之外的系统资源进行调度,其中包含不可抢占资源(如打印机)的分配。进程调度仅负责对 CPU 进行分配,而 CPU 属于可抢占的资源。

有两种基本的进程调度方式,即抢占方式和非抢占方式。前者指就绪队列中一旦有优先级高于当前正在运行的进程出现时,系统便立即把 CPU 分配给高优先级的进程,并保存被抢占了 CPU 的进程的有关状态信息,以便以后恢复。而对于非抢占方式,一旦 CPU 分给了某进程,即使

就绪队列中出现了优先级比它高的进程,高优先级进程也不能抢占现行进程的 CPU。在任意时刻,处于运行状态的进程数至多等于计算机系统 CPU 的个数。

最基本的进程调度算法有以下几种。

(1) 先来先服务调度算法

先来先服务 (first-come first-served, FCFS) 调度算法按照进程就绪的先后顺序来调度进程,到达得越早,就越先执行。获得 CPU 的进程,未遇到其他情况时,一直运行下去,系统只需设置一个先进先出的队列。在管理相同优先级的就绪队列时,这种方法是一种最常见策略,并且在没有其他信息时,也是一种最合理的策略。

(2) 时间片轮转调度算法

时间片轮转 (round-robin algorithm, RR) 调度算法是系统把所有就绪进程按先后次序排队, CPU 总是优先分配给就绪队列中的第一个就绪进程,并分配给它一个固定的时间片 (如 50 ms)。当该运行进程用完规定的时间片时,被迫释放 CPU 给处于就绪队列中的下一个进程,并分配给这个进程相同的时间片。每个运行完时间片的进程,当未遇到任何阻塞时,就被放回到就绪队列的尾部,并等待下次轮到它时再投入运行。

当某个正在运行的进程的时间片尚未用完,而此时由于进程发出 I/O 请求受到阻塞,进程被送到相应阻塞队列而非就绪队列的尾部。只有等它所需要的 I/O 操作完成之后,才能重新返回到就绪队列的尾部,等待再次被调度后投入运行。

(3) 优先级调度算法

进程调度最常用的一种简单方法,是把 CPU 分配给就绪队列中具有最高优先级的就绪进程,即优先级调度算法 (priority-scheduling algorithm)。根据 CPU 是否可被抢占,优先级调度分为抢占式 (preemptive) 和非抢占式 (non-preemptive) 优先级调度算法两种。

进程的优先级通常根据以下因素确定:进程类型 (比如系统进程比用户进程具有较高的优先级)、运行时间 (通常规定进程优先级与进程所需运行时间成反比) 及作业的优先级 (根据作业的优先级来决定其所属进程的优先级)。上面是静态优先级法,每个进程的优先级在其生存周期内是一直不变的。进程也可以有动态优先级,其优先级在该进程的运行生命周期内可以改变。这样就能更精确地控制 CPU 的响应时间,在分时系统中,其意义尤为重要。比如多级反馈队列轮转法就是动态优先级调度算法。

除上面算法外还有其他一些进程调度算法。实际操作系统的调度相当复杂,都不是孤立地采用某一种算法,而是将几种算法结合起来使用并进行优化,这样可以得到效率更高的算法。

20 世纪 80 年代中期,人们提出了比进程更小的能独立运行的基本单位——线程 (thread),用它来提高程序的并行程度,减少系统开销,从而可进一步提高系统的吞吐量。许多现代操作系统都在内核级提供了对线程的支持。

操作系统的主要目标之一就是充分利用系统资源,为此,它允许多个进程并发执行,并且共享系统的软硬件资源。由于各进程互相独立地动态获得,不断申请和释放系统中的软硬件资源,这就有可能使系统出现这样一种状态:其中若干个进程均因互相“无知地”等待对方所占有的资源而无限地等待。这种状态称为死锁。因此,在操作系统中还要建立相应的机制来处理进程之间的通信和同步等问题,以便解决并发进程运行时可能会由于资源不能满足而产生的死锁的检测及其预防、避免和恢复等问题。在此不再详述了。



1.3.3 存储管理

内存是计算机系统的重要资源之一,它为多道程序所共享,也是各程序的竞争对象,因此,对存储器(注:本节所说的存储器均指内存)这个资源进行有效的组织、管理和分配,也是操作系统的主要任务之一。

1. 存储管理的功能和地址重定位

(1) 存储管理的功能

在多道程序系统中,存储管理一般应包括以下一些功能。

① 地址变换。要把用户程序中的相对地址转换成实际内存空间的绝对地址。

② 内存分配。根据各用户程序的需要以及内存空间的实际大小,按照一定的策略划分内存,以便分配给各个程序使用。

③ 存储共享与保护。由于各用户程序与操作系统同在内存,因此,一方面允许各用户程序能够共享系统或用户的程序和数据,另一方面又要求各程序之间互不干扰或破坏对方。

④ 存储器扩充。由于多道程序共享内存,使内存资源尤为紧张,这就要求操作系统根据各时刻用户程序允许的情况合理地利用内存,以便确保当前需要的程序和数据在内存,而其余部分可以暂时放在外存中,等确实需要时再调入内存。

(2) 地址重定位

一般来说,用户在编写程序时并不知道自己的程序在执行时放在内存空间的什么区域,因此不可能用内存中的实际地址(称为物理地址)来编写程序,只能相对于某个基准地址(通常为0地址)来编写程序、安排指令和数据的位置,这种在用户程序中所用的地址通常称为相对地址(或逻辑地址)。当用户程序进入内存执行时,又必须要把用户程序中的所有相对地址(逻辑地址)转换成内存中的实际地址(物理地址),否则用户程序无法执行。这就是所谓的地址变换(又称地址映射)。在进行地址变换时,必须修改程序中所有与地址有关的项,也就是说,要对程序中的指令地址以及指令中有关地址的部分(称为有效地址)进行调整,这个调整过程称为地址重定位。

地址重定位建立用户程序的逻辑地址与物理地址之间的对应关系,实现方式包括静态地址重定位和动态地址重定位。静态地址重定位是在程序执行之前由操作系统的重定位装入程序完成,程序必须占用连续的内存空间,且一旦装入内存后,程序不便于移动。动态地址重定位则在程序执行期间进行,由专门的硬件机构来完成,通常采用一个重定位寄存器(relocation register,其内容是程序装入内存的起始地址),在每次进行存储访问时,将取出的逻辑地址加上重定位寄存器的内容形成物理地址。动态地址重定位的优点是不要求程序装入固定的内存空间,在内存中允许程序再次移动位置,而且可以部分地装入程序运行,同时也便于多个作业共享同一程序的副本。因此,动态地址重定位技术被广泛采用。

下面简单介绍几种基本的存储管理技术。

2. 连续存储管理

这种存储管理也称为界地址存储管理。其基本特点是,内存空间被划分成一个个分区,一个作业占一个分区,即系统和用户作业都以分区为单位享用内存。

在这种内存分配方式中,地址重定位采用静态地址重定位方法,分区的存储保护可采用上、

下界寄存器保护方式。为了实现地址的转换,需要设置一个基址寄存器(也称重定位寄存器)BR与限长寄存器(也称界限寄存器)LR。具体的地址变换过程如下:

当一个作业被调入内存运行时,首先给这个作业分配一个内存分区,同时将该分区的首地址送到BR,该分区的长度送到LR。在该作业运行过程中,将指令中的有效地址转换成实际的内存地址,其转换的关系为

$$\text{实际内存地址 } D = \text{BR} + \text{指令中的有效地址}$$

如果 $\text{BR} \leq D < \text{BR} + \text{LR}$, 则按地址D进行访问;如果 $D < \text{BR}$ 或 $D \geq \text{BR} + \text{LR}$, 则说明地址越界错。

在分区分配方式中,分区的大小可以是固定的(称为固定分区),也可以是可变的(称为可变分区或动态分区)。

在固定分区分配方式中,系统把内存划分成若干大小固定的分区,一个分区可以分给一个作业使用,直到某个作业完成后,才把其所用的分区归还系统。固定分区的大小是根据系统要处理的作业的一般规模来确定,如果分区太大,会造成内存空间的浪费,但如果分区太小,则对于稍大一些的作业就无法调入内存运行。固定分区存储管理的最大优点是简单,要求的硬件支持少;缺点是容易产生内部碎片。分配给某作业的分区中,未被使用的空闲部分称为分区的内部碎片(internal fragments)。如图1.12(a)所示,假定每个作业分区大小为32KB,但作业1实际只需要20KB,这样会产生12KB的内部碎片。

在可变分区分配方式中,在作业调入内存时建立一个大小恰好与作业匹配的分区,这样就避免了每个分区对存储空间利用不充分的问题。可变分区虽然避免了固定分区中每个分区都可能有余空间的情况,但由于它的空闲区域仍是离散的,可变式分区会出现外部碎片(external fragments)。系统运行某个时刻内存分配如图1.12(b)所示,有3个空闲区1、2和3,大小分别为10KB、12KB和10KB,合计32KB。如果当前有一个大小为30KB的进程要运行,则该进程无法装入进来,因为没有大于或等于30KB的连续内存。

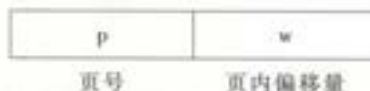


图 1.12 内部和外部碎片例

3. 分页式存储管理

在分页存储管理方式中,作业空间被划分为页,实际的内存空间被划分为块,其中页的大小与块的大小相等。当某个作业被调入内存运行时,由重定位机构将作业中的页映射到内存空间对应的块上,从而实现地址的转换。

为了具体实现地址转换,在分页系统中,用户程序指令中的有效地址的结构如下:



当作业提出存储分配请求时,系统首先根据存储块大小把作业分成若干页。每一页可存储在内存的任意一个空白块内。这样,只要建立起程序的逻辑页和内存的存储块之间的对应关系,借助动态地址重定位技术,分散在不连续物理存储块中的用户作业就能够正常运行。

(1) 分页式存储管理的地址重定位

分页式存储管理通常会在内存中为每个作业开辟一块特定区域,建立起作业的逻辑页与存



储块之间的对应关系表,称为页面映像表或简称页表(page table)。最简单的页表只包含页号、块号两个内容。页表的起始地址和长度放在该作业的进程控制块 PCB 中。当前运行作业的页表由一个专用的控制寄存器(页表始址寄存器)来指定。当要运行一个新作业时,就将该作业的页表始址、长度从进程控制块中取出。

在作业执行过程中,由硬件地址分页机构自动将每条程序指令中的逻辑地址解释成两部分——页号 p 和页内地址 w 。通过页号查页表得到存储块号 b ,与页内地址 w 组合在一起形成物理地址,访问内存后得到操作数据。

(2) 分页式存储保护

分页式存储管理中的存储信息保护可从两个方面实现,一方面是在进行地址变换时,产生的页号应小于页表长度,否则视为越界访问,这类似于基址—限长存储保护;另一方面,可在页表中增加存取控制和存储保护的信息;对每一个存储块,可允许 4 种保护方式:禁止做任何操作、只能执行、只能读、能读/写。当要访问某页时,先判断该页的存取控制和存储保护信息是否允许。添加了存取控制信息的页表表项如下:

页号	块号	存取控制信息
----	----	--------

分页式存储管理的最大优点是能有效解决碎片问题,内存利用率高,内存分配与回收算法也比较简单;缺点是采用动态地址变换机构增加了硬件成本,也降低了处理机速度。

4. 分段式存储管理及段页式存储管理

(1) 分段式存储管理

分段式(segmentation)存储管理较好地解决了程序和数据的共享以及程序动态链接等问题。在分段式存储管理中,作业的地址空间由若干个逻辑分段组成,每一分段是一组逻辑意义完整的信息集合,并有自己的名字(段名)。每一段都是以 0 开始的连续的一维地址空间,整个作业则构成了二维地址空间。

分段式存储管理是以段为基本单位分配内存的,且每一段必须分配连续的内存空间,但各段之间不要求连续。由于各段的长度不一样,所以分配的内存空间大小也不一样。分段式存储管理的逻辑地址结构如下:

段号 s	段内位移 w
--------	----------

分段式存储管理方式管理由若干段组成的作业,并且按段来进行存储分配。分段式存储管理的关键在于如何把分段地址结构变成一维的地址结构。与分页式存储管理一样,分段式存储管理采用动态重定位技术来进行地址转换。分页的优点体现在内存空间的管理上,而分段的优点体现在地址空间的管理上。

(2) 段页式存储管理

分页存储管理能有效地提高内存的利用率,而分段存储管理能够反映程序的逻辑结构以满足用户的需要,并且可以实现段的共享。段页式存储(segmentation with paging)管理则是分页和分段两种存储管理方式的结合,它同时具备两者的优点。

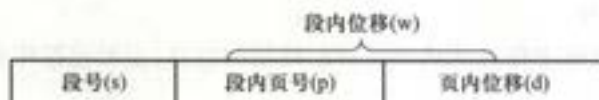
段页式存储管理是目前使用较多的一种存储管理方式,它有如下特点。

① 作业地址空间进行段式管理,即将作业地址空间分成若干个逻辑分段,每段都有自己的段名。

② 每段内再分成若干大小固定的页,每段都从零开始为自己的各页依次编写连续的页号。

③ 对内存空间的管理仍然和分页存储管理一样,将其分成若干个与页面大小相同的物理块,对内存空间的分配是以物理块为单位的。

④ 作业的逻辑地址包括3个部分:段号、段内页号和页内位移。其结构如下:



对上述3个部分的逻辑地址来说,用户可见的仍是段号 s 和段内位移 w ,由地址变换机构将段内位移 w 的高几位作为段内页号 p ,低几位作为页内位移 d 。

为实现地址变换,段页式系统设立了段表和页表。系统为每个作业建立一张段表,并为每个段建立一张页表。段表表项中至少包含段号、页表起始地址和页表长度等信息。其中,页表起始地址指出了该段的页表在内存中的起始存放地址。页表表项中至少应包括页号和块号等信息。此外,为了指出运行作业的段表起始地址和段表的长度,硬件需要有一个段表控制寄存器。

5. 虚拟存储器管理

一般来说,一个作业的大小不能超过实际内存空间的大小,实际内存空间是用户进行程序设计时可以利用的最大空间。前面所讨论的分区(固定分区和可变式分区)存储管理和分页、分段式存储管理技术,都要求作业在执行之前必须一次性地全部装入内存,这就要求作业的逻辑地址空间不能比实际的内存空间大,否则无法装入内存运行。但在实际上,根据程序的时间局部性和空间局部性,在作业运行过程中可以只让当前用到的信息进入内存,其他当前未用的信息留在外存;而当作业进一步运行需要用到外存中的信息时,再把已经用过但暂时还不会用到的信息换到外存,把当前要用的信息换到已空出的内存区中,从而给用户提供一个比实际内存空间大得多的地址空间。对于用户来说,这个特别大的地址空间就好像是可以自由使用的内存空间一样。这种大容量的地址空间并不是真实的存储空间,而是虚拟的,因此,称这样的存储器为虚拟存储器(virtual memory)。用于支持虚拟存储器的外存称为后备存储器。虚拟存储器是对主存的逻辑扩展。从程序员的角度看,外存被看作逻辑存储空间,访问的地址是一个逻辑地址(虚地址)。虚拟存储器使存储系统既具有相当于外存的容量又有接近于主存的访问速度。

(1) 请求页式存储管理

与分页式存储管理不同,如果作业的所有页面并不是一次全部装入,而是根据作业运行时的实际要求只装入了目前运行所要用到的一些页,其余的页仍保存在外存,等到需要时再请求系统调入,这种分页管理称为请求页式存储管理。

请求页式存储管理与分页式存储管理在内存块的分配与回收、存储保护等方面都十分相似,不同之处在于地址重定位问题:在请求页式存储管理的地址重定位时,可能会出现所需页面不在内存的情况。此时,可以在页表表项中增加一个状态位,状态位用以表示当前页是否在内存中。比如,状态位为 N 表示该页不在内存中,状态位为 Y 表示该页已在内存中。

按照分页式管理的处理,程序中的逻辑地址变换时,根据页号可查页表,如该页在内存,查找



得到相应块号,转换成物理地址,执行程序指令;如该页不在内存,则会引起缺页中断(page fault)。发生缺页中断后,系统处理中断,将该页从外存读入,并修改页表项中的状态位,填入实际内存块号,之后再转换成物理地址,继续运行程序指令。

在请求分页系统中,为了实现地址转换,需要建立一些表,它们既可以由硬件寄存器实现,也可以借助内存单元来实现:

① 作业表 JT。记录每个作业的状态与资源使用的信息,主要包括作业号、页表大小、页表地址等。

② 每一个作业要有一个页表 PMT。记录该作业每一页的页号(从第 0 页开始)以及该页是否在内存的标志,如果某页已进入内存,则还记录该页在内存中的块号。

③ 存储分块表 MBT。记录内存空间中每一块的使用情况,系统实际上就是按照这个表的内容来具体分配或释放内存块。

图 1.13 表示了请求分页系统中页表、存储分块表与作业表三者之间的关系。由图 1.13 可以看出,系统中共有 4 个作业,开始时,作业 1、2、3 的所有页面以及作业 4 的第 0、1 页已经在内存,而作业 4 的第 2、3 页不在内存。

在需要将页面从外存调入时,内存中如果没有空闲块,就需要进行页面置换:选定某个页面(被置换的页),有时可能要把该页面写到外存中,修改相应的标志,把需要调入的页面写到这个页面上,并修改相应的页标志。发生页面置换时,如果被置换的页面在进入内存后未被修改过,则不必写回外存;如果已被修改,则必须把该页面写回外存。因此,请求页式管理在页表中一般还有修改位,以标志该页在内存中是否被修改过。

当发生缺页中断时,如果内存中已无空闲块,就要把已在内存的一些页面置换出去。页面置换算法即确定淘汰内存中的某些页所采用的策略。常用的页面置换算法包括以下几种。

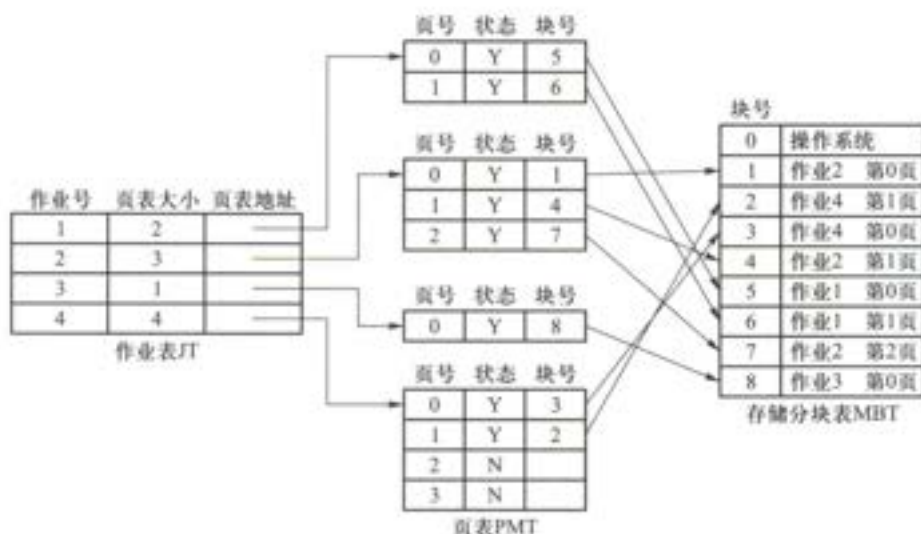


图 1.13 请求分页系统管理机构示意图

① 最优算法(optimal algorithm, OPT)。最优算法是最理想的页面置换算法:从内存中移出

以后不再使用的页面;如果没有这样的页面,则选择以后最长时间内不再需要访问的页面。由于页面访问后续的顺序一般难以预知,所以最优算法只具有理论上的意义。

② 先进先出算法(first in first out, FIFO)。先进先出算法总是先淘汰那些驻留在内存时间最长的页面,即先进入内存的页面会先被置换掉,因为最先进入内存的页面不再被访问的可能性最大。这种算法实现起来比较简单。

③ 最近最久未使用算法(least recently used, LRU)。如果某一页被访问了,那么它很可能马上又会被访问;反之,如果某一页很长时间没有被访问,那么最近也不太可能会被访问。最近最久未使用算法考虑了程序的局部性原理,当需要置换一页时,选择最近一段时间最久未使用的页面予以淘汰。LRU 算法能够比较普遍地适用于各种类型的程序,但是实现起来比较困难。在实际中得到广泛应用的是一种近似 LRU 的简单而有效的算法。

请求页式存储管理具有以下一些优点:

由于提供了大容量的虚拟存储器,用户的地址空间不再受内存大小的限制,大大方便了用户的程序设计;可以容纳更多的作业进入系统,更有利于多道程序的运行。另外,由于作业地址空间中的各页面都是按照需要调入内存的,不用的信息不会调入内存,很少用的信息也只是短时间驻留在内存,因此更有效地利用了内存。

(2) 请求段式存储管理

与请求页式存储管理相似,基于程序局部性原理,段式逻辑地址空间中的程序段在运行时也并不需要全部装入内存,而是在需要时再调入内存。这样,在程序运行时,首先调入一个或若干个程序段运行,在运行过程中调用到其他段时,会产生段错误(segmentation fault),操作系统就根据该段长度在内存分配一个连续的分区给该段使用。若内存中没有足够大的空闲分区,则考虑进行段的紧凑(compaction)或将某个段或某些段淘汰出去。对应于请求页式存储管理,这种存储管理技术称为请求段式存储管理(demand segmentation)。

同样的道理,还有请求段页式存储管理,在此不再详述。

1.3.4 文件管理

操作系统的重要功能之一是为计算机系统提供数据存储、数据处理和数据管理的功能。数据通常以文件形式存放在磁盘或其他外部存储介质上。在现代操作系统中,用户的程序和数据、操作系统自身的程序和数据,甚至各种外部设备,都是以文件形式提供给用户的,其使用也相当频繁。因此文件管理系统在操作系统中占有非常重要的地位。

1. 文件及文件系统

所谓文件(file),是指一组带标识(标识即为文件名)的、在逻辑上有完整意义的信息项的序列。文件所包括的范围很广。例如,用户作业、源程序和目标程序、初始数据和输出结果等,都是以文件的形式存在的;系统软件的资源,如汇编程序、编译程序和连接装配程序,以及编辑程序、调试程序和诊断程序等实用程序,也都是以文件的形式存在的。各类文件都是由文件系统来统一管理的。

所谓文件系统(file system),是指负责存取和管理文件信息的软件机构。借助于文件系统,用户可以简单、方便地使用文件,而不必考虑文件存储空间分配,也无须知道文件的具体存放位置,文件的存储和访问均由文件系统自动处理。同时,在文件系统中,通过文件的存取权限,对



文件提供保护措施,并提供转储功能,为文件复制后备副本等。总之,文件系统一方面要方便用户,实现对文件的“按名存取”;另一方面要实现对文件存储空间的组织、分配和文件信息的存储,并且要对文件提供保护和有效的检索等功能。

随着操作系统的不断发展,越来越多的功能强大的文件系统不断涌现。这里列出一些常用的具有代表性的文件系统。

① EXT2/4:Linux 最为常用的文件系统。

② NFS:网络文件系统,允许多台计算机之间共享文件系统,易于从网络中的计算机上存取文件。

③ HPFS:高性能文件系统,是 IBM OS/2 的文件系统。

④ FAT:经过了 MS-DOS 及后来的 Windows 3.x、Windows 9x、Windows NT、Windows 2000/XP 和 OS/2 等操作系统的不断改进,它已经发展出包含 FAT12、FAT16 和 FAT32 的庞大家族。

⑤ NTFS:NTFS 是微软为了配合 Windows NT 的推出而设计的文件系统,为系统提供了极大的安全性和可靠性保障。

(1) 文件类型

为了有效、方便地组织和管理文件,常对文件进行分类,以提高文件的处理速度以及更好地实现文件的保护和共享。文件依不同标准可以有多种分类方式,按用途分为系统文件、库文件(由标准的和非标准的子程序库构成的文件)和用户文件;按性质分为普通文件、目录文件和特殊文件(比如在 UNIX 系统中,所有的输入/输出设备都被看作是特殊的文件);按保护级别分为只读文件、读写文件、可执行文件和不保护文件(用户具有一切权限);按文件数据的形式分为源文件、目标文件和可执行文件。

(2) 文件系统模型

文件系统的传统模型为层次模型,该模型由许多不同的层组成。每一层都会使用下一层的功能特性来创建新的功能,为上一层服务。模型的每一层都在下层的基础上向上层提供更多的功能,由下至上逐层扩展,从而形成一个功能完备、层次清晰的文件系统。

文件系统层次模型对支持单个文件系统比较合适。现代操作系统一般可同时支持多个文件系统,采用了如 Sun 公司的虚拟文件系统(virtual file system, VFS)类技术,可以支持多种文件系统,如 EXT2、FAT 和 NTFS 等。

2. 文件的组织结构

(1) 文件的逻辑结构

文件的逻辑结构就是从用户观点出发所见到的文件结构,通常分为两种:记录式文件和流式文件。记录式文件在逻辑上总是被看成一组顺序记录的集合,是一种有结构的文件组织,并且根据记录长度可分成定长记录文件和变长记录文件。流式文件又称无结构文件,是由一组相关信息组合成的有序字符流。这种文件的长度直接按字节计算。

(2) 文件的物理结构

文件的物理结构是指文件在外部存储介质上的存放形式,也叫文件的存储结构。它对文件的存取方法有较大的影响。文件在逻辑上看都是连续的,但在物理介质上存放时却不一定连续。基本的文件物理存储组织形式包括顺序结构、链接结构和索引结构。顺序结构把一个逻辑上连续的文件信息存放在连续编号的物理块中,只需给出首块块号和文件长度;链接结

构把一个逻辑上连续的文件分散地存放在不同的物理块中,在各物理块中设立一个指针(称为连接字),它指示该文件的下一个物理块,最后一个块连接字为 NULL,表示该块是文件结尾;索引结构实现中,系统为每个文件建立一个索引表,其中的表项指出存放该文件的各个物理块号,索引表在文件属性说明项中指出。索引方式可以方便地进行随机存取。当文件很大时,单一索引表结构无法满足灵活性和节省内存的需要,这时可使用多重索引结构(又称多级索引结构)。

3. 文件目录管理

(1) 文件目录概念

为了根据文件名存取文件,必须建立文件名与文件在外存空间中的物理地址的对应,表示这种对应关系的数据结构称为文件目录(directory)。把若干文件目录组织在一起,以文件的形式保存在外存上,就形成了目录文件。

每一个文件在文件目录中登记为一项,作为文件系统建立和维护文件的清单。每个文件的文件目录项又称文件控制块(file control block,FCB)。FCB一般应该包括以下内容。

① 有关文件存取控制的信息。如文件名、用户名、文件主存取权限、授权者存取权限、文件类型和文件属性,即可读写文件、执行文件、只读文件等。

② 有关文件结构的信息。文件的逻辑结构,如记录类型、记录个数、记录长度等;文件的物理结构,如文件所在设备名、文件物理结构类型、记录存放在外存的相对位置或文件第一块的物理块号,也可指出文件索引的所在位置等。

③ 有关文件使用的信息。已打开该文件的进程数、文件被修改的情况、文件最大长度和当前大小等。

④ 有关文件管理的信息。如文件建立日期、最近修改日期和最后访问日期,以及记账信息等。

每当创建一个新文件时,系统就要为它建立一个FCB,多个文件的FCB便组成了文件目录。当用户要求存取某个文件时,系统查找目录文件,先找到对应的文件目录,然后比较文件名就可找到相应文件的FCB(文件目录项),再通过FCB中的文件信息就能存取文件。在文件目录实现中,为了减少检索文件访问的物理块数,会把文件目录项中的文件名和其他管理信息分开,后者单独组成称为索引节点(i-node或inode,也称i节点)的一个数据结构。这样不仅加快了目录检索速度,而且也便于实现文件的共享,有利于系统的控制和管理。

(2) 文件目录结构

为了便于对文件进行存取和管理,系统要建立一个用于存放每个文件的有关信息的文件目录。文件系统的基本功能之一,就是负责文件目录的编排、维护和检索。

根据文件目录的组织结构,可以将目录分为单级目录、二级目录、多级层次目录、无环图结构目录和图状结构目录等。

① 单级目录。单级目录(single-level directory)又称简单文件目录,它是一个线性表,在这个线性表的每一个目录项中包含以下一些信息:文件名、有关文件结构的信息(包括逻辑结构和物理结构)、有关存取控制的信息、有关管理的信息等。

目录项中究竟包括哪些内容,这要根据系统的要求而定。由于各个用户使用同一个目录表,因此,要防止不同用户对各自的文件取相同的名字。一旦文件名重复,就无法“按名存取”所需



要的文件。这是简单文件目录的一个缺点。

② 二级目录。二级目录(two-level directory)结构允许每个用户建立各自的名字空间,并通过建立相应的总文件目录来管理这些名字空间。各个用户的名字空间构成了各个用户文件目录(user file directory, UFD),而管理这些用户目录的总文件目录称为主目录(master file directory, MFD)。通常,在主目录中的各项说明了用户目录的名字、目录大小以及所在物理位置等信息。而各用户目录中的各项说明了各文件的具体位置和其他一些属性。二级目录结构如图 1.14 所示。

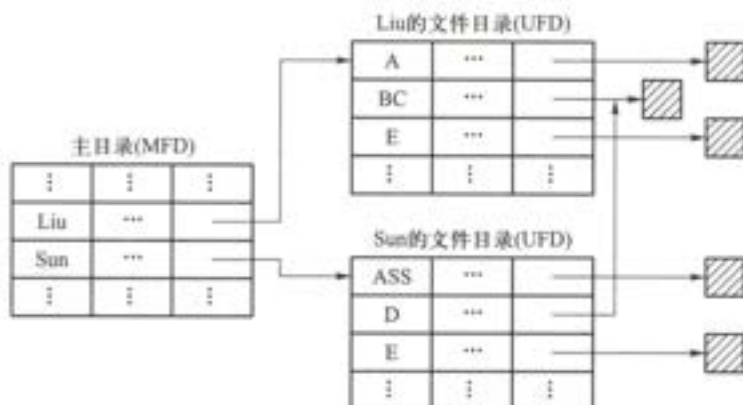


图 1.14 二级目录结构

在多用户情况下,采用二级目录结构是比较方便的。当一个新用户要建立文件时,系统为其在主目录中分配一个表目,并为其分配一个存放二级目录的存储空间,同时要为新建的文件在二级目录中分配一个表目,并分配文件存储空间。当用户要访问一个文件时,先按用户名在主目录中找到该用户的二级目录,然后在二级目录中按文件名找出该文件的起始地址并进行访问。

显然,二级目录结构解决了文件重名的问题,也可以解决文件的连接问题。实际上,只要把目录指针指向要连接的文件登记项即可实现文件共享。例如,在图 1.14 中,用户 Sun 用名字 D 连接了用户 Liu 的文件 BC。

二级目录具有以下特点:提高了文件检索速度(用户名大大缩小了需要检索的文件数量),部分地允许文件名重名。但对同一用户,也不能有两个同名的文件存在。

③ 多级层次目录。多级层次目录也叫树结构目录(tree-structured directories),是二级目录的推广。在多级层次目录中,有一个根目录和许多分目录。分目录不但可以包含文件,还可以包含下一级的分目录。这样推广就形成了多级层次目录。

相比二级目录,多级层次目录具有以下优点:既可方便用户查找文件,又可以把不同类型和不同用途的文件分类;允许文件重名;不但不同用户可以使用相同名称的文件,同一用户也可使用相同名称的文件;利用多级层次结构关系,可以更方便地制定保护文件的存取权限,有利于文件的保护。

多级层次目录也有其缺点,比如不能直接支持文件或目录的共享。为了使文件或目录可以被不同的目录所共享,进一步形成了结构更复杂的无环结构目录和图状结构目录等。

(3) 存取权限

存取权限可以通过建立访问控制表(access control list, ACL)和存取权限表(capability list)来实现。访问控制表是以文件为单位建立的,存取权限表是以用户或用户组为单位建立的。存取权限表将一个用户或用户组所要存取的文件集中起来存入一张表中,其中每个表目指明用户(用户组)对相应文件的存取权限:读(r),写(w)和执行(x)。

大型文件系统主要采用两个措施来进行安全性保护:一是对文件和目录的权限设置;二是对文件和目录进行加密。对文件和目录的权限进行设置,就能使用户可以共享相应权限的文件数据,不仅可以为不同用户完成共同任务提供基础,而且还节省了大量的磁盘空间。

4. 文件空闲区的组织

一个大容量的文件存储器要为系统和许多用户所共享。能自动地为用户文件分配存储空间,这是文件系统的又一重要功能。下面介绍几种文件存储空间管理的方案。

(1) 空闲文件项和空闲区表

空闲文件项是一种最简单的空闲区管理方法。在这种方法中,空闲区与其他文件目录放在一张表中。在分配时,系统依次扫描这个目录表,从标记为空闲的项中寻找长度满足要求的项,然后把相应项的空闲标记去掉,填上文件名。在删除文件时,只要把文件名栏标记为空闲即可。

在这种空闲区管理方案中,由于空闲区与真正的目录混在一起,因此,无论是分配空间还是查找目录,效率都不高。另外,如果空闲区比所申请的区要大,则多余的部分有可能被浪费。为解决这些问题,可以采用空闲区表的方法,即将空闲区项抽出来单独构成一张表,这样可以减少目录管理的复杂性,提高文件查找和空闲区查找的速度。

(2) 空闲块链

所谓空闲块链,是指将所有空闲块链接在一起。当需要空闲块时,从链头依次摘取一(些)块,并将链头指针依次指向后面的空闲块。当文件被删除而释放空闲块时,只需将被释放的空闲块挂到空闲块链的链头即可。

(3) 位示图

位示图的方法是用若干字节构成一张表,表中的每一个二进制位对应一个物理块,并依次顺序编号。如果位标记为“1”,则表示对应的物理块已分配;位标记为“0”,则表示对应物理块为空闲。在存储分配时,只要把找到的空闲块位标记改为“1”;释放时,只要把相应的位标记改为“0”即可。

(4) 空闲块成组链接法

在 UNIX 操作系统中,采用改进空闲块链方法来组织管理文件存储空间。其方法是,将所有的空闲块进行分组,再通过指针将组与组之间链接起来。这种空闲块的管理方法称为成组链接法。在此就不再详述了。

1.3.5 I/O 设备管理

计算机系统中外部设备种类众多,速度差异极大,接口方式也有很大不同,这使设备管理成为操作系统中最为庞杂和琐碎的部分。为了方便用户使用各种外部设备,设备管理要能为不同设备提供统一界面、发挥系统并行性且方便使用,从而使 I/O 设备被高效使用。为此,设备管理通常应具有以下基本功能:外围设备中断处理,缓冲区管理,外围设备的分配,外围设备驱动调度等。而且,为了进一步提高系统效率,操作系统还有虚拟设备及其实现等功能。

1. 输入/输出软件的层次结构

输入/输出软件的设计目标就是将软件组织成一种层次结构,底层的软件用来屏蔽输入/输出硬件的细节,从而实现上层的设备无关性(即设备独立性),高层软件则要为用户提供一个统一、规范、方便的接口。操作系统实现时把输入/输出软件分成以下层次:中断处理程序、设备驱动程序、与设备无关的 I/O 软件、用户层的输入/输出软件,如图 1.15 所示,图中的箭头表示控制流的方向。



图 1.15 I/O 系统的层次结构

当用户程序从文件中读一个数据块时,需要通过操作系统的系统调用来执行此操作。设备无关的 I/O 软件首先在数据块缓冲区中查找此数据块。若未找到,则调用设备驱动程序向硬件提出相应的请求。用户进程随即阻塞,直至数据块读出。当磁盘操作结束时,硬件发一个中断给 CPU,操作系统响应中断而执行中断处理程序。中断处理程序从设备获得返回状态值,并唤醒被阻塞的用户进程来结束此次 I/O 请求处理。随后用户进程将继续进行。

2. 中断处理过程

外部设备完成了 I/O 操作后,设备控制器便向 CPU 发出一个中断请求,CPU 响应后便转向中断处理程序。在单 CPU 机器上的中断处理过程如下:

- ① 检查 CPU 响应中断的条件是否满足;
- ② 如果满足,CPU 响应中断,立即关中断(避免中断被打断);
- ③ 保存被中断进程的 CPU 环境:比如当前处理机的状态字 PSW 和程序计数器 PC 等的内容及其他寄存器,如段寄存器和通用寄存器等(因在中断处理时可能会用到这些寄存器);
- ④ 分析产生中断的原因,转入相应设备的中断处理程序;
- ⑤ 执行中断处理程序;
- ⑥ 恢复被中断进程的 CPU 现场,返回被中断的程序;
- ⑦ 开中断,CPU 继续执行。

I/O 操作完成后,驱动程序必须检查本次 I/O 操作中是否发生了错误,以便向上层软件报告。

3. 设备驱动程序

不同类型的设备甚至同一类型的不同品牌或型号的设备都可能有不同的设备驱动程序。设备驱动程序(device driver)是指驱动物理设备和 DMA 控制器或 I/O 控制等直接进行 I/O 操作的子程序集合。设备驱动程序主要负责启动对应设备进行 I/O 操作。设备驱动程序有如下功能:

- ① 可将接收到的抽象要求(如打开或关闭)转换为具体要求;
- ② 接受用户的 I/O 请求:设备驱动程序将用户的 I/O 请求排在请求队列的队尾,检查 I/O 请求的合法性,了解 I/O 设备的状态,传递有关参数等;
- ③ 取出请求队列中的队首请求,将相应设备分配给它,然后启动该设备工作,完成指定的 I/O 操作;
- ④ 处理来自设备的中断:及时响应由控制器或通道发来的中断请求,并根据其中断类型调用相应的中断程序进行处理。

4. 与设备无关的 I/O 软件

为了实现设备独立性,就必须在与硬件紧密相关的驱动程序之上设置一层与设备无关的软件。与设备无关的软件提供适用于所有设备的常用 I/O 功能,并向用户层软件提供一个统一的接口,其主要功能如下:

- ① 向用户层软件提供统一接口。无论哪种设备,它们向用户所提供的接口相同,例如对各种设备的读操作,在应用程序中都用 read,而写操作则都用 write;
- ② 设备命名。设备无关程序负责将设备名映射到相应的设备驱动程序上;
- ③ 设备保护。操作系统为各个用户赋予不同的设备访问权限,以实现对设备的保护;
- ④ 提供一个独立于设备的块。设备无关软件屏蔽了不同设备使用的数据块大小可能不同的特点,向用户软件提供了统一的逻辑块大小。

此外,与设备无关的软件还负责对独占设备的分配与回收、设备使用缓冲区的管理及设备的差错控制(设备驱动程序无法处理的错误)等。

5. 用户层的 I/O 软件

用户层的 I/O 软件是 I/O 系统软件分层中的最上层,它面向程序员,负责与用户和设备无关的 I/O 软件通信。当接收到用户的 I/O 指令后,该层会把具体的请求发送到设备无关的 I/O 软件进行进一步的处理。它主要包含用于 I/O 操作的库例程和 SPOOLing 系统(simultaneous peripheral operations on-line,即在联机情况下实现的同时外围操作)。用户层的输入/输出软件完全屏蔽了具体的硬件细节,向用户提供统一的接口。

在多道程序设计中,SPOOLing 系统是将一台独占设备改造为共享设备的一种行之有效的技术。例如,在打印一个文件之前,进程首先产生完整的待打印文件并将其放在 SPOOLing 目录下。当进程把该文件放到 SPOOLing 系统中之后,就可以认为打印过程已经完成。实际的打印作业由 SPOOLing 系统守护进程(daemon)进行处理,只有该守护进程能够真正使用打印机设备文件。

用户层 I/O 软件还会使用到缓冲技术。尽管随着计算机技术的发展,外设的速度也在不断提高,但很多设备与 CPU 的速度相差甚远。使用缓冲技术有几个方面的好处:首先,引入缓冲技术可以改善 CPU 和 I/O 设备之间速度不匹配的情况,比如上面打印中设置了缓冲区,则程序输出的数据先送到缓冲区,然后由打印机慢慢输出,使 CPU 和打印机得以并行工作;其次,虽然通道技术和中断技术可为计算机系统的并行活动提供支持,但往往由于通道数量不足而产生“瓶颈”现象,缓冲技术的引入可以减少占用通道的时间,明显提高 CPU、通道和 I/O 设备的并行程度;再次,缓冲技术的引入还可以减少外设对 CPU 的中断次数,放宽 CPU 对中断响应时间的限制。缓冲技术的引入提高了系统的处理能力和设备的利用率。

缓冲技术的实现主要是设置合适的缓冲区。缓冲区可以用硬件寄存器来实现硬缓冲,另外较经济的方式是在内存中设置软缓冲,缓冲区的大小一般与盘块的大小一样。缓冲区的个数可根据数据输入/输出的速率和加工处理的速率之间的差异情况来确定。

6. 设备的分配与回收

为了方便系统的管理,进程在使用资源时必须首先向设备管理程序提出资源申请,只要是可能和安全的,便由设备分配程序根据相应的分配算法为进程分配资源。如果申请进程得不到它所申请的资源,将被放入相应资源的等待队列中,直到所需要的资源被释放后才会分配给某个等



待的进程。如果进程得到了它所需要的资源,就使用该资源完成相关的操作,使用完之后通知系统,系统将及时回收这些资源,以便其他进程使用。

习题 1

- 下列计算机中整数的表示法中,可以直接作加减运算的是
A) 原码 B) 反码 C) 补码 D) 偏移码
- 裸机指的是
A) 没有软件系统的计算机 B) 没有应用软件的计算机系统
C) 放在露天的计算机 D) 缺少外部设备的计算机
- 下列存储器中,访问速度最快的是
A) 磁带 B) 磁盘 C) USB D) 内存存储器
- 如果作业的逻辑地址空间大于计算机实际的内存空间,则应采用的存储管理技术是
A) 请求分页式存储管理 B) 分区存储管理
C) 分段式存储管理 D) 段页式存储管理
- 主存储器和 CPU 之间增加高速缓冲存储器的目的是
A) 扩大主存储器的容量
B) 扩大 CPU 中通用寄存器的数量
C) 解决 CPU 和主存之间的速度匹配问题
D) 既扩大主存容量又扩大 CPU 通用寄存器数量
- 允许在一台主机上同时连接多台终端,且多个用户可以通过各自的终端同时交互地使用计算机的操作系统是
A) 分时操作系统 B) 网络操作系统 C) 实时操作系统 D) 分布式操作系统
- 多道程序设计技术是指
A) 将多个程序用多个 CPU 同时运行
B) 允许多个程序同时进入内存并运行
C) 将一个程序分成多个小程序用多个 CPU 运行
D) 将一个程序分成多个小程序用一个 CPU 分别运行
- 操作系统管理进程所使用的数据结构是
A) 进程控制块 PCB B) 文件控制块 FCB
C) 设备控制块 DCB D) 目录控制块
- 操作系统提供了进程管理、设备管理、文件管理和
A) 存储器管理 B) 通信管理 C) 用户管理 D) 数据管理
- 文件系统中用于管理文件的是
A) 目录 B) 指针 C) 页表 D) 堆栈结构

第2章

数据结构与算法

2.1 算法

2.1.1 算法的基本概念

所谓算法是指解题方案的准确而完整的描述。

对于一个问题,如果可以通过一个计算机程序,在有限的存储空间内运行有限长的时间而得到正确的结果,则称这个问题是算法可解的。但算法不等于程序,也不等于计算方法。当然,程序也可以作为算法的一种描述,但程序通常还需考虑很多与方法和分析无关的细节问题,这是因为在编写程序时要受到计算机系统运行环境的限制。通常,程序的编制不可能优于算法的设计。

作为一个算法,一般应具有以下几个基本特征。

(1) 可行性(effectiveness)

算法的可行性包括以下两个方面:

① 算法中的每一个步骤必须能够实现。如在算法中不允许执行分母为0的操作,在实数范围内不可能求一个负数的平方根等。

② 算法执行的结果要能够达到预期的目的。

针对实际问题设计的算法,人们总是希望能够得到满意的结果。但一个算法又总是在某个特定的计算工具上执行的,因此,算法在执行过程中往往要受到计算工具的限制,使执行结果产生偏差。例如,在进行数值计算时,如果某计算工具具有7位有效数字(如程序设计语言中的单精度运算),则在计算下列三个量

$$A=10^{12}, B=1, C=-10^{12}$$

的和时,如果采用不同的运算顺序,就会得到不同的结果,即

$$A+B+C=10^{12}+1+(-10^{12})=0$$

$$A+C+B=10^{12}+(-10^{12})+1=1$$

而在数学上, $A+B+C$ 与 $A+C+B$ 是完全等价的。因此,算法与计算公式是有差别的。在设计一个



算法时,必须要考虑它的可行性,否则是不会得到满意结果的。

(2) 确定性(definiteness)

算法的确定性,是指算法中的每一个步骤都必须是有明确定义的,不允许有模棱两可的解释,也不允许有多义性。这一性质也反映了算法与数学公式的明显差别。在解决实际问题时,可能会出现这样的情况:针对某种特殊问题,数学公式是正确的,但按此数学公式设计的计算过程可能会使计算机系统无所适从。这是因为根据数学公式设计的计算过程只考虑了正常使用情况,而当出现异常情况时,此计算过程就不能适应了。

(3) 有穷性(finiteness)

算法的有穷性,是指算法必须能在有限的时间内做完,即算法必须能在执行有限个步骤之后终止。数学中的无穷级数,在实际计算时只能取有限项,即计算无穷级数值的过程只能是有穷的。因此,一个数的无穷级数表示只是一个计算公式,而根据精度要求确定的计算过程才是有穷的算法。

算法的有穷性还应包括合理的执行时间的含义。因为,如果一个算法需要执行千万年,显然失去了实用价值。

(4) 拥有足够的情报

一个算法是否有效,还取决于为算法所提供的情报是否足够。通常,算法中的各种运算总是要施加到各个运算对象上,而这些运算对象又可能具有某种初始状态,这是算法执行的起点或是依据。因此,一个算法执行的结果总是与输入的初始数据有关,不同的输入将会有不同的结果输出。当输入不够或输入错误时,算法本身也就无法执行或导致执行有错。一般来说,当算法拥有足够的情报时,此算法才是有效的,而当提供的情报不够时,算法可能无效。

综上所述,所谓算法,是一组严谨地定义运算顺序的规则,并且每一个规则都是有效的,且是明确的,此顺序将在有限的次数下终止。

2.1.2 算法设计基本方法

计算机解题的过程实际上是在实施某种算法,这种算法称为计算机算法。计算机算法不同于人工处理的方法。

本小节介绍工程上常用的几种算法设计方法,在实际应用时,各种方法之间往往存在着一定的联系。

(1) 列举法

列举法的基本思想是,根据提出的问题,列举所有可能的情况,并用问题中给定的条件检验哪些是需要的,哪些是不需要的。因此,列举法常用于解决“是否存在”或“有多少种可能”等类型的问题,例如求解不定方程的问题。

列举法的特点是算法比较简单。但当列举的可能情况较多时,执行列举算法的工作量将会很大。因此,在用列举法设计算法时,使方案优化,尽量减少运算工作量,是应该重点注意的。通常,在设计列举算法时,只要对实际问题进行详细的分析,将与问题有关的知识条理化、完备化、系统化,从中找出规律;或对所有可能的情况进行分类,引出一些有用的信息,是可以大大减少列举量的。

下面举例说明利用列举算法解决问题时如何对算法进行优化。

例 2.1 今有鸡母一,值钱三;鸡翁一,值钱二;鸡雏一,值钱半。凡百钱买百鸡,问鸡母、鸡翁、鸡雏各几何?

假设买母鸡 I 只、公鸡 J 只、小鸡 K 只。根据题意,粗略的列举算法描述如下:

```
FOR I=0 TO 100 STEP 1 DO
  FOR J=0 TO 100 STEP 1 DO
    FOR K=0 TO 100 STEP 1 DO
      IF ((I+J+K==100)AND(3*I+2*J+0.5*K==100.0)) THEN
        PRINT I, J, K
      |
    END
```

(注:本算法描述采用的是一种大家都能理解的算法描述语言书写的,并不是用某种具体的程序设计语言编写的程序。本章中以后均采用这种描述语言来描述算法。)

在这个算法中,共嵌套有三层循环,每层循环各需要循环 101 次,因此,总循环次数为 101^3 , 大约为 100 万次。但只要对问题进行分析,发现可以对这个算法进行优化,可以减少大量不必要的循环次数。

首先,考虑到母鸡为 3 钱一只,因此,母鸡最多只能买 33 只,即算法中的外循环没有必要从 0 到 100,而只需要从 0 到 33 就可以了。

其次,考虑到公鸡为 2 钱一只,因此,公鸡最多只能买 50 只。又考虑到对公鸡的列举是在算法的第二层循环中,此时已经买了 I 只母鸡,且买一只母鸡的价钱相当于买 1.5 只公鸡。因此,由第一层循环已经确定买 I 只母鸡的前提下,公鸡最多只能买 $50-1.5I$,即第二层对 J 的循环只需从 0 到 $50-1.5I$ 就可以了。

最后,考虑到买的总鸡数为 100,而由第一层循环已确定买 I 只母鸡,由第二层循环已确定买 J 只公鸡,因此,买小鸡的数量只能是 $K=100-I-J$,即第三层循环已经没有必要了。

经过以上分析,可以将上述算法修改如下:

```
FOR I=0 TO 33 STEP 1 DO
  FOR J=0 TO 50-1.5*I STEP 1 DO
    K=100-I-J
    IF (3*I+2*J+0.5*K==100.0) THEN
      PRINT I, J, K
    |
  END
```

不难分析,经修改后的算法的列举量(即循环次数)为

$$\sum_{I=0}^{33} (51 - 1.5I) \approx 894$$

这个算法的执行结果如下:



2	30	68
5	25	70
8	20	72
11	15	74
14	10	76
17	5	78
20	0	80

列举原理是计算机应用领域中十分重要的原理。许多实际问题,若采用人工列举是不可想象的,但由于计算机的运算速度快,擅长重复操作,可以很方便地进行大量列举。列举算法虽然是一种比较笨拙而原始的方法,其运算量比较大,但在有些实际问题中(如寻找路径、查找、搜索等问题),局部使用列举法却是很有效的,因此,列举算法是计算机算法中的一个基础算法。

(2) 归纳法

归纳法的基本思想是,通过列举少量的特殊情况,经过分析,最后找出一般的关系。显然,归纳法要比列举法更能反映问题的本质,并且可以解决列举量为无限的问题。但是,从一个实际问题中总结归纳出一般的关系,并不是一件容易的事情,尤其是要归纳出一个数学模型更为困难。从本质上讲,归纳就是通过观察一些简单而特殊的情况,最后总结出一般性的结论。

归纳是一种抽象,即从特殊现象中找出一般关系。但由于在归纳的过程中不可能对所有情况进行列举,因此,最后由归纳得到的结论还只是一种猜测,还需要对这种猜测加以必要的证明。实际上,通过精心观察而得到的猜测得不到证实或最后证明猜测是错的,也是常有的事。

(3) 递推

所谓递推,是指从已知的初始条件出发,逐次推出所要求的各中间结果和最后结果。其中初始条件或是问题本身已经给定,或是通过对问题的分析与化简而确定。递推本质上属于归纳法,工程上许多递推关系式实际上是通过分析对实际问题的分析与归纳而得到的,因此,递推关系式往往是归纳的结果。

递推算法在数值计算中是极为常见的。但是,对于数值型的递推算法必须要注意数值计算的稳定性问题。

(4) 递归

人们在解决一些复杂问题时,为了降低问题的复杂程度(如问题的规模等),一般总是将问题逐层分解,最后归结为一些最简单的问题。这种将问题逐层分解的过程,实际上并没有对问题进行求解,而只是当解决了最后那些最简单的问题后,再沿着原来分解的逆过程逐步进行综合,这就是递归的基本思想。由此可以看出,递归的基础也是归纳。在工程实际中,有许多问题就是用递归来定义的,数学中的许多函数也是用递归来定义的。递归在可计算性理论和算法设计中占有很重要的地位。

下面用一个简单的例子来说明递归的基本思想。

例 2.2 编写一个过程,对于输入的参数 n ,依次打印输出自然数 1 到 n 。

这是一个很简单的问题,实际上不用递归就能解决。对应的算法描述如下:

```
wrt(int n)
```

```

FOR k=1 TO n STEP 1 DO PRINT k
RETURN

```

这个问题还可以用递归来解决。对应的算法描述如下：

```

wrt1 (int n)
|
|  IF (n≠0) THEN
|  |
|  |  wrt1 (n-1)
|  |  PRINT n
|  |
|  |  RETURN
|
|

```

在递归过程 $wrt1()$ 中, n 是形参。在开始执行过程 $wrt1()$ 时, 首先要判断形参变量值(开始时为 n) 是否不等于 0, 如果不等于 0, 则将形参值减 1 (即 $n-1$) 后作为新的实参再调用过程 $wrt1()$; 在调用过程 $wrt1()$ 时, 又需判断形参值(此时已变为 $n-1$) 是否不等于 0, 如果不等于 0, 则又将形参值减 1 (即 $n-2$) 后作为新的实参再次调用过程 $wrt1()$ ……以此类推, 直到过程 $wrt1()$ 的形参值等于 0 为止。此时, 由于在先前各层的过程调用中, 过程 $wrt1()$ 实际上没有执行完, 即各层中的形参值还没有被打印输出, 这就需要逐层返回, 以便打印输出各层中的输入参数 $1, 2, \dots, n$ 。为此, 在递归算法的执行过程中, 需要记忆各层调用中的参数, 以便在逐层返回时恢复这些参数继续进行处理。具体来说, 在函数 $wrt1()$ 开始执行后, 随着各次的递归调用, 逐次记忆各层调用中的输入参数 $n, n-1, n-2, \dots, 2, 1$, 在逐层返回时, 又依次(按记忆的相反次序)将这些参数打印输出。

在程序设计中, 递归是一个很有用的工具。对于一些比较复杂的问题, 设计成递归算法其结构清晰, 可读性也强。

由上例可以看出, 自己调用自己的过程称为递归调用过程。

递归分为直接递归与间接递归两种。如果一个算法 P 显式地调用自己则称为直接递归。如果算法 P 调用另一个算法 Q , 而算法 Q 又调用算法 P , 则称为间接递归调用。

递归是很重要的算法设计方法之一。实际上, 递归过程能将一个复杂的问题归结为若干个较简单的问题, 然后将这些较简单的问题再归结为更简单的问题, 这个过程可以一直做下去, 直到最简单的问题为止。

有些实际问题, 既可以归纳为递推算法, 又可以归纳为递归算法。但递推与递归的实现方法是大不一样的。递推是从初始条件出发, 逐次推出所需求的结果; 而递归则是从算法本身到达递归边界的。通常, 递归算法要比递推算法清晰易读, 其结构比较简练。特别是在许多比较复杂的问题中, 很难找到从初始条件推出所需结果的全过程, 此时, 设计递归算法要比递推算法容易得多。但递归算法的执行效率比较低。

(5) 减半递推技术



实际问题的复杂程度往往与问题的规模有着密切的联系。因此,利用分治法解决这类实际问题是有效的。所谓分治法,就是对问题分而治之。工程上常用的分治法是减半递推技术。

所谓“减半”,是指将问题的规模减半,而问题的性质不变;所谓“递推”,是指重复“减半”的过程。

下面举例说明利用减半递推技术设计算法的基本思想。

例 2.3 设方程 $f(x)=0$ 在区间 $[a,b]$ 上有实根,且 $f(a)$ 与 $f(b)$ 异号。利用二分法求该方程在区间 $[a,b]$ 上的一个实根。

用二分法求方程实根的减半递推过程如下:

首先取给定区间的中点 $c=(a+b)/2$ 。

然后判断 $f(c)$ 是否为 0。若 $f(c)=0$,则说明 c 即为所求的根,求解过程结束;如果 $f(c) \neq 0$,则根据以下原则将原区间减半:

若 $f(a)f(c)<0$,则取原区间的前半部分;

若 $f(b)f(c)<0$,则取原区间的后半部分。

最后判断减半后的区间长度是否已经很小:

若 $|a-b|<\varepsilon$,则过程结束,取 $(a+b)/2$ 为根的近似值;

若 $|a-b|\geq\varepsilon$,则重复上述的减半过程。

(6) 回溯法

前面讨论的递推和递归算法本质上是对实际问题进行归纳的结果,而减半递推技术也是归纳法的一个分支。在工程上,有些实际问题很难归纳出一组简单的递推公式或直观的求解步骤,并且也不能进行无限的列举。对于这类问题,一种有效的方法是“试”。通过对问题的分析,找出一个解决问题的线索,然后沿着这个线索逐步试探,对于每一步的试探,若试探成功,就得到问题的解,若试探失败,就逐步回退,换别的路线再进行试探。这种方法称为回溯法。回溯法在处理复杂数据结构方面有着广泛的应用。

2.1.3 算法复杂度

算法的复杂度主要包括时间复杂度和空间复杂度。

1. 算法的时间复杂度

所谓算法的时间复杂度,是指执行算法所需要的计算工作量。

为了能够比较客观地反映出一个算法的效率,在度量一个算法的工作量时,不仅应该与所使用的计算机、程序设计语言以及程序编制者无关,而且还应该与算法实现过程中的许多细节无关。为此,可以用算法在执行过程中所需基本运算的执行次数来度量算法的工作量。基本运算反映了算法运算的主要特征,因此,用基本运算的次数来度量算法工作量是客观的也是实际可行的,有利于比较同一问题的几种算法的优劣。例如,在考虑两个矩阵相乘时,可以将两个实数之间的乘法运算作为基本运算,而对于所用的加法(或减法)运算忽略不计。又如,当需要在一个表中进行查找时,可以将两个元素之间的比较作为基本运算。

算法所执行的基本运算次数还与问题的规模有关。例如,两个 20 阶矩阵相乘与两个 10 阶矩阵相乘,所需要的基本运算(即两个实数的乘法)次数显然是不同的,前者需要更多的运算次数。因此,在分析算法的工作量时,还必须对问题的规模进行度量。

综上所述,算法的工作量用算法所执行的基本运算次数来度量,而算法所执行的基本运算次数是问题规模的函数,即

$$\text{算法的工作量} = f(n)$$

其中 n 是问题的规模。例如,两个 n 阶矩阵相乘所需要的基本运算(即两个实数的乘法)次数为 n^2 ,即计算工作量为 n^2 ,也就是时间复杂度为 n^2 。

在具体分析一个算法的工作量时,还会存在这样的问题:对于一个固定的规模,算法所执行的基本运算次数还可能与特定的输入有关,而实际上又不可能将所有可能情况下算法所执行的基本运算次数都列举出来。例如,“在长度为 n 的一维数组中查找值为 x 的元素”,若采用顺序搜索法,即从数组的第一个元素开始,逐个与被查值 x 进行比较。显然,如果第一个元素恰为 x ,则只需要比较 1 次。但如果 x 为数组的最后一个元素,或者 x 不在数组中,则需要比较 n 次才能得到结果。因此,在这个问题的算法中,其基本运算(即比较)的次数与具体的被查值 x 有关。

在同一个问题规模下,如果算法执行所需的基本运算次数取决于某一特定输入时,可以用以下两种方法来分析算法的工作量。

(1) 平均性态(average behavior)

所谓平均性态分析,是指用各种特定输入下的基本运算次数的加权平均值来度量算法的工作量。

设 x 是有可能输入中的某个特定输入, $p(x)$ 是 x 出现的概率(即输入为 x 的概率), $t(x)$ 是算法在输入为 x 时所执行的基本运算次数,则算法的平均性态定义为

$$A(n) = \sum_{x \in D_n} p(x)t(x)$$

其中 D_n 表示当规模为 n 时,算法执行时所有可能输入的集合。这个式子中的 $t(x)$ 可以通过分析算法来加以确定;而 $p(x)$ 必须由经验或用算法中有关的一些特定信息来确定,通常是不能解析地加以计算的。如果确定 $p(x)$ 比较困难,则会给平均性态的分析带来困难。

(2) 最坏情况复杂性(worst-case complexity)

所谓最坏情况分析,是指在规模为 n 时,算法所执行的基本运算的最大次数。它定义为

$$W(n) = \max_{x \in D_n} t(x)$$

显然, $W(n)$ 的计算要比 $A(n)$ 的计算方便得多。由于 $W(n)$ 实际上是给出了算法工作量的一个上界,因此,它比 $A(n)$ 更具有实用价值。

下面通过一个例子来说明算法复杂度的平均性态分析与最坏情况分析。

例 2.4 采用顺序搜索法,在长度为 n 的一维数组中查找值为 x 的元素。即从数组的第一个元素开始,逐个与被查值 x 进行比较。基本运算为 x 与数组元素的比较。

首先考虑平均性态分析。

设被查项 x 在数组中出现的概率为 q 。当需要查找的 x 为数组中第 i 个元素时,则在查找过程中需要做 i 次比较,当需要查找的 x 不在数组中时(即数组中没有 x 这个元素),则需要与数组中所有的元素进行比较。即

$$t_i = \begin{cases} i, & 1 \leq i \leq n \\ n, & i = n+1 \end{cases}$$

其中 $i = n+1$ 表示 x 不在数组中的情况。



如果假设需要查找的 x 出现在数组中每个位置上的可能性是一样的, 则 x 出现在数组中每一个位置上的概率为 q/n (因为前面已经假设 x 在数组中的概率为 q), 而 x 不在数组中的概率为 $1-q$, 即

$$p_i = \begin{cases} q/n, & 1 \leq i \leq n \\ 1-q, & i = n+1 \end{cases}$$

其中 $i=n+1$ 表示 x 不在数组中的情况。

因此, 用顺序搜索法在长度为 n 的一维数组中查找值为 x 的元素, 在平均情况下需要做的比较次数为

$$A(n) = \sum_{i=1}^{n+1} p_i t_i = \sum_{i=1}^n (q/n) i + (1-q)n = (n+1)q/2 + (1-q)n$$

如果已知需要查找的 x 一定在数组中, 此时 $q=1$, 则 $A(n) = (n+1)/2$ 。这就是说, 在这种情况下, 用顺序搜索法在长度为 n 的一维数组中查找值为 x 的元素, 在平均情况下需要检查数组中一半的元素。

如果已知需要查找的 x 有一半的机会在数组中, 此时 $q=1/2$, 则

$$A(n) = [(n+1)/4] + n/2 \approx 3n/4$$

这就是说, 在这种情况下, 用顺序搜索法在长度为 n 的一维数组中查找值为 x 的元素, 在平均情况下需要检查数组中 $3/4$ 的元素。

再考虑最坏情况分析。

在这个例子中, 最坏情况发生在需要查找的 x 是数组中的最后一个元素或 x 不在数组中的时候, 此时显然有

$$W(n) = \max \{t_i \mid 1 \leq i \leq n+1\} = n$$

在上述例子中, 算法执行的工作量是与具体的输入有关的, $A(n)$ 只是它的加权平均值, 而实际上对于某个特定的输入, 其计算工作量未必是 $A(n)$, 且 $A(n)$ 也不一定等于 $W(n)$ 。但在另外一些情况下, 算法的计算工作量与输入无关, 即当规模为 n 时, 在所有可能的输入下, 算法所执行的基本运算次数是一定的, 此时有 $A(n) = W(n)$ 。例如, 两个 n 阶的矩阵相乘, 都需要做 n^3 次实数乘法, 而与输入矩阵的具体元素无关。

2. 算法的空间复杂度

一个算法的空间复杂度, 一般是指执行这个算法所需要的内存空间。

一个算法所占用的存储空间包括算法程序所占的空间、输入的初始数据所占的存储空间以及算法执行过程中所需要的额外空间。其中额外空间包括算法程序执行过程中的工作单元以及某种数据结构所需要的附加存储空间 (例如, 在链式结构中, 除了要存储数据本身外, 还需要存储链接信息)。如果额外空间量相对于问题规模来说是常数, 则称该算法是原地 (in place) 工作的。在许多实际问题中, 为了减少算法所占的存储空间, 通常采用压缩存储技术, 以便尽量减少不必要的额外空间。

2.2 数据结构的基本概念

利用计算机进行数据处理是计算机应用的一个重要领域。在进行数据处理时,实际需要处理的数据元素一般有很多,而这些大量的数据元素都需要存放在计算机中,因此,大量的数据元素在计算机中如何组织,以便提高数据处理的效率,并且节省计算机的存储空间,这是进行数据处理的关键问题。

显然,杂乱无章的数据是不便于处理的。而将大量的数据随意地存放在计算机中,实际上也是“自找苦吃”,对数据处理更是不利。

数据结构作为计算机的一门学科,主要研究和讨论以下三个方面的问题:

- ① 数据集中各数据元素之间所固有的逻辑关系,即数据的逻辑结构;
- ② 在对数据进行处理时,各数据元素在计算机中的存储关系,即数据的存储结构;
- ③ 对各种数据结构进行的运算。

讨论以上问题的主要目的是为了提高数据处理的效率。所谓提高数据处理的效率,主要包括两个方面:一是提高数据处理的速度,二是尽量节省在数据处理过程中所占用的计算机存储空间。

本节主要讨论工程上常用的一些基本数据结构,它们是软件设计的基础。

2.2.1 什么是数据结构

计算机已被广泛用于数据处理。实际问题中的各数据元素之间总是相互关联的。所谓数据处理,是指对数据集中的各元素以各种方式进行运算,包括插入、删除、查找、更改等运算,也包括对数据元素进行分析。在数据处理领域中,建立数学模型有时并不十分重要,事实上,许多实际问题是无法表示成数学模型的。人们最感兴趣的是知道数据集中各数据元素之间存在什么关系,应如何组织它们,即如何表示所需要处理的数据元素。

简单地说,数据结构是指相互有关联的数据元素的集合。例如,向量和矩阵就是数据结构,在这两个数据结构中,数据元素之间有着位置上的关系。又如,图书馆中的图书卡片目录,则是一个较为复杂的数据结构,对于列在各卡片上的各种书之间,可能在主题、作者等问题上相互关联,甚至一本书本身也有不同的相关成分。

数据元素具有广泛的含义。一般来说,现实世界中客观存在的一切个体都可以是数据元素。例如:

描述一年四季的季节名

春、夏、秋、冬

可以作为季节的数据元素:

表示数值的各个数

18、11、35、23、16、...

可以作为数值的数据元素:

表示家庭成员的各成员名



父亲、儿子、女儿

可以作为家庭成员的数据元素。

甚至每一个客观存在的事件,如一次演出、一次借书、一次比赛等也可以作为数据元素。

总之,在数据处理领域中,每一个需要处理的对象都可以抽象成数据元素。数据元素一般简称为元素。

在实际应用中,被处理的数据元素一般有很多,而且,作为某种处理,其中的数据元素一般具有某种共同特征。例如,|春,夏,秋,冬|这四个数据元素有一个共同特征,即它们都是季节名,分别表示了一年中的四个季节,从而这四个数据元素构成了季节名的集合。又如,|父亲,儿子,女儿|这三个数据元素也有一个共同特征,即它们都是家庭的成员名,从而构成了家庭成员名的集合。一般来说,人们不会同时处理特征完全不同且互相之间没有任何关系的各类数据元素,对于具有不同特征的数据元素总是分别进行处理。

一般情况下,在具有相同特征的数据元素集合中,各个数据元素之间存在有某种关系(即联系),这种关系反映了该集合中的数据元素所固有的一种结构。在数据处理领域中,通常把数据元素之间这种固有的关系简单地用前后件关系(或直接前驱与直接后继关系)来描述。

例如,在考虑一年四个季节的顺序关系时,则“春”是“夏”的前件(即直接前驱,下同),而“夏”是“春”的后件(即直接后继,下同)。同样,“夏”是“秋”的前件,“秋”是“夏”的后件;“秋”是“冬”的前件,“冬”是“秋”的后件。

在考虑家庭成员间的辈分关系时,则“父亲”是“儿子”和“女儿”的前件,而“儿子”与“女儿”都是“父亲”的后件。

前后件关系是数据元素之间的一个基本关系,但前后件关系所表示的实际意义随具体对象的不同而不同。一般来说,数据元素之间的任何关系都可以用前后件关系来描述。

1. 数据的逻辑结构

前面提到,数据结构是指反映数据元素之间关系的数据元素集合的表示。更通俗地说,数据结构是指带有结构的数据元素的集合。在此,所谓结构实际上就是指数据元素之间的前后件关系。

由上所述,一个数据结构应包含以下两方面的信息:

- ① 表示数据元素的信息;
- ② 表示各数据元素之间的前后件关系。

在以上所述的数据结构中,其中数据元素之间的前后件关系是指它们的逻辑关系,而与它们在计算机中的存储位置无关。因此,上面所述的数据结构实际上是数据的逻辑结构。

所谓数据的逻辑结构,是指反映数据元素之间逻辑关系的数据结构。

由前面的叙述可以知道,数据的逻辑结构有两个要素:一是数据元素的集合,通常记为 D ;二是 D 上的关系,它反映了 D 中各数据元素之间的前后件关系,通常记为 R 。即一个数据结构可以表示成

$$B = (D, R)$$

其中 B 表示数据结构。为了反映 D 中各数据元素之间的前后件关系,一般用二元组来表示。例如,假设 a 与 b 是 D 中的两个数据,则二元组 (a, b) 表示 a 是 b 的前件, b 是 a 的后件。这样,在

D 中的每两个元素之间的关系都可以用这种二元组来表示。

例 2.5 一年四季的数据结构可以表示成

$$B = (D, R)$$

$$D = \{\text{春}, \text{夏}, \text{秋}, \text{冬}\}$$

$$R = \{(\text{春}, \text{夏}), (\text{夏}, \text{秋}), (\text{秋}, \text{冬})\}$$

例 2.6 家庭成员数据结构可以表示成

$$B = (D, R)$$

$$D = \{\text{父亲}, \text{儿子}, \text{女儿}\}$$

$$R = \{(\text{父亲}, \text{儿子}), (\text{父亲}, \text{女儿})\}$$

例 2.7 n 维向量

$$X = (x_1, x_2, \dots, x_n)$$

也是一种数据结构。即 $X = (D, R)$, 其中数据元素的集合为

$$D = \{x_1, x_2, \dots, x_n\}$$

关系为

$$R = \{(x_1, x_2), (x_2, x_3), \dots, (x_{n-1}, x_n)\}$$

对于一些复杂的数据结构来说, 它的数据元素可以是另一种数据结构。

例如, $m \times n$ 的矩阵

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

是一个数据结构。在这个数据结构中, 矩阵的每一行

$$A_i = (a_{i1}, a_{i2}, \dots, a_{in}), \quad i = 1, 2, \dots, m$$

可以看成是它的一个数据元素。即这个数据结构的数据元素的集合为

$$D = \{A_1, A_2, \dots, A_m\}$$

D 上的一个关系为

$$R = \{(A_1, A_2), (A_2, A_3), \dots, (A_i, A_{i+1}), \dots, (A_{m-1}, A_m)\}$$

显然, 数据结构 A 中的每一个数据元素 $A_i (i = 1, 2, \dots, m)$ 又是另一个数据结构, 即数据元素的集合为

$$D_i = \{a_{i1}, a_{i2}, \dots, a_{in}\}$$

D_i 上的一个关系为

$$R_i = \{(a_{i1}, a_{i2}), (a_{i2}, a_{i3}), \dots, (a_{ij}, a_{i,j+1}), \dots, (a_{i,n-1}, a_{in})\}$$

2. 数据的存储结构

数据处理是计算机应用的一个重要领域, 在实际进行数据处理时, 被处理的各数据元素总是被存放在计算机的存储空间中, 并且, 各数据元素在计算机存储空间中的位置关系与它们的逻辑关系不一定是相同的, 而且一般也不可能相同。例如, 在前面提到的一年四个季节的数据结构中, “春”是“夏”的前件, “夏”是“春”的后件, 但在对它们进行处理时, 在计算机存储空间中,



“春”这个数据元素的信息不一定被存储在“夏”这个数据元素信息的前面,而可能在后面,也可能不是紧邻在前面,而是中间被其他的信息所隔开。又如,在家庭成员的数据结构中,“儿子”和“女儿”都是“父亲”的后件,但在计算机存储空间中,根本不可能将“儿子”和“女儿”这两个数据元素的信息都紧邻存放在“父亲”这个数据元素信息的后面,即在存储空间中与“父亲”紧邻的只可能是其中的一个。由此可以看出,一个数据结构中的各数据元素在计算机存储空间中的位置关系与逻辑关系是有可能不同的。

数据的逻辑结构在计算机存储空间中的存放形式称为数据的存储结构(也称数据的物理结构)。

由于数据元素在计算机存储空间中的位置关系可能与逻辑关系不同,因此,为了表示存放在计算机存储空间中的各数据元素之间的逻辑关系(即前后件关系),在数据的存储结构中,不仅要存放各数据元素的信息,还需要存放各数据元素之间的前后件关系的信息。

一般来说,一种数据的逻辑结构根据需要可以表示成多种存储结构,常用的存储结构有顺序、链接、索引等存储结构。而采用不同的存储结构,其数据处理的效率是不同的。因此,在进行数据处理时,选择合适的存储结构是很重要的。

2.2.2 数据结构的图形表示

一个数据结构除了用二元关系表示外,还可以直观地用图形表示。在数据结构的图形表示中,对于数据集合 D 中的每一个数据元素用中间标有元素值的方框表示,一般称之为数据结点,并简称为结点;为了进一步表示各数据元素之间的前后件关系,对于关系 R 中的每一个二元组,用一条有向线段从前件结点指向后件结点。

例如,一年四季的数据结构可以用如图 2.1 所示的图形来表示。

又如,反映家庭成员间辈分关系的数据结构可以用如图 2.2 所示的图形表示。



图 2.1 一年四季数据结构的图形表示

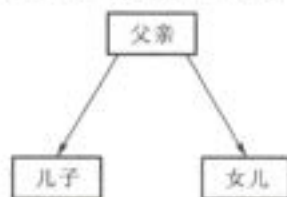


图 2.2 家庭成员间辈分关系数据结构的图形表示

显然,用图形方式表示一个数据结构是很方便的,并且也比较直观。有时在不会引起误会的情况下,在前件结点到后件结点连线上的箭头可以省去。例如,在图 2.2 中,即使将“父亲”结点与“儿子”结点连线上的箭头以及“父亲”结点与“女儿”结点连线上的箭头都去掉,同样表示了“父亲”是“儿子”与“女儿”的前件,“儿子”与“女儿”均是“父亲”的后件,而不会引起误会。

例 2.8 用图形表示数据结构 $B=(D,R)$, 其中

$$D = \{d_i | 1 \leq i \leq 7\} = \{d_1, d_2, d_3, d_4, d_5, d_6, d_7\}$$

$$R = \{(d_1, d_2), (d_1, d_7), (d_2, d_4), (d_3, d_6), (d_4, d_5)\}$$

这个数据结构的图形表示如图 2.3 所示。

在数据结构中,没有前件的结点称为根结点;没有后件的结点称为终端结点(也称为叶子结点)。例如,在图 2.1 所示的数据结构中,元素“春”所在的结点(简称为结点“春”,下同)为根结点,结点“冬”为终端结点;在图 2.2 所示的数据结构中,结点“父亲”为根结点,结点“儿子”与“女儿”均为终端结点;在图 2.3 所示的数据结构中,有两个根结点 d_1 与 d_2 ,有三个终端结点 d_6 、 d_7 、 d_5 。数据结构中除了根结点与终端结点外的其他结点一般称为内部结点。

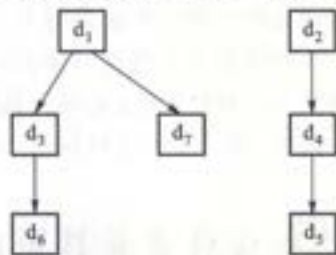


图 2.3 例 2.8 数据结构的图形表示

通常,一个数据结构中的元素结点可能是在动态变化的。根据需要或在处理过程中,可以在一个数据结构中增加一个新结点(称为插入运算),也可以删除数据结构中的某个结点(称为删除运算)。插入与删除是对数据结构的两种基本运算。除此之外,对数据结构的运算还有查找、分类、合并、分解、复制和修改等。在对数据结构的处理过程中,不仅数据结构中的结点(即数据元素)个数在动态地变化,而且,各数据元素之间的关系也有可能在动态地变化。例如,一个无序表可以通过排序处理而变成有序表;一个数据结构中的根结点被删除后,它的某一个后件可能就变成了根结点;在一个数据结构中的终端结点后插入一个新结点后,则原来的那个终端结点就不再是终端结点而成为内部结点了。有关数据结构的基本运算将在后面讲到具体数据结构时再介绍。

2.2.3 线性结构与非线性结构

如果在一个数据结构中一个数据元素都没有,则称该数据结构为空的数据结构。在一个空的数据结构中插入一个新的元素后就变为非空;在只有一个数据元素的数据结构中,将该元素删除后就变为空的数据结构。

根据数据结构中各数据元素之间前后件关系的复杂程度,一般将数据结构分为两大类型:线性结构与非线性结构。

如果一个非空的数据结构满足下列两个条件:

- ① 有且只有一个根结点;
- ② 每一个结点最多有一个前件,也最多有一个后件。

则称该数据结构为线性结构。线性结构又称线性表。

由此可以看出,在线性结构中,各数据元素之间的前后件关系是很简单的。如例 2.5 中的一年四季这个数据结构,以及例 2.7 中的 n 维向量数据结构,它们都属于线性结构。

特别需要说明的是,在一个线性结构中插入或删除任何一个结点后还应是线性结构。根据这一点,如果一个数据结构满足上述两个条件,但当在此数据结构中插入或删除任何一个结点后就不满足这两个条件了,则该数据结构不能称为线性结构。例如,图 2.4 所示的数据结构显然是满足上述两个条件的,但它不属于线性结构这个类型,因为如果在这个数据结构中删除结点 A 后,就不满足上述的条件(1)。

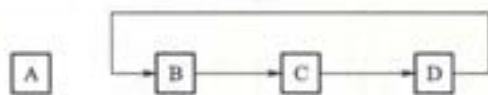


图 2.4 不是线性结构的数据结构特例

如果一个数据结构不是线性结构,则称之为非线性结构。如例 2.6 中反映家庭成员间辈分关系的数据结构,以及例 2.8 中的数据结构,它们都不是线性结构,而是属于非线性结构。显然,



在非线性结构中,各数据元素之间的前后件关系要比线性结构复杂,因此,对非线性结构的存储与处理比线性结构要复杂得多。

线性结构与非线性结构都可以是空的数据结构。一个空的数据结构究竟是属于线性结构还是属于非线性结构,这要根据具体情况来确定。如果对该数据结构的运算是按线性结构的规则来处理的,则属于线性结构;否则属于非线性结构。

2.3 线性表及其顺序存储结构

2.3.1 线性表的基本概念

线性表(linear list)是最简单、最常用的一种数据结构。

线性表由一组数据元素构成。数据元素的含义很广泛,在不同的具体情况下,它可以有不同的含义。例如,一个 n 维向量 $(x_1, x_2, \dots, x_n)$ 是一个长度为 n 的线性表,其中的每一个分量就是一个数据元素。又如,英文小写字母表 $(a, b, c, \dots, z)$ 是一个长度为26的线性表,其中的每一个小写字母就是一个数据元素。再如,一年中的四个季节(春,夏,秋,冬)是一个长度为4的线性表,其中的每一个季节名就是一个数据元素。

矩阵也是一个线性表,只不过它是一个比较复杂的线性表。在矩阵中,既可以把每一行看成是一个数据元素(即一个行向量为一个数据元素),也可以把每一列看成是一个数据元素(即一个列向量为一个数据元素)。其中每一个数据元素(一个行向量或一个列向量)实际上又是一个简单的线性表。

数据元素可以是简单项(如上述例子中的数、字母、季节名等)。在稍微复杂的线性表中,一个数据元素还可以由若干个数据项组成。例如,某班的学生情况登记表是一个复杂的线性表,表中每一个学生的情况就组成了线性表中的每一个元素,每一个数据元素包括姓名、学号、性别、年龄和健康状况5个数据项,如表2.1所示。在这种复杂的线性表中,由若干数据项组成的数据元素称为记录(record),而由多个记录构成的线性表又称为文件(file)。因此,上述学生情况登记表就是一个文件,其中每一个学生的情况就是一个记录。

表 2.1 学生情况登记表

姓名	学号	性别	年龄	健康状况
王强	800356	男	19	良好
刘建平	800357	男	20	一般
赵军	800361	女	19	良好
葛文华	800367	男	21	较差
...	...	...	...	...

综上所述,线性表是由 $n(n \geq 0)$ 个数据元素 $a_1, a_2, \dots, a_n$ 组成的一个有限序列,表中的每一个数据元素,除了第一个外,有且只有一个前件,除了最后一个外,有且只有一个后件。即线性表或是一个空表,或可以表示为

$$(a_1, a_2, \dots, a_i, \dots, a_n)$$

其中 $a_i (i=1, 2, \dots, n)$ 是属于数据对象的元素, 通常也称其为线性表中的一个结点。

显然, 线性表是一种线性结构。数据元素在线性表中的位置只取决于它们自己的序号, 即数据元素之间的相对位置是线性的。

非空线性表有如下一些结构特征:

① 有且只有一个根结点 a_1 , 它无前件;

② 有且只有一个终端结点 a_n , 它无后件;

③ 除根结点与终端结点外, 其他所有结点有且只有一个前件, 也有且只有一个后件。线性表中结点的个数 n 称为线性表的长度。当 $n=0$ 时, 称为空表。

2.3.2 线性表的顺序存储结构

在计算机中存放线性表, 一种最简单的方法是顺序存储, 也称为顺序分配。

线性表的顺序存储结构具有以下两个基本特点:

① 线性表中所有元素所占的存储空间是连续的;

② 线性表中各数据元素在存储空间中是按逻辑顺序依次存放的。

由此可以看出, 在线性表的顺序存储结构中, 其前后件两个元素在存储空间中是紧邻的, 且前件元素一定存储在后件元素的前面。

在线性表的顺序存储结构中, 如果线性表中各数据元素所占的存储空间(字节数)相等, 则要在该线性表中查找某一个元素是很方便的。

假设线性表中的第一个数据元素的存储地址(指第一个字节的地址, 即首地址)为 $ADR(a_1)$, 每一个数据元素占 k 个字节, 则线性表中第 i 个元素 a_i 在计算机存储空间中的存储地址为

$$ADR(a_i) = ADR(a_1) + (i - 1)k$$

即在顺序存储结构中, 线性表中每一个数据元素在计算机存储空间中的存储地址由该元素在线性表中的位置序号唯一确定。一般来说, 长度为 n 的线性表

$$(a_1, a_2, \dots, a_i, \dots, a_n)$$

在计算机中的顺序存储结构如图 2.5 所示。

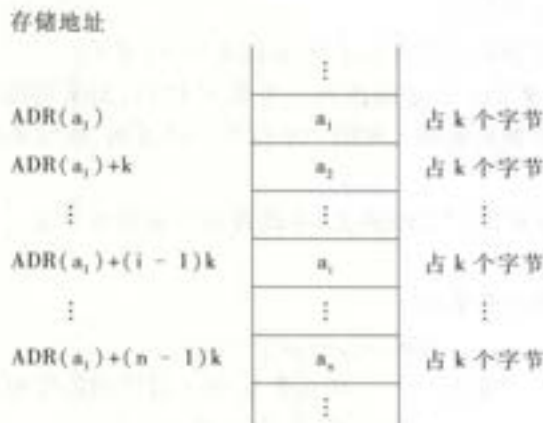


图 2.5 线性表的顺序存储结构



在程序设计语言中,通常定义一个一维数组来表示线性表的顺序存储空间。因为程序设计语言中的一维数组与计算机中实际的存储空间结构是类似的,这就便于用程序设计语言对线性表进行各种运算处理。

在用一维数组存放线性表时,该一维数组的长度通常要定义得比线性表的实际长度大一些,以便对线性表进行各种运算,特别是插入运算。在一般情况下,如果线性表的长度在处理过程中是动态变化的,则在开辟线性表的存储空间时要考虑到线性表在动态变化过程中可能达到的最大长度。如果开始时所开辟的存储空间太小,则在线性表动态增长时可能会出现存储空间不够而无法再插入新的元素;但如果开始时所开辟的存储空间太大,而实际上又用不着那么大的存储空间,则会造成存储空间的浪费。在实际应用中,可以根据线性表动态变化过程中的一般规模来决定开辟的存储空间量。

在线性表的顺序存储结构下,可以对线性表进行各种处理。主要的运算有以下几种:

- ① 在线性表的指定位置处加入一个新的元素(即线性表的插入);
- ② 在线性表中删除指定的元素(即线性表的删除);
- ③ 在线性表中查找某个(或某些)特定的元素(即线性表的查找);
- ④ 对线性表中的元素进行整序(即线性表的排序);
- ⑤ 按要求将一个线性表分解成多个线性表(即线性表的分解);
- ⑥ 按要求将多个线性表合并成一个线性表(即线性表的合并);
- ⑦ 复制一个线性表(即线性表的复制);
- ⑧ 逆转一个线性表(即线性表的逆转)等。

下面分别讨论线性表在顺序存储结构下的插入与删除的问题。

2.3.3 顺序表的插入运算

首先举一个例子来说明如何在顺序存储结构的线性表中插入一个新元素。

例 2.9 图 2.6(a) 为一个长度为 8 的线性表顺序存储在长度为 10 的存储空间中。现在要求在第 2 个元素(即 18)之前插入一个新元素 87。其插入过程如下:

首先从最后一个元素开始直到第 2 个元素,将其中的每一个元素均依次往后移动一个位置,然后将新元素 87 插入到第 2 个位置。

插入一个新元素后,线性表的长度变成了 9,如图 2.6(b) 所示。

如果再要在线性表的第 9 个元素之前插入一个新元素 14,则采用类似的方法:将第 9 个元素往后移动一个位置,然后将新元素插入到第 9 个位置。插入后,线性表的长度变成了 10,如图 2.6(c) 所示。

现在,为线性表开辟的存储空间已经满了,不能再插入新的元素了。如果再要插入,则会造成为“上溢”的错误。

一般来说,设长度为 n 的线性表为

$$(a_1, a_2, \dots, a_i, \dots, a_n)$$

现要在线性表的第 i 个元素 a_i 之前插入一个新元素 b ,插入后得到长度为 $n+1$ 的线性表为

$$(a'_1, a'_2, \dots, a'_i, a'_{i+1}, \dots, a'_n, a'_{n+1})$$

则插入前后的两线性表中的元素满足如下关系:

$$a'_j = \begin{cases} a_j & 1 \leq j \leq i-1 \\ b & j = i \\ a_{j-1} & i+1 \leq j \leq n+1 \end{cases}$$

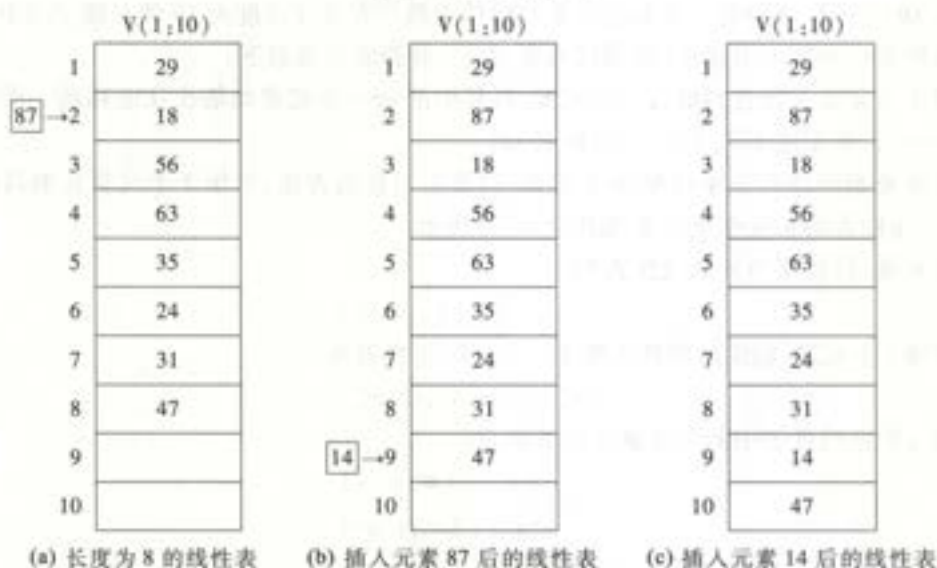


图 2.6 线性表在顺序存储结构下的插入

在一般情况下,要在第 i ($1 \leq i \leq n$) 个元素之前插入一个新元素时,首先要从最后一个(即第 n 个)元素开始,直到第 i 个元素之间共 $n - i + 1$ 个元素依次向后移动一个位置,移动结束后,第 i 个位置就被空出,然后将新元素插入到第 i 项。插入结束后,线性表的长度就增加了 1。

显然,在线性表采用顺序存储结构时,如果插入运算在线性表的末尾进行,即在第 n 个元素之后(可以认为是在第 $n+1$ 个元素之前)插入新元素,则只要在表的末尾增加一个元素即可,不需要移动表中的元素;如果要在线性表的第 1 个元素之前插入一个新元素,则需要移动表中所有的元素。在一般情况下,如果插入运算在第 i ($1 \leq i \leq n$) 个元素之前进行,则原来第 i 个元素之后(包括第 i 个元素)的所有元素都必须移动。在平均情况下,要在线性表中插入一个新元素,需要移动表中一半的元素。因此,在线性表顺序存储的情况下,要插入一个新元素,其效率是很低的,特别是在线性表比较大的情况下更为突出,由于数据元素的移动而消耗较多的处理时间。

假设线性表的存储空间为 $V(1:m)$,线性表的长度为 n ($n \leq m$),插入的位置为 i (i 表示在第 i 个元素之前插入),插入新元素。则插入的过程如下:

① 首先处理以下三种异常情况:

- 当存储空间已满(即 $n=m$)时为“上溢”错误,不能进行插入,算法结束;
- 当 $i > n$ 时,认为在最后一个元素之后(即第 $n+1$ 个元素之前)插入;
- 当 $i < 1$ 时,认为在第 1 个元素之前插入。

② 然后从最后一个元素开始,直到第 i 个元素,其中每一个元素均往后移动一个位置。

③ 最后将新元素插入到第 i 个位置,并且将线性表的长度增加 1。



2.3.4 顺序表的删除运算

首先举一个例子来说明如何在顺序存储结构的线性表中删除一个元素。

例 2.10 图 2.7(a) 为一个长度为 8 的线性表顺序存储在长度为 10 的存储空间中。现在要求删除线性表中的第 1 个元素(即删除元素 29)。其删除过程如下:

从第 2 个元素开始直到最后一个元素,将其中的每一个元素均依次往前移动一个位置。此时,线性表的长度变成了 7,如图 2.7(b) 所示。

如果再要删除线性表中的第 6 个元素,则采用类似的方法:将第 7 个元素往前移动一个位置。此时,线性表的长度变成了 6,如图 2.7(c) 所示。

一般来说,设长度为 n 的线性表为

$$(a_1, a_2, \dots, a_i, \dots, a_n)$$

现要删除第 i 个元素,删除后得到长度为 $n-1$ 的线性表为

$$(a'_1, a'_2, \dots, a'_j, \dots, a'_{n-1})$$

则删除前后的两线性表中的元素满足如下关系:

$$a'_j = \begin{cases} a_j & 1 \leq j \leq i-1 \\ a_{j+1} & 1 \leq j \leq n-1 \end{cases}$$

V(1:10)	
1	29
2	18
3	56
4	63
5	35
6	24
7	31
8	47
9	
10	

(a) 长度为 8 的线性表

V(1:10)	
1	18
2	56
3	63
4	35
5	24
6	31
7	47
8	
9	
10	

(b) 删除元素 29 后的线性表

V(1:10)	
1	18
2	56
3	63
4	35
5	24
6	47
7	
8	
9	
10	

(c) 删除元素 31 后的线性表

图 2.7 线性表在顺序存储结构下的删除

在一般情况下,要删除第 $i(1 \leq i \leq n)$ 个元素时,则要从第 $i+1$ 个元素开始,直到第 n 个元素之间共 $n-i$ 个元素依次向前移动一个位置。删除结束后,线性表的长度就减小了 1。

显然,在线性表采用顺序存储结构时,如果删除运算在线性表的末尾进行,即删除第 n 个元素,则不需要移动表中的元素;如果要删除线性表中的第 1 个元素,则需要移动表中所有的元素。在一般情况下,如果要删除第 $i(1 \leq i \leq n)$ 个元素,则原来第 i 个元素之后的所有元素都必须依次往前移动一个位置。在平均情况下,要在线性表中删除一个元素,需要移动表中一半的元素。因此,在线性表顺序存储的情况下,要删除一个元素,其效率也是很低的,特别是在线性表比较大的情况下更为突出,由于数据元素的移动而消耗较多的处理时间。

假设线性表的存储空间为 $V(1:m)$,线性表的长度为 $n(n \leq m)$,删除的位置为 i (表示删除

第 i 个元素)。则删除的过程如下:

① 首先处理以下两种异常情况:

- 当线性表为空(即 $n=0$)时为“下溢”错误,不能进行删除,算法结束;
- 当 $i<1$ 或 $i>n$ 时,认为不能进行删除,算法结束。

② 然后从第 $i+1$ 个元素开始,直到最后一个元素,其中每一个元素均依次往前移动一个位置。

③ 最后将线性表的长度减小 1。

由线性表在顺序存储结构下的插入与删除运算可以看出,线性表的顺序存储结构对于小线性表或者其中元素不常变动的线性表来说是合适的,因为顺序存储的结构比较简单。但这种顺序存储的方式对于元素经常需要变动的大线性表就不太合适了,因为插入与删除的效率比较低。

2.4 栈和队列

2.4.1 栈及其基本运算

1. 什么是栈

栈实际上也是线性表,只不过是一种特殊的线性表。

为了认识栈这种数据结构,下面先看一个例子。

图 2.8 中给出了具有嵌套调用关系的五个程序,其中主程序要调用子程序 SUB1, SUB1 要调用子程序 SUB2, SUB2 要调用子程序 SUB3, SUB3 要调用子程序 SUB4, SUB4 不再调用其他子程序了。

MAIN	SUB1	SUB2	SUB3	SUB4
.....				
CALL SUB1	CALL SUB2	CALL SUB3	CALL SUB4	
A ₁	B ₁	C ₁	D ₁	
.....				
END	RETURN	RETURN	RETURN	RETURN

图 2.8 主程序与子程序之间的调用关系

下面来看一看计算机系统是如何处理它们之间的调用关系的。关键是要正确处理好执行过程中的调用层次和返回的路径,这就需要记忆每一次调用时的返回点。计算机系统在处理时要用一个线性表来动态记忆调用过程中的路径,其基本原则如下:

① 在开始执行程序前,建立一个线性表,其初始状态为空;

② 当发生调用时,将当前调用的返回点地址(在图 2.8 中用语句标号表示)插入到线性表的末尾;

③ 当遇到从某个子程序返回时,其返回点地址从线性表的末尾取出(即删除线性表的最后



一个元素)。

根据以上原则,可以跟踪图 2.8 中五个程序在执行过程中的调用顺序以及线性表中元素变化的情况如下:

① 主程序开始执行前,建立一个空线性表。即线性表的状态为()。

② 开始执行主程序 MAIN。当要调用子程序 SUB1 时,先将本次调用的返回点地址 A 插入到线性表的末尾。此时,线性表的状态为(A)。

③ 开始调用执行子程序 SUB1。当要调用子程序 SUB2 时,先将本次调用的返回点地址 B 插入到线性表的末尾。此时,线性表的状态为(A,B)。

④ 开始调用执行子程序 SUB2。当要调用子程序 SUB3 时,先将本次调用的返回点地址 C 插入到线性表的末尾。此时,线性表的状态为(A,B,C)。

⑤ 开始调用执行子程序 SUB3。当要调用子程序 SUB4 时,先将本次调用的返回点地址 D 插入到线性表的末尾。此时,线性表的状态为(A,B,C,D)。

由上述逐步调用的过程可以看出,每次发生调用时,都需要将当前调用的返回点地址插入到线性表的末尾,而这种对线性表的插入操作是不需要移动线性表中原有元素的。并且,各返回点地址在线性表中的存放顺序恰好是调用顺序。

⑥ 开始调用执行子程序 SUB4。由于子程序 SUB4 不再调用其他子程序,执行完子程序 SUB4 后要返回到子程序 SUB3 的地址 D 处。其中返回点地址 D 取自线性表的最后一个元素。取出 D 后,线性表的状态为(A,B,C)。

⑦ 返回到子程序 SUB3 的 D 处继续执行,执行完子程序 SUB3 后要返回到子程序 SUB2 的地址 C 处。其中返回点地址 C 取自线性表的最后一个元素。取出 C 后,线性表的状态为(A,B)。

⑧ 返回到子程序 SUB2 的 C 处继续执行,执行完子程序 SUB2 后要返回到子程序 SUB1 的地址 B 处。其中返回点地址 B 取自线性表的最后一个元素。取出 B 后,线性表的状态为(A)。

⑨ 返回到子程序 SUB1 的 B 处继续执行,执行完子程序 SUB1 后要返回到主程序 MAIN 的地址 A 处。其中返回点地址 A 取自线性表的最后一个元素。取出 A 后,线性表变空,即线性表的状态为()。

⑩ 返回到主程序 MAIN 的 A 处继续执行,直到主程序 MAIN 执行完成。

由上述逐步返回的过程可以看出,当由子程序返回到上一个调用程序时,需要从线性表的末尾取出返回点地址,即从线性表中删除最后一个元素,而这种对线性表的删除操作也是不需要移动线性表中其他数据元素的。由于各返回点地址在线性表中的存放顺序恰好是对应的调用顺序,因此,每次从线性表的末尾取出的返回点地址正好对应了各次调用的正确的返回顺序。

由这个例子可以看出,在这种特殊的线性表中,其插入与删除运算都只在线性表的一端进行。即在这种线性表的结构中,一端是封闭的,不允许进行插入与删除元素;另一端是开口的,允许插入与删除元素。在顺序存储结构下,对这种类型线性表的插入与删除运算是不需要移动表中其他数据元素的。这种线性表称为栈。

栈(stack)是限定在一端进行插入与删除的线性表。

在栈中,允许插入与删除的一端称为栈顶,而不允许插入与删除的另一端称为栈底。栈顶元素总是最后被插入的元素,从而也是最先能被删除的元素;栈底元素总是最先被插入的元素,从

而也是最后才能被删除的元素。即栈是按照“先进后出”(first in last out, FILO)或“后进先出”(last in first out, LIFO)的原则组织数据的,因此,栈也被称为“先进后出”表或“后进先出”表。由此可以看出,栈具有记忆作用。

通常用指针 top 来指示栈顶的位置,用指针 $bottom$ 指向栈底。

往栈中插入一个元素称为入栈运算,从栈中删除一个元素(即删除栈顶元素)称为退栈运算。栈顶指针 top 动态反映了栈中元素的变化情况。

图 2.9 是栈的示意图。

栈这种数据结构在日常生活中也是常见的。例如,子弹夹是一种栈的结构,最后压入的子弹总是最先被弹出,而最先压入的子弹最后才能被弹出。又如,在用一端为封闭另一端为开口的容器装物品时,也是遵循“先进后出”或“后进先出”原则的。

2. 栈的顺序存储及其运算

与一般的线性表一样,在程序设计语言中,用一维数组 $S(1:m)$ 作为栈的顺序存储空间,其中 m 为栈的最大容量。

通常,栈底指针指向栈空间的低地址一端(即数组的起始地址这一端)。

图 2.10(a)是容量为 10 的栈顺序存储空间,栈中已有 6 个元素;图 2.10(b)与图 2.10(c)分别为入栈与退栈后的状态。

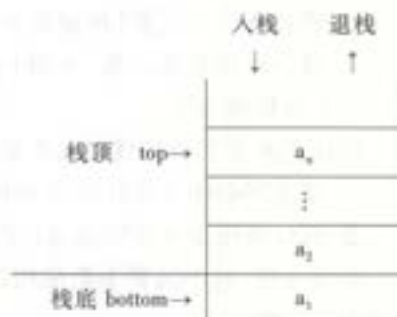


图 2.9 栈示意图



图 2.10 栈在顺序存储结构下的运算

在栈的顺序存储空间 $S(1:m)$ 中, $S(bottom)$ 通常为栈底元素(在栈非空的情况下), $S(top)$ 为栈顶元素。 $top=0$ 表示栈空; $top=m$ 表示栈满。

栈的基本运算有三种:入栈、退栈与读栈顶元素。下面分别介绍在顺序存储结构下栈的这三种运算。

(1) 入栈运算

入栈运算是指在栈顶位置插入一个新元素。操作过程如下:

① 首先判断栈顶指针是否已经指向存储空间的最后一个位置。如果是,则说明栈空间已满,不可能再进行入栈操作(这种情况称为栈“上溢”错误),算法结束。



- ② 然后将栈顶指针进一(即 top 加 1)。
- ③ 最后将新元素 x 插入栈顶指针指向的位置。

(2) 退栈运算

退栈运算是指取出栈顶元素并赋给一个指定的变量。操作过程如下:

① 首先判断栈顶指针是否为 0。如果是,则说明栈空,不可能进行退栈操作(这种情况称为栈“下溢”错误),算法结束。

② 然后将栈顶元素(栈顶指针指向的元素)赋给一个指定的变量。

③ 最后将栈顶指针退一(即 top 减 1)。

(3) 读栈顶元素

读栈顶元素是指将栈顶元素赋给一个指定的变量。操作过程如下:

① 首先判断栈顶指针是否为 0。如果是,则说明栈空,读不到栈顶元素,算法结束。

② 然后将栈顶元素赋给指定的变量 y 。

必须注意,这个运算不删除栈顶元素,只是将它的值赋给一个变量,因此,在这个运算中栈顶指针不会改变。

2.4.2 队列及其基本运算

1. 什么是队列

在计算机系统中,如果一次只能执行一个用户程序,则在多个用户程序需要执行时,这些用户程序必须先按照到来的顺序进行排队等待。这通常是由计算机操作系统来进行管理的。

在操作系统中,用一个线性表来组织管理用户程序的排队执行,原则是:

- ① 初始时线性表为空;
- ② 当有用户程序来到时,将该用户程序加到线性表的末尾进行等待;
- ③ 当计算机系统执行完当前的用户程序后,就从线性表的头部取出一个用户程序执行。

由这个例子可以看出,在这种线性表中,需要加入的元素总是插入到线性表的末尾,并且又总是从线性表的头部取出(删除)元素。这种线性表称为队列。

队列(queue)是指允许在一端进行插入,而在另一端进行删除的线性表。允许插入的一端称为队尾,通常用一个称为尾指针(rear)的指针指向队尾元素,即尾指针总是指向最后被插入的元素;允许删除的一端称为排头(也称为队头),通常也用一个排头指针(front)指向排头元素的前一个位置。显然,在队列这种数据结构中,最先插入的元素将最先能够被删除,反之,最后插入的元素将最后才能被删除。因此,队列又称为“先进先出”(first in first out, FIFO)或“后进后出”(last in last out, LILO)的线性表,它体现了“先来先服务”的原则。在队列中,队尾指针 rear 与排头指针 front 共同反映了队列中元素动态变化的情况。图 2.11 是具有 6 个元素的队列示意图。

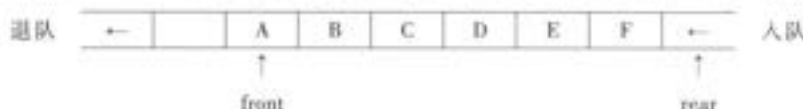


图 2.11 具有 6 个元素的队列示意图

往队列的队尾插入一个元素称为入队运算,从队列的排头删除一个元素称为退队运算。

图 2.12 是在队列中进行插入与删除的示意图。由图 2.12 可以看出,在队列的末尾插入一个元素(入队运算)只涉及队尾指针 $rear$ 的变化,而要删除队列中的排头元素(退队运算)只涉及排头指针 $front$ 的变化。

与栈类似,在程序设计语言中,用一维数组作为队列的顺序存储空间。



图 2.12 队列运算示意图

2. 循环队列及其运算

在实际应用中,队列的顺序存储结构一般采用循环队列的形式。

所谓循环队列,就是将队列存储空间的最后一个位置绕到第一个位置,形成逻辑上的环状空间,供队列循环使用,如图 2.13 所示。在循环队列结构中,当存储空间的最后一个位置已被使用而再要进行入队运算时,只要存储空间的第一个位置空闲,便可将元素加到第一个位置,即将存储空间的第一个位置作为队尾。

在循环队列中,用队尾指针 $rear$ 指向队列中的队尾元素,用排头指针 $front$ 指向排头元素的前一个位置,因此,从排头指针 $front$ 指向的后一个位置直到队尾指针 $rear$ 指向的位置之间所有的元素均为队列中的元素。

循环队列的初始状态为空,即 $rear = front = m$,如图 2.13 所示。

循环队列主要有两种基本运算:入队运算与退队运算。

每进行一次入队运算,队尾指针就进一。当队尾指针 $rear = m + 1$ 时,则置 $rear = 1$ 。

每进行一次退队运算,排头指针就进一。当排头指针 $front = m + 1$ 时,则置 $front = 1$ 。

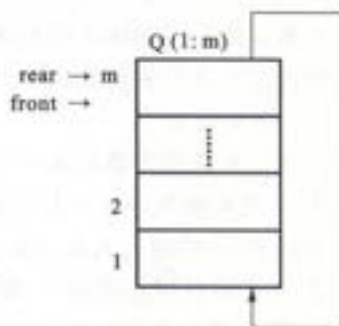


图 2.13 循环队列存储空间示意图

图 2.14(a) 是一个容量为 8 的循环队列存储空间,且其中已有 6 个元素。图 2.14(b) 是在图 2.14(a) 的循环队列中又加入了 2 个元素后的状态。图 2.14(c) 是在 2.14(b) 的循环队列中退出了 1 个元素后的状态。

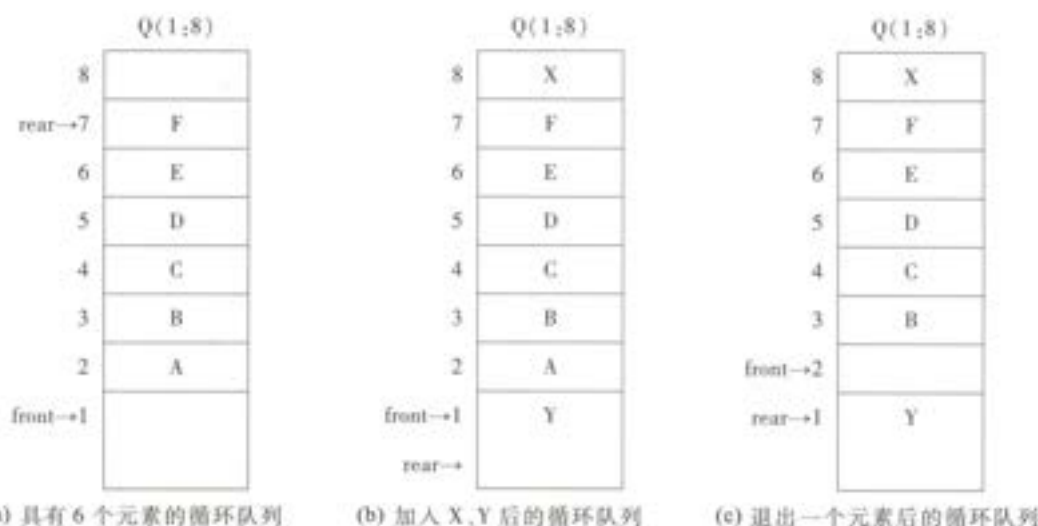


图 2.14 循环队列运算例

由图 2.14 中循环队列动态变化的过程可以看出,当循环队列满时有 $front = rear$,而当循环队列空时也有 $front = rear$ 。即在循环队列中,当 $front = rear$ 时,不能确定是队列满还是队列空。在实际使用循环队列时,为了能区分队列满还是队列空,通常还需增加一个标志 s , s 值的定义如下:

$$s = \begin{cases} 0 & \text{表示队列空} \\ 1 & \text{表示队列非空} \end{cases}$$

由此可以得出队列空与队列满的条件如下:

队列空的条件为 $s = 0$;

队列满的条件为 $s = 1$ 且 $front = rear$ 。

下面具体介绍循环队列入队与退队的运算。

假设循环队列的初始状态为空,即: $s = 0$, 且 $front = rear = m$ 。

(1) 入队运算

入队运算是指在循环队列的队尾加入一个新元素。操作过程如下:

① 首先判断循环队列是否满。当循环队列非空 ($s = 1$) 且队尾指针等于排头指针时,说明循环队列已满,不能进行入队运算。这种情况称为“上溢”。此时算法结束。

② 然后将队尾指针进一 (即 $rear = rear + 1$), 并当 $rear = m + 1$ 时置 $rear = 1$ 。

③ 最后将新元素 x 插入队尾指针指向的位置,并且置循环队列非空标志。

(2) 退队运算

退队运算是指在循环队列的排头位置退出一个元素并赋给指定的变量。操作过程如下:

① 首先判断循环队列是否为空。当循环队列为空 ($s = 0$) 时,不能进行退队运算。这种情况称为“下溢”。此时算法结束。

② 然后将排头指针进一 (即 $front = front + 1$), 并当 $front = m + 1$ 时置 $front = 1$ 。

③ 再将排头指针指向的元素赋给指定的变量。

④ 最后判断退队后循环队列是否为空。当 $front = rear$ 时置循环队列空标志 (即 $s = 0$)。

2.5 线性链表

2.5.1 线性链表的基本概念

前面主要讨论了线性表的顺序存储结构以及在顺序存储结构下的运算。线性表的顺序存储结构具有简单、运算方便等优点,特别是对于小线性表或长度固定的线性表,采用顺序存储结构的优越性更为突出。

但是,线性表的顺序存储结构在某些情况下就显得不那么方便,运算效率不那么高。实际上,线性表的顺序存储结构存在以下几方面的缺点:

① 在一般情况下,要在顺序存储的线性表中插入一个新元素或删除一个元素时,为了保证插入或删除后的线性表仍然为顺序存储,则在插入或删除过程中需要移动大量的数据元素。在平均情况下,为了在顺序存储的线性表中插入或删除一个元素,需要移动线性表中约一半的元素;在最坏情况下,则需要移动线性表中所有的元素。因此,对于大的线性表,特别是元素的插入或删除很频繁的情况下,采用顺序存储结构是很不方便的,插入与删除运算的效率都很低。

② 当为一个线性表分配顺序存储空间后,如果出现线性表的存储空间已满,但还需要插入新的元素时,就会发生“上溢”错误。在这种情况下,如果在原线性表的存储空间后找不到与之连续的可用空间,则会导致运算的失败或中断。显然,这种情况的出现对运算是很不利的。也就是说,在顺序存储结构下,线性表的存储空间不便于扩充。

③ 在实际应用中,往往是同时有多个线性表共享计算机的存储空间,例如,在一个处理中,可能要用到若干个线性表(包括栈与队列)。在这种情况下,存储空间的分配将是一个难题。如果将存储空间平均分配给各线性表,则有可能造成有的线性表的空间不够用,而有的线性表的空间根本用不着或用不满,这就使得在有的线性表空间无用而处于空闲的情况下,另外一些线性表的操作由于“上溢”而无法进行。这种情况实际上是计算机的存储空间得不到充分利用。如果多个线性表共享存储空间,对每一个线性表的存储空间进行动态分配,则为了保证每一个线性表的存储空间连续且顺序分配,会导致在对某个线性表进行动态分配存储空间时,必须要移动其他线性表中的数据元素。这就是说,线性表的顺序存储结构不便于对存储空间的动态分配。

由于线性表的顺序存储结构存在以上这些缺点,因此,对于大的线性表,特别是元素变动频繁的大线性表不宜采用顺序存储结构,而是采用下面要介绍的链式存储结构。

假设数据结构中的每一个数据结点对应于一个存储单元,这种存储单元称为存储结点,简称结点。

在链式存储方式中,要求每个结点由两部分组成:一部分用于存放数据元素值,称为数据域;另一部分用于存放指针,称为指针域。其中指针用于指向该结点的前一个或后一个结点(即前件或后件)。

在链式存储结构中,存储数据结构的存储空间可以不连续,各数据结点的存储顺序与数据元素之间的逻辑关系可以不一致,而数据元素之间的逻辑关系是由指针域来确定的。

链式存储方式既可用于表示线性结构,也可用于表示非线性结构。在用链式结构表示较复



杂的非线性结构时,其指针域的个数要多一些。

1. 线性链表

线性表的链式存储结构称为线性链表。

为了适应线性表的链式存储结构,计算机存储空间被划分为一个一个小块,每一小块占若干字节,通常称这些小块为存储结点。

为了存储线性表中的每一个元素,一方面要存储数据元素的值,另一方面要存储各数据元素之间的前后件关系。为此目的,将存储空间中的每一个存储结点分为两部分:一部分用于存储数据元素的值,称为数据域;另一部分用于存放下一个数据元素的存储序号(即存储结点的地址),即指向后件结点,称为指针域。由此可知,在线性链表中,存储空间的结构如图 2.15 所示。

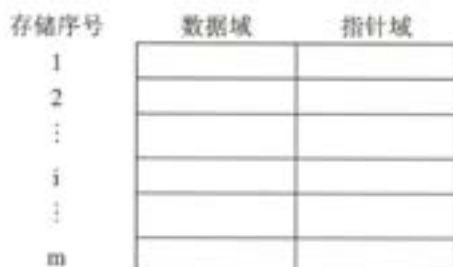


图 2.15 线性链表的存储空间



图 2.16 线性链表的一个存储结点

在线性链表中,用一个专门的指针 HEAD 指向线性链表中第一个数据元素的结点(即存放线性表中第一个数据元素的存储结点的序号)。线性表中最后一个元素没有后件,因此,线性链表中最后一个结点的指针域为空(用 NULL 或 0 表示),表示链表终止。线性链表中存储结点的结构如图 2.16 所示。线性链表的逻辑结构如图 2.17 所示。

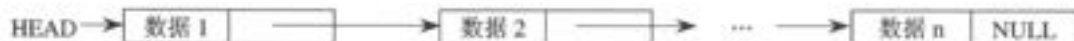


图 2.17 线性链表的逻辑结构

下面举一个例子来说明线性链表的存储结构。

设线性表为 $(a_1, a_2, a_3, a_4, a_5)$, 存储空间具有 10 个存储结点, 该线性表在存储空间中的存储情况如图 2.18(a) 所示。为了直观地表示该线性链表中各元素之间的前后件关系, 还可以用如图 2.18(b) 所示的逻辑状态来表示, 其中每一个结点上面的数字表示该结点的存储序号(简称结点号)。

一般来说, 在线性表的链式存储结构中, 各数据结点的存储序号是不连续的, 并且各结点在存储空间中的位置关系与逻辑关系也不一致。在线性链表中, 各数据元素之间的前后件关系是由各结点的指针域来指示的, 指向线性表中第一个结点的指针 HEAD 称为头指针, 当 HEAD = NULL(或 0) 时称为空表。

对于线性链表, 可以从头指针开始, 沿各结点的指针扫描到链表中的所有结点。读者可自行写出一个算法, 从头指针开始, 依次输出各结点值。

上面讨论的线性链表又称为线性单链表。在这种链表中, 每一个结点只有一个指针域, 由这



图 2.18 线性链表例

个指针只能找到后件结点,但不能找到前件结点。因此,在这种线性链表中,只能顺指针向链尾方向进行扫描,这对于某些问题的处理会带来不便,因为在这种链接方式下,由某一个结点出发,只能找到它的后件,而为了找出它的前件,必须从头指针开始重新寻找。

为了弥补线性单链表的这个缺点,在某些应用中,对线性链表中的每个结点设置两个指针,一个称为左指针(Llink),用以指向其前件结点;另一个称为右指针(Rlink),用以指向其后件结点。这样的线性链表称为双向链表,其逻辑状态如图 2.19 所示。



图 2.19 双向链表示意图

2. 带链的栈

栈也是线性表,也可以采用链式存储结构。图 2.20 是栈在链式存储时的逻辑状态示意图。

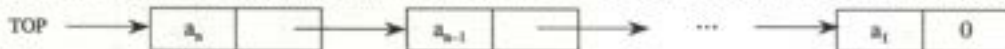


图 2.20 带链的栈

在实际应用中,带链的栈可以用来收集计算机存储空间中所有空闲的存储结点,这种带链的栈称为可利用栈。由于可利用栈链接了计算机存储空间中所有的空闲结点,因此,当计算机系统或用户程序需要存储结点时,就可以从中取出栈顶结点,如图 2.21(b)所示;当计算机系统或用户程序释放一个存储结点(该元素从表中删除)时,则要将该结点放回可利用栈的栈顶,如图 2.21(a)所示。由此可知,计算机中的所有可利用空间都可以以结点为单位链接在可利用栈中。随着其他线性链表中结点的插入与删除,可利用栈处于动态变化之中,即可利用栈经常要进行退栈与入栈操作。

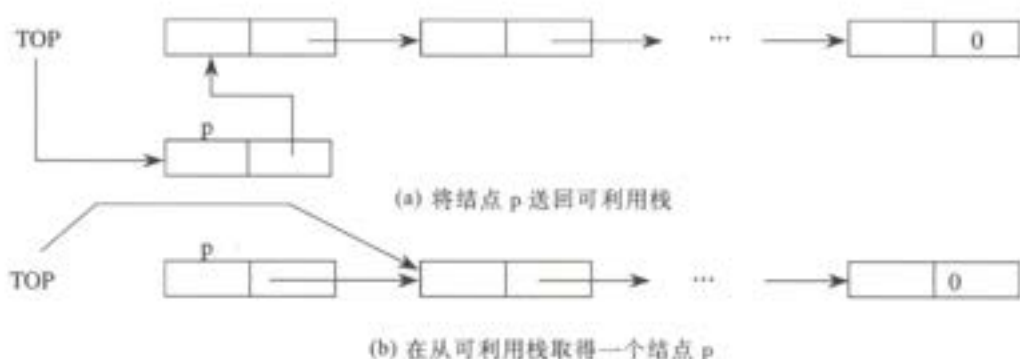


图 2.21 可利用栈及其运算

与顺序栈一样,带链栈的基本操作有以下几个:

- ① 栈的初始化。即建立一个空栈。
- ② 入栈运算。是指在栈顶位置插入一个新元素。
- ③ 退栈运算。是指取出栈顶元素并赋给一个指定的变量。
- ④ 读栈顶元素。是指将栈顶元素赋给一个指定的变量。

3. 带链的队列

与栈类似,队列也是线性表,也可以采用链式存储结构。图 2.22(a)是队列在链式存储时的逻辑状态示意图。图 2.22(b)是将新结点 p 插入队列的示意图。图 2.22(c)是将排头结点 p 退出队列的示意图。

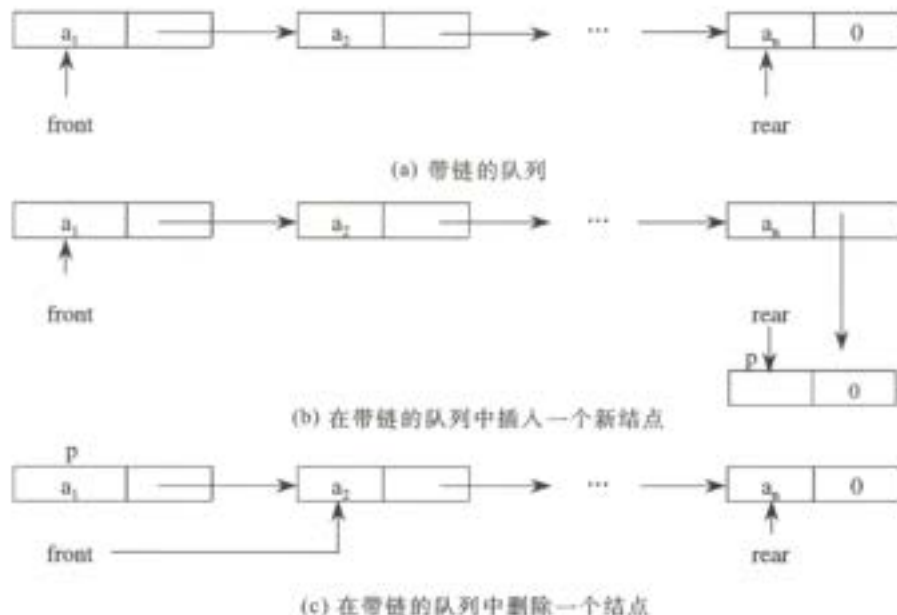


图 2.22 带链的队列及其运算

与顺序队列一样,带链队列的基本操作有以下几个:

- ① 队列的初始化。即建立一个空队列。
- ② 入队运算。是指在循环队列的队尾加入一个新元素。
- ③ 退队运算。是指在循环队列的排头位置退出一个元素并赋给指定的变量。

2.5.2 线性链表的基本运算

线性链表的运算主要有以下几个:

- ① 在线性链表中包含指定元素的结点之前插入一个新元素。
- ② 在线性链表中删除包含指定元素的结点。
- ③ 将两个线性链表按要求合并成一个线性链表。
- ④ 将一个线性链表按要求进行分解。
- ⑤ 逆转线性链表。
- ⑥ 复制线性链表。
- ⑦ 线性链表的排序。
- ⑧ 线性链表的查找。

本小节主要讨论线性链表的插入与删除。

1. 在线性链表中查找指定元素

在对线性链表进行插入或删除的运算中,总是首先需要找到插入或删除的位置,这就需要对线性链表进行扫描查找,在线性链表中寻找包含指定元素值的前一个结点位置。当找到包含指定元素的前一个结点后,就可以在该结点后插入新结点或删除该结点后的一个结点。

在非空线性链表中寻找包含指定元素值 x 的前一个结点 p 的基本方法如下:

从头指针指向的结点开始往后沿指针进行扫描,直到后面已没有结点或下一个结点的数据域为 x 为止。因此,由这种方法找到的结点 p 有两种可能:当线性链表中存在包含元素 x 的结点时,则找到的 p 为第一次遇到的包含元素 x 的前一个结点序号;当线性链表中不存在包含元素 x 的结点时,则找到的 p 为线性链表中的最后一个结点号。

2. 线性链表的插入

线性链表的插入是指在链式存储结构下的线性表中插入一个新元素。

为了要在线性链表中插入一个新元素,首先要给该元素分配一个新结点,以便用于存储该元素的值。新结点可以从可利用栈中取得。然后将存放新元素值的结点链接到线性链表中指定的位置。

假设可利用栈与线性链表如图 2.23(a) 所示。现在要在线性链表中包含元素 x 的结点之前插入一个新元素 b 。其插入过程如下:

① 从可利用栈取得一个结点,设该结点号为 p (即取得结点的存储序号存放在变量 p 中),并置结点 p 的数据域为插入的元素值 b 。经过这一步后,可利用栈的状态如图 2.23(b) 所示。

② 在线性链表中寻找包含元素 x 的前一个结点,设该结点的存储序号为 q 。线性链表如图 2.23(b) 所示。

③ 最后将结点 p 插入到结点 q 之后。为了实现这一步,只要改变以下两个结点的指针域内容:

a. 使结点 p 指向包含元素 x 的结点(即结点 q 的后件结点)。

b. 使结点 q 的指针域内容改为指向结点 p 。

这一步的结果如图 2.23(c) 所示。此时插入就完成。

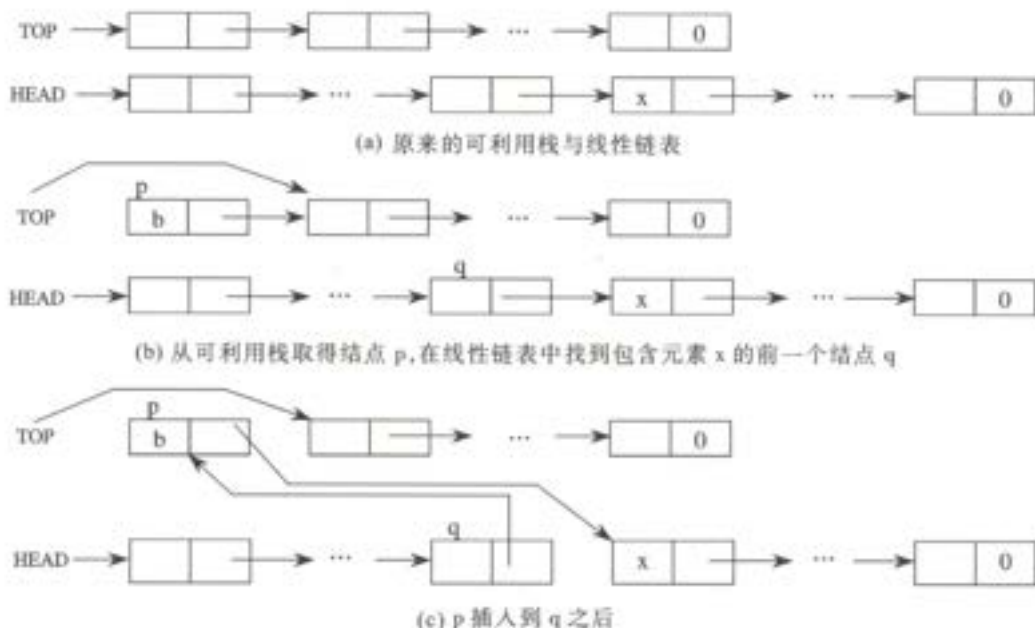


图 2.23 线性链表的插入

由线性链表的插入过程可以看出,由于插入的新结点取自于可利用栈,因此,只要可利用栈不空,在线性链表插入时总能取到存储插入元素的新结点,不会发生“上溢”的情况。而且,由于可利用栈是公用的,多个线性链表可以共享它,从而很方便地实现了存储空间的动态分配。另外,线性链表在插入过程中不发生数据元素移动的现象,只需改变有关结点的指针即可,从而提高了插入的效率。

3. 线性链表的删除

线性链表的删除是指在链式存储结构下的线性表中删除包含指定元素的结点。

为了在线性链表中删除包含指定元素的结点,首先要在线性链表中找到这个结点,然后将要删除结点放回到可利用栈。

假设可利用栈与线性链表如图 2.24(a) 所示。现在要在线性链表中删除包含元素 x 的结点,其删除过程如下:

① 在线性链表中寻找包含元素 x 的前一个结点,设该结点序号为 q 。

② 将结点 q 后的结点 p 从线性链表中删除,即让结点 q 的指针指向包含元素 x 的结点 p 的指针指向的结点。

经过上述两步后,线性链表如图 2.24(b) 所示。

③ 将包含元素 x 的结点 p 送回可利用栈。经过这一步后,可利用栈的状态如图 2.24(c) 所

示。此时,线性链表的删除运算完成。

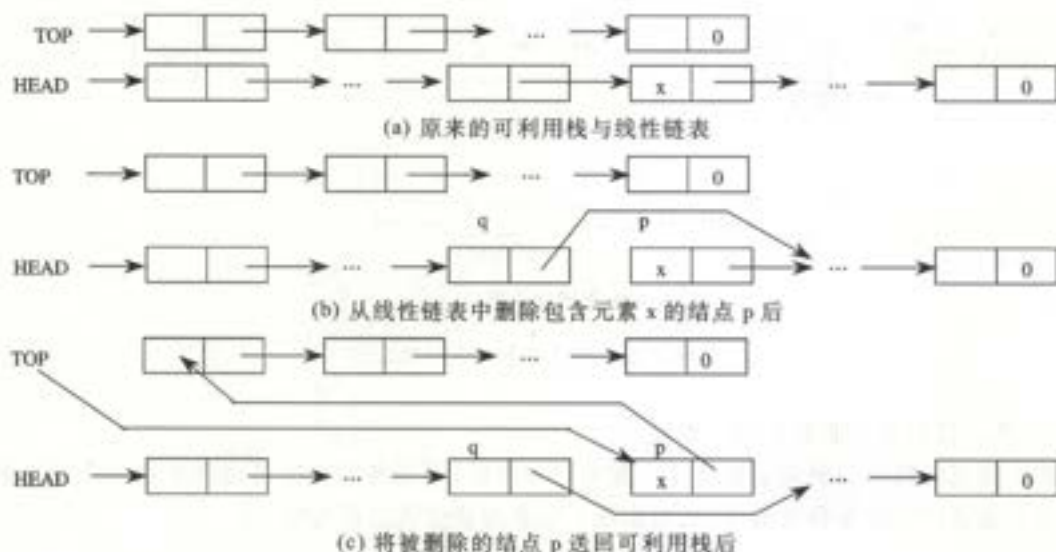


图 2.24 线性链表的删除

从线性链表的删除过程可以看出,在线性链表中删除一个元素后,不需要移动表的数据元素,只需改变被删除元素所在结点的前一个结点的指针域即可。另外,由于可利用栈是用于收集计算机中所有的空闲结点,因此,当从线性链表中删除一个元素后,该元素的存储结点就变为空闲,应将该空闲结点送回到可利用栈。

2.5.3 循环链表

前面所讨论的线性链表中,其插入与删除的运算虽然比较方便,但还存在一个问题,在运算过程中对于空表和对第一个结点的处理必须单独考虑,使空表与非空表的运算不统一。为了克服线性链表的这个缺点,可以采用另一种链接方式,即循环链表(circular linked list)的结构。

循环链表的结构与前面所讨论的线性链表相比,具有以下两个特点:

① 在循环链表中增加了一个表头结点,其数据域为任意或者根据需要来设置,指针域指向线性表的第一个元素的结点。循环链表的头指针指向表头结点。

② 循环链表中最后一个结点的指针域不是空,而是指向表头结点。即在循环链表中,所有结点的指针构成了一个环状链。

图 2.25 是循环链表的示意图。其中图 2.25(a)是一个非空的循环链表,图 2.25(b)是一个空的循环链表。在此,所谓的空表与非空表是针对线性表中的元素而言。

在实际应用中,循环链表与线性单链表相比主要有以下两个方面的优点:

① 在循环链表中,只要指出表中任何一个结点的位置,就可以从它出发访问到表中其他所有的结点。而线性单链表做不到这一点。

② 由于在循环链表中设置了一个表头结点,因此,在任何情况下循环链表中至少有一个结

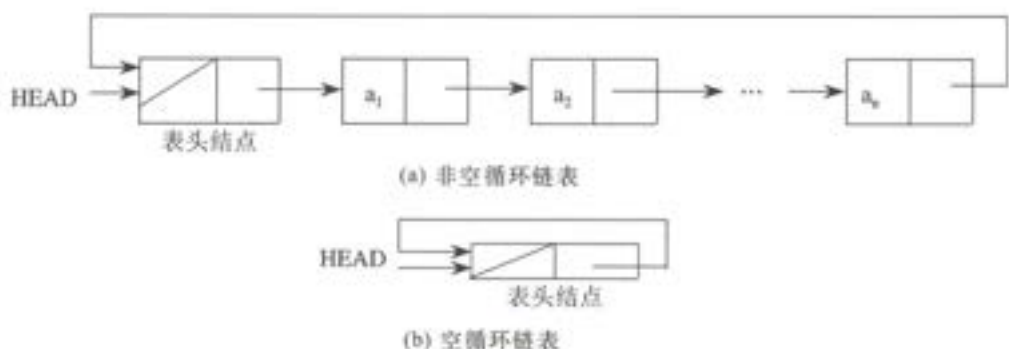


图 2.25 循环链表的逻辑状态

点存在,从而使空表与非空表的运算统一。

循环链表的插入与删除运算要比一般单链表简单,不用考虑在空链表和在第一个结点前插入以及空链表的删除等特殊情况,从而实现了空表与非空表运算的统一。

2.6 树与二叉树

2.6.1 树的基本概念

树(tree)是一种简单的非线性结构。在树这种数据结构中,所有数据元素之间的关系具有明显的层次特性。图 2.26 表示了一棵一般的树。由图 2.26 可以看出,在用图形表示树这种数据结构时,很像自然界中的树,只不过是一棵倒长的树,因此,这种数据结构就用“树”来命名。

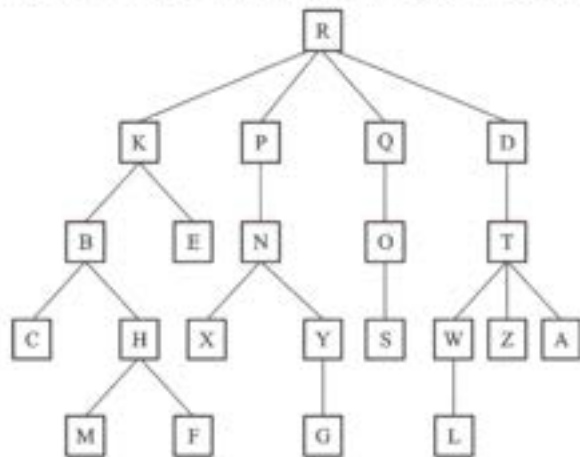


图 2.26 一般的树

在树的图形表示中,总是认为在用直线连起来的两端结点中,上端结点是前件,下端结点是后件,这样,表示前后件关系的箭头就可以省略。

在现实世界中,能用树这种数据结构表示的例子有很多。例如,图 2.27 中的树表示了学校行政关系结构;图 2.28 中的树反映了一本书的层次结构。由于树具有明显的层次关系,因此,具有层次关系的数据都可以用树这种数据结构来描述。在所有的层次关系中,人们最熟悉的是血缘关系,按血缘关系可以很直观地理解树结构中各数据元素结点之间的关系,因此,在描述树结构时,也经常使用血缘关系中的一些术语。

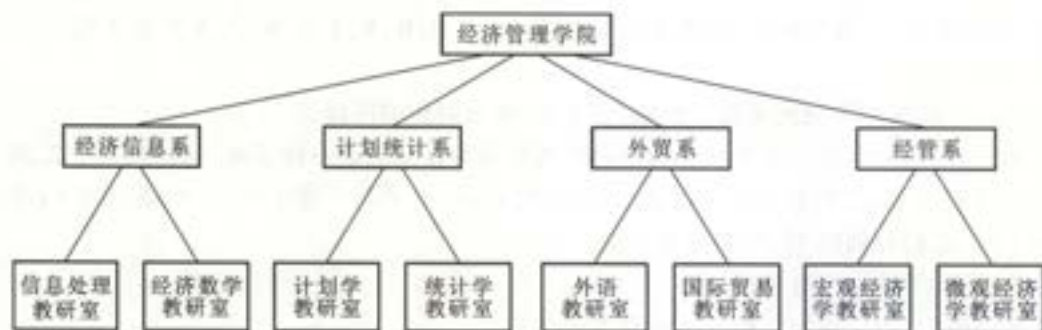


图 2.27 学校行政层次结构树



图 2.28 书的层次结构树

下面介绍树这种数据结构中的一些基本特征,同时介绍有关树结构的基本术语。

在树结构中,每一个结点只有一个前件,称为父结点,没有前件的结点只有一个,称为树的根结点,简称为树的根。例如,在图 2.26 中,结点 R 是树的根结点。

在树结构中,每一个结点可以有多个后件,它们都称为该结点的子结点。没有后件的结点称为叶子结点。例如,在图 2.26 中,结点 C、M、F、E、X、G、S、L、Z、A 均为叶子结点。

在树结构中,一个结点所拥有的后件个数称为该结点的度。例如,在图 2.26 中,根结点 R 的度为 4;结点 T 的度为 3;结点 K、B、N、H 的度为 2;结点 P、Q、D、O、Y、W 的度为 1。叶子结点的度为 0。在树中,所有结点中的最大的度称为树的度。例如,图 2.26 所示的树的度为 4。

在树中,一个结点的度为 n ,则表示该结点有 n 个分支,而每一个分支指向一个后件。因此,在树中,除根结点外,每一个结点(包括所有叶子结点)都有一个唯一的分支指向它。由此可知,在树中,树中的结点数即为树中所有结点的度之和再加 1。例如,在图 2.26 所示的树中,度为 4 的结点有 1 个(结点 R),度为 3 的结点有 1 个(结点 T),度为 2 的结点有 4 个(结点 K、B、N、H),度为 1 的结点有 6 个(结点 P、Q、D、O、Y、W),度为 0 的结点(叶子结点)有 10 个(结点 E、C、X、S、Z、A、M、F、G、L),现在树中共有 22 个结点,而树中所有结点的度之和为 $4 \times 1 + 3 \times 1 + 2 \times 4 + 1 \times 6 +$



$0 \times 10 = 21$, 即树中的结点数 = 树中所有结点的度之和 + 1。

前面已经说过, 树结构具有明显的层次关系, 即树是一种层次结构。在树结构中, 一般按下原则分层:

根结点在第 1 层。

同一层上所有结点的所有子结点都在下一层。例如, 在图 2.26 中, 根结点 R 在第 1 层; 结点 K、P、Q、D 在第 2 层; 结点 B、E、N、O、T 在第 3 层; 结点 C、H、X、Y、S、W、Z、A 在第 4 层; 结点 M、F、G、L 在第 5 层。

树的最大层次称为树的深度。例如, 图 2.26 所示的树的深度为 5。

在树中, 以某结点的一个子结点为根构成的树称为该结点的一棵子树。例如, 在图 2.26 中: 结点 R 有 4 棵子树, 它们分别以 K、P、Q、D 为根结点; 结点 P 有 1 棵子树, 其根结点为 N; 结点 T 有 3 棵子树, 它们分别以 W、Z、A 为根结点。

在树中, 叶子结点没有子树。

在计算机中, 可以用树结构来表示算术表达式。

在一个算术表达式中, 有运算符和运算对象。一个运算符可以有若干个运算对象。例如, 取正 (+) 与取负 (-) 运算符只有一个运算对象, 称为单目运算符; 加 (+)、减 (-)、乘 (*)、除 (/)、乘幂 (**) 运算符有两个运算对象, 称为双目运算符; 三元函数 $f(x, y, z)$ 中的 f 为函数运算符, 它有三个运算对象, 称为三目运算符。一般来说, 多元函数运算符有多个运算对象, 称为多目运算符。算术表达式中的一个运算对象可以是子表达式, 也可以是单变量 (或单变数)。例如, 在表达式 $a * b + c$ 中, 运算符 “+” 有两个运算对象, 其中 $a * b$ 为子表达式, c 为单变量; 而在子表达式 $a * b$ 中, 运算符 “*” 有两个运算对象 a 和 b , 它们都是单变量。

用树来表示算术表达式的原则如下:

- ① 表达式中的每一个运算符在树中对应一个结点, 称为运算符结点。
- ② 运算符的每一个运算对象在树中为该运算符结点的子树 (在树中的顺序为从左到右)。
- ③ 运算对象中的单变量均为叶子结点。

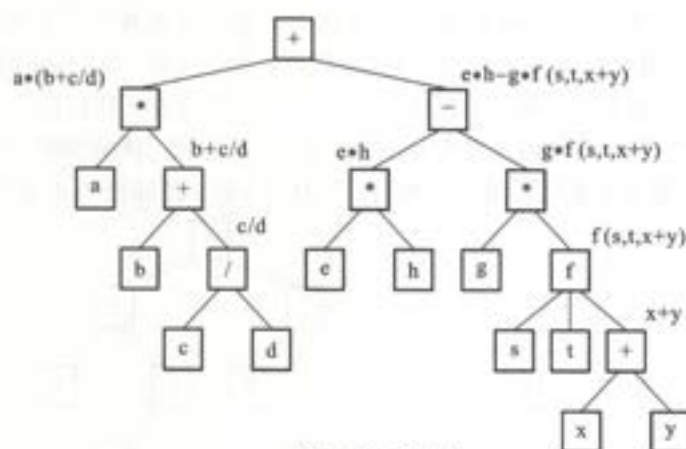
根据以上原则, 可以将表达式

$$a * (b + c / d) + e * h - g * f(s, t, x + y)$$

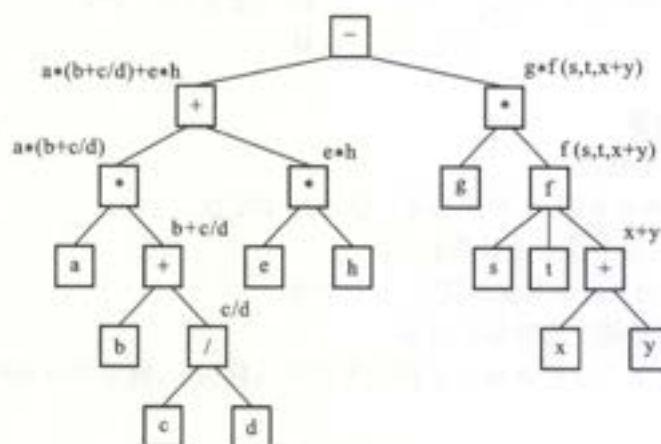
用如图 2.29 所示的树来表示。表示表达式的树通常称为表达式树。由图 2.29 可以看出, 表示一个表达式的表达式树是不唯一的, 如上述表达式可以表示成如图 2.29(a) 和图 2.29(b) 两种表达式树。

树在计算机中通常用多重链表表示。多重链表中的每个结点描述了树中对应结点的信息, 而每个结点中的链域 (即指针域) 个数将随树中该结点的度而定, 其一般结构如图 2.30 所示。

在表示树的多重链表中, 由于树中每个结点的度一般是不同的, 因此, 多重链表中各结点的链域个数也就不同, 这将导致对树进行处理的算法很复杂。如果用定长的结点来表示树中的每个结点, 即取树的度作为每个结点的链域个数, 这就可以使对树的各种处理算法大大简化。但在这种情况下, 容易造成存储空间的浪费, 因为有可能在很多结点中存在空链域。



(a) 表达式树之一



(b) 表达式树之二

图 2.29 $a * (b + c/d) + e * h - g * f(s, t, x + y)$ 的两种表达式树

value(值)	degree(度)	link ₁	link ₂	...	link _n
----------	-----------	-------------------	-------------------	-----	-------------------

图 2.30 树链表中的结点结构

2.6.2 二叉树及其基本性质

1. 什么是二叉树

二叉树(binary tree)是一种很有用的非线性结构。二叉树不同于前面介绍的树结构,但它与树结构很相似,并且,树结构的所有术语都可以用到二叉树这种数据结构上。

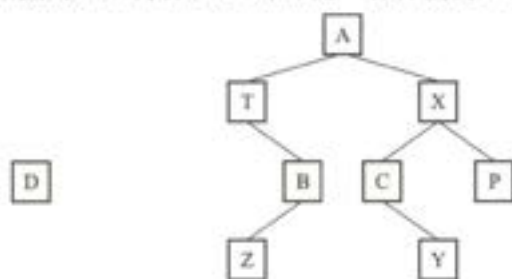
二叉树具有以下两个特点:

- ① 非空二叉树只有一个根结点;
- ② 每一个结点最多有两棵子树,且分别称为该结点的左子树与右子树。



由以上特点可以看出,在二叉树中,每一个结点的度最大为2,即所有子树(左子树或右子树)也均为二叉树,而树结构中的每一个结点的度可以是任意的。另外,二叉树中的每一个结点的子树被明显地分为左子树与右子树。在二叉树中,一个结点可以只有左子树而没有右子树,也可以只有右子树而没有左子树。当一个结点既没有左子树也没有右子树时,该结点即是叶子结点。

图 2.31(a) 是一棵只有根结点的二叉树,图 2.31(b) 是一棵深度为 4 的二叉树。



(a) 只有根结点的二叉树

(b) 深度为 4 的二叉树

图 2.31 二叉树例

2. 二叉树的基本性质

二叉树具有以下几个性质:

性质 1 在二叉树的第 k 层上,最多有 2^{k-1} ($k \geq 1$) 个结点。

根据二叉树的特点,这个性质是显然的。

性质 2 深度为 m 的二叉树最多有 $2^m - 1$ 个结点。

深度为 m 的二叉树是指二叉树共有 m 层。

根据性质 1,只要将第 1 层到第 m 层上的最大的结点数相加,就可以得到整个二叉树中结点数的最大值,即

$$2^{1-1} + 2^{2-1} + \cdots + 2^{m-1} = 2^m - 1$$

性质 3 在任意一棵二叉树中,度为 0 的结点(即叶子结点)总是比度为 2 的结点多一个。

对于这个性质说明如下:

假设二叉树中有 n_0 个叶子结点, n_1 个度为 1 的结点, n_2 个度为 2 的结点,则二叉树中总的结点数为

$$n = n_0 + n_1 + n_2 \quad (1)$$

由于在二叉树中除了根结点外,其余每一个结点都有唯一的一个分支进入。设二叉树中所有进入分支的总数为 m ,则二叉树中总的结点数为

$$n = m + 1 \quad (2)$$

又由于二叉树中这 m 个进入分支是分别由非叶子结点射出的。其中度为 1 的每个结点射出 1 个分支,度为 2 的每个结点射出 2 个分支。因此,二叉树中所有度为 1 与度为 2 的结点射出的分支总数为 $n_1 + 2n_2$ 。而在二叉树中,总的射出分支数应与总的进入分支数相等,即

$$m = n_1 + 2n_2 \quad (3)$$

将(3)代入(2)式有

$$n = n_1 + 2n_2 + 1 \quad (4)$$

最后比较(1)式和(4)式有

$$n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$$

化简后得

$$n_0 = n_2 + 1$$

即：在二叉树中，度为0的结点（即叶子结点）总是比度为2的结点多一个。

例如，在图2.31(b)所示的二叉树中，有3个叶子结点，有2个度为2的结点，度为0的结点数比度为2的结点多一个。

性质4 具有 n 个结点的二叉树，其深度至少为 $\lceil \log_2 n \rceil + 1$ ，其中 $\lceil \log_2 n \rceil$ 表示取 $\log_2 n$ 的整数部分。

这个性质可以由性质2直接得到。

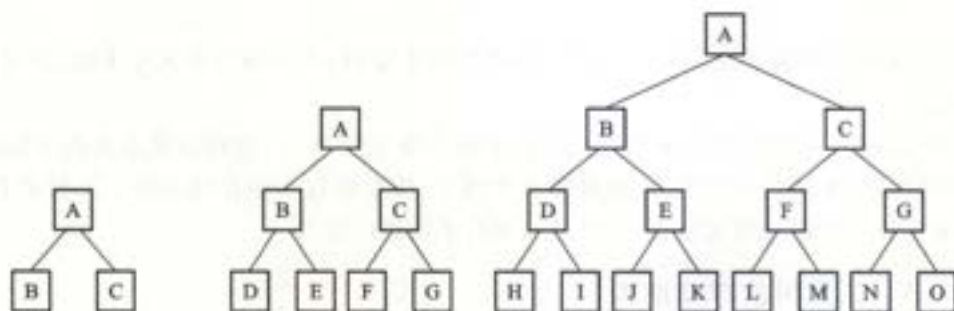
3. 满二叉树与完全二叉树

满二叉树与完全二叉树是两种特殊形态的二叉树。

(1) 满二叉树

所谓满二叉树是指这样的一种二叉树：除最后一层外，每一层上的所有结点都有两个子结点。这就是说，在满二叉树中，每一层上的结点数都达到最大值，即在满二叉树的第 k 层上有 2^{k-1} 个结点，且深度为 m 的满二叉树有 $2^m - 1$ 个结点。

图2.32(a)、(b)、(c)分别是深度为2、3、4的满二叉树。



(a) 深度为2的满二叉树 (b) 深度为3的满二叉树 (c) 深度为4的满二叉树

图 2.32 满二叉树

(2) 完全二叉树

所谓完全二叉树是指这样的二叉树：除最后一层外，每一层上的结点数均达到最大值；在最后一层上只缺少右边的若干结点。

更确切地说，如果从根结点起，对二叉树的结点自上而下、自左至右用自然数进行连续编号，则深度为 m 且有 n 个结点的二叉树，当且仅当其每一个结点都与深度为 m 的满二叉树中编号从1到 n 的结点一一对应时，称之为完全二叉树。

图2.33(a)、(b)分别是深度为3、4的完全二叉树。

对于完全二叉树来说，叶子结点只可能在层次最大的两层上出现；对于任何一个结点，若其右分支下的子孙结点的最大层次为 p ，则其左分支下的子孙结点的最大层次或为 p ，或为 $p+1$ 。

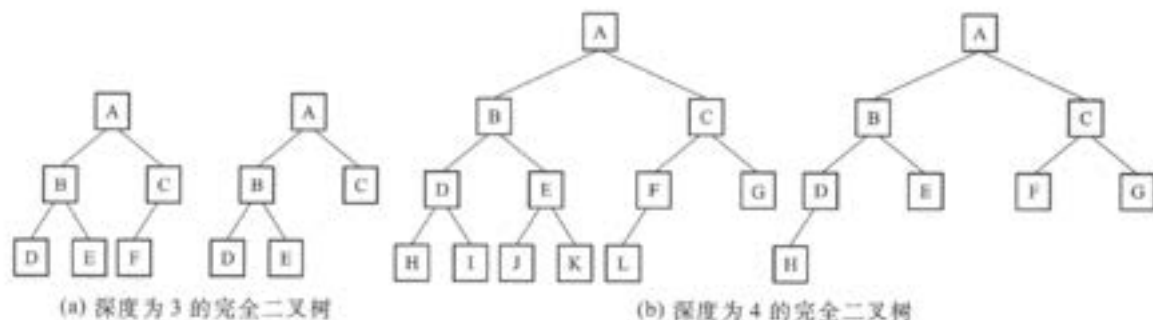


图 2.33 完全二叉树

由满二叉树与完全二叉树的特点可以看出,满二叉树也是完全二叉树,而完全二叉树一般不是满二叉树。

完全二叉树还具有以下两个性质:

性质5 具有 n 个结点的完全二叉树的深度为 $\lceil \log_2 n \rceil + 1$ 。

性质6 设完全二叉树共有 n 个结点。如果从根结点开始,按层序(每一层从左到右)用自然数 $1, 2, \dots, n$ 给结点进行编号,则对于编号为 k ($k=1, 2, \dots, n$) 的结点有以下结论:

① 若 $k=1$,则该结点为根结点,它没有父结点;若 $k>1$,则该结点的父结点编号为 $\text{INT}(k/2)$ 。

② 若 $2k \leq n$,则编号为 k 的结点的左子结点编号为 $2k$;否则该结点无左子结点(显然也没有右子结点)。

③ 若 $2k+1 \leq n$,则编号为 k 的结点的右子结点编号为 $2k+1$;否则该结点无右子结点。

根据完全二叉树的这个性质,如果按从上到下、从左到右顺序存储完全二叉树的各结点,则很容易确定每一个结点的父结点、左子结点和右子结点的位置。

2.6.3 二叉树的存储结构

在计算机中,二叉树通常采用链式存储结构。

与线性链表类似,用于存储二叉树中各元素的存储结点也由两部分组成:数据域与指针域。但在二叉树中,由于每一个元素可以有两个后件(即两个子结点),因此,用于存储二叉树的存储结点的指针域有两个:一个用于指向该结点的左子结点的存储地址,称为左指针域;另一个用于指向该结点的右子结点的存储地址,称为右指针域。图 2.34 为二叉树存储结点的示意图。其中: $L(i)$ 为结点 i 的左指针域,即 $L(i)$ 为结点 i 的左子结点的存储地址; $R(i)$ 为结点 i 的右指针域,即 $R(i)$ 为结点 i 的右子结点的存储地址; $V(i)$ 为数据域。

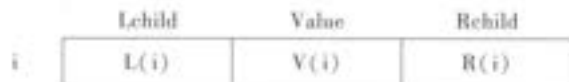


图 2.34 二叉树存储结点的结构

由于二叉树的存储结构中每一个存储结点有两个指针域,因此,二叉树的链式存储结构也称为二叉链表。图 2.35(a)、(b)、(c)分别表示了一棵二叉树、二叉链表的逻辑状态、二叉链表的

物理状态。其中 BT 称为二叉链表的头指针,用于指向二叉树根结点(即存放二叉树根结点的存储地址)。

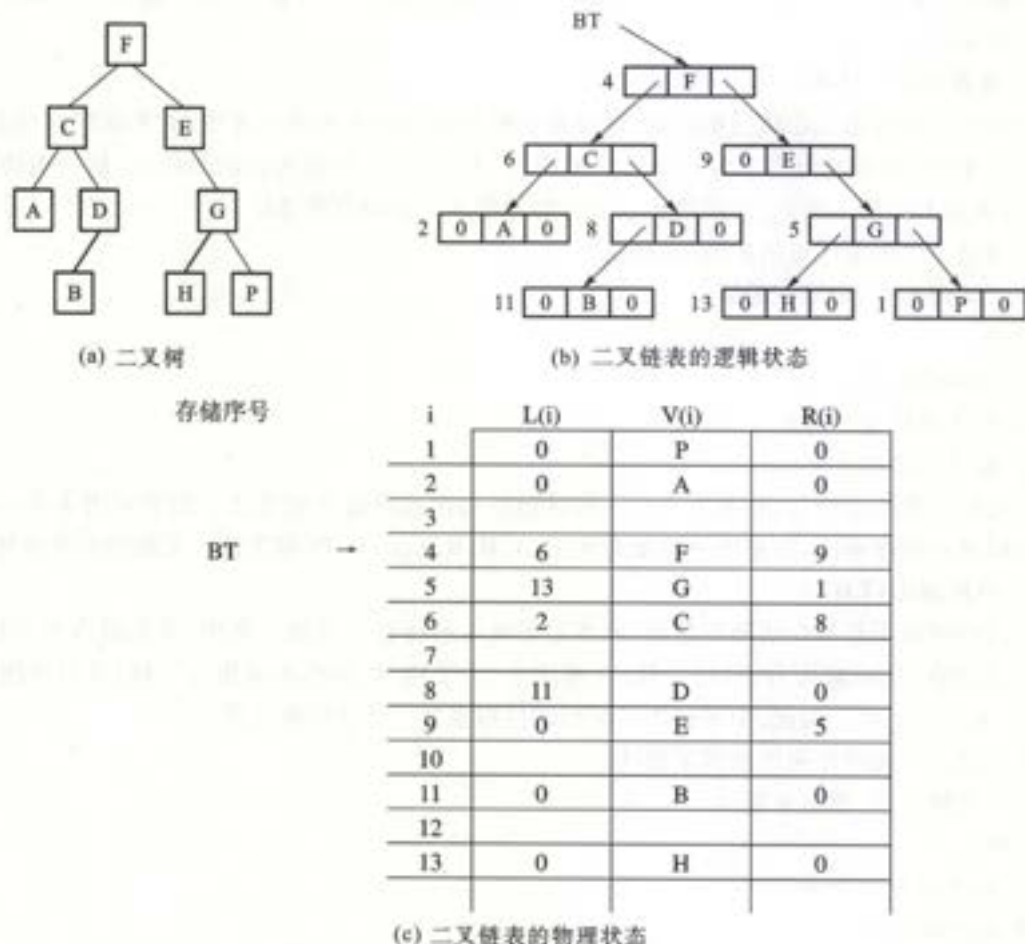


图 2.35 二叉树的链式存储结构

对于满二叉树与完全二叉树来说,根据完全二叉树的性质 6,可以按层序进行顺序存储,这样,不仅节省了存储空间,又能方便地确定每一个结点的父结点与左右子结点的位置,但顺序存储结构对于一般的二叉树不适用。

2.6.4 二叉树的遍历

二叉树的遍历是指不重复地访问二叉树中的所有结点。

由于二叉树是一种非线性结构,因此,对二叉树的遍历要比遍历线性表复杂得多。在遍历二叉树的过程中,当访问到某个结点时,再往下访问可能有两个分支,那么先访问哪一个分支呢?对于二叉树来说,需要访问根结点、左子树上的所有结点、右子树上的所有结点,在这三者中,究竟先访问哪一个?也就是说,遍历二叉树的方法实际上是要确定访问各结点的顺序,以便不重不



漏地访问到二叉树中的所有结点。

在遍历二叉树的过程中,一般先遍历左子树,然后再遍历右子树。在先左后右的原则下,根据访问根结点的次序,二叉树的遍历可以分为三种:前序遍历、中序遍历、后序遍历。下面分别介绍这三种遍历的方法。

1. 前序遍历(DLR)

所谓前序遍历是指在访问根结点、遍历左子树与遍历右子树这三者中,首先访问根结点,然后遍历左子树,最后遍历右子树;并且,在遍历左、右子树时,仍然先访问根结点,然后遍历左子树,最后遍历右子树。因此,前序遍历二叉树的过程是一个递归的过程。

下面是二叉树前序遍历的简单描述:

若二叉树为空,则结束返回。

否则:

- ① 访问根结点;
- ② 前序遍历左子树;
- ③ 前序遍历右子树。

在此特别要注意的是,在遍历左右子树时仍然采用前序遍历的方法。如果对图 2.35(a) 中的二叉树进行前序遍历,则遍历的结果为 F,C,A,D,B,E,G,H,P(称为该二叉树的前序序列)。

2. 中序遍历(LDR)

所谓中序遍历是指在访问根结点、遍历左子树与遍历右子树这三者中,首先遍历左子树,然后访问根结点,最后遍历右子树;并且,在遍历左、右子树时,仍然先遍历左子树,然后访问根结点,最后遍历右子树。因此,中序遍历二叉树的过程也是一个递归的过程。

下面是二叉树中序遍历的简单描述:

若二叉树为空,则结束返回。

否则:

- ① 中序遍历左子树;
- ② 访问根结点;
- ③ 中序遍历右子树。

在此也要特别注意的是,在遍历左右子树时仍然采用中序遍历的方法。如果对图 2.35(a) 中的二叉树进行中序遍历,则遍历结果为 A,C,B,D,F,E,H,G,P(称为该二叉树的中序序列)。

3. 后序遍历(LRD)

所谓后序遍历是指在访问根结点、遍历左子树与遍历右子树这三者中,首先遍历左子树,然后遍历右子树,最后访问根结点,并且,在遍历左、右子树时,仍然先遍历左子树,然后遍历右子树,最后访问根结点。因此,后序遍历二叉树的过程也是一个递归的过程。

下面是二叉树后序遍历的简单描述:

若二叉树为空,则结束返回。否则:

- ① 后序遍历左子树;
- ② 后序遍历右子树;
- ③ 访问根结点。

在此也要特别注意的是,在遍历左右子树时仍然采用后序遍历的方法。如果对图 2.35(a)

中的二叉树进行后序遍历,则遍历结果为 A,B,D,C,H,P,G,E,F(称为该二叉树的后序序列)。

由上述对二叉树的三种遍历方法可以看出,如果知道了某二叉树的前序序列和中序序列,则可以唯一地恢复该二叉树。同样,如果知道了某二叉树的后序序列和中序序列,则也可以唯一地恢复该二叉树。但如果只知道某二叉树的前序序列和后序序列,是不能唯一恢复该二叉树的。

例如,假设某二叉树的前序序列为 DBACFEG,中序序列为 ABCDEFG,则恢复该二叉树的分析过程如下:

由于在前序遍历二叉树中首先访问根结点,因此,前序序列中的第一个结点为二叉树的根结点,即 D 为二叉树的根结点。又由于在中序遍历中访问根结点的次序为居中,而访问左子树上的结点为居先,访问右子树上的结点为最后,因此,在中序序列中以根结点(D)为分界线,前面的子序列(ABC)一定在左子树中,后面的子序列(EFG)一定在右子树中。同样的道理,对于已经划分出的每一个子序列,位于前序序列最前面的一个结点为该子树的根结点,而在中序序列中位于该根结点前面的结点构成左子树上的结点子序列,位于该根结点后面的结点构成右子树上的结点子序列。这个处理过程直到所有子序列为空为止。

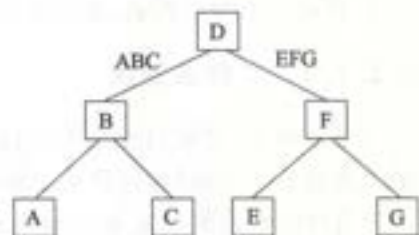


图 2.36 二叉树的恢复过程

根据以上分析过程,该二叉树的恢复过程如图2.36所示。

用类似的分析方法,在已知二叉树的后序序列和中序序列的情况下恢复该二叉树的过程如下:

由于在后序遍历二叉树中访问根结点在最后,因此,后序序列中的最后一个结点为二叉树的根结点。又由于在中序遍历中访问根结点的次序为居中,而访问左子树上的结点为居先,访问右子树上的结点为最后,因此,在中序序列中以根结点为分界线,前面的子序列一定在左子树中,后面的子序列一定在右子树中。同样的道理,对于已经划分出的每一个子序列,位于后序序列最后的一个结点为该子树的根结点,而在中序序列中位于该根结点前面的结点构成左子树上的结点子序列,位于该根结点后面的结点构成右子树上的结点子序列。这个处理过程直到所有子序列为空为止。

2.7 查找技术

查找是数据处理领域中的一个重要内容,查找的效率将直接影响到数据处理的效率。

所谓查找是指在一个给定的数据结构中查找某个指定的元素。通常,根据不同的数据结构,应采用不同的查找方法。

2.7.1 顺序查找

顺序查找又称顺序搜索。顺序查找一般是指在线性表中查找指定的元素,其基本方法如下:

从线性表的第一个元素开始,依次将线性表中的元素与被查元素进行比较,若相等则表示找到(即查找成功);若线性表中所有的元素都与被查元素进行了比较但都不相等,则表示线性表



中没有要找的元素(即查找失败)。

在进行顺序查找过程中,如果线性表中的第一个元素就是被查找元素,则只需做一次比较就查找成功,查找效率最高;但如果被查的元素是线性表中的最后一个元素,或者被查元素根本不在线性表中,则为了查找这个元素需要与线性表中所有的元素进行比较,这是顺序查找的最坏情况。在平均情况下,利用顺序查找法在线性表中查找一个元素,大约要与线性表中一半的元素进行比较。

由此可以看出,对于大的线性表来说,顺序查找的效率是很低的。虽然顺序查找的效率不高,但在下列两种情况下也只能采用顺序查找:

① 如果线性表为无序表(即表中元素的排列是无序的),则不管是顺序存储结构还是链式存储结构,都只能用顺序查找。

② 即使是有序线性表,如果采用链式存储结构,也只能用顺序查找。

2.7.2 二分法查找

二分法查找(又称对分查找)只适用于顺序存储的有序表。在此所说的有序表是指线性表中的元素按值非递减排列(即从小到大,但允许相邻元素值相等)。

设有序线性表的长度为 n ,被查元素为 x ,则对分查找的方法如下:

将 x 与线性表的中间项进行比较:

若中间项的值等于 x ,则说明查到,查找结束;

若 x 小于中间项的值,则在线性表的前半部分(即中间项以前的部分)以相同的方法进行查找;

若 x 大于中间项的值,则在线性表的后半部分(即中间项以后的部分)以相同的方法进行查找。

这个过程一直进行到查找成功或子表长度为 0(说明线性表中没有这个元素)为止。

显然,当有序线性表为顺序存储时才能采用二分查找,并且,二分查找的效率要比顺序查找高得多。可以证明,对于长度为 n 的有序线性表,在最坏情况下,二分查找只需要比较 $\log_2 n$ 次,而顺序查找需要比较 n 次。

2.8 排序技术

排序也是数据处理的重要内容。所谓排序是指将一个无序序列整理成按值非递减顺序排列的有序序列。排序的方法有很多,根据待排序序列的规模以及对数据处理的要求,可以采用不同的排序方法。本节主要介绍一些常用的排序方法。

排序可以在各种不同的存储结构上实现。在本节所介绍的排序方法中,其排序的对象一般认为是顺序存储的线性表,在程序设计语言中就是一维数组。

2.8.1 交换类排序法

所谓交换类排序法是指借助数据元素之间的互相交换进行排序的一种方法。冒泡排序法与快速排序法都属于交换类的排序方法。

1. 冒泡排序法

冒泡排序法是一种最简单的交换类排序方法,它是通过相邻数据元素的交换逐步将线性表

变成有序。

冒泡排序法的基本过程如下：

首先，从表头开始往后扫描线性表，在扫描过程中逐次比较相邻两个元素的大小。若相邻两个元素中，前面的元素大于后面的元素，则将它们互换，称之为消去了一个逆序。显然，在扫描过程中，不断地将两相邻元素中的大者往后移动，最后就将线性表中的最大者换到了表的最后，这也是线性表中最大元素应有的位置。

然后，从后到前扫描剩下的线性表，同样，在扫描过程中逐次比较相邻两个元素的大小。若相邻两个元素中，后面的元素小于前面的元素，则将它们互换，这样就又消去了一个逆序。显然，在扫描过程中，不断地将两相邻元素中的小者往前移动，最后就将剩下线性表中的最小者换到了表的最前面，这也是线性表中最小元素应有的位置。

对剩下的线性表重复上述过程，直到剩下的线性表变空为止，此时的线性表已经变为有序。

在上述排序过程中，对线性表的每一次来回扫描后，都将其中的最大者沉到了表的底部，最小者像气泡一样冒到表的前头。冒泡排序由此而得名，且冒泡排序又称下沉排序。

假设线性表的长度为 n ，则在最坏情况下，冒泡排序需要经过 $n/2$ 遍的从前往后的扫描和 $n/2$ 遍的从后往前的扫描，需要的比较次数为 $n(n-1)/2$ 。但这个工作量不是必需的，一般情况下要小于这个工作量。

图 2.37 是冒泡排序的示意图。图中有方框的元素位置表示扫描过程中最后一次发生交换的位置。由图 2.37 可以看出，整个排序实际上只用了 2 遍从前往后的扫描和 2 遍从后往前的扫描就完成。

原序列	5	1	7	3	1	6	9	4	2	8	6
第 1 遍(从前往后)	5	→ 1	7	→ 3	→ 1	→ 6	9	→ 4	→ 2	→ 8	→ 6
结果	1	5	3	1	6	7	4	2	8	6	9
(从后往前)	1	5	→ 3	→ 1	6	→ 7	→ 4	→ 2	8	→ 6	9
结果	1	1	5	3	2	6	7	4	6	8	9
第 2 遍(从前往后)	1	1	5	→ 3	→ 2	6	7	→ 4	→ 6	8	9
结果	1	1	3	2	5	6	4	6	7	8	9
(从后往前)	1	1	3	→ 2	5	→ 6	→ 4	6	7	8	9
结果	1	1	2	3	4	5	6	6	7	8	9
第 3 遍(从前往后)	1	1	2	3	4	5	6	6	7	8	9
最后结果	1	1	2	3	4	5	6	6	7	8	9

图 2.37 冒泡排序过程示意图

2. 快速排序法

在前面所讨论的冒泡排序法中，由于在扫描过程中只对相邻两个元素进行比较，因此，在互换两个相邻元素时只能消除一个逆序。如果通过两个（不是相邻的）元素的交换，能够消除线性表中的多个逆序，就会大大加快排序的速度。显然，为了通过一次交换能消除多个逆序，就不能像冒泡排序法那样对相邻两个元素进行比较，因为这只能使相邻两个元素进行交换，从而只能消除一个逆序。下面介绍的快速排序法可以实现通过一次交换而消除多个逆序。



快速排序法也是一种互换类的排序方法,但由于它比冒泡排序法的速度快,因此称之为快速排序法。

快速排序法的基本思想如下:

从线性表中选取一个元素,设为 T ,将线性表后面小于 T 的元素移到前面,而前面大于 T 的元素移到后面,结果就将线性表分成了两部分(称为两个子表), T 插入到其分界线的位置处,这个过程称为线性表的分割。通过对线性表的一次分割,就以 T 为分界线,将线性表分成了前后两个子表,且前面子表中的所有元素均不大于 T ,而后面子表中的所有元素均不小于 T 。

如果对分割后的各子表再按上述原则进行分割,并且,这种分割过程可以一直做下去,直到所有子表为空为止,则此时的线性表就变成了有序表。

由此可知,快速排序法的关键是对线性表进行分割,以及对各分割出的子表再进行分割,这个过程如图 2.38 所示。

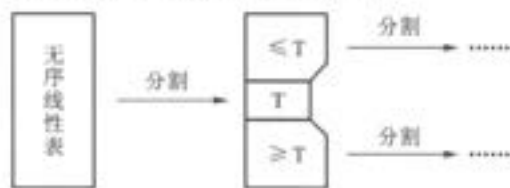


图 2.38 快速排序示意图

在对线性表或子表进行实际分割时,可以按如下步骤进行:

首先,在表的第一个、中间一个与最后一个元素中选取中项,设为 $P(k)$,并将 $P(k)$ 赋给 T ,再将表中的第一个元素移到 $P(k)$ 的位置上。

然后设置两个指针 i 和 j 分别指向表的起始与最后的位置。反复操作以下两步:

- ① 将 j 逐渐减小,并逐次比较 $P(j)$ 与 T ,直到发现一个 $P(j) < T$ 为止,将 $P(j)$ 移到 $P(i)$ 的位置上。
- ② 将 i 逐渐增大,并逐次比较 $P(i)$ 与 T ,直到发现一个 $P(i) > T$ 为止,将 $P(i)$ 移到 $P(j)$ 的位置上。

上述两个操作交替进行,直到指针 i 与 j 指向同一个位置(即 $i=j$)为止,此时将 T 移到 $P(i)$ 的位置上。

在快速排序过程中,随着对各子表不断地进行分割,划分出的子表会越来越多,但一次又只能对一个子表进行再分割处理,需要将暂时不分割的子表记忆起来,这就要用一个栈来实现。在对某个子表进行分割后,可以将分割出的后一个子表的第一个元素与最后一个元素的位置压入栈中,而继续对前一个子表进行再分割;当分割出的子表为空时,可以从栈中退出一个子表(实际上只是该子表的第一个元素与最后一个元素的位置)进行分割。这个过程直到栈空为止,此时说明所有子表为空,没有子表再需要分割,排序就完成了。

由此可以看出,快速排序可以设计成一个递归算法,上述处理过程由系统自动完成。

快速排序在最坏情况下需要进行 $n(n-1)/2$ 次比较,但实际的排序效率要比冒泡排序高得多。

2.8.2 插入类排序法

冒泡排序法与快速排序法本质上都是通过数据元素的交换来逐步消除线性表中的逆序。本节讨论另一类排序的方法,即插入类排序法。

1. 简单插入排序法

所谓插入排序,是指将无序序列中的各元素依次插入到已经有序的线性表中。

我们可以想象,在线性表中,只包含第 1 个元素的子表显然可以看成是有序表。接下来的问题是,从线性表的第 2 个元素开始直到最后一个元素,逐次将其中的每一个元素插入到前面已经有序的子表中。一般来说,假设线性表中前 $j-1$ 个元素已经有序,现在要将线性表中第 j 个元素插入到前面的有序子表中,插入过程如下:

首先将第 j 个元素放到一个变量 T 中,然后从有序子表的最后一个元素(即线性表中第 $j-1$ 个元素)开始,往前逐个与 T 进行比较,将大于 T 的元素均依次向后移动一个位置,直到发现一个元素不大于 T 为止,此时就将 T (即原线性表中的第 j 个元素)插入到刚移出的空位置上,有序子表的长度就变为 j 了。

图 2.39 给出了插入排序的示意图。图中画有方框的元素表示刚被插入到有序子表中。

在简单插入排序法中,每一次比较后最多移掉一个逆序,因此,这种排序方法的效率与冒泡排序法相同。在最坏情况下,简单插入排序需要 $n(n-1)/2$ 次比较。

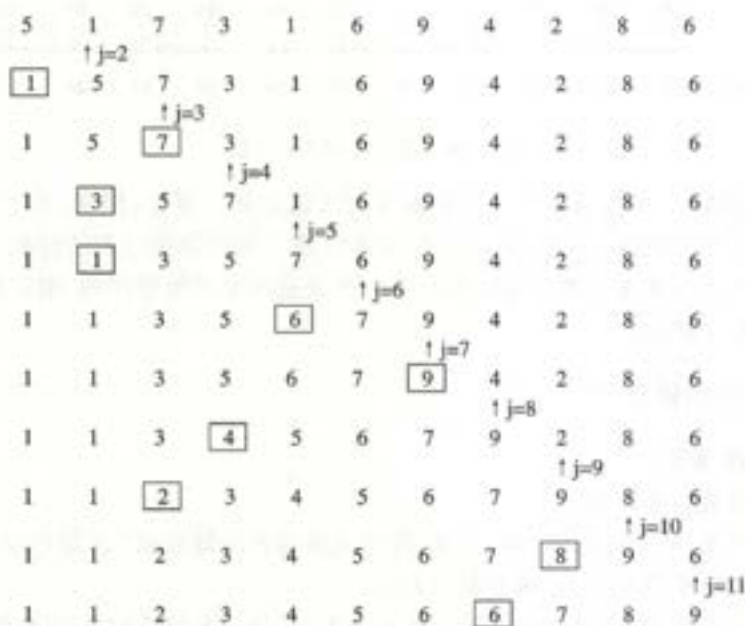


图 2.39 简单插入排序示意图

2. 希尔排序法

希尔排序法(shell sort)属于插入类排序,但它对简单插入排序做了较大的改进。

希尔排序法的基本思想如下:

将整个无序序列分割成若干小的子序列分别进行插入排序。

子序列的分割方法如下:

将相隔某个增量 h 的元素构成一个子序列。在排序过程中,逐次减小这个增量,最后当 h 减到 1 时,进行一次插入排序,排序就完成。

增量序列一般取 $h_k = n/2^k (k=1, 2, \dots, [\log_2 n])$, 其中 n 为待排序序列的长度。

图 2.40 为希尔排序法的示意图。

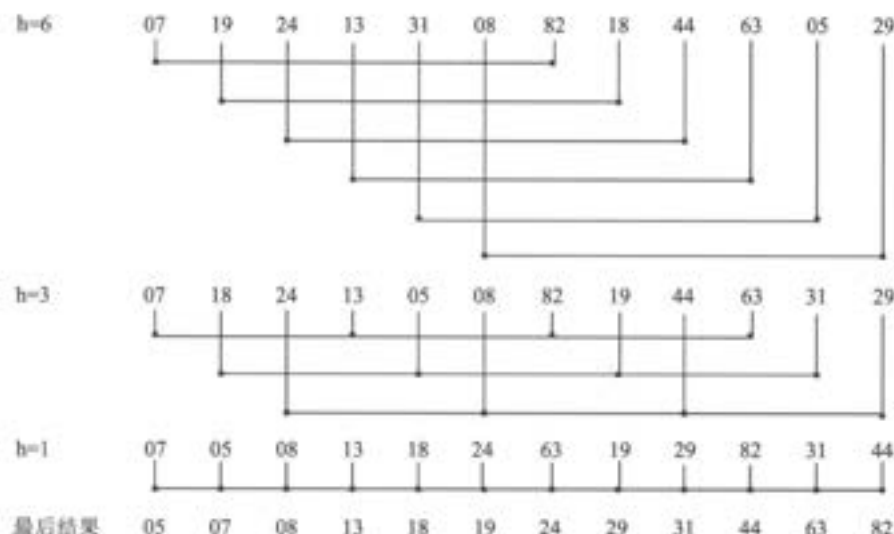


图 2.40 希尔排序法示意图

在希尔排序过程中,虽然对于每一个子表采用的仍是插入排序,但是,在子表中每进行一次比较就有可能移去整个线性表中的多个逆序,从而改善了整个排序过程的性能。

希尔排序的效率与所选取的增量序列有关。如果选取上述增量序列,则在最坏情况下,希尔排序所需要的比较次数为 $O(n^{1.5})$ 。

2.8.3 选择类排序法

1. 简单选择排序法

选择排序法的基本思想如下:

扫描整个线性表,从中选出最小的元素,将它交换到表的最前面(这是它应有的位置);然后对剩下的子表采用同样的方法,直到子表空为止。

对于长度为 n 的序列,选择排序需要扫描 $n-1$ 遍,每一遍扫描均从剩下的子表中选出最小的元素,然后将该最小的元素与子表中的第一个元素进行交换。图 2.41 是这种排序法的示意图,图中有方框的元素是刚被选出来的最小元素。

原序列	89	21	56	48	85	16	19	47
第 1 遍选择	16	21	56	48	85	89	19	47
第 2 遍选择	16	19	56	48	85	89	21	47
第 3 遍选择	16	19	21	48	85	89	56	47
第 4 遍选择	16	19	21	47	85	89	56	48
第 5 遍选择	16	19	21	47	48	89	56	85
第 6 遍选择	16	19	21	47	48	56	89	85
第 7 遍选择	16	19	21	47	48	56	85	89

图 2.41 简单选择排序法示意图

简单选择排序法在最坏情况下需要比较 $n(n-1)/2$ 次。

2. 堆排序法

堆排序法属于选择类的排序方法。

堆的定义如下：

具有 n 个元素的序列 $(h_1, h_2, \dots, h_n)$ ，当且仅当满足

$$\begin{cases} h_i \geq h_{2i} \\ h_i \geq h_{2i+1} \end{cases} \quad \text{或} \quad \begin{cases} h_i \leq h_{2i} \\ h_i \leq h_{2i+1} \end{cases}$$

($i=1, 2, \dots, n/2$) 时称之为堆。本节只讨论满足前者条件的堆。

由堆的定义可以看出，堆顶元素（即第一个元素）必为最大项。

在实际处理中，可以用一维数组 $H(1:n)$ 来存储堆序列中的元素，也可以用完全二叉树来直观地表示堆的结构。例如，序列 $(91, 85, 53, 36, 47, 30, 24, 12)$ 是一个堆，它所对应的完全二叉树如图 2.42 所示。由图 2.42 可以看出，在用完全二叉树表示堆时，树中所有非叶子结点值均不小于其左、右子树的根结点值，因此，堆顶（完全二叉树的根结点）元素必为序列的 n 个元素中的最大项。

在具体讨论堆排序法之前，先讨论这样一个问题：在一棵具有 n 个结点的完全二叉树[用一维数组 $H(1:n)$ 表示]中，假设结点 $H(m)$ 的左右子树均为堆，现要将以 $H(m)$ 为根结点的子树也调整为堆。这是调整建堆的问题。

例如，假设图 2.43(a) 是某完全二叉树的一棵子树。显然，在这棵子树中，根结点 47 的左、右子树均为堆。现在为了将整个子树调整为堆，首先将根结点 47 与其左、右子树的根结点值进行比较，此时由于左子树根结点 91 大于右子树根结点 53，且它又大于根结点 47，因此，根据堆的条件，应将元素 47 与 91 交换，如图 2.43(b) 所示。经过这一次交换后，破坏了原来左子树的堆结构，需要对左子树再进行调整，将元素 85 与 47 进行交换，调整后的结果如图 2.43(c) 所示。

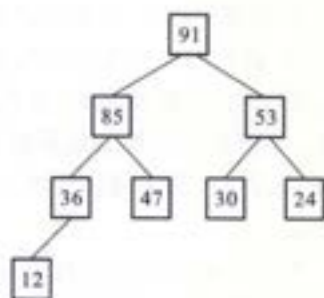


图 2.42 堆顶元素为最大的堆

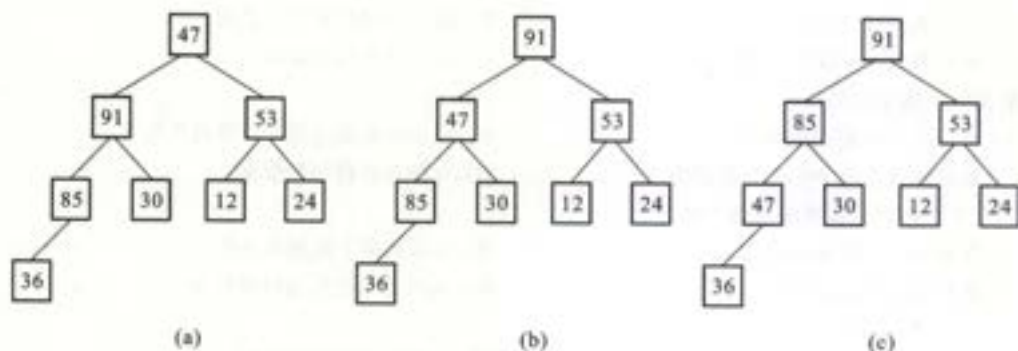


图 2.43 调整建堆示意图

由这个例子可以看出，在调整建堆的过程中，总是将根结点值与左、右子树的根结点值进行比较，若不满足堆的条件，则将左、右子树根结点值中的大者与根结点值进行交换。这个调整过

程一直做到所有子树均为堆为止。

有了调整建堆的算法后,就可以将一个无序序列建成为堆。

假设无序序列 $H(1:n)$ 以完全二叉树表示。从完全二叉树的最后一个非叶子结点(即第 $n/2$ 个元素)开始,直到根结点(即第一个元素)为止,对每一个结点进行调整建堆,最后就可以得到与该序列对应的堆。

根据堆的定义,可以得到堆排序的方法如下:

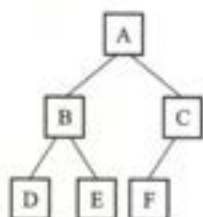
① 首先将一个无序序列建成堆。

② 然后将堆顶元素(序列中的最大项)与堆中最后一个元素交换(最大项应该在序列的最后)。不考虑已经换到最后的那个元素,只考虑前 $n-1$ 个元素构成的子序列,显然,该子序列已不是堆,但左、右子树仍为堆,可以将该子序列调整为堆。反复做第(2)步,直到剩下的子序列为空为止。

堆排序的方法对于规模较小的线性表并不适合,但对于较大规模的线性表来说是很有效的。在最坏情况下,堆排序需要比较的次数为 $O(n \log_2 n)$ 。

习题 2

- 算法的时间复杂度是指
 - 执行算法程序所需要的时间
 - 算法程序的长度
 - 算法执行过程中所需要的基本运算次数
 - 算法执行过程中所需要的所有运算次数
 - 算法程序中的指令条数
- 算法的空间复杂度是指
 - 算法程序的长度
 - 算法程序中的指令条数
 - 算法程序所占的存储空间
 - 算法执行过程中所需要的存储空间
 - 算法所处理的数据量
- 下列叙述中正确的是
 - 线性表是线性结构
 - 栈与队列是非线性结构
 - 循环链表是非线性结构
 - 二叉树是线性结构
- 数据的存储结构是指
 - 数据所占的存储空间量
 - 数据的逻辑结构在计算机中的表示
 - 数据在计算机中的顺序存储方式
 - 存储在外存中的数据
- 下列关于队列的叙述中正确的是
 - 在队列中只能插入数据
 - 在队列中只能删除数据
 - 队列是先进先出的线性表
 - 队列是先进后出的线性表
- 下列关于栈的叙述中正确的是
 - 在栈中只能插入数据
 - 在栈中只能删除数据
 - 栈是先进先出的线性表
 - 栈是先进后出的线性表
 - 栈是一种非线性结构
- 设有下列二叉树:



对此二叉树进行中序遍历的结果为

- A) ABCDEF B) DBEAF C) ABDECF D) DEBFCA
8. 在深度为 5 的满二叉树中, 叶子结点的个数为
A) 32 B) 31 C) 16 D) 15
9. 设一棵二叉树的中序序列为 DBEAF, 前序序列为 ABDECF, 则后序序列为
A) ABCDEF B) FEDCBA C) DEBFCA
10. 设一棵完全二叉树共有 700 个结点, 则该完全二叉树中的叶子结点数为
A) 351 B) 350 C) 349 D) 348
11. 对长度为 n 的线性表进行顺序查找, 在最坏情况下所需要的比较次数为
A) $n+1$ B) n C) $(n+1)/2$ D) $n/2$
12. 在长度为 n 的线性表中寻找最大项, 在最坏情况下所需要的比较次数为
A) $n+1$ B) $n-1$ C) n D) $n/2$
13. 设树 T 的度为 4, 其中度为 1, 2, 3, 4 的结点个数分别为 4, 2, 1, 1。则树 T 中的叶子结点数为
A) 8 B) 7 C) 6 D) 5
14. 在长度为 n 的有序线性表中进行二分查找, 在最坏情况下所需要的比较次数为
A) $n-1$ B) $n/2$ C) $(n-1)/2$ D) $\log_2 n$
15. 下列排序法中, 在最坏情况下时间复杂度最小的是
A) 快速排序 B) 希尔排序 C) 堆排序 D) 冒泡排序
16. 设循环队列为 $Q(1:m)$, 其初始状态为 $front=rear=m$ 。经过一系列入队与退队运算后, $front=30, rear=$
10. 现要在该循环队列中作顺序查找, 最坏情况下需要比较的次数为
A) 19 B) 20 C) $m-19$ D) $m-20$
17. 设数据集合为 $D=\{1, 3, 5, 7, 9\}$, D 上的关系为 R 。下列数据结构 $B=(D, R)$ 中为非线性结构的是
A) $R=\{(5, 1), (7, 9), (1, 7), (9, 3)\}$ B) $R=\{(9, 7), (1, 3), (7, 1), (3, 5)\}$
C) $R=\{(1, 9), (9, 7), (7, 5), (5, 3)\}$ D) $R=\{(1, 3), (3, 5), (5, 9)\}$

3.1 程序设计方法与风格

程序设计是一门技术,需要相应的理论、技术、方法和工具来支持。就程序设计方法和技术的发展而言,主要经过了结构化程序设计和面向对象的程序设计阶段。

除了好的程序设计方法和技术之外,程序设计风格也是很重要的。因为程序设计风格会深刻地影响软件的质量和可维护性,良好的程序设计风格可以使程序结构清晰合理,使程序代码便于维护,因此,程序设计风格对保证程序的质量是很重要的。

一般来讲,程序设计风格是指编写程序时所表现出的特点、习惯和逻辑思路。程序是由人来编写的,为了测试和维护程序,往往还要阅读和跟踪程序,因此程序设计的风格总体而言应该强调简明清晰,易读易懂,程序必须是可以理解的。可以认为,著名的“清晰第一,效率第二”的论点已成为当今主导的程序设计风格。

要形成良好的程序设计风格,主要应注重和考虑下述一些因素。

1. 源程序文档化

源程序文档化应考虑如下几点:

① 符号名的命名:符号名的命名应具有一定的实际含义,以便于对程序功能的理解。

② 程序注释:正确的注释能够帮助读者理解程序。注释一般分为序言性注释和功能性注释。序言性注释通常位于每个程序的开头部分,它给出程序的整体说明,主要描述内容可以包括:程序标题、程序功能说明、主要算法、接口说明、程序位置、开发简历、程序设计者、复审者、复审日期、修改日期等。功能性注释的位置一般嵌在源程序体之中,主要描述其后的语句或程序做什么。

③ 视觉组织:为使程序的结构一目了然,可以在程序中利用空格、空行、缩进等技巧使程序层次清晰。

2. 数据说明的方法

在编写程序时,需要注意数据说明的风格,以便使程序中的数据说明更易于理解和维护。一般应注意如下几点:

① 数据说明的次序规范化。鉴于程序理解、阅读和维护的需要,使数据说明次序固定,可以使数据的属性容易查找,也有利于测试、排错和维护。

② 说明语句中变量安排有序化。当一个说明语句说明多个变量时,变量按照字母顺序排列为好。

③ 使用注释来说明复杂数据的结构。

3. 语句的结构

程序应该简单易懂,语句构造应该简单直接,不应该为提高效率而把语句复杂化。一般应注意如下:

- ① 在一行内只写一条语句;
- ② 程序编写应优先考虑清晰性;
- ③ 除非对效率有特殊要求,程序编写要做到清晰第一,效率第二;
- ④ 首先要保证程序正确,然后才要求提高速度;
- ⑤ 避免使用临时变量而使程序的可读性下降;
- ⑥ 避免不必要的转移;
- ⑦ 尽可能使用库函数;
- ⑧ 避免采用复杂的条件语句;
- ⑨ 尽量减少使用“否定”条件的条件语句;
- ⑩ 数据结构要有利于程序的简化;
- ⑪ 要模块化,使模块功能尽可能单一化;
- ⑫ 利用信息隐蔽,确保每一个模块的独立性;
- ⑬ 从数据出发去构造程序;
- ⑭ 不要修补不好的程序,要重新编写。

4. 输入和输出

输入和输出信息是用户直接关心的,输入和输出方式和格式应尽可能方便用户的使用,因为系统能否被用户接受,往往取决于输入和输出的风格。无论是批处理的输入和输出方式,还是交互式的输入和输出方式,在设计和编程时都应该考虑如下原则:

- ① 对所有的输入数据都要检验数据的合法性;
- ② 检查输入项的各种重要组合的合理性;
- ③ 输入格式要简单,以使得输入的步骤和操作尽可能简单;
- ④ 输入数据时,应允许使用自由格式;
- ⑤ 应允许默认值;
- ⑥ 输入一批数据时,最好使用输入结束标志;
- ⑦ 在以交互式输入/输出方式进行输入时,要在屏幕上使用提示符明确提示输入的请求,同时在数据输入过程中和输入结束时,应在屏幕上给出状态信息;
- ⑧ 当程序设计语言对输入格式有严格要求时,应保持输入格式与输入语句的一致性;给所有的输出加注释,并设计输出报表格式。



3.2 结构化程序设计

由于软件危机的出现,人们开始研究程序设计方法,其中最受关注的是结构化程序设计方法。20世纪70年代提出了“结构化程序设计(structured programming)”的思想和方法。结构化程序设计方法引入了工程思想和结构化思想,使大型软件的开发和编程都得到了极大的改善。

3.2.1 结构化程序设计的原则

结构化程序设计方法的主要原则可以概括为自顶向下,逐步求精,模块化,限制使用 goto 语句。

(1) 自顶向下

程序设计时,应先考虑总体,后考虑细节;先考虑全局目标,后考虑局部目标。不要一开始就过多追求众多的细节,先从最上层总目标开始设计,逐步使问题具体化。

(2) 逐步求精

对复杂问题,应设计一些子目标作过渡,逐步细化。

(3) 模块化

一个复杂问题,肯定是由若干稍简单的问题构成。模块化是把程序要解决的总目标分解为分目标,再进一步分解为具体的小目标,把每个小目标称为一个模块。

(4) 限制使用 goto 语句

实际上,结构化程序设计方法的起源来自对 GOTO 语句的认识和争论。肯定的结论是,在块和进程的非正常出口处往往需要用 GOTO 语句,使用 GOTO 语句会使程序执行效率较高;在合成程序目标时,GOTO 语句往往是有用的,如返回语句用 GOTO。否定的结论是,GOTO 语句是有害的,是造成程序混乱的祸根,程序的质量与 GOTO 语句的数量成反比,应该在所有高级程序设计语言中取消 GOTO 语句。取消 GOTO 语句后,程序易理解、易排错、易维护,程序容易进行正确性证明。作为争论的结论,1974年 Knuth 发表了令人信服的总结,并证实了:

① 滥用 GOTO 语句确实有害,应尽量避免;

② 完全避免使用 GOTO 语句也并非是个明智的方法,有些地方使用 GOTO 语句,会使程序流程更清楚、效率更高;

③ 争论的焦点不应该放在是否取消 GOTO 语句,而应该放在用什么样的程序结构上。

其中最关键的是,肯定以提高程序清晰性为目标的结构化方法。

3.2.2 结构化程序的基本结构与特点

结构化程序设计方法是程序设计的先进方法和工具。采用结构化程序设计方法编写程序,可使程序结构良好、易读、易理解、易维护。1966年,Boehm 和 Jacopini 证明了程序设计语言仅仅使用顺序、选择和循环三种基本控制结构就足以表达出各种其他形式结构的程序设计方法。

(1) 顺序结构

顺序结构是一种简单的程序设计,它是最基本、最常用的结构,如图 3.1 所示。顺序结构是

顺序执行结构,所谓顺序执行,就是按照程序语句行的自然顺序,一条语句一条语句地执行程序。

(2) 选择结构

选择结构又称为分支结构,它包括简单选择和多分支选择结构,这种结构可以根据设定的条件,判断应该选择哪一条分支来执行相应的语句序列。图 3.2 列出了包含 2 个分支的简单选择结构。图 3.2 中“exp”表示条件表达式。

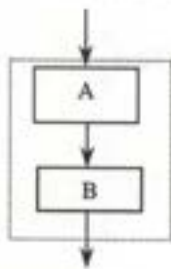


图 3.1 顺序结构

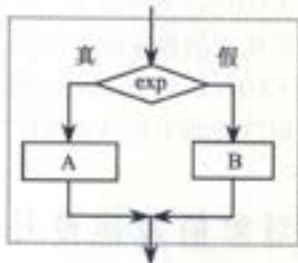


图 3.2 选择结构

(3) 循环结构

循环结构根据给定的条件,判断是否需要重复执行某一相同的或类似的程序段,利用循环结构可简化大量的程序行。在程序设计语言中,循环结构对应两类循环语句,对先判断后执行循环体的称为当型循环结构,如图 3.3 所示。对先执行循环体后判断的称为直到型循环结构,如图 3.4 所示。

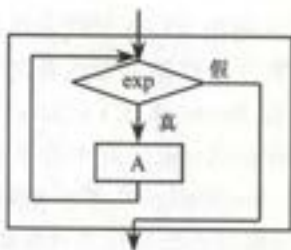


图 3.3 当型循环结构

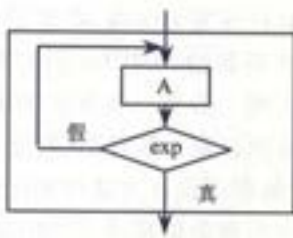


图 3.4 直到型循环结构

总之,遵循结构化程序的设计原则,按结构化程序设计方法设计出的程序具有明显的优点,其一,程序易于理解、使用和维护。程序员采用结构化编程方法,便于控制、降低程序的复杂性,因此容易编写程序。便于验证程序的正确性,结构化程序清晰易读,可理解性好,程序员能够进行逐步求精、程序证明和测试,以确保程序的正确性,程序容易阅读并被人理解,便于用户的使用和维护。其二,提高了编程工作的效率,降低了软件开发成本。由于结构化编程方法能够把错误控制到最低限度,因此能够减少调试和查错时间。结构化程序是由一些为数不多的基本结构模块组成,这些模块甚至可以由机器自动生成,从而极大地减轻了编程工作量。

3.2.3 结构化程序设计原则和方法的应用

基于对结构化程序设计原则、方法以及结构化程序基本构成结构的掌握和了解,在结构化程序设计的具体实施中,要注意把握如下要素:

- ① 使用程序设计语言中的顺序、选择、循环等有限的控制结构表示程序的控制逻辑；
- ② 选用的控制结构只准许有一个入口和一个出口；
- ③ 程序语句组成容易识别的块，每块只有一个入口和一个出口；
- ④ 复杂结构应该用嵌套的基本控制结构进行组合嵌套来实现；
- ⑤ 语言中所没有的控制结构，应该采用前后一致的方法来模拟；
- ⑥ 严格控制 GOTO 语句的使用。其意思是指：
 - a. 用一个非结构化的程序设计语言去实现一个结构化的构造；
 - b. 若不使用 GOTO 语句会使功能模糊；
 - c. 在某种可以改善而不是损害程序可读性的情况下。

3.3 面向对象的程序设计

3.3.1 关于面向对象方法

自 20 世纪 80 年代中期开始，面向对象 (Object Oriented) 方法已经发展成为主流的软件开发方法。面向对象方法的形成同结构化方法一样，起源于实现语言，首先对面向对象的程序设计语言开展研究，随之逐渐形成面向对象分析和设计方法。面向对象方法和技术历经 40 多年的研究和发展，已经越来越成熟和完善，应用也越来越深入和广泛。

面向对象的软件开发方法在 20 世纪 60 年代后期首次提出，以 60 年代末挪威奥斯陆大学和挪威计算中心共同研制的 SIMULA 语言为标志，面向对象方法的基本要点首次在 SIMULA 语言中得到了表达和实现。后来一些著名的面向对象语言 (如 Smalltalk、C++、Java、Eiffel) 的设计者都曾从 SIMULA 得到启发。随着 20 世纪 80 年代美国加州的 Xerox 研究中心推出 Smalltalk 语言和环境，使面向对象程序设计方法得到比较完善的实现。Smalltalk-80 等一系列描述能力较强、执行效率较高的面向对象编程语言的出现，标志着面向对象的方法与技术开始走向实用。

面向对象方法的本质，就是主张从客观世界固有的事物出发来构造系统，提倡用人类在现实生活中常用的思维方法来认识、理解和描述客观事物，强调最终建立的系统能够映射问题域，也就是说，系统中的对象以及对象之间的关系能够如实地反映问题域中的固有事物及其关系。

面向对象方法之所以日益受到人们的重视和应用，成为流行的软件开发方法，是源于面向对象方法的以下主要优点。

1. 与人类习惯的思维方法一致

传统的程序设计技术是面向过程的，其核心思想是以算法为核心，把数据和过程作为相互独立的部分，数据代表问题空间中的客体，程序则用于处理这些数据，在计算机内部数据和程序是分开存放的，这样的做法往往会发生使用错误的数据调用正确的程序模块的情况。其原因是，传统的程序设计方法忽略了数据和操作之间的内在联系，用这种方法设计出来的软件系统其解空间与问题空间不一致，使人感到难以理解。实际上，用计算机解决的问题都是现实世界中的问题，这些问题无非由一些相互间存在一定联系的事物所组成，每个具体的事物都具有行为和属性两方面的特征。因此，把描述事物静态属性的数据结构和表示事物动态行为的操作放在一起构

成一个整体,才能完整、自然地表示客观世界中的实体。

面向对象方法和技术以对象为核心。对象是由数据和容许的操作组成的封装体,与客观实体有直接的对应关系。对象之间通过传递消息互相联系,以模拟现实世界中不同事物彼此之间的联系。

面向对象的设计方法与传统的面向过程的方法有本质不同,这种方法的基本原理是,使用现实世界的概念抽象地思考问题从而自然地解决问题。它强调模拟现实世界中的概念而不强调算法,它鼓励开发者在软件开发的绝大部分过程中都用应用领域的概念去思考。

2. 稳定性好

面向对象方法基于构造问题领域的对象模型,以对象为中心构造软件系统。它的基本做法是用对象模拟问题领域中的实体,以对象间的联系刻画实体间的联系。因为面向对象的软件系统的结构是根据问题领域的模型建立起来的,而不是基于对系统应完成的功能的分解,所以,当对系统的功能需求变化时并不会引起软件结构的整体变化,往往仅需要做一些局部性的修改。由于现实世界中的实体是相对稳定的,因此,以对象为中心构造的软件系统也是比较稳定的。而传统的软件开发方法以算法为核心,开发过程基于功能分析和功能分解。用传统方法所建立起来的软件系统的结构紧密地依赖于系统所要完成的功能,当功能需求发生变化时将引起软件结构的整体修改。事实上,用户需求变化大部分是针对功能的,因此,这样的软件系统是不稳定的。

3. 可重用性好

软件重用是指在不同的软件开发过程中重复使用相同或相似软件元素的过程。重用是提高软件生产率的最主要的方法。

传统的软件重用技术是利用标准函数库,也就是试图用标准函数库中的函数作为“预制件”来建造新的软件系统。但是,标准函数缺乏必要的“柔性”,不能适应不同应用场合的不同需要,并不是理想的可重用的软件成分。实际的库函数往往仅提供最基本、最常用的功能,在开发一个新的软件系统时,通常多数函数是开发者自己编写的,甚至绝大多数函数都是新编的。

使用传统方法学开发软件时,人们强调的是功能抽象,认为具有功能内聚性的模块是理想的模块,也就是说,如果一个模块完成一个且只完成一个相对独立的子功能,那么这个模块就是理想的可重用模块,而且这样的模块也更容易维护。基于这种认识,通常尽量把标准函数库中的函数做成功能内聚的。但是,事实上具有功能内聚性的模块并不是自含的和独立的,相反,它必须在数据上运行。如果要重用这样的模块,则相应的数据也必须重用。如果新产品中的数据与最初产品中的数据不同,则要么修改数据要么修改这个模块。

事实上,离开了操作数据便无法处理,而脱离了数据的操作也是毫无意义的,我们应该对数据和操作同样重视。在面向对象方法中所使用的对象,其数据和操作是作为平等伙伴出现的。因此,对象具有很强的自含性,此外,对象所固有的封装性,使得对象的内部实现与外界隔离,具有较强的独立性。由此可见,对象提供了比较理想的模块化机制和比较理想的可重用的软件成分。

面向对象的软件开发技术在利用可重用的软件成分构造新的软件系统时,有很大的灵活性。有两种方法可以重复使用一个对象类:一种方法是创建该类的实例,从而直接使用它;另一种方法是从它派生出一个满足当前需要的新类。继承性机制使得子类不仅可以重用其父类的数据结构和程序代码,而且可以在父类代码的基础上方便地修改和扩充,这种修改并不影响对原有类的使用。可见,面向对象的软件开发技术所实现的可重用性是自然的和准确的。



4. 易于开发大型软件产品

当开发大型软件产品时,组织开发人员的方法不恰当往往是出现问题的主要原因。用面向对象范型开发软件时,可以把一个大型产品看作是一系列本质上相互独立的小产品来处理,这就不仅降低了开发的技术难度,而且也使得对开发工作的管理变得容易。这就是为什么对于大型软件产品来说,面向对象范型优于结构化范型的原因之一。许多软件开发公司的经验都表明,当把面向对象技术用于大型软件开发时,软件成本明显地降低了,软件的整体质量也提高了。

5. 可维护性好

用传统的开发方法和面向过程的方法开发出来的软件很难维护,是长期困扰人们的一个严重问题,是软件危机的突出表现。

由于下述因素的存在,使得用面向对象的方法开发的软件可维护性好。

(1) 用面向对象的方法开发的软件稳定性比较好

如前所述,当对软件的功能或性能的要求发生变化时,通常不会引起软件的整体变化,往往只需对局部做一些修改。由于对软件的改动较小且限于局部,自然比较容易实现。

(2) 用面向对象的方法开发的软件比较容易修改

在面向对象方法中,核心是类(对象),它具有理想的模块机制,独立性好,修改一个类通常很少会牵扯到其他类。如果仅修改一个类的内部实现部分(私有数据成员或成员函数的算法),而不修改该类的对外接口,则可以完全不影响软件的其他部分。

面向对象技术特有的继承机制,使得对所开发的软件的修改和扩充比较容易实现,通常只需从已有类派生出一些新类,无须修改软件原有成分。

面向对象技术的多态性机制,使得当扩充软件功能时对原有代码的修改进一步减少,需要增加的新代码也比较少。

(3) 用面向对象的方法开发的软件比较容易理解

在维护已有软件的时候,首先需要对原有软件与此次修改有关的部分有深入理解,才能正确地完成维护工作。传统软件之所以难以维护,在很大程度上是因为修改所涉及的部分分散在软件各个地方,需要了解的面很广,内容很多,而且传统软件的解空间与问题空间的结构很不一致,更增加了理解原有软件的难度和工作量。

面向对象的技术符合人们习惯的思维方式,用这种方法所建立的软件系统的结构与问题空间的结构基本一致。因此,面向对象的软件系统比较容易理解。

对面向对象软件系统进行修改和扩充,通常是通过在原有类的基础上派生出一些新类来实现。由于对象类有很强的独立性,当派生新类的时候通常不需要详细了解基类中操作的实现算法。因此,了解原有系统的工作量可以大幅度降低。

(4) 易于测试和调试

为了保证软件质量,对软件进行维护之后必须进行必要的测试,以确保要求修改或扩充的功能已正确地实现了,而且没有影响到软件未修改的部分。如果测试过程中发现了错误,还必须通过调试改正过来。显然,软件是否易于测试和调试,是影响软件可维护性的一个重要因素。

对用面向对象的方法开发的软件进行维护,往往是通过从已有类派生出一些新类来实现。因此,维护后的测试和调试工作也主要围绕这些新派生出来的类进行。类是独立性很强的模块,向类的实例发消息即可运行它,观察它是否能正确地地完成相应的工作,因此对类的测试通常比较

容易实现。

3.3.2 面向对象方法的基本概念

关于面向对象方法,对其概念有许多不同的看法和定义,但是都涵盖对象及对象属性与方法、类、继承、多态性几个基本要素。下面分别介绍面向对象方法中这几个重要的基本概念,这些概念是理解和使用面向对象方法的基础和关键。

1. 对象(object)

对象是面向对象方法中最基本的概念。对象可以用来表示客观世界中的任何实体,也就是说,应用领域中有意义的、与所要解决的问题有关系的任何事物都可以作为对象,它既可以是具体的物理实体的抽象,也可以是人为的概念,或者是任何有明确边界和意义的东西。例如,一个人、一家公司、一个窗口、贷款和借款等,都可以作为一个对象。总之,对象是对问题域中某个实体的抽象,设立某个对象就反映了软件系统保存有关它的信息并具有与它进行交互的能力。

面向对象的程序设计方法中涉及的对象是系统中用来描述客观事物的一个实体,是构成系统的一个基本单位,它由一组表示其静态特征的属性和它可执行的一组操作组成。

例如,一辆汽车是一个对象,它包含了汽车的属性(如颜色、型号、载重量等)及其操作(如启动、制动等)。一个窗口是一个对象,它包含了窗口的属性(如大小、颜色、位置等)及其操作(如打开、关闭等)。

客观世界中的实体通常都既具有静态的属性,又具有动态的行为,因此,面向对象方法学中的对象是由描述该对象属性的数据以及可以对这些数据施加的所有操作封装在一起构成的统一体。对象可以做的操作表示它的动态行为,在面向对象分析和面向对象设计中,通常把对象的操作也称为方法或服务。

属性即对象所包含的信息,它在设计对象时确定,一般只能通过执行对象的操作来改变。如对象 Person 的属性有姓名、年龄、体重等。不同对象的同一属性可以具有相同或不同的属性值。如张三的年龄为 19,李四的年龄为 20。张三、李四是两个不同的对象,他们共同的属性“年龄”的值不同。要注意的是,属性值应该指的是纯粹的数据值,而不能指对象。

操作描述了对象执行的功能,若通过消息传递,还可以为其他对象使用。操作的过程对外是封闭的,即用户只能看到这一操作实施后的结果。这相当于事先已经设计好的各种过程,只需要调用就可以了,用户不必去关心这一过程是如何编写的。事实上,这个过程已经封装在对象中,用户也看不到。对象的这一特性,即是对象的封装性。

对象有如下一些基本特点:

- ① 标识唯一性。指对象是可区分的,并且由对象的内在本质来区分,而不是通过描述来区分。
- ② 分类性。指可以将具有相同属性和操作的对象抽象成类。
- ③ 多态性。指同一个操作可以是不同对象的行为。
- ④ 封装性。从外面看只能看到对象的外部特性,即只需知道数据的取值范围和可以对该数据施加的操作,根本无须知道数据的具体结构以及实现操作的算法。对象的内部,即处理能力的实行和内部状态,对外是不可见的。从外面不能直接使用对象的处理能力,也不能直接修改其内部状态,对象的内部状态只能由其自身改变。



⑤ 模块独立性好。对象是面向对象的软件的基本模块,它是由数据及可以对这些数据施加的操作所组成的统一体,而且对象是以数据为中心的,操作围绕对其数据所需做的处理来设置,没有无关的操作。从模块的独立性考虑,对象内部各种元素彼此结合得很紧密,内聚性强。

2. 类(class)和实例(instance)

将属性、操作相似的对象归为类,也就是说,类是具有共同属性、共同方法的对象的集合。所以,类是对象的抽象,它描述了属于该对象类型的所有对象的性质,而一个对象则是其对应类的一个实例。

要注意的是,当使用“对象”这个术语时,既可以指一个具体的对象,也可以泛指一般的对象,但是,当使用“实例”这个术语时,必然是指一个具体的对象。

例如,Integer 是一个整数类,它描述了所有整数的性质。因此任何整数都是整数类的对象,而一个具体的整数“123”是类 Integer 的一个实例。

由类的定义可知,类是关于对象性质的描述,它同对象一样,包括一组数据属性和在数据上的一组合法操作。例如,一个面向对象的图形程序在屏幕左下角显示一个半径 3 cm 的红颜色的圆,在屏幕中部显示一个半径 4 cm 的绿颜色的圆,在屏幕右上角显示一个半径 1 cm 的黄颜色的圆。这三个圆心位置、半径大小和颜色均不相同的圆,是三个不同的对象。但是,它们都有相同的属性(圆心坐标、半径、颜色)和相同的操作(显示自己、放大缩小半径、在屏幕上移动位置,等等)。因此,它们是同一类事物,可以用“Circle 类”来定义。

3. 消息(message)

面向对象的世界是通过对象与对象间彼此的相互合作来推动的,对象间的这种相互合作需要一个机制协助进行,这样的机制称为“消息”。消息是一个实例与另一个实例之间传递的信息,它请求对象执行某一处理或回答某一要求的信息,它统一了数据流和控制流。消息的使用类似于函数调用,消息中指定了某一个实例,一个操作名和一个参数表(可空)。接收消息的实例执行消息中指定的操作,并将形式参数与参数表中相应的值结合起来。消息传递过程中,由发送消息的对象(发送对象)的触发操作产生输出结果,作为消息传送到接收消息的对象(接收对象),引发接收消息的对象一系列的操作。所传送的消息实质上是接收对象所具有的操作/方法名称,有时还包括相应参数,图 3.5 表示了消息传递的概念。



图 3.5 消息传递示意图

消息中只包含传递者的要求,它告诉接收者需要做哪些处理,但并不指示接收者应该怎样完成这些处理。消息完全由接收者解释,接收者独立决定采用什么方式完成所需的处理,发送者对接收者不起任何控制作用。一个对象能够接收不同形式、不同内容的多个消息;相同形式的消息可以送往不同的对象,不同的对象对于形式相同的消息可以有不同的解释,能够做出不同的反应。一个对象可以同时往多个对象传递信息,两个对象也可以同时向某个对象传递消息。

例如,一个汽车对象具有“行驶”这项操作,那么要让汽车以时速 50 公里行驶的话,需传递给汽车对象“行驶”及“时速 50 公里”的消息。

通常,一个消息由下述三部分组成:

- ① 接收消息的对象的名称;
- ② 消息标识符(也称为消息名);
- ③ 零个或多个参数。

例如,MyCircle 是一个半径 4cm、圆心位于(100,200)的 Circle 类的对象,也就是 Circle 类的一个实例,当要求它以绿颜色在屏幕上显示自己时,在 C++语言中应该向它发下列消息:

MyCircle.Show(GREEN);

其中,MyCircle 是接收消息的对象的名字,Show 是消息名,Green 是消息的参数。

4. 继承 (inheritance)

继承是面向对象的方法的一个主要特征。继承是使用已有的类定义作为基础建立新类的定义技术。已有的类可当作基类来引用,则新类相应地可当作派生类来引用。

广义地说,继承是指能够直接获得已有的性质和特征,而不必重复定义它们。

面向对象软件技术的许多强有力的功能和突出的优点,都来源于把类组成一个层次结构的系统:一个类的上层可以有父类,下层可以有子类。这种层次结构系统的一个重要性质是继承性,一个类直接继承其父类的描述(数据和操作)或特性,子类自动地共享基类中定义的数据和方法。

为了更深入、具体地理解继承性的含义,图 3.6 示出了实现继承机制的原理。



图 3.6 实现继承机制的原理图

图 3.6 中以 A、B 两个类为例,其中类 B 是从类 A 派生出来的子类,它除了具有自己定义的特性(数据和操作)之外,还从父类 A 继承特性。当创建类 A 的实例 a1 的时候,a1 以类 A 为样板建立实例变量。

当创建类 B 的实例 b1 的时候,b1 既要类 B 为样板建立实例变量,又要以类 A 为样板建立实例变量,b1 所能执行的操作既有类 B 中定义的方法,又有类 A 中定义的方法,这就是继承。

继承具有传递性,如果类 C 继承类 B,类 B 继承类 A,则类 C 继承类 A。因此,一个类实际上继承了它上层的全部基类的特性,也就是说,属于某类的对象除了具有该类所定义的特性外,还具有该类上层全部基类定义的特性。

继承分为单继承与多重继承。单继承是指,一个类只允许有一个父类,即类等级为树形结构。多重继承是指,一个类允许有多个父类。多重继承的类可以组合多个父类的性质构成所需



要的性质。因此,功能更强,使用更方便;但是,使用多重继承时要注意避免二义性。继承性的优点是,相似的对象可以共享程序代码和数据结构,从而大大减少了程序中的冗余信息,提高软件的可重用性,便于软件修改维护。另外,继承性使得用户在开发新的应用系统时不必完全从零开始,可以继承原有的相似系统的功能或者从类库中选取需要的类,再派生出新的类以实现所需要的功能。

5. 多态性 (polymorphism)

对象根据所接收的消息而做出动作,同样的消息被不同的对象接收时可导致完全不同的行动,该现象称为多态性。在面向对象的软件技术中,多态性是指子类对象可以像父类对象那样使用,同样的消息既可以发送给父类对象也可以发送给子类对象。

例如,在两个类 Male(男性)和 Female(女性)都有一项属性为 Friend。一个人的朋友必须属于类 Male 或 Female,这是一个多态性的情况。因为, Friend 指向两个类之一的实例。如果 Tom 的朋友或者是 Mary 或者是 John,类 Male 就不知道 Friend 应该与哪个类关联。这里参照量 Friend 必须是多态的,多态意味着可以关联不同的实例,而实例可以属于不同的类。

多态性机制不仅增加了面向对象软件系统的灵活性,进一步减少了信息冗余,而且显著地提高了软件的可重用性和可扩充性。当扩充系统功能增加新的实体类型时,只需派生出与新实体类相应的新的子类,完全无须修改原有的程序代码,甚至不需要重新编译原有的程序。利用多态性,用户能够发送一般形式的消息,而将所有的实现细节都留给接收消息的对象。

习题 3

- 结构化程序设计主要强调的是
 - 程序的规模
 - 程序的易读性
 - 程序的执行效率
 - 程序的可移植性
- 对建立良好的程序设计风格,下面描述正确的是
 - 程序应简单、清晰、可读性好
 - 符号名的命名只要符合语法
 - 充分考虑程序的执行效率
 - 程序的注释可有可无
- 在面向对象方法中,一个对象请求另一对象为其服务的方式是通过发送
 - 调用语句
 - 命令
 - 口令
 - 消息
- 信息隐蔽的概念与下述哪一种概念直接相关
 - 软件结构定义
 - 模块独立性
 - 模块类型划分
 - 模块耦合度
- 下面对对象概念描述错误的是
 - 任何对象都必须有继承性
 - 对象是属性和方法的封装体
 - 对象间的通信靠消息传递
 - 操作是对象的动态属性
- 将十进制整数设为整数类 I,则下面属于类 I 的实例的是
 - 0x1f
 - 0.51
 - 51
 - 518E-2

4.1 软件工程基本概念

4.1.1 软件定义与软件特点

计算机软件(software)是计算机系统中与硬件相互依存的另一部分,是包括程序、数据及相关文档的完整集合。其中,程序是软件开发人员根据用户需求开发的、用程序设计语言描述的、适合计算机执行的指令(语句)序列。数据是使程序能正常操纵信息的数据结构。文档是与程序开发、维护和使用有关的图文资料。可见软件由两部分组成:一是机器可执行的程序和数据;二是机器不可执行的,与软件开发、运行、维护、使用等有关的文档。

GB/T11457—2006《信息技术 软件术语》中对软件的定义是:软件是与计算机系统的操作有关的计算机程序、规程及可能的相关文档的完整集合。定义中所指,计算机程序是计算机程序和数据定义的组合;规程是为执行给定的任务应采取的一系列动作的描述;文档概念与前面描述相同。

软件在开发、生产、维护和使用等方面与计算机硬件相比存在明显的差异。深入理解软件的定义需要了解软件的特点:

① 软件是一种逻辑实体,而不是物理实体,具有抽象性。软件的这个特点使它与其他工程对象有着明显的差异。人们可以把它记录在纸上或存储介质上,但却无法看到软件本身的形态,必须通过观察、分析、思考、判断,才能了解它的功能、性能等特性。

② 软件的生产与硬件不同,它没有明显的制作过程。一旦研制开发成功,可以大量拷贝同一内容的副本。所以对软件的质量控制,必须着重在软件开发方面下功夫。

③ 软件在运行、使用期间不存在磨损、老化问题。软件虽然在生存周期后期不会因为磨损而老化,但为了适应硬件、环境以及需求的变化要进行修改,而这些修改又会不可避免地引入错误,导致软件失效率升高,从而使得软件退化。

④ 软件的开发、运行对计算机系统具有依赖性,受计算机系统的限制,这导致了软件移植的问题。



⑤ 软件复杂性高,成本昂贵。软件是人类有史以来生产的复杂度最高的工业产品。软件涉及人类社会的各行各业、方方面面,软件开发常常涉及其他领域的专门知识。软件开发需要投入大量、高强度的脑力劳动,成本高,风险大。

⑥ 软件开发涉及诸多的社会因素。许多软件的开发和运行涉及软件用户的机构设置,体制问题以及管理方式等,甚至涉及人们的观念和心理,软件知识产权及法律等问题。

软件根据应用目标的不同,分类是多种多样的。软件按功能可以分为:应用软件、系统软件、支撑软件(或工具软件)。应用软件是为解决特定领域的应用而开发的软件。例如,事务处理软件,工程与科学计算软件,实时处理软件,嵌入式软件,人工智能软件等应用性质不同的各种软件。系统软件是计算机管理自身资源,提高计算机使用效率并服务于其他程序的软件。如操作系统,编译程序,汇编程序,网络软件,数据库管理系统等。支撑软件是介于系统软件和应用软件之间,协助用户开发软件的工具性软件,包括辅助和支持开发和维护应用软件的软件,如需求分析工具软件,设计工具软件,编码工具软件,测试工具软件,维护工具软件等,也包括辅助管理人员控制开发进程和项目管理的工具软件,如计划进度管理工具软件,过程控制工具软件,质量管理及配置管理工具软件等。

4.1.2 软件危机与软件工程

软件工程概念的出现源自软件危机。

20世纪60年代末以后,“软件危机”这个词频繁出现。所谓软件危机是泛指在计算机软件的开发和维护过程中所遇到的一系列严重问题。实际上,几乎所有的软件都不同程度地存在这些问题。

随着计算机技术的发展和应用领域的扩大,计算机硬件性能/价格比和质量稳步提高,软件规模越来越大,复杂程度不断增加,软件成本逐年上升,质量没有可靠的保证,软件已成为计算机科学发展的“瓶颈”。

具体地说,在软件开发和维护过程中,软件危机主要表现在:

① 软件需求的增长得不到满足。用户对系统不满意的情况经常发生。

② 软件开发成本和进度无法控制。开发成本超出预算,开发周期大大超过规定日期的情况经常发生。

③ 软件质量难以保证。

④ 软件不可维护或维护程度非常低。

⑤ 软件的成本不断提高。

⑥ 软件开发生产率的提高赶不上硬件的发展和应用需求的增长。

总之,可以将软件危机归结为成本、质量、生产率等问题。

分析带来软件危机的原因,宏观方面是由于软件日益深入社会生活的各个层面,对软件需求的增长速度大大超过了技术进步所能带来的软件生产率的提高。而就每一项具体的工程任务来看,许多困难来源于软件工程所面临的任务和其他工程之间的差异以及软件和其他工业产品的不同。

在软件开发和维护过程中,之所以存在这些严重的问题,一方面与软件本身的特点有关,例如,在软件运行前,软件开发过程的进展难以衡量,质量难以评价,因此管理和控制软件开发过程相

当困难;在软件运行过程中,软件维护意味着改正或修改原来的设计;另外,软件的显著特点是规模庞大,复杂度超线性增长,在开发大型软件时,要保证高质量,极端复杂困难,不仅涉及技术问题(如分析方法、设计方法、版本控制),更重要的是必须有严格而科学的管理。另一方面与软件开发和维护方法不正确有关,这是主要原因。

为了消除软件危机,通过认真研究解决软件危机的方法,认识到软件工程是使计算机软件走向工程科学的途径,逐步形成了软件工程的观念,开辟了工程学的新兴领域——软件工程学。软件工程就是试图用工程、科学和数学的原理与方法研制、维护计算机软件的有关技术及管理方法。

关于软件工程的定义,在 GB/T11457—2006《信息技术 软件工程术语》中将其定义为“应用计算机科学理论和技术以及工程管理原则和方法,按预算和进度,实现满足用户要求的软件产品的定义、开发、发布和维护的工程或进行研究的学科”。

1968 年在北大西洋公约组织会议(NATO 会议)上,讨论摆脱软件危机的办法,软件工程(Software Engineering)作为一个概念首次被提出,这在软件技术发展史上是一件大事。其后的几十年里,各种有关软件工程的技术、思想、方法和概念不断地被提出,软件工程逐步发展成为一门独立的科学。在会议上,德国人 Fritz Bauer 认为:“软件工程是建立并使用完善的工程化原则,以较经济的手段获得能在实际机器上有效运行的可靠软件的一系列方法”。

1993 年,IEEE(Institute of Electrical & Electronic Engineers,电气和电子工程师学会)给出了一个更加综合的定义:“将系统化的、规范的、可度量的方法应用于软件的开发、运行和维护的过程,即将工程化应用于软件中”。此后,IEEE 多次给出软件工程的定义。

这些软件工程的定义都是强调在软件开发过程中需要应用工程化原则。

软件工程包括 3 个要素,即方法、工具和过程。方法是完成软件工程项目的手段;工具支持软件的开发、管理、文档生成;过程支持软件开发的各个环节的控制、管理。

软件工程的进步和学科的发展是近几十年软件产业迅速发展的重要原动力。从根本上来讲,其目的是研究软件的开发技术,软件工程的名称意味着用工业化的开发方法来替代小作坊式的开发模式。但是,几十年的软件开发和软件发展的实践证明,软件开发是既不同于其他工业工程,也不同于科学研究的一组活动。软件不是自然界的有形物体,它作为人类智慧的产物有其本身的特点,所以软件工程的方法、概念、目标等都在发展,有的与最初的想法有了一定的差距。但是认识和学习过去和现在的发展演变,真正掌握软件开发技术的成就,并为进一步发展软件开发技术,以适应时代对软件的更高期望是有极大意义的。

软件工程的核心理念是把软件产品(就像其他工业产品一样)看作是一个工程产品来处理。把需求计划、可行性研究、工程审核、质量监督等工程化的概念引入到软件生产当中,以期达到工程项目的三个基本要素:进度、经费和质量的目标。同时,软件工程也注重研究不同于其他工业产品生产的一些独特特性,并针对软件的特点提出了许多有别于一般工业工程技术的一些技术方法。代表性的有结构化的方法、面向对象方法和软件开发模型及软件开发过程等。

特别地,从经济学的意义上来说,考虑到软件庞大的维护费用远比软件开发费用要高,因而开发软件不能只考虑开发期间的费用,而且应考虑软件生命周期内的全部费用。因此,软件生命周期的概念就变得特别重要。在考虑软件费用时,不仅仅要降低开发成本,更要降低整个软件生命周期的总成本。



4.1.3 软件过程与软件生命周期

1. 软件过程

ISO 9000 定义:软件过程是把输入转化为输出的一组彼此相关的资源和活动。

定义给出了软件过程的两方面内涵。

其一,软件过程是指为获得软件产品,在软件工具支持下由软件工程师完成的一系列软件工程活动。软件过程通常包含4种基本活动:

- ① P(plan)——软件规格说明。规定软件的功能及其运行时的限制。
- ② D(do)——软件开发或软件设计与实现。生产满足规格说明的软件。
- ③ C(check)——软件确认。确认软件能够满足客户提出的要求。
- ④ A(action)——软件演进。为满足客户的变更要求,软件必须在使用进的过程中演进。

事实上,软件过程是一个软件开发机构针对某类软件产品为自己规定的工作步骤,它应当是科学的、合理的,否则必将影响软件产品的质量。

通常把用户的要求转变成软件产品的过程也叫作软件开发过程。此过程包括对用户的要求进行分析,解释成软件需求,把需求变换成设计,把设计用代码来实现并进行代码测试,有些软件还需要进行代码安装和交付运行。

其二,从软件开发的观点看,它就是使用适当的资源(包括人员、计算机硬件、软件工具、时间等),为开发软件进行的一组开发活动,在过程结束时将输入(用户要求)转化为输出(软件产品)。

所以,软件过程是将软件工程的方法和工具综合起来,以达到合理、及时地进行计算机软件开发的目的。软件过程应确定方法使用的顺序、要求交付的文档资料、为保证质量和适应变化所需要的管理、软件开发各个阶段完成的任务。

2. 软件生命周期 (software life cycle)

通常,将软件产品从提出、实现、使用维护到停止使用退役的过程称为软件生命周期。也就是说,软件产品从考虑其概念开始,到该软件产品不能使用为止的整个时期都属于软件生命周期。一般包括可行性研究与需求分析、设计、编码、测试、交付使用以及维护等活动,如图4.1所示。这些活动可以有重复,执行时也可以有迭代。

还可以将软件生命周期分为如图4.1所示的软件定义、软件开发及软件运行维护三个阶段。

软件定义阶段的任务是:确定软件开发工作必须完成的目标;确定工程的可行性。

软件开发阶段的任务是:具体完成设计和实现定义阶段所定义的软件,通常包括总体设计、详细设计、编码和测试。其中总体设计和详细设计又称为系统设计,编码和测试又称为系统实现。

软件维护阶段的任务是:使软件在运行中持久地满足用户的



图 4.1 软件生命周期

需要。具体地说,当软件在使用过程中发现错误时应加以改正,当环境改变时应修改软件以适应新的环境,当用户有新的需求时应及时改进软件以满足用户需求的变更。

下面介绍图 4.1 所示的软件生命周期中各阶段的基本任务。

① 可行性研究与计划制定。确定待开发软件系统的开发目标和总的要求,给出它的功能、性能、可靠性以及接口等方面的可能方案,制定完成开发任务的实施计划。

② 需求分析。对待开发软件提出的需求进行分析并给出详细定义。编写软件规格说明书及初步的用户手册,提交评审。

③ 软件设计。系统设计人员和程序设计人员应该在反复理解软件需求的基础上,给出软件的结构、模块的划分、功能的分配以及处理流程。在系统比较复杂的情况下,设计阶段可分解成概要设计阶段和详细设计阶段。编写概要设计说明书、详细设计说明书和测试计划初稿,提交评审。

④ 编码。编码是把软件设计转换成计算机可以接受的程序代码。即完成源程序的编码,编写用户手册、操作手册等面向用户的文档,编写单元测试计划。

⑤ 软件测试。在设计测试用例的基础上,检验软件的各个组成部分。编写测试分析报告。

⑥ 运行和维护。将已交付的软件投入运行,并在运行使用中不断地维护,根据新提出的需求进行必要而且可能的扩充和删改。

4.1.4 软件工程的目标与原则

1. 软件工程的目标

软件工程的目标是,在给定成本、进度的前提下,开发出具有有效性、可靠性、可理解性、可维护性、可重用性、可适应性、可移植性、可追踪性和可操作性且满足用户需求的产品。

软件工程需要达到的基本目标应是:付出较低的开发成本;达到要求的软件功能;取得较好的软件性能;开发的软件易于移植;需要较低的维护费用;能按时完成开发,及时交付使用。

基于软件工程的目标,软件工程的理论和技术性研究的内容主要包括:软件开发技术和软件工程管理。

(1) 软件开发技术

软件开发技术包括:软件开发方法学、开发过程、开发工具和软件工程环境,其主体内容是软件开发方法学。软件开发方法学是根据不同的软件类型,按不同的观点和原则,对软件开发中应遵循的策略、原则、步骤和必须产生的文档资料都作出规定,从而使软件的开发能够进入规范化和工程化的阶段,以克服早期的手工方法生产中的随意性和非规范性做法。

(2) 软件工程管理

软件工程管理包括:软件管理学、软件工程经济学、软件心理学等内容。

软件工程管理是软件按工程化生产时的重要环节,它要求按照预先制定的计划、进度和预算执行,以实现预期的经济效益和社会效益。统计数据表明,多数软件开发项目的失败,并不是由于软件开发技术方面的原因,它们的失败是由于不适当的管理造成的。因此人们对软件项目管理重要性的认识有待提高。软件管理学包括人员组织、进度安排、质量保证、配置管理、项目计划等。

软件工程经济学是研究软件开发中成本的估算、成本效益分析的方法和技术,用经济学的基

本原理来研究软件工程开发中的经济效益问题。

软件心理学是软件工程领域具有挑战性的一个全新的研究视角,它是从个体心理、人类行为、组织行为和企业文化等角度来研究软件管理和软件工程的。

2. 软件工程的原则

为了达到上述的软件工程目标,在软件开发过程中,必须遵循软件工程的基本原则。这些原则适用于所有的软件项目。这些基本原则包括抽象、信息隐蔽、模块化、局部化、确定性、一致性、完备性和可验证性。

① 抽象。抽取事物最基本的特性和行为,忽略非本质细节。采用分层次抽象,自顶向下,逐层细化的办法控制软件开发过程的复杂性。

② 信息隐蔽。采用封装技术,将程序模块的实现细节隐藏起来,使模块接口尽量简单。

③ 模块化。模块是程序中相对独立的成分,一个独立的编程单位,应有良好的接口定义。模块的大小要适中,模块过大会使模块内部的复杂性增加,不利于对模块的理解和修改,也不利于模块的调试和重用。模块太小会导致整个系统表示过于复杂,不利于控制系统的复杂性。

④ 局部化。要求在一个物理模块内集中逻辑上相互关联的计算资源,保证模块间具有松散的耦合关系,模块内部有较强的内聚性,这有助于控制解的复杂性。

⑤ 确定性。软件开发过程中所有概念的表达应是确定的、无歧义且规范的。这有助于人与人的交互不会产生误解和遗漏,以保证整个开发工作的协调一致。

⑥ 一致性。包括程序、数据和文档的整个软件系统的各模块应使用已知的概念、符号和术语;程序内外部接口应保持一致,系统规格说明与系统行为应保持一致。

⑦ 完备性。软件系统不丢失任何重要成分,完全实现系统所需的功能。

⑧ 可验证性。开发大型软件系统需要对系统自顶向下,逐层分解。系统分解应遵循容易检查、测评、评审的原则,以确保系统的正确性。

4.1.5 软件开发工具与软件开发环境

现代软件工程方法之所以得以实施,其重要的保证是软件开发工具和环境的保证,使软件在开发效率、工程质量等多方面得到改善。软件工程鼓励研制和采用各种先进的软件开发方法、工具和环境。工具和环境的使用进一步提高了软件的开发效率、维护效率和软件质量。

1. 软件开发工具

早期的软件开发除了一般的程序设计语言外,尚缺少工具的支持,致使编程工作量大,质量和进度难以保证,导致人们将很多的精力和时间花费在程序的编制和调试上,而在更重要的软件的需求和设计上反而得不到必要的精力和时间投入。软件开发工具的完善和发展将促进软件开发方法的进步和完善,促进软件开发的高速度和高质量。软件开发工具的发展是从单项工具的开发逐步向集成工具发展的,软件开发工具为软件工程方法提供了自动的或半自动的软件支撑环境。同时,软件开发方法的有效应用也必须得到相应工具的支持,否则方法将难以有效地实施。

2. 软件开发环境

软件开发环境(或称软件工程环境)是全面支持软件开发全过程的软件工具集合。这些软件工具按照一定的方法或模式组合起来,支持软件生命周期内的各个阶段和各项任务的完成。



计算机辅助软件工程(computer aided software engineering, CASE)是当前软件开发环境中富有特色的研究工作和发展方向。CASE 将各种软件工具、开发机器和一个存放开发过程信息的中心数据库组合起来,形成软件工程环境。CASE 的成功产品将最大限度地降低软件开发的技术难度并使软件开发的质量得到保证。

4.2 结构化分析方法

软件开发方法是软件开发过程所遵循的方法和步骤,其目的在于有效地得到一些工作产品,即程序和文档,并且满足质量要求。软件开发方法包括分析方法、设计方法和程序设计方法。

结构化方法经过 30 多年的发展,已经成为系统、成熟的软件开发方法之一。结构化方法包括已经形成了配套的结构化分析方法、结构化设计方法和结构化编程方法,其核心和基础是结构化程序设计理论。

4.2.1 需求分析与需求分析方法

1. 需求分析

软件需求是指用户对目标软件系统在功能、行为、性能、设计约束等方面的期望。需求分析的任务是发现需求、求精、建模和定义需求的过程。需求分析将创建所需的数据模型、功能模型和控制模型。

(1) 需求的定义

1997 年 IEEE 在《软件工程标准词汇表》中对需求定义如下:

- ① 用户解决问题或达到目标所需的条件或权能;
- ② 系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或权能;
- ③ 一种反映①或②所描述的条件或权能的文档说明。

该定义从两方面阐述了需求的含义:一是从用户角度所要求的系统应具有的功能,是系统的外部行为;二是从系统开发者角度所要求的系统应具有的功能,是系统的内部特性。定义还强调了需求一定要文档化。GB/T11457—2006《信息技术 软件工程术语》采用了 IEEE 的定义。

由需求的定义可知,需求及需求分析的内容包括:提炼、分析和仔细审查已收集到的需求;确保所有利益相关者都明白其含义并找出其中的错误、遗漏或其他不足的地方;从用户最初的非形式化需求到满足用户对软件产品的要求的映射;对用户意图不断进行提示和判断。

(2) 需求分析阶段的工作

需求分析阶段的工作,可以概括为四个方面:

① 需求获取。需求获取的目的是确定对目标系统的各方面需求。涉及的主要任务是建立获取用户需求的方法框架,并支持和监控需求获取的过程。

需求获取涉及的关键问题有:对问题空间的理解;人与人之间的通信;不断变化的需求。

需求获取是在同用户的交流过程中不断收集、积累用户的各种信息,并且通过认真理解用户的各项要求,澄清那些模糊的需求,排除不合理的,从而较全面地提炼系统的功能性与非功能性需求。一般功能性与非功能性需求包括系统功能、物理环境、用户界面、用户因素、资源、安全性、



质量保证及其他约束。

要特别注意的是,在需求获取过程中,容易产生诸如与用户存在交流障碍,相互误解,缺乏共同语言,理解不完整,忽视需求变化,混淆目标和需求等问题,这些问题都将直接影响到需求分析和系统后续开发的成败。

② 需求分析。对获取的需求进行分析和综合,最终给出系统的解决方案和目标系统的逻辑模型。

③ 编写需求规格说明书。需求规格说明书作为需求分析的阶段成果,可以为用户、分析人员和设计人员之间的交流提供方便,可以直接支持目标软件系统的确认,又可以作为控制软件开发进程的依据。

④ 需求评审。在需求分析的最后一步,对需求分析阶段的工作进行复审,验证需求文档的一致性、可行性、完整性和有效性。

2. 需求分析方法

常见的需求分析方法有:

① 结构化分析方法。主要包括:面向数据流的结构化分析方法(structured analysis, SA),面向数据结构的 Jackson 方法(Jackson system development method, JSD),面向数据结构的结构化数据系统开发方法(data structured system development method, DSSD)。

② 面向对象的分析方法(object-oriented analysis method, OOA)。从需求分析建立的模型的特性来分,需求分析方法又分为静态分析方法和动态分析方法。

4.2.2 结构化分析方法

1. 关于结构化分析方法

结构化分析方法是结构化程序设计理论在软件需求分析阶段的运用。它是 20 世纪 70 年代中期倡导的基于功能分解的分析方法,其目的是帮助弄清用户对软件的需求。

对于面向数据流的结构化分析方法,按照 DeMarco 的定义,“结构化分析就是使用数据流图(DFD)、数据字典(DD)、结构化英语、判定表和判定树等工具,来建立一种新的、称为结构化规格说明的目标文档。”

结构化分析方法的实质是着眼于数据流,自顶向下,逐层分解,建立系统的处理流程,以数据流图和数据字典为主要工具,建立系统的逻辑模型。

结构化分析的步骤如下:

- ① 通过对用户的调查,以软件的需求为线索,获得当前系统的具体模型;
- ② 去掉具体模型中非本质因素,抽象出当前系统的逻辑模型;
- ③ 根据计算机的特点分析当前系统与目标系统的差别,建立目标系统的逻辑模型;
- ④ 完善目标系统并补充细节,写出目标系统的软件需求规格说明;
- ⑤ 评审直到确认完全符合用户对软件的需求。

2. 结构化分析的常用工具

(1) 数据流图(data flow diagram, DFD)

数据流图是描述数据处理过程的工具,是需求理解的逻辑模型的图形表示,它直接支持系统的功能建模。

数据流图从数据传递和加工的角度,来刻画数据流从输入到输出的移动变换过程。数据流图中的主要图形元素与说明如下:

- 加工(转换)。输入数据经加工变换产生输出。
- 数据流。沿箭头方向传送数据的通道,一般在旁边标注数据流名。
- ▬存储文件(数据源)。表示处理过程中存放各种数据的文件。
- 数据的源点和终点。表示系统和环境的接口,属系统之外的实体。

一般通过对实际系统的了解和分析后,使用数据流图为系统建立逻辑模型。建立数据流图的步骤如下:

第1步:由外向里。先画系统的输入输出,然后画系统的内部。

第2步:自顶向下。顺序完成顶层、中间层、底层数据流图。

第3步:逐层分解。

数据流图的建立从顶层开始,顶层的数据流图形式如图4.2所示。顶层数据流图应该包含所有相关外部实体,以及外部实体与软件中间的数据流,其作用主要是描述软件的作用范围,对总体功能、输入、输出进行抽象描述,并反映软件和系统、环境的关系。



图 4.2 顶层数据流图

对复杂系统的表达应采用控制复杂度策略,需要按照问题的层次结构逐步分解细化,使用分层的数据流图表达这种结构关系,分层的数据流图的形式如图4.3所示。

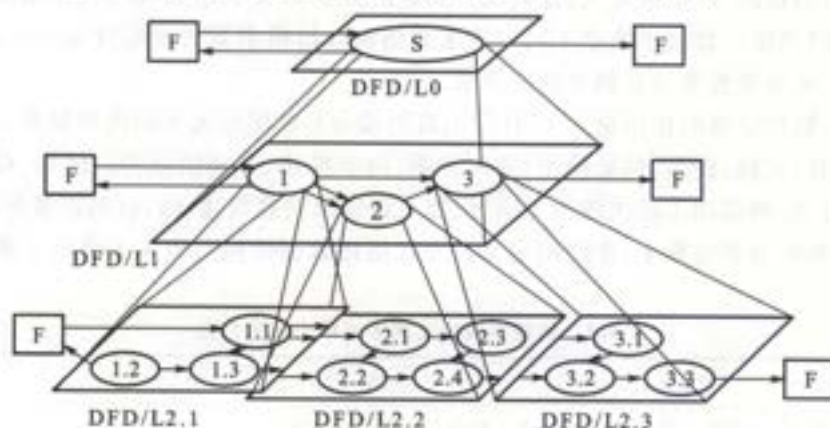


图 4.3 分层数据流图

为保证构造的数据流图表达完整、准确、规范,应遵循以下数据流图的构造规则和注意事项:



① 对加工处理建立唯一、层次性的编号,且每个加工处理通常要求既有输入又有输出。

② 数据存储之间不应该有数据流。

③ 数据流图的一致性。它包括数据守恒和数据存储文件的使用,即某个处理用以产生输出的数据没有输入,即出现遗漏,另一种是一个处理的某些输入并没有在处理中使用以产生输出;数据存储(文件)应被数据流图中的处理读和写,而不是仅读不写或仅写不读。

④ 父图、子图关系与平衡规则。相邻两层 DFD 之间具有父、子关系,子图代表了父图中某个加工的详细描述,父图表示了子图间的接口。子图个数不大于父图中的处理个数。所有子图的输入、输出数据流和父图中相应处理的输入、输出数据流必须一致。

图 4.4 是银行取款业务的数据流图。

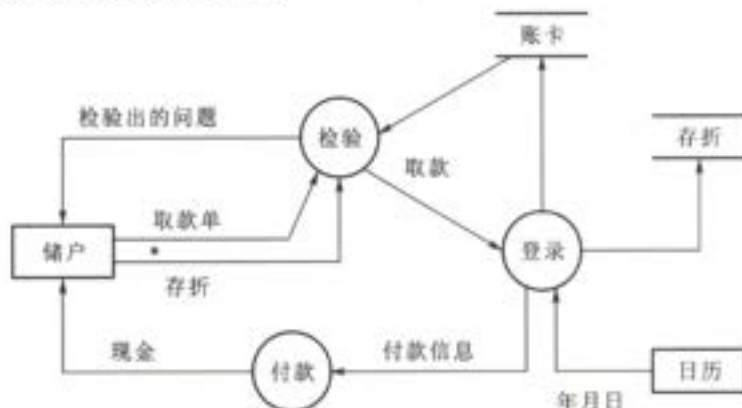


图 4.4 银行取款业务的数据流图

(2) 数据字典(data dictionary, DD)

数据字典是结构化分析方法的核心。数据字典是对所有与系统相关的数据元素的一个有组织的列表,以及精确的、严格的定义,使得用户和系统分析员对于输入、输出、存储成分和中间计算结果有共同的理解。数据字典把不同的需求文档和分析模型紧密地结合在一起,与各模型的图形表示配合,能清楚地表达数据处理的要求。

概括地说,数据字典的作用是对 DFD 中出现的被命名的图形元素的确切解释。通常数据字典包含的信息有:名称、别名、何处使用/如何使用、内容描述、补充信息等。例如,对加工的描述应包括:加工名、反映该加工层次的加工编号、加工逻辑及功能简述、输入/输出数据流等。

在数据字典的编制过程中,常使用定义式方式描述数据结构。表 4.1 给出了常用的定义式符号。

表 4.1 数据字典定义式方式中出现的符号

符号	含义
=	表示“等于”,“定义为”,“由什么构成”
[... ...]	表示“或”,即选择括号中用“ ”号分隔的各项中的某一项
+	表示“与”,“和”
n m	表示“重复”,即括号中的项要重复若干次,n,m 是重复次数的上下限

续表

符号	含义
(...)	表示“可选”，即括号中的项可以没有
**	表示“注释”
..	连接符

例如，银行取款业务的数据流图中，存储文件“存折”的 DD 定义如下：

存折 = 户名 + 所号 + 账号 + 开户日 + 性质 + (印密) + 1 | 存取行 | 50

户名 = 2 | 字母 | 24

所号 = “001”.. “999”

账号 = “00000001”.. “99999999”

开户日 = 年 + 月 + 日

性质 = “1”.. “6”

印密 = “0”

存取行 = 日期 + (摘要) + 支出 + 存入 + 余额 + 操作 + 复核

日期 = 年 + 月 + 日

年 = “00”.. “99”

月 = “01”.. “12”

日 = “01”.. “31”

摘要 = 1 | 字母 | 4

支出 = 金额

金额 = “0000000.01”.. “9999999.99”

操作 = “00001”.. “99999”

(3) 判定树

使用判定树进行描述时，应先从问题定义的文字描述中分清哪些是判定的条件，哪些是判定的结论，根据描述材料中的连接词找出判定条件之间的从属关系、并列关系、选择关系，根据它们构造判定树。

例如，某货物托运管理系统中，对发货情况的处理要依赖检查发货单，检查发货单受货物托运金额、欠款等条件的约束，可以使用类似分段函数的形式来描述这些约束和处理。对这种约束条件的描述，如果使用自然语言，表达易出现不准确和不清晰。如果使用如图 4.5 所示的判定树来描述，则简洁清晰。



图 4.5 “检查发货单”判定树



(4) 判定表

判定表与判定树相似,当数据流图中的加工要依赖于多个逻辑条件的取值,即完成该加工的一组动作是由于某一组条件取值的组合而引发的,使用判定表描述比较适宜。

判定表由四部分组成,如图4.6所示。其中标识为①的左上部称基本条件项,列出了各种可能的条件。标识为②的右上部称条件项,列出了各种可能的条件组合。标识为③的左下部称基本动作项,列出了所有的操作。标识为④的右下部称动作项,列出了在对应的条件组合下所选的操作。

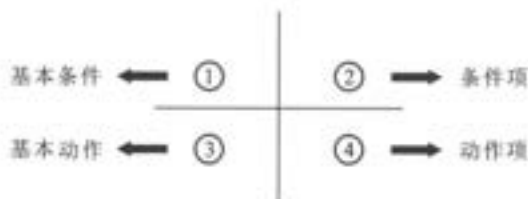


图4.6 判定表组成

图4.7为“检查发货单”判定表,其中“√”表示满足对应条件项时执行的操作。

判定表或判定树是以图形形式描述数据流图的加工逻辑,它结构简单,易读易懂。尤其遇到组合条件的判定,利用判定表或判定树可以使问题的描述清晰,而且便于直接映射到程序代码。在表达一个加工逻辑时,判定树、判定表都是好的描述工具,根据需要还可以交叉使用。

		1	2	3	4
条件	发货单金额	> \$ 500	> \$ 500	≤ \$ 500	≤ \$ 500
	拖欠情况	>60 天	≤60 天	>60 天	≤60 天
操作	不发出批准书	√			
	发出批准书		√	√	√
	发出发货单		√	√	√
	发出拖欠报告			√	

图4.7 “检查发货单”判定表

4.2.3 软件需求规格说明书

软件需求规格说明 (software requirement specification, SRS) 是描述需求中的重要文档,是软件需求分析的主要成果。GB/T9385—2008 是最新的软件需求规格说明规范。

1. 软件需求规格说明书的作用

- ① 便于用户、开发人员进行理解和交流。
- ② 反映出用户问题的结构,可以作为软件开发工作的基础和依据。
- ③ 作为确认测试和验收的依据。
- ④ 为成本估算和编制计划进度提供基础。



⑤ 软件不断改进的基础。

2. 软件需求规格说明书的内容

软件需求规格说明应重点描述软件的目标,软件的功能需求、性能需求、外部接口、属性及约束条件等。功能需求是软件需求规格说明,给出软件要执行什么功能的详尽描述。性能需求是指定量地描述软件系统应满足的具体性能需求,即各种软件功能的速度、响应时间、恢复时间。外部接口指软件如何与人、系统的硬件及其他硬件和其他软件进行交互。属性是指与软件有关的质量属性,如正确性、可用性、可靠性、安全性、可维护性等。约束条件包括影响软件实现的各种设计约束,如使用的标准、编程语言、数据库完整性方针、资源限制、运行环境等方面的要求。

3. 软件需求规格说明的特点

软件需求规格说明是确保软件质量的有力措施,衡量软件需求规格说明书质量好坏的标准、标准的优先级及标准的内涵是:

- ① 正确性。体现待开发系统的真实要求。
- ② 无歧义性。对每一个需求只有一种解释,其陈述具有唯一性。
- ③ 完整性。包括全部有意义的需求,功能的、性能的、设计的、约束的,属性或外部接口等方面的需求。
- ④ 可验证性。描述的每一个需求都是可以验证的,即存在有限代价的有效过程验证确认。
- ⑤ 一致性。各个需求的描述不矛盾。
- ⑥ 可理解性。需求说明必须简明易懂,尽量少包含计算机的概念和术语,以使用户和软件人员都能接受它。
- ⑦ 可修改性。SRS 的结构风格在需求有必要改变时是易于实现的。
- ⑧ 可追踪性。每一个需求的来源、流向是清晰的,当产生和改变文件编制时,可以方便地引证每一个需求。

软件需求规格说明是一份在软件生命周期中至关重要的文档,它在开发早期就为尚未诞生的软件系统建立了一个可见的逻辑模型,它可以保证开发工作的顺利进行,因而应及时地建立并保证它的质量。

作为设计的基础和验收的依据,软件需求规格说明应该是精确而无二义性的,需求说明书越精确,则以后出现错误、混淆、反复的可能性越小。用户能看懂需求说明书,并且发现和指出其中的错误是保证软件系统质量的关键,因而需求说明必须简明易懂,尽量少包含计算机的概念和术语,以使用户和软件人员双方都能接受它。

4.3 结构化设计方法

经过需求分析,软件系统确定了要“做什么”的目标,进而进入软件设计,来解决和确定“怎样做”的问题。



4.3.1 软件设计的基本概念

1. 软件设计的基础

软件设计是软件工程的重要阶段,是一个把软件需求转换为软件表示的过程。软件设计的基本目标是用比较抽象概括的方式确定目标系统如何完成预定的任务,即软件设计是确定系统的物理模型。

软件设计的重要地位和地位概括为以下几点:

① 软件开发阶段(设计、编码、测试)占据软件项目开发总成本绝大部分,是在软件开发中形成质量的关键环节;

② 软件设计是开发阶段最重要的步骤,是将需求准确地转化为完整的软件产品或系统的唯一途径;

③ 软件设计作出的决策,最终影响软件实现的成败;

④ 设计是软件工程和软件维护的基础。

从技术观点来看,软件设计包括软件结构设计、数据设计、接口设计、过程设计。其中,结构设计是定义软件系统各主要部件之间的关系;数据设计是将分析时创建的模型转化为数据结构的定义;接口设计是描述软件内部、软件和协作系统之间以及软件与人之间如何通信;过程设计则是把系统结构部件转换成软件的过程性描述。

从工程管理角度来看,软件设计分两步完成:概要设计和详细设计。概要设计(又称结构设计)将软件需求转化为软件体系结构、确定系统级接口、全局数据结构或数据库模式;详细设计确立每个模块的实现算法和局部数据结构,用适当方法表示算法和数据结构的细节。

软件设计的一般过程是:软件设计是一个迭代的过程;先进行高层次的结构设计;后进行低层次的过程设计;穿插进行数据设计和接口设计。

2. 软件设计的基本原理

软件设计过程中应遵循软件工程的基本原理。软件设计的基本原理和有关概念可概括为:

(1) 抽象

人类在认识复杂问题时使用的很有效的思维工具是抽象。抽象是把事物本质的共同特性提取出来而暂时忽略它们的细节及它们之间的差异。软件设计中考虑模块化解决方案时,可以定义多个抽象级别。抽象的层次从软件的总体设计到详细设计到编码逐层降低。软件工程过程的每一步都是对软件求解的上一层抽象的一次精化。

(2) 逐步求精和模块化

逐步求精和模块化概念与抽象密切相关。逐步求精是人们解决复杂问题时常采用的一种方法,即化繁为简,分而治之,将大而复杂的问题分解成许多容易解决的小问题,则原来的问题就容易解决。在软件设计中,这种逐步求精就是对模块的划分。一般将一个软件系统模块化划分后形成的结构是模块的分层结构。

(3) 信息隐蔽和局部化

信息隐蔽是指,所设计的模块使得其所包含的信息(过程和数据)对于不需要这些信息的模块是不能访问的。局部化与信息隐蔽概念密切相关。所谓局部化,是指把一些关系密切的软件元素物理地放得彼此靠近,例如在模块中使用局部数据就是如此。实际上,隐蔽的不是有关模块



的一切信息,而是模块的实现细节。局部化有助于实现信息隐蔽。

(4) 模块独立性

模块独立性的概念是抽象、模块化、信息隐蔽和局部化的直接结果。模块的独立性是指软件模块的编写和修改应使其具有独立功能,且与其他模块的关联尽可能少。换句话说讲,设计软件结构时,应使得每个模块仅完成一个相对独立的特定子功能,并且与其他模块关联最少且接口简单。

模块的独立程度是评价设计好坏的重要度量标准,模块的独立程度由两个定性标准度量,即模块间的耦合性和模块内的内聚性。

耦合性是对模块间互相连接的紧密程度的度量。一个模块的耦合性越强(越高)则该模块的独立性越弱(越低)。所以软件设计应该建立模块之间耦合程度尽可能低的系统,这样对某一模块的设计、编码、测试和维护就不需要对系统其他模块有很多的了解。

内聚性是对一个模块内部各个元素间彼此结合的紧密程度的度量。内聚性是信息隐蔽和局部化概念的扩展。一个模块的内聚性越强(越高)则该模块的独立性越强。

模块的内聚性越高,模块间的耦合性就越低,可见模块的耦合性和内聚性是相互关联的。因此,好的软件设计,应尽量做到高内聚、低耦合。

3. 结构化设计方法

与结构化需求分析方法相对应的是结构化设计方法。结构化设计就是采用最佳的可能方法设计系统的各个组成部分以及各成分之间的内部联系的技术。也就是说,结构化设计是这样一个过程,它决定用哪些方法把哪些部分联系起来,才能解决好某个具体有清楚定义的问题。

结构化设计方法的基本思想是将软件设计成由相对独立、单一功能的模块组成的结构。下面重点以面向数据流的结构化方法为例讨论结构化设计方法。

4.3.2 概要设计

1. 概要设计的任务

软件概要设计(总体设计)的基本任务是:

(1) 设计软件系统结构

在需求分析阶段,已经把系统分解成层次结构,而在概要设计阶段,需要进一步分解,划分为模块以及模块的层次结构。划分的具体过程是:

- ① 采用某种设计方法,将一个复杂的系统按功能划分成模块。
- ② 确定每个模块的功能。
- ③ 确定模块之间的调用关系。
- ④ 确定模块之间的接口,即模块之间传递的信息。
- ⑤ 评价模块结构的质量。

(2) 数据结构及数据库设计

数据设计是实现需求定义和规格说明过程中提出的数据对象的逻辑表示。数据设计的具体任务是:确定输入、输出文件的详细数据结构;结合算法设计,确定算法所必需的逻辑数据结构及其操作;确定对逻辑数据结构所必需的那些操作的程序模块,限制和确定各个数据设计决策的影



响范围;需要与操作系统或调度程序接口所必需的控制表进行数据交换时,确定其详细的数据结构和使用规则;数据的保护性设计:防卫性、一致性、冗余性设计。

数据设计中应注意掌握以下设计原则:

- ① 用于功能和行为的系统分析原则也应用于数据。
- ② 应该标识所有的数据结构以及其上的操作。
- ③ 应当建立数据字典,并用于数据设计和程序设计。
- ④ 低层的设计决策应该推迟到设计过程的后期。
- ⑤ 只有那些需要直接使用数据结构、内部数据的模块才能看到该数据的表示。
- ⑥ 应该开发一个由有用的数据结构和应用于其上的操作组成的库。
- ⑦ 软件设计和程序设计语言应该支持抽象数据类型的规格说明和实现。

(3) 编写概要设计文档

在概要设计阶段,需要编写的文档有概要设计说明书、数据库设计说明书、集成测试计划等。

(4) 概要设计文档评审

在概要设计中,对设计部分是否完整地实现了需求中规定的功能、性能等要求,设计方案的可行性,关键的处理及内外部接口定义正确性、有效性,各部分之间的一致性等都要进行评审,以免在以后的设计中出现大的问题而返工。

常用的软件结构设计工具是结构图(structure chart, SC),也称程序结构图。使用结构图描述软件系统的层次和分块结构关系,它反映了整个系统的功能实现以及模块与模块之间的联系与通信,是未来程序中的控制层次体系。

结构图是描述软件结构的图形工具。结构图的基本图符如图4.8所示。

模块用一个矩形表示,矩形内注明模块的功能和名字;箭头表示模块间的调用关系。在结构图中还可以用带注释的箭头表示模块调用过程中来回传递的信息。如果希望进一步标明传递的信息是数据还是控制信息,则可用带实心圆的箭头表示传递的是控制信息,用带空心圆的箭头表示传递的是数据。



图4.8 结构图基本图符

根据结构化设计思想,结构图构成的基本形式如图4.9所示。

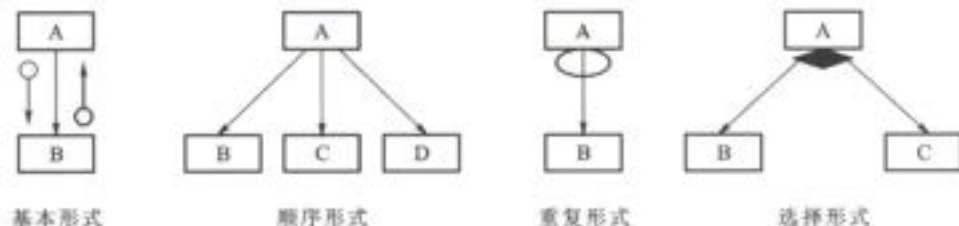


图4.9 结构图构成的基本形式

经常使用的结构图有四种模块类型:传入模块、传出模块、变换模块和协调模块。其表示形式和含义如图4.10所示。

下面通过图4.11进一步了解程序结构图的有关术语。

深度:表示控制的层数。

上级模块、从属模块：上、下两层模块 a 和 b，且有 a 调用 b，则 a 是上级模块，b 是从属模块。

宽度：整体控制跨度（最大模块数的层）的表示。

扇入：调用一个给定模块的模块个数。

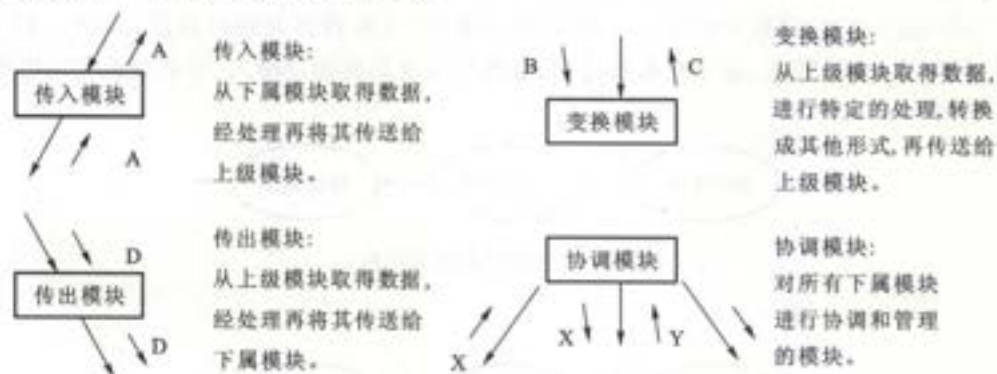


图 4.10 传入模块、传出模块、变换模块和协调模块的表示形式和含义

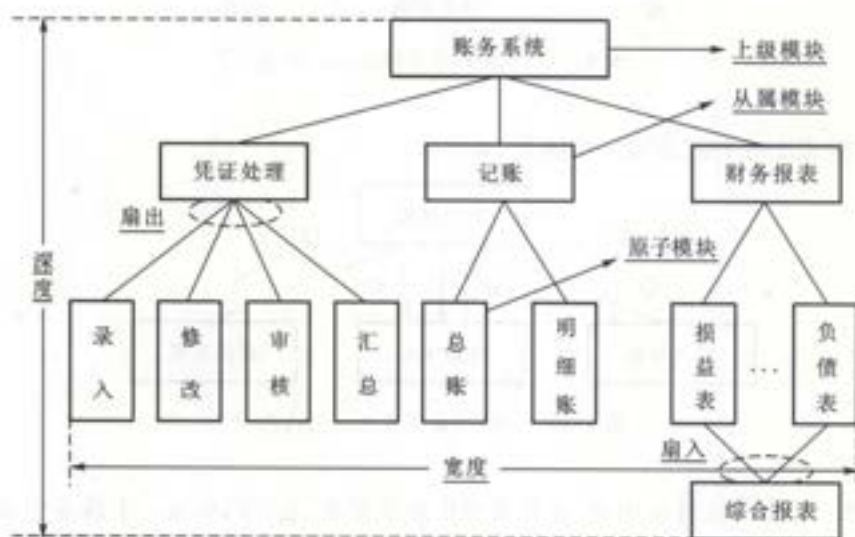


图 4.11 简单财务管理系统结构图

扇出：一个模块直接调用的其他模块数。

原子模块：树中位于叶子结点的模块。

2. 面向数据流的结构化设计方法

在需求分析阶段，主要是分析信息在系统中加工和流动的情况。面向数据流的设计方法定义了一些不同的映射方法，利用这些映射方法可以把数据流图变换成结构图表示的软件结构。首先需要了解数据流图表示的数据处理的类型，然后针对不同类型分别进行分析处理。

(1) 数据流类型

典型的数据流类型有两种:变换型和事务型。

① 变换型。变换型是指信息沿输入通路进入系统,同时由外部形式变换成内部形式,进入系统的信息通过变换中心,经加工处理以后再沿输出通路变换成外部形式离开软件系统。变换型数据处理工作过程大致分为三步,即取得数据、变换数据和输出数据,如图 4.12 所示。相应于取得数据、变换数据、输出数据的过程,变换型系统结构图由输入、中心变换和输出等三部分组成,如图 4.13 所示。



图 4.12 变换型数据流结构



图 4.13 变换型系统结构图的组成

变换型数据流图映射的结构图如图 4.14 所示。

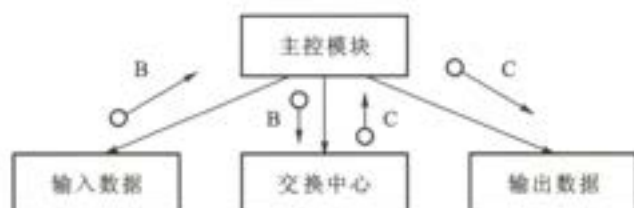


图 4.14 变换型数据流系统结构图

② 事务型。在很多软件应用中,存在某种作业数据流,它可以引发一个或多个处理,这些处理能够完成该作业要求的功能,这种数据流就叫做事务。事务型数据流的特点是接收一项事务,根据事务处理的特点和性质,选择分派一个适当的处理单元(事务处理中心),然后给出结果。这类数据流归为特殊的一类,称为事务型数据流,如图 4.15 所示。在一个事务型数据流中,事务中心接收数据,分析每个事务以确定它的类型,根据事务类型选取一条活动通路。

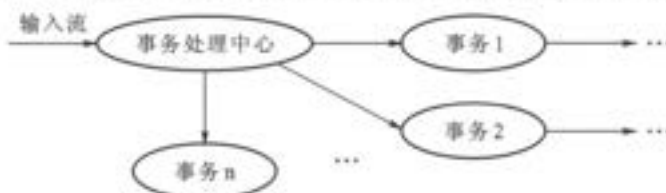


图 4.15 事务型数据流结构

事务型数据流图映射的结构图如图 4.16 所示。



图 4.16 事务型数据流系统结构图

在事务型数据流系统结构图中,事务中心模块按所接收的事务类型,选择某一事务处理模块执行,各事务处理模块并列。每个事务处理模块可能要调用若干个操作模块,而操作模块又可能调用若干个细节模块。

(2) 面向数据流设计方法的实施要点与设计过程

面向数据流的结构设计过程和步骤是:

第1步:分析、确认数据流图的类型,区分是事务型还是变换型。

第2步:说明数据流的边界。

第3步:把数据流图映射为程序结构。对于事务流区分事务中心和数据接收通路,将它映射成事务结构。对于变换流,区分输出和输入分支,并将其映射成变换结构。

第4步:根据设计准则对产生的结构进行细化和求精。

下面分别讨论变换型和事务型数据流图转换成程序结构图的实施步骤。

① 变换型。将变换型映射成结构图,又称为变换分析。其步骤如下:

第1步:确定数据流图是否具有变换特性。一般地说,一个系统中所有的信息流都可以认为是变换流,但是,当遇有明显的事务特性的信息流时,建议采用事务分析方法进行设计。这时应该观察在整个数据流图中哪种属性占优势,先确定数据流的全局特性。此外还应把具有全局特性的不同特点的局部区域孤立出来,根据这些子数据流的特点做部分的处理。

第2步:确定输入流和输出流的边界,划分出输入、变换和输出,独立出变换中心。

第3步:进行第一级分解,将变换型映射成软件结构(参见图4.15),其中输入数据处理模块协调对所有输入数据的接收;变换中心控制模块管理对内部形式的数据的所有操作;输出数据处理控制模块协调输出信息的产生过程。

第4步:按上述步骤如出现事务流也可按事务流的映射方式对各个子流进行逐级分解,直至分解到基本功能。

第5步:对每个模块写一个简要说明,内容为该模块的接口描述、模块内部的信息、过程陈述、包括的主要判定点及任务等。

第6步:利用软件结构的设计原则对软件结构进一步转化。

② 事务型。将事务型映射成结构图,又称为事务分析。其步骤如下:

事务分析的设计步骤与变换分析设计步骤大致类似,主要差别仅在于由数据流图到软件结



构的映射方法不同(参见图4.14和图4.16)。它是将事务中心映射成为软件结构中发送分支的调度模块,将接收通路映射成软件结构的接收分支。

3. 设计的准则

大量软件设计的实践证明,以下的设计准则可以借鉴为设计的指导和对软件结构图进行优化。这些准则是:

① 提高模块独立性。对软件结构应着眼于改善模块的独立性,依据降低耦合提高内聚的原则,通过把一些模块取消或合并来修改程序结构。

② 模块规模适中。经验表明,当模块增大时,模块的可理解性迅速下降。但是当对大的模块分解时,不应降低模块的独立性。因为,当对一个大的模块分解时,有可能会增加模块间的依赖。

③ 深度、宽度、扇出和扇入适当。如果深度过大,则说明有的控制模块可能简单了。如果宽度过大,则说明系统的控制过于集中。而扇出过大则意味模块过分复杂,需要控制和协调过多的下级模块,这时应适当增加中间层次。扇出太小则可以把下级模块进一步分解成若干个子功能模块,或者合并到上级模块中去。扇入越大则共享该模块的上级模块数目越多。

经验表明,好的软件设计结构通常顶层高扇出,中间层扇出较少,底层高扇入。

④ 使模块的作用域在该模块的控制域内。模块的作用域是指模块内一个判定的作用范围,凡是受这个判定影响的所有模块都属于这个判定的作用域。模块的控制域是指这个模块本身以及所有直接或间接从属于它的模块的集合。在一个设计得很好的系统中,所有受某个判定影响的模块应该都从属于做出判定的那个模块,最好局限于做出判定的那个模块本身及它的直属下级模块。对于那些不满足这一条件的软件结构,修改的办法是:将判定点上移或者将那些在作用范围内但是不在控制范围内的模块移到控制范围以内。

⑤ 应减少模块的接口和界面的复杂性。模块的接口复杂是软件容易发生错误的一个主要原因。应该仔细设计模块接口,使得信息传递简单并且和模块的功能一致。

⑥ 设计成单入口、单出口的模块。

⑦ 设计功能可预测的模块。如果一个模块可以当作一个“黑盒”,也就是不考虑模块的内部结构和处理过程,则这个模块的功能就是可以预测的。

4.3.3 详细设计

详细设计的任务,是为软件结构图中的每一个模块确定实现算法和局部数据结构,用某种选定的表达工具表示算法和数据结构的细节。表达工具可以由设计人员自由选择,但它应该具有描述过程细节的能力,而且能够使程序员在编程时便于直接翻译成程序设计语言的源程序。本节重点对过程设计进行讨论。

在过程设计阶段,要对每个模块规定的功能以及算法的设计,给出适当的算法描述,即确定模块内部的详细执行过程,包括局部数据组织、控制流、每一步具体处理要求和各种实现细节等。其目的是确定应该怎样来具体实现所要求的系统。

常见的过程设计工具有:

图形工具:程序流程图,N-S图,PAD图,HIPO图。

表格工具:判定表。

语言工具:PDL(伪码)。

下面讨论其中几种主要的工具:

1. 程序流程图

程序流程图是一种传统的、应用广泛的软件过程设计表示工具,通常也称为程序框图。程序流程图表达直观、清晰,易于学习掌握,且独立于任何一种程序设计语言。

构成程序流程图的最基本图符及含义如图 4.17 所示。

按照结构化程序设计的要求,程序流程图构成的任何程序描述限制为如图 4.18 所示的 5 种控制结构。



图 4.17 程序流程图的基本图符

图 4.18 所示的程序流程图构成的 5 种控制结构的含义是:

顺序型:几个连续的加工步骤依次排列构成;

选择型:由某个逻辑判断式的取值决定选择两个加工中的一个;

先判断重复型:先判断循环控制条件是否成立,成立则执行循环体语句;

后判断重复型:重复执行某些特定的加工,直到控制条件成立;

多分支选择型:列举多种加工情况,根据控制变量的取值,选择执行其中之一。

通过把程序流程图的 5 种基本控制结构相互组合或嵌套,可以构成任何复杂的程序流程图。

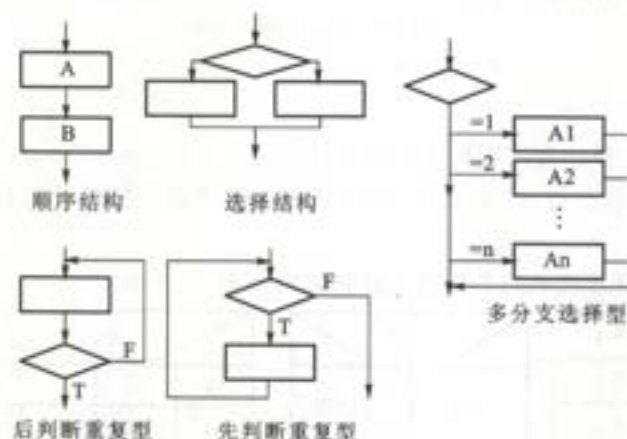


图 4.18 程序流程图构成的 5 种控制结构

例如,下面是简单托运货物运费计算的问题。

设货物重量 x , 客户信息 y , 输入 x, y 后, 计算运费的具体要求是:

如果 $0 < x \leq 15$ (设为条件 1), 则用公式 1 计算后, 循环 3 次完成同样的“记账”和“输出”操作, 然后程序结束;

如果 $x > 15$ (设为条件 2), 则用公式 2 计算后, 循环 3 次完成同样的“记账”和“输出”操作, 然后程序结束。

该问题的程序流程图描述如图 4.19 所示。

程序流程图虽然简单易学, 但是若程序员不受任何约束, 随意转移控制, 会破坏结构化设计的原则, 而且程序流程图不易表示数据结构。

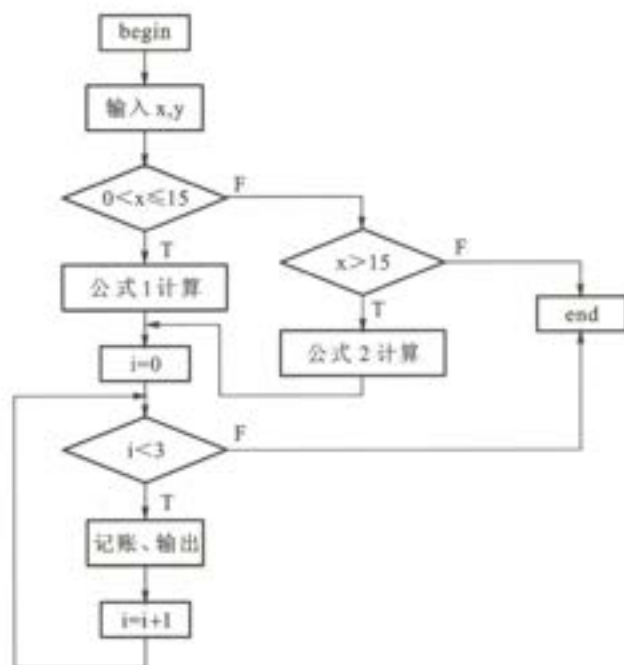


图 4.19 程序流程图示例

2. N-S 图

为了避免流程图在描述程序逻辑时的随意性与灵活性,1973 年 Nossi 和 Shneiderman 发表了题为“结构化程序的流程图技术”的文章,提出了用方框图来代替传统的程序流程图,通常也把这种图称为 N-S 图。

N-S 图的基本图符及表示的 5 种基本控制结构如图 4.20 所示。

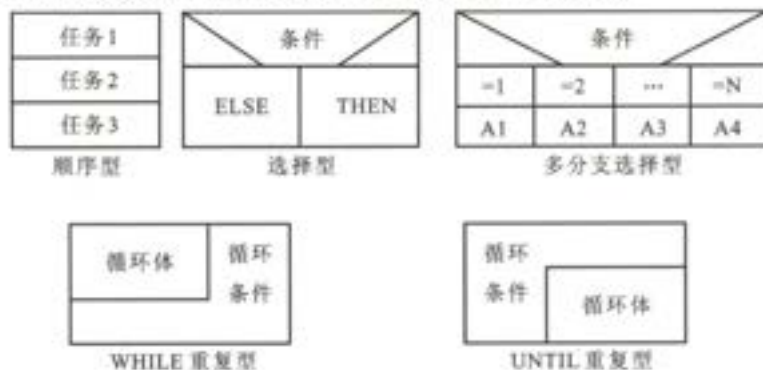


图 4.20 N-S 图图符与构成的 5 种控制结构

例如,上述问题程序的 N-S 图描述如图 4.21 所示。

N-S 图有以下特征:

- ① 每个构件具有明确的功能域;

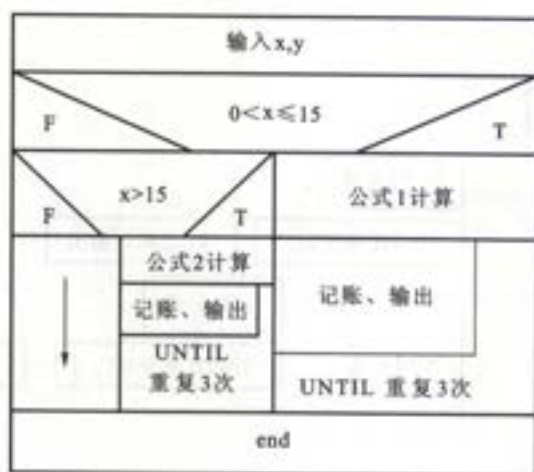


图 4.21 N-S 图示例

- ② 控制转移必须遵守结构化设计要求；
- ③ 易于确定局部数据和(或)全局数据的作用域；
- ④ 易于表达嵌套关系和模块的层次结构。

3. PAD 图

PAD 图是问题分析图(problem analysis diagram)的英文缩写。它是继程序流程图和方框图之后,提出的又一种主要用于描述软件详细设计的图形表示工具。

PAD 图的基本图符及表示的 5 种基本控制结构,如图 4.22 所示。



图 4.22 PAD 图图符与构成的 5 种控制结构

例如,上述问题程序的 PAD 图描述如图 4.23 所示。

PAD 图有以下特征:

- ① 结构清晰,结构化程度高;
- ② 易于阅读;
- ③ 最左端的纵线是程序主干线,对应程序的第一层结构;每增加一层 PAD 图向右扩展一条纵线,故程序的纵线数等于程序层数;
- ④ 程序执行:从 PAD 图最左主干线上端结点开始,自上而下、自左向右依次执行,程序终止于最左主干线。

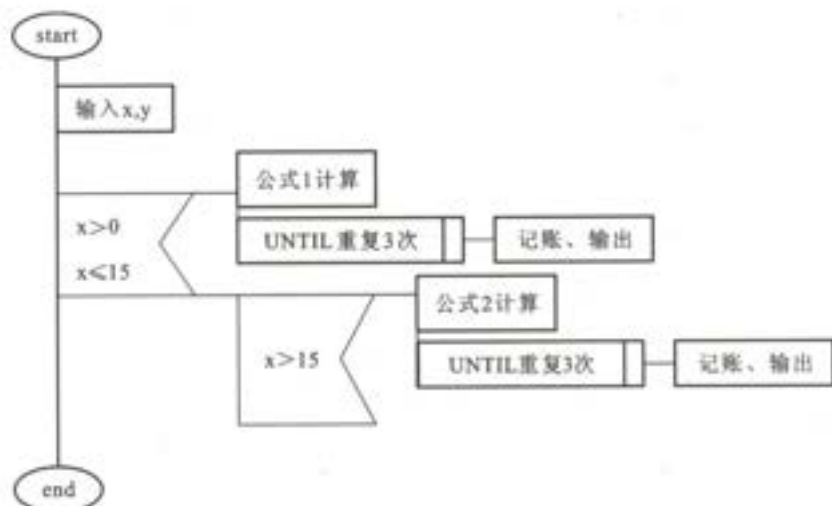


图 4.23 PAD 图示例

4. PDL (procedure design language)

过程设计语言(PDL)也称为结构化的英语和伪码,它是一种混合语言,采用英语的词汇和结构化程序设计语言的语法,类似编程语言。

用 PDL 表示的基本控制结构的常用词汇如下:

顺序;

条件: IF/THEN/ELSE/ENDIF

循环: DO WHILE/ENDDO

循环: REPEAT UNTIL/ENDREPEAT

分支: CASE_OF/WHEN/SELECT/WHEN/SELECT/ENDCASE

例如,上述问题程序的描述如下,它是类似 C 语言的 PDL。

```

/* 计算运费 */
count ( );
| 输入 x; 输入 y;
| if (0 < x <= 15) 条件 1 | 公式 1 计算; call sub; |
|   else if (x > 15) | 公式 2 计算; call sub; |
|
| sub ( );
|   for (i = 1, 3, i++) do | 记账; 输出; |
|

```

PDL 可以由编程语言转换得到,也可以是专门为过程描述而设计的。但应具备以下特征:

- ① 有为结构化构成元素、数据说明和模块化特征提供的关键词语法;
- ② 处理部分的描述采用自然语言语法;
- ③ 可以说明简单和复杂的数据结构;
- ④ 支持各种接口描述的子程序定义和调用技术。

4.4 软件测试

随着计算机软硬件技术的发展,计算机的应用领域越来越广泛,方方面面的应用对软件的功能要求也就越来越强,而且软件的复杂程度也就越来越高。但是,如何才能确保软件的质量并保证软件的可靠性呢?无疑,通过对软件产品进行必要的测试是非常重要的一个环节。软件测试也是在软件投入运行前对软件需求、设计、编码的最后审核。

软件测试的投入,包括人员和资金投入是巨大的,通常其工作量、成本占软件开发总工作量、总成本的比重很大,而且具有很高的组织管理和技术难度。

软件测试是保证软件质量的重要手段,其主要过程涵盖了整个软件生命期的过程,包括需求定义阶段的需求测试、编码阶段的单元测试、集成测试以及后期的确认测试、系统测试,验证软件是否合格、能否交付用户使用等。

4.4.1 软件测试的目的和定义

1990年在IEEE 610.12标准中给出了软件测试的定义:

① 在规定条件下运行系统或构件的过程;在此过程中观察和记录结果,并对系统或构件的某些方面给出评价。

② 软件项目的过程:检测现有状况和所需状况的不同(即bug),并评估软件项目的特性。

GB/T11457—2006《信息技术 软件工程术语》中采用了IEEE的定义。定义表明软件测试是一个过程,其目的是评价系统或构件的某些方面,以评估是否满足规定的需求;评估项目的特点,以验证期望的结果与实际的结果之间有无差别。换句话说,该定义说明了软件测试是一项验证和评估活动,其目的是基于满足规定的需求来保证软件的质量。

关于软件测试的目的,Grenford J. Myers在《The Art of Software Testing》一书中给出了更深刻的阐述:

软件测试是为了发现错误而执行程序的过程;

一个好的测试用例是指很可能找到迄今为止尚未发现的错误的用例;

一个成功的测试是发现了至今尚未发现的错误的测试。

Myers的观点告诉人们测试要以查找错误为中心,而不是为了演示软件的正确功能。

4.4.2 软件测试的准则

鉴于软件测试的重要性,要做好软件测试,设计出有效的测试方案和好的测试用例,软件测试人员需要充分理解和运用软件测试的一些基本准则:

1. 所有测试都应追溯到需求

软件测试的目的是发现错误,而最严重的错误不外乎是导致程序无法满足用户需求的错误。

2. 严格执行测试计划,排除测试的随意性

软件测试应当制定明确的测试计划并按照计划执行。测试计划应包括:所测软件的功能、输入和输出、测试内容、各项测试的目的和进度安排、测试资料、测试工具、测试用例的选择、资源要



求、测试的控制方式和过程等。

3. 充分注意测试中的群集现象

经验表明,程序中存在错误的概率与该程序中已发现的错误数成正比。这一现象说明,为了提高测试效率,测试人员应该集中对付那些错误群集的程序。

4. 程序员应避免检查自己的程序

为了达到好的测试效果,应该由独立的第三方来构造测试。因为从心理学角度讲,程序人员或设计方在测试自己的程序时,要采取客观的态度是程度不同地存在障碍的。

5. 穷举测试不可能

所谓穷举测试是指把程序所有可能的执行路径都进行检查的测试。但是,即使规模较小的程序,其路径排列数也是相当大的,在实际测试过程中不可能穷尽每一种组合。这说明,测试只能证明程序中有错误,不能证明程序中没有错误。

6. 妥善保存相关资料

妥善保存测试计划、测试用例、出错统计和最终分析报告,为维护提供方便。

4.4.3 软件测试方法与技术综述

软件测试的方法和技术是多种多样的。对于软件测试方法和技术,可以从不同的角度加以分类。

若从是否需要执行被测软件的角度,可以分为静态测试和动态测试方法。若按照功能可以分为白盒测试和黑盒测试方法。

1. 静态测试与动态测试

(1) 静态测试

静态测试包括代码检查、静态结构分析、代码质量度量等。静态测试可以由人工进行,充分发挥人的逻辑思维优势,也可以借助软件工具自动进行。经验表明,使用人工测试能够有效地发现 30% 到 70% 的逻辑设计和编码错误。

代码检查主要检查代码和设计的一致性,包括代码的逻辑表达的正确性,代码结构的合理性等方面。这项工作可以发现违背程序编写标准的问题,程序中不安全、不明确和模糊的部分,找出程序中不可移植部分、违背程序编程风格的问题,包括变量检查、命名和类型审查、程序逻辑审查、程序语法检查和程序结构检查等内容。代码检查包括代码审查、代码走查、桌面检查、静态分析等具体方式。

代码审查:小组集体阅读、讨论检查代码。

代码走查:小组成员通过用“脑”研究、执行程序来检查代码。

桌面检查:由程序员自己检查自己编写的程序。程序员在程序通过编译之后,进行单元测试之前,对源代码进行分析、检验,并补充相关文档,目的是发现程序的错误。

静态分析:对代码的机械性、程式化的特性分析方法,包括控制流分析、数据流分析、接口分析、表达式分析。

(2) 动态测试

静态测试不实际运行软件,主要通过人工进行。动态测试是基于计算机的测试,是为了发现错误而执行程序的过程。或者说,是根据软件开发各阶段的规格说明和程序的内部结构而精心设计一批测试用例(即输入数据及其预期的输出结果),并利用这些测试用例去运行程序,以发

现程序错误的过程。

设计高效、合理的测试用例是动态测试的关键。测试用例(test case)是为测试设计的数据。测试用例由测试输入数据和与之对应的预期输出结果两部分组成。测试用例的格式为:

[(输入值集),(输出值集)]

下面重点讨论动态的白盒测试方法和黑盒测试方法。

2. 白盒测试方法与测试用例设计

白盒测试方法也称结构测试或逻辑驱动测试。它是根据软件产品的内部工作过程,检查内部成分,以确认每种内部操作符合设计规格要求。白盒测试把测试对象看作一个打开的盒子,允许测试人员利用程序内部的逻辑结构及有关信息来设计或选择测试用例,对程序所有的逻辑路径进行测试。通过在不同点检查程序的状态来了解实际的运行状态是否与预期的一致。所以,白盒测试是在程序内部进行,主要用于完成软件内部操作的验证。

白盒测试的基本原则是:保证所测模块中每一独立路径至少执行一次;保证所测模块所有判断的每一分支至少执行一次;保证所测模块每一循环都在边界条件和一般条件下至少各执行一次;验证所有内部数据结构的有效性。

按照白盒测试的基本原则,“白盒”法是穷举路径测试。在使用这一方案时,测试者必须检查程序的内部结构,从检查程序的逻辑着手,得出测试数据。贯穿程序的独立路径数是天文数字,但即使每条路径都测试了仍然可能有错误。第一,穷举路径测试绝不能查出程序是否违反了设计规范,即程序本身是个错误的程序;第二,穷举路径测试不可能查出程序中因遗漏路径而出错;第三,穷举路径测试可能发现不了一些与数据相关的错误。

白盒测试的主要方法有逻辑覆盖测试、基本路径测试等。

(1) 逻辑覆盖测试

逻辑覆盖是泛指一系列以程序内部的逻辑结构为基础的测试用例设计技术。通常所指的程序中的逻辑表示有判断、分支、条件等几种表示方式。

① 语句覆盖。选择足够的测试用例,使得程序中每个语句至少都能被执行一次。

例 4.1 设有程序流程图表示的程序如图 4.24 所示。

按照语句覆盖的测试要求,对图 4.24 的程序设计如下测试用例 1 和测试用例 2。

测试用例 1:

输入 (i,j)	输出 (i,j,x)
(10,10)	(10,10,10)

测试用例 2:

输入 (i,j)	输出 (i,j,x)
(10,15)	(10,15,15)

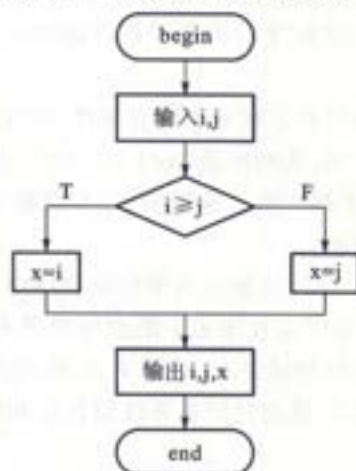


图 4.24 程序流程图



语句覆盖是逻辑覆盖中基本的覆盖,尤其对单元测试来说。但是语句覆盖往往没有关注判断中的条件有可能隐含的错误。

② 基本路径覆盖。执行足够的测试用例,使程序中所有可能的路径都至少经历一次。

例 4.2 设有程序流程图表示的程序如图 4.25 所示。

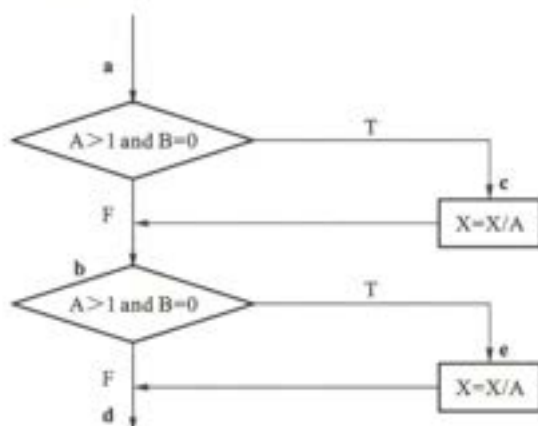


图 4.25 程序流程图

对图 4.25 的程序设计如表 4.2 列出的一组测试用例,就可以覆盖该程序的全部 4 条路径: ace, abd, abe, acd。

表 4.2 一组测试用例

测试用例	通过路径	测试用例	通过路径
[(A=2, B=0, X=3), (输出略)]	(ace)	[(A=2, B=1, X=1), (输出略)]	(abe)
[(A=1, B=0, X=1), (输出略)]	(abd)	[(A=3, B=0, X=1), (输出略)]	(acd)

③ 判定覆盖。使设计的测试用例保证程序中每个判断的每个取值分支(T或F)至少经历一次。

根据判定覆盖的要求,对如图 4.26 所示的程序,如果其中包含条件 $i \geq j$ 的判断为真值(即为“T”)和为假值(即为“F”)的程序执行路径至少经历一次,仍然可以使用例 4.1 的测试用例 1 和测试用例 2。

程序每个判断中若存在多个联立条件,仅保证判断的真假值往往会导致某些单个条件的错误不能被发现。例如,某判断是“ $x < 1$ 或 $y > 5$ ”,其中只要一个条件取值为真,无论另一个条件是否正确,判断的结果都为真。这说明,仅有判断覆盖还无法保证能查出在判断的条件中的错误,需要更强的逻辑覆盖。

④ 条件覆盖。设计的测试用例保证程序中每个判断的每个条件的可能取值至少执行一次。

例 4.3 设有程序流程图表示的程序如图 4.26 所示。

按照条件覆盖的测试要求,对图 4.26 的程序判断框中的条件 $i \geq j$ 和条件 $j < 5$ 设计如下测试用例 1 和测试用例 2,就能保证该条件取真值和取假值的情况至少执行一次。

测试用例 1:

输入	输出
(i,j)	(i,j,x)
(3,2)	(3,2,3)

测试用例 2:

输入	输出
(i,j)	(i,j,x)
(5,10)	(5,10,10)

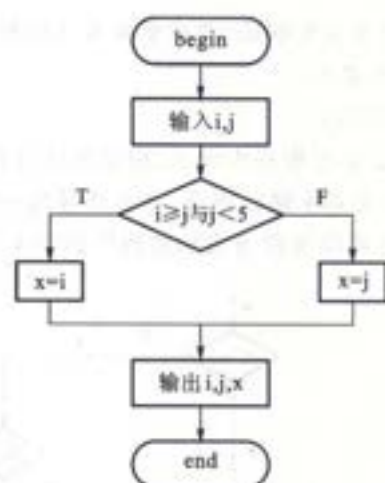


图 4.26 程序流程图

条件覆盖深入到判断中的每个条件,但是可能会忽略全面的判断覆盖的要求。有必要考虑判断-条件覆盖。

⑤ 判断-条件覆盖。设计足够的测试用例,使判断中每个条件的所有可能取值至少执行一次,同时每个判断的所有可能取值分支至少执行一次。

例 4.4 设有程序流程图表示的程序如图 4.27 所示。

按照判断-条件覆盖的测试要求,对图 4.27 程序的两个判断框的每个取值分支至少经历一次,同时两个判断框中的三个条件的所有可能取值至少执行一次,设计如下测试用例 1、测试用例 2 和测试用例 3,就能保证满足判断-条件覆盖。

测试用例 1:

输入	输出
(i,j,x)	(i,j,x)
(3,2,0)	(3,2,0)

测试用例 2:

输入	输出
(i,j,x)	(i,j,x)
(6,4,0)	(6,4,6)

测试用例 3:

输入	输出
(i,j,x)	(i,j,x)
(5,10,0)	(5,10,10)

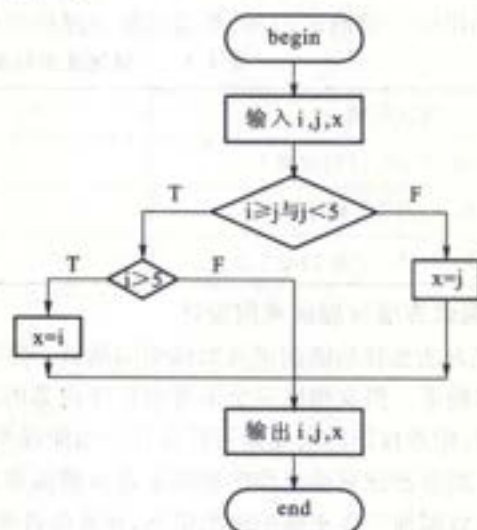


图 4.27 程序流程图



判断-条件覆盖也有缺陷,对质量要求高的软件单元,可根据情况提出多重条件组合覆盖以及其他更高的覆盖要求。

(2) 基本路径测试

基本路径测试的思想和步骤是,根据软件过程性描述中的控制流程确定程序的环路复杂度度量,用此度量定义基本路径集合,并由此导出一组测试用例对每一条独立执行路径进行测试。

例 4.5 设有程序流程图表示的程序如图 4.28 所示。

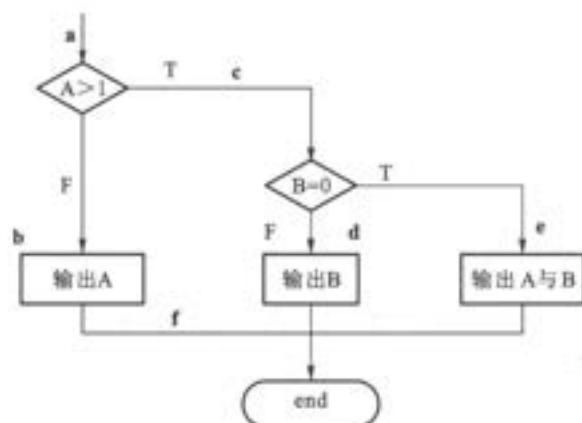


图 4.28 程序流程图

对图 4.28 的程序流程图确定程序的环路复杂度,方法是:

环路复杂度 = 程序流程图中的判断框^①个数+1

则环路复杂度的值即为要设计测试用例的基本路径数,图 4.28 所示的程序环路复杂度为 3,设计如表 4.3 列出的一组测试用例,覆盖的基本路径是:abf, acef, acdf。

表 4.3 一组测试用例及通过的路径

测试用例	通过路径
[(A=-2, B=0), (输出略)]	(abf)
[(A=5, B=0), (输出略)]	(acef)
[(A=5, B=5), (输出略)]	(acdf)

3. 黑盒测试方法与测试用例设计

黑盒测试方法也称功能测试或数据驱动测试。黑盒测试是对软件已经实现的功能是否满足需求进行测试和验证。黑盒测试完全不考虑程序内部的逻辑结构和内部特性,只依据程序的需求和功能规格说明,检查程序的功能是否符合它的功能说明。所以,黑盒测试是在软件接口处进行,完成功能验证。黑盒测试只检查程序功能是否按照需求规格说明书的规定正常使用,程序是否能适当地接收输入数据而产生正确的输出信息,并且保持外部信息(如数据库或文件)的完整性。

黑盒测试主要诊断功能不对或遗漏、界面错误、数据结构或外部数据库访问错误、性能错误、初始化和终止条件错。

^① 判断框是仅有一个单一条件的判断框。

黑盒测试方法主要有等价类划分法、边界值分析法、错误推测法、因果图法等,主要用于软件确认测试。

(1) 等价类划分法

等价类划分法是一种典型的黑盒测试方法。它是将程序的所有可能的输入数据划分成若干部分(即若干等价类),然后从每个等价类中选取数据作为测试用例。对每一个等价类,各个输入数据对发现程序中的错误的概率都是相等的,因此只需从每个等价类中选取一些有代表性的测试用例进行测试而发现错误。

使用等价类划分法设计测试方案,首先需要划分输入集合的等价类。等价类包括:

① 有效等价类:合理、有意义的输入数据构成的集合。可以检验程序中符合规定的功能、性能。

② 无效等价类:不合理、无意义的输入数据构成的集合。可以检验程序中不符合规定的功能、性能。

为此,需要研究程序的功能说明,从而确定输入数据的有效等价类和无效等价类。

等价类划分法实施步骤分为两步:

第1步:划分等价类;

第2步:根据等价类选取相应的测试用例。

例 4.6 程序实现输入3个边长(设为 a, b, c),判断能否构成三角形。对该程序考虑等价类划分法。

满足测试三角形构成条件程序的等价类划分如表4.4所示。

表 4.4 三角形边的等价类划分

输入条件	有效等价类	无效等价类
① 边长 a, b, c 限制	$a > 0$ 或 $b > 0$ 或 $c > 0$	$a \leq 0$ 或 $b \leq 0$ 或 $c \leq 0$
② 边长关系限制	$a + b > c$ 或 $b + c > a$ 或 $a + c > b$	$a + b \leq c$ 或 $b + c \leq a$ 或 $a + c \leq b$

根据表4.4划分的等价类,可以设计以下的测试用例:

对满足输入条件①和②的有效等价类设计的测试用例:

$[(a = 3, b = 4, c = 5), (\text{符合三角形构成条件})]$

对满足输入条件①的无效等价类设计的测试用例:

$[(a = -3, b = 4, c = 5), (\text{无效输入})]$

对满足输入条件②的无效等价类设计的测试用例:

$[(a = 3, b = 4, c = 8), (\text{无效输入})]$

划分等价类常用的几条原则是:

若输入条件规定了确切的取值范围,则可划分出一个有效等价类和两个无效等价类。

若输入条件规定了输入值的集合(或有“必须如何”的条件),可确定一个有效等价类和一个无效等价类。

若输入条件是一个布尔量,则可确定一个有效等价类和一个无效等价类。

若输入数据是一组值,且程序要对每个值分别处理。可为每个输入值确定一个有效等价类和一个无效等价类。



若规定了输入数据必须遵守一定规则,则可确定一个有效等价类和若干个无效等价类。

若已划分的等价类中各元素在程序中处理方式不同,须将该等价类进一步划分(更小的等价类)。

(2) 边界值分析法

边界值分析法是对各种输入、输出范围的边界情况设计测试用例的方法。

经验表明,程序错误最容易出现在输入或输出范围的边界处,而不是在输入范围的内部。因此针对各种边界情况设计测试用例,可以查出更多的错误。

使用边界值分析方法设计测试用例,确定边界情况应考虑选取正好等于、刚刚大于或刚刚小于边界的值作为测试数据,这样发现程序中错误的概率较大。

边界值分析方法的使用要注意以下几点:

① 如果输入条件规定了取值范围或数据个数,则可选择正好等于边界值、刚刚在边界范围内和刚刚超越边界外的值进行测试;

② 针对规格说明的每个输入条件,使用上述原则;

③ 对于有序数列,选择第一个和最后一个作为测试数据。

例如,对例4.6中的判断三角形构成的程序,如果在等价类划分法中加入边界值分析的思想,即选取该等价类的边界值,则会使等价类划分法更有效。考虑等价类划分法加入边界值分析的例4.6的测试用例可以设计如下:

对满足输入条件①的无效等价类设计的测试用例:

[(a=0, b=4, c=5), (无效输入)]

或[(a=3, b=0, c=5), (无效输入)]

或[(a=3, b=4, c=0), (无效输入)]

对满足输入条件②的无效等价类设计的测试用例:

[(a=3, b=4, c=7), (无效输入)]

或[(a=9, b=4, c=5), (无效输入)]

或[(a=3, b=8, c=5), (无效输入)]

一般多用边界值分析法来补充等价类划分方法。

(3) 错误推测法

人们可以靠经验和直觉推测程序中可能存在的各种错误,从而有针对性地编写检查这些错误的例子,这就是错误推测法。

错误推测法的基本想法是:列举出程序中所有可能有的错误和容易发生错误的特殊情况,根据它们选择测试用例。错误推测法针对性强,可以直接切入可能的错误,直接定位,是一种非常实用、有效的方法。但是它需要丰富的经验和专业知识。

错误推测法的实施步骤一般是,对被测软件首先列出所有可能有的错误和易错情况表,然后基于该表设计测试用例。

例如,一般程序中输入为“0”或输出为“0”的情形是易错情况,测试者可以设计输入值为0的测试情况,以及使输出强迫为0的测试情况。

例如,要测试一个排序子程序,特别需要检查的情况是:输入表为空;输入表只含有一个元素;输入表的所有元素的值都相同;输入表已经排过序。这些情况都是在程序设计时可能忽略的

特殊情况。

实际上,无论是使用白盒测试方法还是黑盒测试方法,或是其他测试方法,针对一种方法设计的测试用例,仅仅是易于发现某种类型的错误,对其他类型的错误不易发现。所以没有一种用例设计方法能适用全部的测试方案,而是各有所长。综合使用各种方法来确定合适的测试方案,应该考虑在测试成本和测试效果之间的一个合理折中。

4.4.4 软件测试的策略

软件测试是保证软件质量的重要手段,软件测试是一个过程,其测试流程是该过程规定的程序,目的是使软件测试工作系统化。软件测试的策略是指在软件测试过程中对各种活动的组织和管理。

软件测试过程一般按4个步骤进行,即单元测试、集成测试、验收测试(确认测试)和系统测试。通过这些步骤的实施来验证软件是否合格,能否交付用户使用。

1. 单元测试

单元测试是对软件设计的最小单位——模块(程序单元)进行正确性检验的测试。单元测试的目的是发现各模块内部可能存在的各种错误。

单元测试的依据是详细设计说明书和源程序。

单元测试的技术可以采用静态分析和动态测试。对动态测试通常以白盒动态测试为主,辅之以黑盒测试。

单元测试主要针对模块的下列5个基本特性进行:

① 模块接口测试——测试通过模块的数据流。例如,检查模块的输入参数和输出参数、全局量、文件属性与操作等都属于模块接口测试的内容。

② 局部数据结构测试。例如,检查局部数据说明的一致性,数据的初始化,数据类型的一致性以及数据的下溢、上溢等。

③ 重要的执行路径的检查。

④ 出错处理测试。检查模块的错误处理功能。

⑤ 影响以上各点及其他相关点的边界条件测试。

单元测试是针对某个模块,这样的模块通常并不是一个独立的程序,因此模块自己不能运行,而要靠辅助其他模块调用或驱动。同时,模块自身也会作为驱动模块去调用其他模块,也就是说,单元测试要考虑它和外界的联系,必须在一定的环境下进行,这些环境可以是真实的也可以是模拟的。模拟环境是单元测试常用的。

所谓模拟环境就是在单元测试中,用一些辅助模块去模拟与被测模块相联系的其他模块,即为被测模块设计和搭建驱动模块和桩模块,如图4.29所示。

其中,驱动(driver)模块相当于被测模块的主程序。它接收测试数据,并传给被测模块,输出实际测试结果。桩(stub)模块通常用于代替被测模块调用的其他模块,其作用仅做少量的数据操作,是一个模拟子程序,不必将子模块的所有功能带入。



图 4.29 单元测试的测试环境



2. 集成测试

集成测试是测试和组装软件的过程。它是在把模块按照设计要求组装起来的同时进行测试,主要目的是发现与接口有关的错误。集成测试的依据是概要设计说明书。

集成测试所涉及的内容包括:软件单元的接口测试、全局数据结构测试、边界条件和非法输入的测试等。

集成测试时将模块组装成程序通常采用两种方式:非增量方式组装与增量方式组装。

非增量方式也称为一次性组装方式。将测试好的每一个软件单元一次组装在一起再进行整体测试。

增量方式是将已经测试好的模块逐步组装成较大系统,在组装过程中边连接边测试,以发现连接过程中产生的问题。最后通过增值,逐步组装到所要求的软件系统。

增量方式包括自顶向下、自底向上、自顶向下与自底向上相结合的混合增量方法。

(1) 自顶向下的增量方式

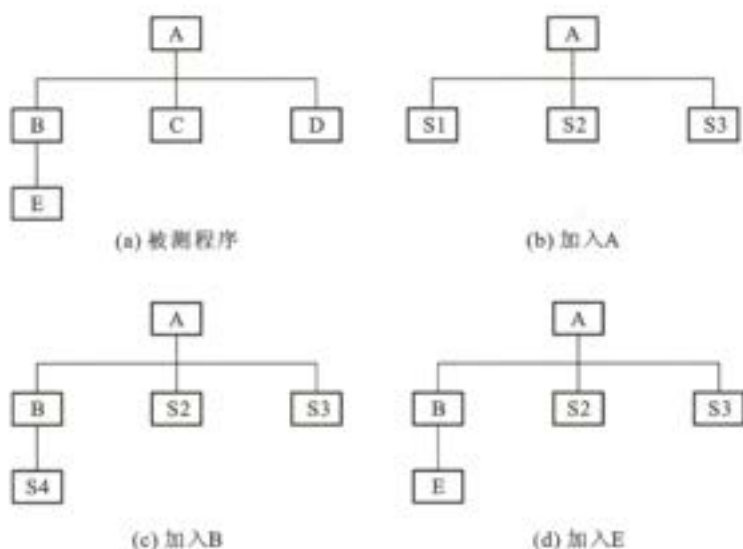
将模块按系统程序结构,从主控模块(主程序)开始,沿控制层次自顶向下地逐个把模块连接起来。自顶向下的增量方式在测试过程中能较早地验证主要的控制点和判断点。

自顶向下集成的过程与步骤如下:

- ① 主控模块作为测试驱动器。直接附属于主控模块的各模块全都用桩模块代替。
- ② 按照一定的组装次序,每次用一个真模块取代一个附属的桩模块。
- ③ 当装入每个真模块时都要进行测试。
- ④ 做完每一组测试后再用一个真模块代替另一个桩模块。
- ⑤ 可以进行回归测试(即重新再做过去做过的全部或部分测试),以便确定没有新的错误发生。

例 4.7 对图 4.30(a) 所示程序结构进行自顶向下的增量方式组装测试。

自顶向下的增量方式的组装过程如图 4.30(b)~(f) 所示。



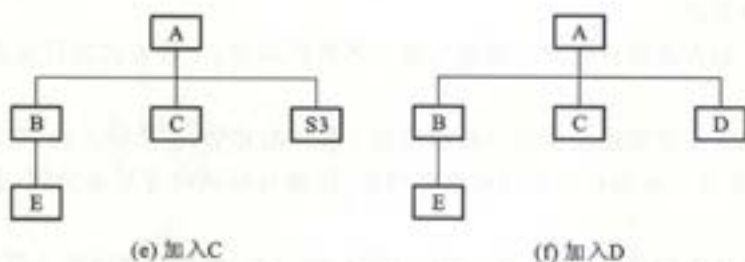


图 4.30 自顶向下的增量测试

(2) 自底向上的增量方式

自底向上集成测试方法是从软件结构中最底层的、最基本的软件单元开始进行集成和测试。在模块的测试过程中需要从子模块得到的信息可以直接运行子模块得到。由于在逐步向上组装过程中下层模块总是存在的,因此不再需要桩模块,但是需要调用这些模块的驱动模块。

自底向上集成的过程与步骤如下:

- ① 低层的模块组成簇,以执行某个特定的软件子功能。
- ② 编写一个驱动模块作为测试的控制程序,和被测试的簇连在一起,负责安排测试用例的输入及输出。
- ③ 对簇进行测试。
- ④ 拆去各个小簇的驱动模块,把几个小簇合并成大簇,再重复做②、③及④步。这样在软件结构上逐步向上组装。

例 4.8 对图 4.31(a) 所示程序结构进行自底向上的增量方式的组装测试。

自底向上的增量方式的组装过程如图 4.31(b) - (d) 所示。

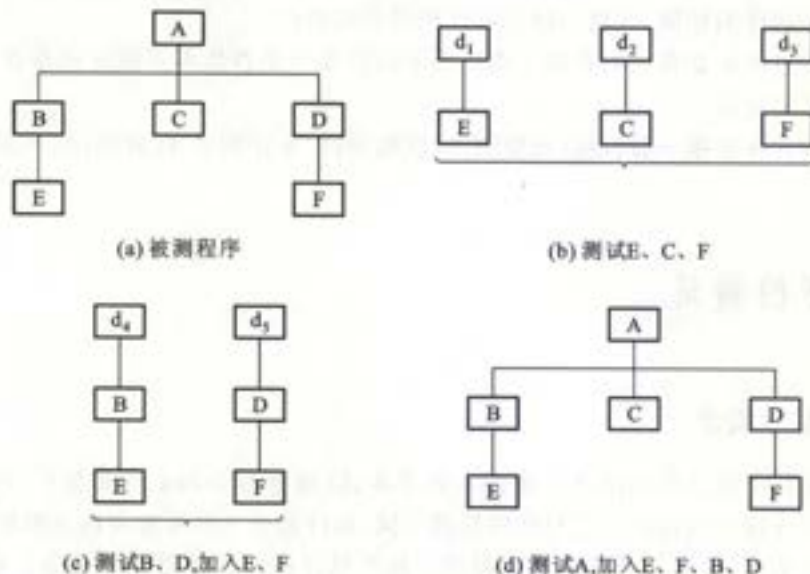


图 4.31 自底向上的增量测试



(3) 混合增量方式

自顶向下增量的方式和自底向上增量的方式各有优缺点,一种方式的优点是另一种方式的缺点。

自顶向下测试的主要优点是能较早显示出整个程序的轮廓,主要缺点是,当测试上层模块时使用桩模块较多,很难模拟出真实模块的全部功能,使部分测试内容被迫推迟,直至换上真实模块后再补充测试。

自底向上测试从下层模块开始,设计测试用例比较容易,但是在测试的早期不能显示出程序的轮廓。

针对自顶向下、自底向上方法各自的优点和不足,人们提出了自顶向下和自底向上相结合、从两头向中间逼近的混合式组装方法,被形象地称为“三明治”方法。这种方式,结合考虑软件总体结构的良好设计原则,在程序结构的高层使用自顶向下方式,在程序结构的低层使用自底向上方式。

3. 确认测试

确认测试的任务是验证软件的功能和性能及其他特性是否满足了需求规格说明中确定的各种需求,以及软件配置是否完全、正确。

确认测试的实施首先运用黑盒测试方法,对软件进行有效性测试,即验证被测软件是否满足需求规格说明确认的标准。复审的目的在于保证软件配置齐全、分类有序,以及软件配置所有成分的完备性、一致性、准确性和可操作性,并且包括软件维护所必需的细节。

4. 系统测试

系统测试是将通过测试确认的软件,作为整个基于计算机系统的一个元素,与计算机硬件、外设、支持软件、数据和人员等其他系统元素组合在一起,在实际运行(使用)环境下对计算机系统进行一系列的集成测试和确认测试。由此可知,系统测试必须在目标环境下运行,其功用在于评估系统环境下软件的性能,发现和捕捉软件中潜在的错误。

系统测试的目的是在真实的系统工作环境下检验软件是否能与系统正确连接,发现软件与系统需求不一致的地方。

系统测试的具体实施一般包括:功能测试、性能测试、操作测试、配置测试、外部接口测试、安全性测试等。

4.5 程序的调试

4.5.1 基本概念

在对程序进行了成功的测试之后将进入程序调试(通常称 debug,即排错)。程序调试的任务是诊断和改正程序中的错误。它与软件测试不同,软件测试是尽可能多地发现软件中的错误。先要发现软件的错误,然后借助于一定的调试工具去执行找出软件错误的具体位置。软件测试贯穿整个软件生命期,调试主要在开发阶段。

由程序调试的概念可知,程序调试活动由两部分组成,其一是根据错误的迹象确定程序中错

误的确切性质、原因和位置。其二是对程序进行修改,排除这个错误。

1. 程序调试的基本步骤

(1) 错误定位

从错误的外部表现形式入手,研究有关部分的程序,确定程序中出错位置,找出错误的内在原因。确定错误位置占据了软件测试绝大部分的工作量。

从技术角度来看,错误的特征和查找错误的难度在于:

① 现象与原因所处的位置可能相距很远。就是说,现象可能出现在程序的一个部位,而原因可能在离此很远的另一个位置。高耦合的程序结构中这种情况更为明显。

② 当纠正其他错误时,这一错误所表现出的现象可能会消失或暂时性消失,但并未实际排除。

③ 现象可能并不是由错误引起的(如舍入误差)。

④ 现象可能是由于一些不容易发现的人为错误引起的。

⑤ 错误现象可能时有时无。

⑥ 现象是由于难以再现的输入状态(例如实时应用中输入顺序不确定)引起的。

⑦ 现象可能是周期出现的。如在软件、硬件结合的嵌入式系统中常常遇到。

(2) 修改设计和代码,以排除错误

排错是软件开发过程中一项艰苦的工作,这也决定了调试工作是一个具有很强技术性和技巧性的工作。软件工程人员在分析测试结果的时候会发现,软件运行失效或出现问题,往往只是潜在错误的外部表现,而外部表现与内在原因之间常常没有明显的联系。如果要找出真正的原因,排除潜在的错误,不是一件易事。因此可以说,调试是通过现象,找出原因的一个思维分析的过程。

(3) 进行回归测试,防止引进新的错误

因为修改程序可能带来新的错误,重复进行暴露这个错误的原始测试或某些有关测试,以确认该错误是否被排除、是否引进了新的错误。如果所做的修正无效,则撤销这次改动,重复上述过程,直到找到一个有效的解决办法为止。

2. 程序调试的原则

在软件调试方面,许多原则实际上是心理学方面的问题。因为调试活动由对程序中错误的定性、定位和排错两部分组成,因此调试原则也从以下两个方面考虑。

(1) 确定错误的性质和位置时的注意事项

① 分析思考与错误征兆有关的信息。

② 避开死胡同。如果程序调试人员在调试中陷入困境,最好暂时把问题抛开,留到后面适当的时间再去考虑,或者向其他人讲解这个问题,去寻求新的解决思路。

③ 只把调试工具当作辅助手段来使用。利用调试工具,可以帮助思考,但不能代替思考。因为调试工具给人提供的是一种无规律的调试方法。

④ 避免用试探法,最多只能把它当作最后手段。这是一种碰运气的盲目的动作,它的成功率很小,而且还常把新的错误带到问题中来。

(2) 修改错误的原则

① 在出现错误的地方,很可能还有别的错误。经验表明,错误有群集现象,当在某一程序段



发现有错误时,在该程序段中还存在别的错误的概率也很高。因此,在修改一个错误时,还要观察和检查相关的代码,看是否还有别的错误。

② 修改错误的一个常见失误是只修改了这个错误的征兆或这个错误的表现,而没有修改错误本身。如果提出的修改不能解释与这个错误有关的全部现象,那就表明了只修改了错误的一部分。

③ 注意修正一个错误的同时有可能会引入新的错误。人们不仅需要注意不正确的修改,而且还要注意看起来是正确的修改可能会带来的副作用,即引进新的错误。因此在修改了错误之后,必须进行回归测试。

④ 修改错误的过程将迫使人们暂时回到程序设计阶段。修改错误也是程序设计的一种形式。一般说来,在程序设计阶段所使用的任何方法都可以应用到错误修正的过程中来。

⑤ 修改源代码程序,不要改变目标代码。

4.5.2 软件调试方法

调试的关键在于推断程序内部的错误位置及原因。从是否跟踪和执行程序的角度,类似于软件测试,软件调试可以分为静态调试和动态调试。软件测试中讨论的静态分析方法同样适用静态调试。静态调试主要指通过人的思维来分析源程序代码和排错,是主要的调试手段,而动态调试是辅助静态调试的。主要的调试方法可以采用:

1. 强行排错法

作为传统的调试方法,其过程可概括为,设置断点、程序暂停、观察程序状态、继续运行程序。这是目前使用较多、效率较低的调试方法。涉及的调试技术主要是设置断点和监视表达式。例如:

① 通过内存全部打印来排错。

② 在程序特定部位设置打印语句——断点法。输出存储器内容,就是在程序执行到某一行时,计算机自动停止运行,并保留这时各变量的状态,方便检查、校对。

③ 自动调试工具。可供利用的典型的语言功能有打印出语句执行的追踪信息、追踪子程序调用,以及指定变量的变化情况。自动调试工具的功能是设置断点,当程序执行到某个特定的语句或某个特定的变量值改变时,程序暂停执行。程序员可在终端上观察程序此时的状态。

应用以上任何一种技术之前,都应当对错误的征兆进行全面彻底的分析,得出对出错位置及错误性质的推测,再使用一种适当的排错方法来检验推测的正确性。

2. 回溯法

该方法适合于小规模程序的排错。即一旦发现了错误,先分析错误征兆,确定最先发现“症状”的位置。然后,从发现“症状”的地方开始,沿程序的控制流程,逆向跟踪源程序代码,直到找到错误根源或确定错误产生的范围。

回溯法对于小程序很有效,往往能把错误范围缩小到程序中的一小段代码,仔细分析这段代码不难确定出错的准确位置。但随着源代码行数的增加,潜在的回溯路径数目很多,回溯会变得很困难,而且实现这种回溯的开销大。

3. 原因排除法

原因排除法是通过演绎和归纳,以及二分法来实现的。

演绎法是一种从一般原理或前提出发,经过排除和精化的过程来推导出结论的思考方法。演绎法排错是测试人员首先根据已有的测试用例,设想及枚举出所有可能出错的原因作为假设。然后再用原始测试数据或新的测试,从中逐个排除不可能正确的假设。最后,再用测试数据验证余下的假设确定出错的原因。

归纳法是一种从特殊推断出一般的系统化思考方法。其基本思想是从一些线索(错误征兆或与错误发生有关的数据)着手,通过分析寻找到潜在的原因,从而找出错误。

二分法实现的基本思想是,如果已知每个变量在程序中若干个关键点的正确值,则可以使用定值语句(如赋值语句、输入语句等)在程序中的某点附近给这些变量赋正确值,然后运行程序并检查程序的输出。如果输出结果是正确的,则错误原因在程序的前半部分;反之,错误原因在程序的后半部分。对错误原因所在的部分重复使用这种方法,直到将出错范围缩小到容易诊断的程度为止。

上面的每一种方法都可以使用调试工具来辅助完成。例如,可以使用带调试功能的编译器、动态调试器、自动测试用例生成器以及交叉引用工具等。

需要注意的一个实际问题是,调试的成果是排错,为了修改程序中错误,往往会采用“补丁程序”来实现,而这种做法会引起整个程序质量的下降,但是从目前程序设计发展的状况看,对大规模的程序的修改和质量保证,又不失为一种可行的方法。

习题 4

- 在软件生命周期中,能准确地确定软件系统必须做什么和必须具备哪些功能的阶段是
A) 概要设计 B) 详细设计 C) 可行性分析 D) 需求分析
- 下面不属于软件工程的3个要素的是
A) 工具 B) 过程 C) 方法 D) 环境
- 检查软件产品是否符合需求定义的过程称为
A) 确认测试 B) 集成测试 C) 验证测试 D) 验收测试
- 数据流图用于抽象描述一个软件的逻辑模型,数据流图由一些特定的图符构成。下列图符名标识的图符不属于数据流图合法图符的是
A) 控制流 B) 加工 C) 数据存储 D) 源和潭
- 下面不属于软件设计原则的是
A) 抽象 B) 模块化 C) 自底向上 D) 信息隐蔽
- 程序流程图(PFD)中的箭头代表的是
A) 数据流 B) 控制流 C) 调用关系 D) 组成关系
- 下列工具中为需求分析常用工具的是
A) PAD B) PFD C) N-S D) DFD
- 在结构化方法中,软件功能分解属于下列软件开发中的阶段是
A) 详细设计 B) 需求分析 C) 总体设计 D) 编程调试
- 软件调试的目的是
A) 发现错误 B) 改正错误 C) 改善软件的性能 D) 挖掘软件的潜能



10. 软件需求分析阶段的工作,可以分为四个方面:需求获取,需求分析,编写需求规格说明书,以及
- A) 阶段性报告 B) 需求评审 C) 总结 D) 都不正确
11. 下面属于系统软件的是
- A) 户籍管理系统 B) 数据库管理系统 C) 演示软件 D) 杀毒软件
12. 下面对软件生命周期的描述中正确的是
- A) 软件产品从提出、实现、使用维护到停止使用退役的过程
- B) 软件的设计与实现阶段
- C) 软件的开发与管理
- D) 软件的实现和维护

第5章

数据库设计基础

数据库技术是计算机领域的一个重要分支。在计算机应用的三大领域(科学计算、数据处理和过程控制)中,数据处理约占其中的70%,而数据库技术就是作为一门数据处理技术发展起来的。随着计算机应用的普及和深入,数据库技术变得越来越重要了,而了解、掌握数据库系统的基本概念和基本技术是应用数据库技术的前提。本章首先介绍数据库系统的基础知识,然后对基本数据模型进行讨论,特别是其中的E-R模型和关系模型;之后再介绍关系代数及其在关系数据库中的应用,并对关系的规范化理论作了简单说明;最后,较为详细地讨论了数据库的设计过程。

5.1 数据库系统的基本概念

计算机科学与技术的发展,计算机应用的深入与拓展,使得数据库在计算机应用中的地位与作用日益重要,它在商业中、事务处理中占有主导地位。近年来在统计领域、在多媒体领域以及智能化应用领域中的地位与作用也变得十分重要。随着网络应用的普及,它在网络中的应用也日渐重要。因此,数据库已成为构成一个计算机应用系统的重要的支持性软件。

5.1.1 数据、数据库、数据库管理系统

1. 数据

数据(data)实际上就是描述事物的符号记录。

计算机中的数据一般分为两部分,其中一部分与程序仅有短时间的交互关系,随着程序的结束而消亡,它们称为临时性(transient)数据,这类数据一般存放于计算机内存中;而另一部分数据则对系统起着长期持久的作用,它们称为持久性(persistent)数据。数据库系统中处理的就是这种持久性数据。

软件中的数据是有一定结构的。首先,数据有型(type)与值(value)之分,数据的型给出了数据表示的类型,如整型、实型、字符型等,而数据的值给出了符合给定型的值,如整型值15。随着应用需求的扩大,数据的型有了进一步的扩大,它包括了将多种相关数据以一定结构方式组合构成特定的数据框架,这样的数据框架称为数据结构(data structure),数据库中在特定条件下称



之为数据模式(data schema)。

在过去的软件系统中是以程序为主体,而数据则以私有形式从属于程序,此时数据在系统中是分散的、凌乱的,这也造成了数据管理的混乱,如数据冗余度高、数据一致性差以及数据的安全性差等多种弊病。自数据库系统出现以来,数据在软件系统中的地位产生了变化,在数据库系统及数据库应用系统中数据已占有主体地位,而程序已退居附属地位。在数据库系统中需要对数据进行集中、统一的管理,以达到数据被多个应用程序共享的目标。

2. 数据库

数据库(database, DB)是数据的集合,它具有统一的结构形式并存放于统一的存储介质内,是多种应用数据的集成,并可被各个应用程序所共享。

数据库存放数据是按数据所提供的数据库模式存放的,它能构造复杂的数据结构以建立数据间内在联系与复杂的关系,从而构成数据的全局结构模式。

数据库中的数据具有“集成”“共享”之特点,亦即是数据库集中了各种应用的数据,进行统一的构造与存储,而使它们可被不同应用程序所使用。

3. 数据库管理系统

数据库管理系统(database management system, DBMS)是数据库的机构,它是一种系统软件,负责数据库中的数据组织、数据操纵、数据维护、控制及保护和数据服务等。数据库中的数据是具有海量级的数据,并且其结构复杂,因此需要提供管理工具。数据库管理系统是数据库系统的核心,它主要有如下几方面的具体功能:

① 数据模式定义。数据库管理系统负责为数据库构建模式,也就是为数据库构建其数据框架。

② 数据存取的物理构建。数据库管理系统负责为数据模式的物理存取及构建提供有效的存取方法与手段。

③ 数据操纵。数据库管理系统为用户使用数据库中的数据提供方便,它一般提供查询、插入、修改以及删除数据的功能。此外,它自身还具有做简单算术运算及统计的能力,而且还可以与某些过程性语言结合,使其具有强大的过程性操作能力。

④ 数据的完整性、安全性定义与检查。数据库中的数据具有内在语义上的关联性与一致性,它们构成了数据的完整性,数据的完整性是保证数据库中数据正确的必要条件,因此必须经常检查以维护数据的正确。

数据库中的数据具有共享性,而数据共享可能会引发数据的非法使用,因此必须要对数据正确使用作出必要的规定,并在使用时做检查,这就是数据的安全性。

数据完整性与安全性的维护是数据库管理系统的基本功能。

⑤ 数据库的并发控制与故障恢复。数据库是一个集成、共享的数据集合体,它能为多个应用程序服务,所以就存在着多个应用程序对数据库的并发操作。在并发操作中如果不加控制和管理,多个应用程序间就会相互干扰,从而对数据库中的数据造成破坏。因此,数据库管理系统必须对多个应用程序的并发操作做必要的控制以保证数据不受破坏,这就是数据库的并发控制。

数据库中的数据一旦遭受破坏,数据库管理系统必须有能力及时进行恢复,这就是数据库的故障恢复。

⑥ 数据的服务。数据库管理系统提供对数据库中数据的多种服务功能,如数据拷贝、转存、

重组、性能监测、分析等。

为完成以上六个功能,数据库管理系统一般提供相应的数据语言(data language),它们是:

数据定义语言(data definition language, DDL)。该语言负责数据的模式定义与数据的物理存取构建。

数据操纵语言(data manipulation language, DML)。该语言负责数据的操纵,包括查询及增、删、改等操作。

数据控制语言(data control language, DCL)。该语言负责数据完整性、安全性的定义与检查以及并发控制、故障恢复等功能,包括系统初启程序、文件读写与维护程序、存取路径管理程序、缓冲区管理程序、安全性控制程序、完整性检查程序、并发控制程序、事务管理程序、运行日志管理程序、数据库恢复程序等。

上述数据语言按其使用方式具有两种结构形式:

交互式命令语言。它的语言简单,能在终端上即时操作,它又称为自含型或自主型语言。

宿主型语言。它一般可嵌入某些宿主语言(host language)中,如 C/C++、Java 和 COBOL 等高级过程性语言中。

关系数据库中普遍使用了结构化查询语言 SQL(structured query language),该语言是一种介于关系代数和关系演算之间的非过程性操作语言,它不仅具有丰富的查询功能,还兼具数据定义和数据控制功能,是集 DDL、DML 和 DCL 于一体的关系数据库语言。SQL 是高级的非过程性编程语言,允许用户在高层数据结构之上工作。SQL 不要求用户指定对数据的存放方法,也不需要用户了解具体的数据存放方式,所以,具有完全不同底层结构的不同数据库系统都可以使用相同的结构化查询语言作为数据输入与管理的接口。SQL 语言也可以嵌入到其他高级语言中使用。SQL 语言简洁,易学易用,数据统计方便直观,具有极大的灵活性和强大的功能。

此外,数据库管理系统还有为用户提供服务的服务性(utility)程序,包括数据初始装入程序、数据转存程序、性能监测程序、数据库再组织程序、数据转换程序、通信程序等。

目前流行的 DBMS 均为关系数据库系统,比如 Oracle、Sybase 的 PowerBuilder 及 IBM 的 DB2、微软的 SQL Server 及 MySQL 等,它们均为严格意义上的 DBMS 系统。另外有一些小型的数据库,如微软的 Visual FoxPro 和 Access 等,它们只具备数据库管理系统的一些简单功能。

4. 数据库管理员

由于数据库的共享性,对数据库的规划、设计、维护、监视等需要有专人管理,称他们为数据库管理员(database administrator, DBA)。其主要工作如下:

① 数据库设计(database design)。DBA 的主要任务之一是做数据库设计,具体地说是进行数据模式的设计。由于数据库的集成与共享性,因此需要有专门人员(即 DBA)对多个应用的数据需求作全面的规划、设计与集成。

② 数据库维护。DBA 必须对数据库中的数据安全性、完整性、并发控制及系统恢复、数据定期转存等实施与维护。

③ 改善系统性能,提高系统效率。DBA 必须随时监视数据库运行状态,不断调整内部结构,使系统保持最佳状态与最高效率。当效率下降时,DBA 需采取适当的措施,如进行数据库的重组、重构等。



5. 数据库系统

数据库系统(database system, DBS)由如下几部分组成:数据库(数据)、数据库管理系统(软件)、数据库管理员(人员)、系统平台之一——硬件平台(硬件)、系统平台之二——软件平台(软件)。这五个部分构成了一个以数据库为核心的完整的运行实体,称为数据库系统。

在数据库系统中,硬件平台包括:

计算机:它是系统中硬件的基础平台,目前常用的有微型机、小型机、中型机、大型机及巨型机。

网络:过去数据库系统一般建立在单机上,但是近年来它较多的建立在网络上,从目前形势看,数据库系统今后将以建立在网络上为主,而其结构形式又以客户/服务器(C/S)方式与浏览器/服务器(B/S)方式为主。

在数据库系统中,软件平台包括:

操作系统:它是系统的基础软件平台,目前常用的有各种 UNIX(包括 Linux)与 Windows 两种。

数据库系统开发工具:为开发数据库应用程序所提供的工具,它包括过程性程序设计语言如 C/C++、Java 等,也包括可视化开发工具 VB、PB、Delphi 等,它还包括与 Internet Web 有关的 HTML 及 XML 等以及一些专用开发工具。

接口软件:在网络环境下数据库系统中数据库与应用程序,数据库与网络间存在着多种接口,它们需要用接口软件进行连接,否则数据库系统整体就无法运作,这些接口软件包括 ODBC、JDBC、OLEDB、CORBA、COM、DCOM 等。

6. 数据库应用系统(database application system, DBAS)

利用数据库系统进行应用开发可构成一个数据库应用系统,数据库应用系统是数据库系统再加上应用软件及应用界面这三者所组成,具体包括:数据库、数据库管理系统、数据库管理员、硬件平台、软件平台、应用软件、应用界面。其中应用软件是由数据库系统所提供的数据库管理系统(软件)及数据库系统开发工具所书写而成,而应用界面大多由相关的可视化工具开发而成。

数据库应用系统的 7 个部分以一定的逻辑层次结构方式组成一个有机的整体。如果不计数据库管理员(人员)并将应用软件与应用界面记成应用系统,则数据库应用系统的结构如图 5.1 所示。



图 5.1 数据库系统的软硬件层次结构图

下面以一个用户读取某数据记录为例,展示在数据库系统中访问数据的具体执行过程,该过程如图 5.2 所示,对其各个步骤简单说明如下:

① 用户程序中有一条读数据库记录的 DML 语句,当计算机执行到该语句时,即向 DBMS 发出读取相应记录的命令。

② DBMS 接到该命令后,首先访问该用户对应的子模式,检查该操作是否在合法授权范围内及欲读记录的正确性、有效性,若不合法则拒绝执行,并向应用程序状态返回区发出回答状态信息;反之执行下一步。

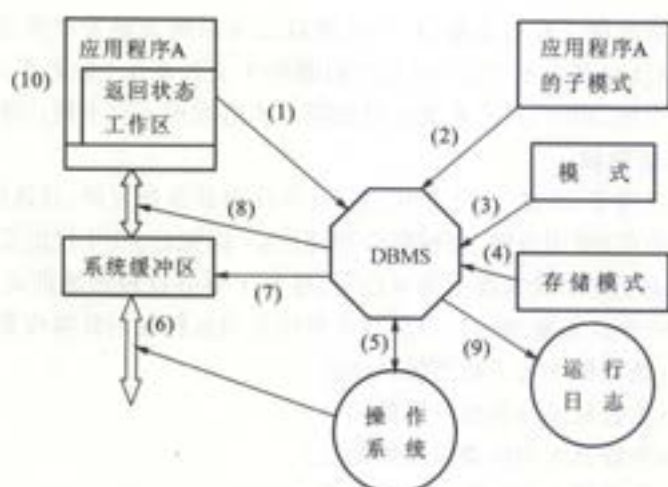


图 5.2 数据库系统访问数据的步骤

③ DBMS 读取模式描述并从子模式映像到全局模式,从而确定所需的逻辑记录类型。

④ DBMS 从逻辑模式映像到存储模式,从而确定读入哪些物理记录以及具体的地址信息。

⑤ DBMS 向操作系统发出从指定地址读取记录的命令。

⑥ 操作系统执行读命令,按指定地址从数据库中把记录读入系统缓冲区,并在操作结束后向 DBMS 作出回答。

⑦ DBMS 按照模式将读入系统缓冲区中的内容映像成用户要求读取的逻辑记录。

⑧ DBMS 将导出的逻辑记录送入用户工作区,并将操作执行情况的状态信息返回给用户。

⑨ DBMS 将已执行的操作载入运行日志。

⑩ 应用程序根据返回的状态信息决定是否利用该数据进行操作等。

如果用户是更新一个记录内容,则执行过程类似。首先读出目标记录,并在用户工作区中进行修改,然后向 DBMS 发出“写回修改数据”的数据库指令即可。

5.1.2 数据库系统的发展

数据管理发展至今已经历了三个阶段:人工管理阶段、文件系统阶段和数据库系统阶段。人工管理阶段是在 20 世纪 50 年代中期以前,主要用于科学计算,硬件无磁盘,直接存取,软件没有操作系统。20 世纪 50 年代后期到 20 世纪 60 年代中期,进入文件系统阶段。20 世纪 60 年代之后,数据管理进入数据库系统阶段。随着计算机应用领域不断扩大,数据库系统的功能和应用范围也愈来愈广,到目前已成为计算机系统的基本及主要的支撑软件。

1. 文件系统阶段

文件系统是数据库系统发展的初级阶段,它提供了简单的数据共享与数据管理能力,但是它无法提供完整的、统一的管理数据和共享的能力。由于它的功能简单,因此它附属于操作系统而不成为独立的软件,目前一般将其看成仅是数据库系统的雏形,而不是真正的数据库系统。

2. 层次数据库与网状数据库系统阶段

从 20 世纪 60 年代末期起,真正的数据库系统——层次数据库与网状数据库开始发展,它们



为统一管理与共享数据提供了有力支撑,这个时期数据库系统蓬勃发展形成了有名的“数据库时代”。但是这两种系统也存在不足,主要是它们脱胎于文件系统,受文件的物理影响较大,对数据库使用带来诸多不便,同时,此类系统的数据模式构造烦琐不宜于推广使用。

3. 关系数据库系统阶段

关系数据库系统出现于20世纪70年代,在80年代得到蓬勃发展,并逐渐取代前两种系统。关系数据库系统结构简单,使用方便,逻辑性强物理性弱,因此在80年代以后一直占据数据库领域的主导地位。但是由于此系统来源于商业应用,适合于事务处理领域而对非事务处理领域应用受到限制,因此在80年代末期兴起与应用技术相结合的各种专用数据库系统。

工程数据库系统:是数据库与工程领域的结合;

图形数据库系统:是数据库与图形应用的结合;

图像数据库系统:是数据库与图像应用的结合;

统计数据库系统:是数据库与工程应用的结合;

知识库系统:是数据库与人工智能应用领域的结合;

分布式数据库系统:是数据库与网络应用的结合;

并行数据库系统:是数据库与多机并行应用的结合;

面向对象数据库系统:是数据库与面向对象方法的结合。

关于数据管理三个阶段中的软硬件背景及处理特点,简单概括在表5.1中。

表5.1 数据管理三个阶段的比较

		人工管理	文件系统	数据库系统
背景	应用背景	科学计算	科学计算、管理	大规模管理
	硬件背景	无直接存取设备	磁盘、磁鼓	大容量磁盘
	软件背景	没有操作系统	有文件系统	有数据库管理系统
	处理方式	批处理	联机实时处理 批处理	联机实时处理 分布处理 批处理
特点	数据管理者	人	文件系统	数据库管理系统
	数据面向对象	某个应用程序	某个应用程序	现实世界
	数据共享程度	无共享 冗余度大	共享性差 冗余度大	共享性大 冗余度小
	数据独立性	不独立,完全依赖于程序	独立性差	具有高度的物理独立性和一定的逻辑独立性
	数据结构化	无结构	记录内有结构 整体无结构	整体结构化,用数据模型描述
	数据控制能力	应用程序自己控制	应用程序自己控制	由DBMS提供数据安全性、完整性、并发控制和恢复

目前,数据库技术也与其他信息技术一样在迅速发展之中,计算机处理能力的增强和越来越广泛的应用是促进数据库技术发展的重要动力。分布式数据库技术是大数据时代云计算技术的基础,是数据的基本存储方式;在大量应用中对数据库管理系统提出了高可靠性、高性能、高可伸缩性(scalability)和高安全性等“四高”要求。一般认为,未来的数据库系统应支持数据管理、对象管理和知识管理,应该具有面向对象的基本特征。在关于数据库的诸多新技术中,下面三种是比较重要的:

面向对象数据库系统:用面向对象方法构筑面向对象数据模型,使其具有比关系数据库系统更为通用的能力;

知识库系统:用人工智能中的方法特别是用谓词逻辑知识表示方法构筑数据模型,使其模型具有特别通用的能力;

关系数据库系统的扩充:利用关系数据库作进一步扩展,使其在模型的表达能力与功能上有进一步的加强,如与网络技术相结合的 Web 数据库、数据仓库及嵌入式数据库等。

5.1.3 数据库系统的基本特点

数据库技术是在文件系统基础上发展产生的,两者都以数据文件的形式组织数据,但由于数据库系统在文件系统之上加入了 DBMS 对数据进行管理,从而使得数据库系统具有以下特点:

1. 数据的集成性

数据库系统的数据集成性主要表现在如下几个方面:

- ① 在数据库系统中采用统一的数据结构方式,如在关系数据库中采用二维表作为统一结构方式。
- ② 在数据库系统中按照多个应用的需要组织全局的、统一的数据结构(即数据模式),数据模式不仅可以建立全局的数据结构,还可以建立数据间的语义联系从而构成一个内在紧密联系的数据整体。

③ 数据库系统中的数据模式是多个应用共同的、全局的数据结构,而每个应用的数据则是全局结构中的一部分,称为局部结构(即视图),这种全局与局部的结构模式构成了数据库系统数据集成性的主要特征。

2. 数据的高共享性与低冗余性

由于数据的集成性使得数据可为多个应用所共享,特别是在网络发达的今天,数据库与网络的结合扩大了数据关系的应用范围。数据的共享自身又可极大地减少数据冗余性,不仅减少了不必要的存储空间,更为重要的是可以避免数据的不一致性。所谓数据的一致性是指在系统中同一数据的不同出现应保持相同的值,而数据的不一致性指的是同一数据在系统的不同拷贝处有不同的值。因此,减少冗余性以避免数据的不同出现是保证系统一致性的基础。

3. 数据独立性

数据独立性是数据与程序间的互不依赖性,即数据库中数据独立于应用程序而不依赖于应用程序。也就是说,数据的逻辑结构、存储结构与存取方式的改变不会影响应用程序。

数据独立性一般分为物理独立性与逻辑独立性两级。

(1) 物理独立性

物理独立性即是数据的物理结构(包括存储结构、存取方式等)的改变,如存储设备的更换、物理存储的更换、存取方式改变等都不影响数据库的逻辑结构,从而不致引起应用程序的变化。

(2) 逻辑独立性



数据库总体逻辑结构的改变,如修改数据模式、增加新的数据类型、改变数据间联系等,不需要相应修改应用程序,这就是数据的逻辑独立性。

4. 数据统一管理与控制

数据库系统不仅为数据提供高度集成环境,同时它还为数据提供统一管理的手段,这主要包含以下三个方面:

- ① 数据的完整性检查:检查数据库中数据的正确性以保证数据的正确。
- ② 数据的安全性保护:检查数据库访问者以防止非法访问。
- ③ 并发控制:控制多个应用的并发访问所产生的相互干扰以保证其正确性。

5.1.4 数据库系统的内部结构体系

数据库系统在其内部具有三级模式及二级映射,三级模式分别是概念级模式、内部级模式与外部级模式,二级映射则分别是概念级到内部级的映射以及外部级到概念级的映射。这种三级模式与二级映射构成了数据库系统内部的抽象结构体系,如图 5.3 所示。



图 5.3 三级模式、两种映射关系图

1. 数据库系统的三级模式

数据模式是数据库系统中数据结构的一种表示形式,它具有不同的层次与结构方式。

(1) 概念模式

概念模式(conceptual schema)是数据库系统中全局数据逻辑结构的描述,是全体用户(应用)公共数据视图。此种描述是一种抽象的描述,它不涉及具体的硬件环境与平台,也与具体的软件环境无关。

概念模式主要描述数据的概念记录类型以及它们间的关系,它还包括一些数据间的语义约束,对它的描述可用 DBMS 中的 DDL 语言定义。

(2) 外模式

外模式(external schema)也称子模式(subschema)或用户模式(user's schema)。它是用户的

数据视图,也就是用户所见到的数据模式,它由概念模式推导而出。概念模式给出了系统全局的数据描述而外模式则给出每个用户的局部数据描述。一个概念模式可以有若干个外模式,每个用户只关心与它有关的模式,这样不仅可以屏蔽大量无关信息而且有利于数据保护。在一般的DBMS中都提供有相关的外模式描述语言(外模式DDL)。

(3) 内模式

内模式(internal schema)又称物理模式(physical schema),它给出了数据库物理存储结构与物理存取方法,如数据存储的文件结构、索引、集簇及Hash等存取方式与存取路径,内模式的物理性主要体现在操作系统及文件级上,它还未深入到设备级上(如磁盘及磁盘操作)。内模式对一般用户是透明的,但它的设计直接影响数据库的性能。DBMS一般提供相关的内模式描述语言(内模式DDL)。

数据模式给出了数据库的数据框架结构,数据是数据库中的真正的实体,但这些数据必须按框架所描述的结构组织,以概念模式为框架所组成的数据库叫概念数据库(conceptual database),以外模式为框架所组成的数据库叫用户数据库(user's database),以内模式为框架所组成的数据库叫物理数据库(physical database)。这三种数据库中只有物理数据库是真实存在于计算机外存中,其他两种数据库并不真正存在于计算机中,而是通过两种映射由物理数据库映射而成。

模式的三个级别层次反映了模式的三个不同环境以及它们的不同要求,其中内模式处于最底层,它反映了数据在计算机物理结构中的实际存储形式,概念模式处于中层,它反映了设计者的数据全局逻辑要求,而外模式处于最外层,它反映了用户对数据的要求。

2. 数据库系统的两级映射

数据库系统的三级模式是对数据的三个级别抽象,它把数据的具体物理实现留给物理模式,使用户与全局设计者不必关心数据库的具体实现与物理背景;同时,它通过两级映射建立了模式间的联系与转换,使得概念模式与外模式虽然并不具备物理存在,但是也能通过映射而获得其实体。此外,两级映射也保证了数据库系统中数据的独立性,亦即数据的物理组织改变与逻辑概念级改变相互独立,使得只要调整映射方式而不必改变用户模式。

(1) 概念模式到内模式的映射

该映射给出了概念模式中数据的全局逻辑结构到数据的物理存储结构间的对应关系,此种映射一般由DBMS实现。

(2) 外模式到概念模式的映射

概念模式是一个全局模式而外模式是用户的局部模式。一个概念模式中可以定义多个外模式,而每个外模式是概念模式的一个基本视图。外模式到概念模式的映射给出了外模式与概念模式的对应关系,这种映射一般也是由DBMS来实现的。

5.2 数据模型

5.2.1 数据模型的基本概念

数据库中的数据模型可以将复杂的现实世界要求反映到计算机数据库中的物理世界,这种



反映是一个逐步转化的过程,它分为两个阶段:由现实世界开始,经历信息世界而至计算机世界,从而完成整个转化。

现实世界(real world):用户为了某种需要,需将现实世界中的部分需求用数据库实现,这样,我们所见到的是客观世界中的划定边界的一个部分环境,它称为现实世界。

信息世界(information world):通过抽象对现实世界进行数据库级上的刻画所构成的逻辑模型叫信息世界。信息世界与数据库的具体模型有关,如层次、网状、关系模型等。

计算机世界(computer world):在信息世界基础上致力于其在计算机物理结构上的描述,从而形成的物理模型叫计算机世界。现实世界的要求只有在计算机世界中才得到真正的物理实现,而这种实现是通过信息世界逐步转化得到的。

数据是现实世界符号的抽象,而数据模型(data model)则是数据特征的抽象,它从抽象层次上描述了系统的静态特征、动态行为和约束条件,为数据库系统的信息表示与操作提供一个抽象的框架。数据模型所描述的内容有三个部分,它们是数据结构、数据操作与数据约束。

(1) 数据结构

数据模型中的数据结构主要描述数据的类型、内容、性质以及数据间的联系等。数据结构是数据模型的基础,数据操作与约束均建立在数据结构上。不同数据结构有不同的操作与约束,因此,一般数据模型的分类均以数据结构的不同而分。

(2) 数据操作

数据模型中的数据操作主要描述在相应数据结构上的操作类型与操作方式。

(3) 数据约束

数据模型中的数据约束主要描述数据结构内数据间的语法、语义联系,它们之间的制约与依存关系,以及数据动态变化的规则,以保证数据的正确、有效与相容。

数据模型按不同的应用层次分成三种类型,它们是概念数据模型(conceptual data model)、逻辑数据模型(logic data model)、物理数据模型(physical data model)。

概念数据模型简称概念模型,它是一种面向客观世界、面向用户的模型;它与具体的数据库管理系统无关,与具体的计算机平台无关。概念模型着重于对客观世界复杂事物的结构描述及它们之间的内在联系的刻画。概念模型是整个数据模型的基础。目前,较为有名的概念模型有E-R模型、扩充的E-R模型、面向对象模型及谓词模型等。

逻辑数据模型又称数据模型,它是一种面向数据库系统的模型,该模型着重于在数据库系统一级的实现。概念模型只有在转换成数据模型后才能在数据库中得以表示。目前,逻辑数据模型也有很多种,较为成熟并先后被人们大量使用过的有:层次模型、网状模型、关系模型、面向对象模型等。

物理数据模型又称物理模型,它是一种面向计算机物理表示的模型,此模型给出了数据模型在计算机上物理结构的表示。

5.2.2 E-R 模型

概念模型是面向现实世界的,它的出发点是有效和自然地模拟现实世界,给出数据的概念化结构。长期以来被广泛使用的概念模型是E-R模型(entity-relationship model)(或实体联系模型),它于1976年由Peter Chen首先提出。该模型将现实世界的要求转化成实体、联系、属性等几个基本概念,以及它们间的两种基本连接关系,并且可以用一种图非常直观地表示出来。

1. E-R 模型的基本概念

(1) 实体

现实世界中的事物可以抽象成为实体,实体是概念世界中的基本单位,它们是客观存在的且又能相互区别的事物。凡是有共性的实体可组成一个集合称为实体集(entity set)。如小赵、小李是实体,他们又均是学生而组成一个实体集。

(2) 属性

现实世界中事物均有一些特性,这些特性可以用属性来表示。属性刻画了实体的特征。一个实体往往可以有若干个属性。每个属性可以有值,一个属性的取值范围称为该属性的值域(value domain)或值集(value set)。如小赵年龄取值为17,小李为19。

(3) 联系

现实世界中事物间的关联称为联系。在概念世界中联系反映了实体集间的一定关系,如工人与设备之间的操作关系,上、下级间的领导关系,生产者与消费者之间的供求关系。

实体集间的联系有多种,就实体集的个数而言有:

① 两个实体集间的联系。两个实体集间的联系是一种最为常见的联系,前面举的例子均属两个实体集间的联系。

② 多个实体集间的联系。这种联系包括三个实体集间的联系以及三个以上实体集间的联系。如工厂、产品、用户这三个实体集间存在着工厂提供产品为用户服务的联系。

③ 一个实体集内部的联系。一个实体集内有若干个实体,它们之间的联系称实体集内部联系。如某公司职工这个实体集内部可以有上、下级联系。

实体集间联系的个数可以是单个也可以是多个。如工人与设备之间有操作联系,另外还可以有维修联系。两个实体集间的联系实际上是实体集间的函数关系,这种函数关系可以有下面几种:

一对一(one to one)的联系,简记为1:1。这种函数关系是常见的函数关系之一,如学校与校长间的联系,一个学校与一个校长间相互一一对应。

一对多(one to many)或多对一(many to one)联系,简记为1:M(1:m)或M:1(m:1)。这两种函数关系实际上是一种函数关系,如学生与其宿舍房间的联系是多对一的联系(反之,则为一对多联系),即多个学生对应一个房间。

多对多(many to many)联系,简记为M:N或m:n。这是一种较为复杂的函数关系,如教师与学生这两个实体集间的教与学的联系是多对多的,因为一个教师可以教授多个学生,而一个学生又可以受教于多个教师。

2. E-R 模型三个基本概念之间的连接关系

E-R 模型由上面三个基本概念组成。由实体、联系、属性三者结合起来才能表示现实世界。

(1) 实体集(联系)与属性间的连接关系

实体是概念世界中的基本单位,属性附属于实体,它本身并不构成独立单位。一个实体可以有若干个属性,实体以及它的所有属性构成了实体的一个完整描述。因此实体与属性间有一定的连接关系。如在人事档案中每个人(实体)可以有:编号、姓名、性别、年龄、籍贯、政治面貌等若干属性,它们组成了一个有关人(实体)的完整描述。

属性有属性域,每个实体可取属性域内的值。一个实体的所有属性取值组成了一个值集叫元组(tuple)。在概念世界中,可以用元组表示实体,也可用它区别不同的实体。如在人事档案



简表 5.2 中,每一行表示一个实体,这个实体可以用一组属性值表示。比如:(101,赵英俊,男,18,浙江,团员),(102,王平,男,21,江苏,党员),这两个元组分别表示两个不同的实体。

实体有型与值之别,一个实体的所有属性构成了这个实体的型,如人事档案中的实体,它的型是由编号、姓名、性别、年龄、籍贯、政治面貌等属性组成,而实体中属性值的集合(即元组)则构成了这个实体的值。

相同型的实体构成了实体集。如表 5.2 中的每一行是一个实体,它们均有相同的型,因此表内诸实体构成了一个实体集。

表 5.2 人事档案简表

编号	姓名	性别	年龄	籍贯	政治面貌
101	赵英俊	男	18	浙江	团员
102	王平	男	21	江苏	党员
103	吴亦奇	女	20	辽宁	群众
104	刘过	男	21	陕西	群众
105	李美丽	女	18	安徽	团员

联系也可以附有属性,联系和它的所有属性构成了联系的一个完整描述,因此,联系与属性间也有连接关系。如有教师与学生两个实体集间的教与学的联系,该联系尚可附有属性“教室号”。

(2) 实体(集)与联系

实体集间可通过联系建立连接关系,一般而言,实体集间无法建立直接关系,它只能通过联系才能建立起连接关系。如教师与学生之间无法直接建立关系,只有通过“教与学”的联系才能在相互之间建立关系。

在 E-R 模型中有三个基本概念以及它们之间的两种基本连接关系。它们将现实世界中的错综复杂的现象抽象成简单明了的几个概念与关系,具有极强的概括性和表达能力。因此,E-R 模型目前已成为表示概念世界的有力工具。

3. E-R 模型的图示法

E-R 模型可以用一种非常直观的图的形式表示,这种图称为 E-R 图(entity-relationship diagram)。在 E-R 图中我们分别用下面不同的几何图形表示 E-R 模型中的三个概念与两个连接关系。

(1) 实体集表示法

在 E-R 图中用矩形表示实体集,在矩形内写上该实体集的名字。如实体集学生(student)、课程(course)可用图 5.4 表示。

(2) 属性表示法

在 E-R 图中用椭圆形表示属性,在椭圆形内写上该属性的名称。如学生有属性:学号(S#)、姓名(Sn)及年龄(Sa),它们可以用图 5.5 表示。

(3) 联系表示法

在 E-R 图中用菱形(内写上联系名)表示联系。如学生与课程间的联系 SC,用图 5.6 表示。



图 5.4 实体集表示法



图 5.5 属性表示法



图 5.6 联系表示法

三个基本概念分别用三种几何图形表示。它们之间的连接关系也可用图形表示。

(4) 实体集(联系)与属性间的连接关系

属性依附于实体集,因此,它们之间有连接关系。在 E-R 图中这种关系可用连接这两个图形间的无向线段表示(一般情况下可用直线)。如实体集 student 有属性 S#(学号)、Sn(学生姓名)及 Sa(学生年龄);实体集 course 有属性 C#(课程号)、Cn(课程名)及 P#(预修课号),此时它们可用图 5.7 连接。

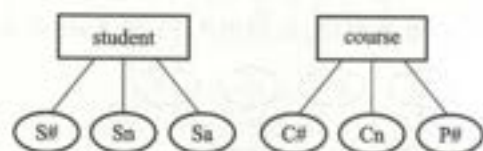


图 5.7 实体集的属性间的连接



图 5.8 联系与属性间的连接

(5) 实体集与联系间的连接关系

在 E-R 图中实体集与联系间的连接关系可用连接这两个图形间的无向线段表示。如实体集 student 与联系 SC 间有连接关系,实体集 course 与联系 SC 间也有连接关系,因此它们之间可用无向线段相连,构成一个如图 5.9 所示的图。

有时为了进一步刻画实体间的函数关系,还可在线段边上注明其对应函数关系,如 1:1, 1:n, n:m 等,如 student 与 course 间有多对多联系,此时在图中可以用图 5.10 所示的形式表示。



图 5.9 实体集与联系间的连接关系



图 5.10 实体集间的联系表示图

实体集与联系间的连接可以有多种,上面所举例子均是两个实体集间联系叫二元联系,也可以是多个实体集间联系,叫多元联系。如工厂、产品与用户间的联系 FPU 是一种三元联系,此种连接关系可用图 5.11 表示。

一个实体集内部可以有联系。如某公司职工(employee)间上、下级管理(manage)的联系,此时,其连接关系可用图 5.12(a)表示。

实体集间可有多种联系。如教师(T)与学生(S)之间可以有教学(E)联系也可有管理(M)联系,此种连接关系可用图 5.12(b)表示。

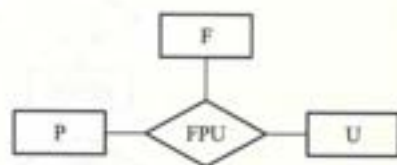


图 5.11 多个实体集间联系的连接方法

由矩形、椭圆形、菱形以及按一定要求相互间连接的线段构成了一个完整的 E-R 图。

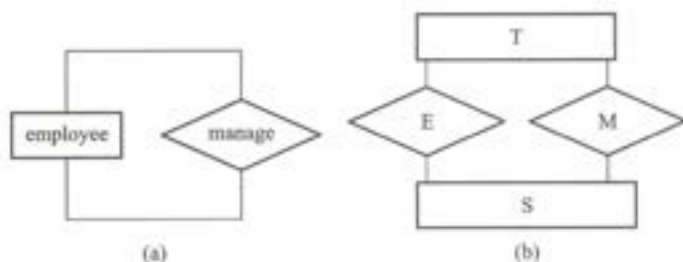


图 5.12 实体集间多种联系

例 5.1 由前面所述的实体集 student, course 以及附属于它们的属性和它们间的联系 SC 以及附属于 SC 的属性 G 构成了一个学生课程联系的概念模型,可用图 5.13 的 E-R 图表示。

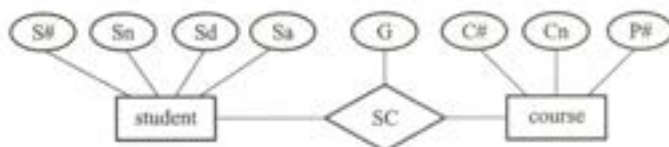


图 5.13 E-R 图的一个实例

在概念上, E-R 模型中的实体、属性与联系是三个有明显区别的不同概念。但是在分析客观世界的具体事物时,对某个具体数据对象,究竟它是实体,还是属性或联系,则是相对的,所做的分析与实际应用的背景以及设计人员的理解有关。这是工程实践中构造 E-R 模型的难点之一。

5.2.3 层次模型

层次模型是最早发展起来的数据库模型。层次模型(hierarchical model)的基本结构是树形结构,这种结构方式在现实世界中很普遍,如家族结构、行政组织机构,它们自顶向下、层次分明。图 5.14 给出了一个学校行政机构图的简化 E-R 图,略去了其中的属性。



图 5.14 一个简化 E-R 图

由图论中树的性质可知,任一树结构均有下列特性:

① 每棵树有且仅有一个无双亲结点,称为根(root)。

② 树中除根外所有结点有且仅有一个双亲。因此,树结构是受到一定限制的,从 E-R 模型观点看,它对于联系也加上了许多限制。

层次数据模型支持的操作主要有查询、插入、删除和更新。在对层次模型进行插入、删除、更新操作时,要满足层次模型的完整性约束条件:进行插入操作时,如果没有相应的双亲结点值就不能插入子女结点值;进行删除操作时,如果删除双亲结点值,则相应的子女结点值也被同时删除;进行更新操作时,应更新所有相应记录,以保证数据的一致性。

层次模型的数据结构比较简单,操作简单;对于实体间联系是固定的且预先定义好的应用系统,层次模型有较高的性能;同时,层次模型还可以提供良好的完整性支持。但由于层次模型形成早,受文件系统影响大,模型受限制多,物理成分复杂,操作与使用均不甚理想,它不适合于表示非层次性的联系;对于插入和删除操作的限制比较多;此外,查询子女结点必须通过双亲结点。

5.2.4 网状模型

网状模型(network model)的出现略晚于层次模型。从图论观点看,网状模型是一个不加任何条件限制的无向图。网状模型在结构上较层次模型好,不像层次模型那样要满足严格的条件。图 5.15 是学校行政机构图中学校与学生联系的简化 E-R 图。

在实现中,网状模型将通用的网络拓扑结构分成一些基本结构。一般采用的分解方法是将一个网络分成若干个二级树,即只有两个层次的树。换句话说,这种树是由一个根及若干个叶所组成。为实现的方便,一般规定根结点与任一叶子结点间的联系均是一对多的联系(包含一对一联系)。

在网状模型的 DBTG(data base task group)标准中,基本结构简单二级树叫系(set),系的基本数据单位是记录(record),它相当于 E-R 模型中的实体(集);记录又可由若干数据项(Data Item)组成,它相当于 E-R 模型中的属性。系有一个首记录(owner record),它相当于简单二级树的根;系同时有若干个成员记录(member record),它相当于简单二级树中的叶;首记录与成员记录之间的联系用有向线段表示(线段方向仅表示由首记录至成员记录的方向,而并不表示搜索方向),在系中首记录与成员记录间是一对多联系(包括一对一联系)。图 5.16 给出了一个系的实例。



图 5.15 一个简化的教学关系 E-R 图

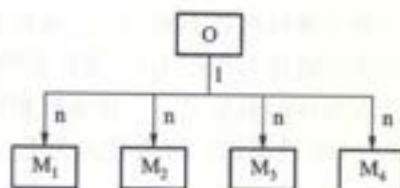


图 5.16 一个系的实例

一般地,现实世界的一个实体结构往往可以由若干个系组成。在网状模型的数据库管理系



统中,一般提供 DDL 语言,用它可以构造系。网状模型中的基本操作是简单二级树中的操作,它包括查询、增加、删除、修改等操作,对于这些操作,不仅需要说明做什么,还需要说明怎么做。比如,在进行查询时,不但要说明查找对象,而且还要规定存取的路径。在 DBTG 报告中,提供了在系上进行操纵的 DML 语言,它们有包括打开(open)、关闭(close)、定位(find)、取(get)、删除(delete)、存储(store)等在内的许多操作。

网状模型明显优于层次模型,不管是数据表示或数据操纵均显示了更高的效率、更为成熟。但是,网状模型数据库系统也有一定的不足,在使用时涉及系统内部的物理因素较多,用户操作使用并不方便,其数据模式与系统实现也均不甚理想。

5.2.5 关系模型

1. 关系的数据结构

关系模型采用二维表来表示,简称表。二维表由表框架(frame)及表的元组(tuple)组成。表框架由 n 个命名的属性(attribute)组成, n 称为属性元数(arity)。每个属性有一个取值范围称为值域(domain)。表框架对应了关系的模式,即类型的概念。

在表框架中按行可以存放数据,每行数据称为元组,实际上,一个元组是由 n 个元组分量所组成,每个元组分量是表框架中每个属性的投影值。一个表框架可以存放 m 个元组, m 称为表的基数(cardinality)。

一个 n 元表框架及框架内 m 个元组构成了一个完整的二维表。表 5.3 给出了有关学生(S)二维表的一个实例。

表 5.3 二维表的一个实例

S#	S _n	S _d	S _a
2001001	张浩然	EE	18
2001002	李一明	EE	19
2001003	王 伟	EE	18
2001004	赵坚强	EE	20

二维表一般满足下面 7 个性质:

- ① 二维表中元组个数是有限的——元组个数有限性。
- ② 二维表中元组均不相同——元组的唯一性。
- ③ 二维表中元组的次序可以任意交换——元组的次序无关性。
- ④ 二维表中元组的分量是不可分割的基本数据项——元组分量的原子性。
- ⑤ 二维表中属性名各不相同——属性名唯一性。
- ⑥ 二维表中属性与次序无关,可任意交换——属性的次序无关性。
- ⑦ 二维表属性的分量具有与该属性相同的值域——分量值域的同—性。

满足以上 7 个性质的二维表称为关系(relation),以二维表为基本结构所建立的模型称为关系模型。

关系模型中的一个重要概念是键(key)或码。键具有标识元组、建立元组间联系等重要作用。在二维表中凡能唯一标识元组的最小属性集称为该表的键或码。

二维表中可能有若干个键,它们称为该表的候选码或候选键(candidate key)。

从二维表的所有候选键中选取一个作为用户使用的键称为主键(primary key)或主码,一般主键也简称键或码。

表A中的某属性集是某表B的键,则称该属性集为A的外键(foreign key)或外码。

表中一定要有键,因为如果表中所有属性的子集均不是键,则表中属性的全集必为键(称为全键),因此也一定有主键。

在关系元组的分量中允许出现空值(null value)以表示信息的空缺。空值用于表示未知的值或不可能出现的值,一般用NULL表示。一般关系数据库系统都支持空值,但是有两个限制,即关系的主键中不允许出现空值,因为如主键为空值则失去了其元组标识的作用;需要定义有关空值的运算。

关系框架与关系元组构成了一个关系。一个语义相关的关系集合构成一个关系数据库(relational database)。关系的框架称为关系模式,而语义相关的关系模式集合构成了关系数据库模式(relational database schema)。

关系模式支持子模式,关系子模式是关系数据库模式中用户所见到的那部分数据模式描述。关系子模式也是二维表结构,关系子模式对应用户数据库称视图(view)。

2. 关系操纵

关系模型的数据操纵即是建立在关系上的数据操纵,一般有查询、增加、删除及修改四种操作。

(1) 数据查询

用户可以查询关系数据库中的数据,它包括一个关系内的查询以及多个关系间的查询。

① 对一个关系内查询的基本单位是元组分量,其基本过程是先定位后操作。所谓定位包括纵向定位与横向定位两部分,纵向定位即是指定关系中的某些属性(称列指定),横向定位即是选择满足某些逻辑条件的元组(称行选择)。通过纵向与横向定位后一个关系中的元组分量即可确定了。在定位后即可进行查询操作,就是将定位的数据从关系数据库中取出并放入至指定内存。

② 对多个关系间的数据查询则可分为三步:第一步,将多个关系合并成一个关系;第二步,对合并后的一个关系作定位;第三步,操作。其中第二步与第三步为对一个关系的查询。对多个关系的合并可分解成两个关系的逐步合并,如有三个关系 R_1, R_2 与 R_3 ,合并过程是先将 R_1 与 R_2 合并成 R_4 ,然后再将 R_4 与 R_3 合并成最终结果 R_5 。

因此,对关系数据库的查询可以分解成一个关系内的属性指定、一个关系内的元组选择、两个关系的合并三个基本定位操作以及一个查询操作。

(2) 数据删除

数据删除的基本单位是一个关系内的元组,它的功能是将指定关系内的指定元组删除。它也分为定位与操作两部分,其中定位部分只需要横向定位而无须纵向定位,定位后即执行删除操作。因此数据删除可以分解为一个关系内的元组选择与关系中元组删除两个基本操作。

(3) 数据插入

数据插入仅对一个关系而言,在指定关系中插入一个或多个元组。在数据插入中不需定位,仅需做关系中元组插入操作,因此数据插入只有一个基本操作。

(4) 数据修改

数据修改是在一个关系中修改指定的元组与属性。数据修改不是一个基本操作,它可以分



解为删除需修改的元组与插入修改后的元组两个更基本的操作。

以上四种操作的对象都是关系,而操作结果也是关系,因此都是建立在关系上的操作。这四种操作可以分解成六种基本操作,称为关系模型的基本操作:

- ① 关系的属性指定;
- ② 关系的元组选择;
- ③ 两个关系合并;
- ④ 一个或多个关系的查询;
- ⑤ 关系中元组的插入;
- ⑥ 关系中元组的删除。

3. 关系中的数据约束

关系模型允许定义三类数据约束,它们是实体完整性约束、参照完整性约束以及用户定义的完整性约束,其中前两种完整性约束由关系数据库系统自动支持。对于用户定义的完整性约束,则由关系数据库系统提供完整性约束语言,用户利用该语言写出约束条件,运行时由系统自动检查。

(1) 实体完整性约束(entity integrity constraint)

该约束要求关系的主键中属性值不能为空值,这是数据库完整性的最基本要求,因为主键是唯一决定元组的,如为空值则其唯一性就成为不可能的了。

(2) 参照完整性约束(reference integrity constraint)

该约束是关系之间相关联的基本约束,它不允许关系引用不存在的元组;即在关系中的外键要么是所关联关系中实际存在的元组,要么就为空值。比如在关系 $S(S\#, SN, SD, SA)$ 与 $SC(S\#, C\#, G)$ 中, SC 中主键为 $(S\#, C\#)$ 而外键为 $S\#$, SC 与 S 通过 $S\#$ 相关联,参照完整性约束要求 SC 中的 $S\#$ 的值必在 S 中有相应元组值,如有 $SC(S13, C8, 70)$,则必在 S 中存在 $S(S13, \dots)$ 。

(3) 用户定义的完整性约束(user defined integrity constraint)

这是针对具体数据环境与应用环境由用户具体设置的约束,它反映了具体应用中数据的语义要求。

实体完整性约束和参照完整性约束是关系数据库所必须遵守的规则,在任何一个关系数据库管理系统(RDBMS)中均由系统自动支持。

5.3 关系代数

关系数据库系统的特点之一是它建立在数学理论的基础之上,有很多数学理论可以表示关系模型的数据操作,其中最为著名的是关系代数(relational algebra)与关系演算(relational calculus)。数学上已经证明两者在功能上是等价的。关系数据库中使用的SQL语言可以支持关系代数和关系演算中的运算和操作。下面将介绍关于关系数据库的理论——关系代数。

1. 关系模型的基本操作

关系是由若干个不同的元组所组成,因此关系可视为元组的集合。 n 元关系是一个 n 元有序组的集合。

设有一个 n 元关系 R , 它有 n 个域, 分别是 $D_1, D_2, \dots, D_n$, 此时, 它们的笛卡尔积是:

$$D_1 \times D_2 \times \dots \times D_n$$

该集合的每个元素都是具有如下形式的 n 元有序组:

$$(d_1, d_2, \dots, d_n) d_i \in D_i (i=1, 2, \dots, n)$$

该集合与 n 元关系 R 有如下联系:

$$R \subseteq D_1 \times D_2 \times \dots \times D_n$$

即 n 元关系 R 是 n 元有序组的集合, 是它的域的笛卡尔积的子集。

关系模型有插入、删除、修改和查询四种操作, 它们又可以进一步分解成六种基本操作:

① 关系的属性指定。指定一个关系内的某些属性, 用它确定关系这个二维表中的列, 它主要用于检索或定位。

② 关系的元组的选择。用一个逻辑表达式给出关系中所满足此表达式的元组, 用它确定关系这个二维表的行, 它主要用于检索或定位。

用上述两种操作即可确定一张二维表内满足一定行、列要求的数据。

③ 两个关系的合并。将两个关系合并成一个关系。用此操作可以不断合并从而可以将若干个关系合并成一个关系, 以建立多个关系间的检索与定位。

用上述三个操作可以进行多个关系的定位。

④ 关系的查询。在一个关系或多个关系间做查询, 查询的结果也为关系。

⑤ 关系元组的插入。在关系中增添一些元组, 用它完成插入与修改。

⑥ 关系元组的删除。在关系中删除一些元组, 用它完成删除与修改。

2. 关系模型的基本运算

由于操作是对关系的运算, 而关系是有序组的集合, 因此, 可以将操作看成是集合的运算。

(1) 插入

设有关系 R 需插入若干元组, 要插入的元组组成关系 R' , 则插入可用集合并运算表示为:

$$R \cup R'$$

(2) 删除

设有关系 R 需删除一些元组, 要删除的元组组成关系 R' , 则删除可用集合差运算表示为:

$$R - R'$$

(3) 修改

修改关系 R 内的元组内容可用下面的方法实现:

① 设需修改的元组构成关系 R' , 则先做删除得:

$$R - R'$$

② 设修改后的元组构成关系 R'' , 此时将其插入即得到结果:

$$(R - R') \cup R''$$

(4) 查询

用于查询的三个操作无法用传统的集合运算表示, 需要引入一些新的运算。

① 投影 (projection) 运算。

对于关系内的域指定可引入新的运算叫投影运算。投影运算是一个一元运算, 一个关系通过投影运算 (并由该运算给出所指定的属性) 后仍为一个关系 R' 。 R' 是这样—个关系, 它是 R 中投影运算所指出的那些域的列所组成的关系。设 R 有 n 个域: $A_1, A_2, \dots, A_n$, 则在 R 上对域



$A_{i_1}, A_{i_2}, \dots, A_{i_n} (A_{i_j} \in \{A_1, A_2, \dots, A_n\})$ 的投影可表示成为下面的一元运算:

$$\pi_{A_{i_1}, A_{i_2}, \dots, A_{i_n}}(R)$$

② 选择(selection)运算。

选择运算也是一个一元运算,关系 R 通过选择运算(并由该运算给出所选择的逻辑条件)后仍为一个关系。这个关系是由 R 中那些满足逻辑条件的元组所组成。设关系的逻辑条件为 F , 则 R 满足 F 的选择运算可写成为:

$$\sigma_F(R)$$

逻辑条件 F 是一个逻辑表达式,它由下面的规则组成。

它可以具有 $\alpha\theta\beta$ 的形式,其中 α, β 是域(变量)或常量,但 α, β 又不能同为常量, θ 是比较符,它可以是 $<, >, \leq, \geq, =$ 及 $\neq$ 。 $\alpha\theta\beta$ 叫基本逻辑条件。

由若干个基本逻辑条件经逻辑运算得到,逻辑运算为 $\wedge$ (并且)、 $\vee$ (或者)及 $\neg$ (否)构成,称为复合逻辑条件。

有了上述两个运算后,我们对一个关系内的任意行、列的数据都可以方便地找到。

③ 笛卡儿积(cartesian product)运算。

对于两个关系的合并操作可以用笛卡儿积表示。设有 n 元关系 R 及 m 元关系 S , 它们分别有 p, q 个元组, 则关系 R 与 S 经笛卡儿积记为 $R \times S$, 该关系是一个 $n+m$ 元关系, 元组个数是 $p \times q$, 由 R 与 S 的有序组组合而成。

表 5.4 给出了两个关系 R, S 的实例以及 R 与 S 的笛卡儿积 $T=R \times S$ 。

表 5.4 关系 R, S 及 $T=R \times S$

R			S		
R_1	R_2	R_3	S_1	S_2	S_3
a	b	c	j	k	l
d	e	f	m	n	o
g	h	i	p	q	r

T=R×S					
R_1	R_2	R_3	S_1	S_2	S_3
a	b	c	j	k	l
a	b	c	m	n	o
a	b	c	p	q	r
d	e	f	j	k	l
d	e	f	m	n	o
d	e	f	p	q	r
g	h	i	j	k	l
g	h	i	m	n	o
g	h	i	p	q	r

3. 关系代数中的扩充运算

关系代数中除了上述几个最基本的运算外,为操纵方便还需增添一些运算,这些运算均可由基本运算导出。常用的扩充运算有交、除、连接及自然连接等。

(1) 交(intersection)运算

关系 R 与 S 经交运算后所得到的关系是由那些既在 R 内又在 S 内的有序组所组成,记为 $R \cap S$ 。

表 5.5 给出了两个关系 R 与 S 及它们经交运算后得到的关系 T 。

表 5.5 关系 R, S 及 $R \cap S$

R				S				$T = R \cap S$			
A	B	C	D	A	B	C	D	A	B	C	D
1	2	3	4	2	2	3	8	1	2	3	4
2	2	5	7	1	2	3	4				
9	0	3	8	9	1	2	3				

交运算可由基本运算推导而得:

$$R \cap S = R - (R - S)$$

(2) 除(division)运算

如果将笛卡儿积运算看作乘运算的话,那么除运算就是它的逆运算。当关系 $T = R \times S$ 时,则可将除运算写成为:

$$T \div R = S \text{ 或 } T/R = S$$

S 称为 T 除以 R 的商(quotient)。

由于除是采用的逆运算,因此除运算的执行是需要满足一定条件的。设有关系 T, R, T 能被除的充分必要条件是: T 中的域包含 R 中的所有属性; T 中有一些域不出现在 R 中。

在除运算中 S 的域由 T 中那些不出现在 R 中的域所组成,对于 S 中任一有序组,由它与关系 R 中每个有序组所构成的有序组均出现在关系 T 中。

表 5.6 给出了关系 R 及一组 S ,对这一组不同的 S 给出了经除法运算后的商 R/S ,从中可以清楚地看出除法的含义及商的内容。

表 5.6 三个除法

R				S		S		S	
A	B	C	D	C	D	C	D	C	D
1	2	3	4	3	4	3	4	3	4
7	8	5	6	5	6			5	6
7	8	3	4					4	2
1	2	5	6						
1	2	4	2						

续表

T		T		T	
A	B	A	B	A	B
1	2	1	2	1	2
7	8	7	8		

除法运算不是基本运算,它可以由基本运算推导而出。设关系 R 有域 $A_1, A_2, \dots, A_n$, 关系 S 有域 $A_{n+1}, A_{n+2}, \dots, A_m$, 此时有:

$$R \div S = \pi_{A_1, A_2, \dots, A_n}(R) - \pi_{A_1, A_2, \dots, A_n}((\pi_{A_1, A_2, \dots, A_n}(R) \times S) - R)$$

除法的定义虽然较复杂,但在实际中,除法的意义还是比较容易理解的。

例 5.2 设关系 R 给出了学生修读课程的情况,关系 S 给出了所有课程号,分别如表 5.7 (a)、(b) 所示。试找出修读所有课程的学生号。

解: 修读所有课程的学生号可用 $T = R \div S$ 表示,结果如表 5.7 (c) 所示。

表 5.7 学生修读课程的除法运算

S#	C#
S ₁	C ₁
S ₁	C ₂
S ₂	C ₁
S ₂	C ₂
S ₂	C ₃
S ₃	C ₂

(a)

C#
C ₁
C ₂
C ₂

(b)

S#
S ₂

(c)

(a)

(b)

(c)

(3) 连接(join)与自然连接(natural join)运算

在数学上,可以用笛卡儿积建立两个关系间的连接,但这样得到的关系庞大,而且数据大量冗余。在实际应用中一般两个相互连接的关系往往须满足一些条件,所得到的结果也较为简单。这样就引入了连接运算与自然连接运算。

连接运算又可称为 θ -连接运算,这是一种二元运算,通过它可以将两个关系合并成一个大关系。设有关系 R, S 以及比较式 $i\theta j$, 其中 i 为 R 中的域, j 为 S 中的域, θ 含义同前。则可以将 R, S 在域 i, j 上的 θ 连接记为:

$$R \bowtie_{i\theta j} S$$

它的含义可用下式定义:

$$R \bowtie_{i\theta j} S = \sigma_{i\theta j}(R \times S)$$

即 R 与 S 的 θ 连接是由 R 与 S 的笛卡儿积中满足限制 $i\theta j$ 的元组构成的关系,一般其元组的数目远远少于 $R \times S$ 的数目。应当注意的是,在 θ 连接中, i 与 j 需具有相同域,否则无法做比较。

在 θ 连接中如果 θ 为 "=", 就称此连接为等值连接,否则称为不等值连接;如 θ 为 "<" 时称

为小于连接;如 θ 为“ $>$ ”时称为大于连接。

例 5.3 设有关系 R、S 分别如表 5.8(a)、(b) 所示,则 $T_1 = R \bowtie_{D=E} S$ 为表 5.8(c) 所示的关系,而 $T_2 = R \bowtie_{D \neq E} S$ 为如表 5.8(d) 所示的关系。

表 5.8 R、S 及 $T_1 = R \bowtie_{D=E} S, T_2 = R \bowtie_{D \neq E} S$

R				S	
A	B	C	D	E	F
1	2	3	4	1	8
3	2	1	8	7	9
7	3	2	1	5	2

(a)

(b)

T_1						T_2					
A	B	C	D	E	F	A	B	C	D	E	F
1	2	3	4	1	8	7	3	2	1	1	8
3	2	1	8	1	8						
3	2	1	8	7	9						
3	2	1	8	5	2						

(c)

(d)

在实际应用中最常用的连接是一个叫自然连接的特例。它满足下面的条件:

- ① 两关系间有公共域;
- ② 通过公共域的相等值进行连接。

设有关系 R、S, R 有域 $A_1, A_2, \dots, A_n$, S 有域 $B_1, B_2, \dots, B_m$, 并且 $A_{i_1}, A_{i_2}, \dots, A_{i_j}$ 与 $B_{j_1}, B_{j_2}, \dots, B_{j_l}$ 分别为相同域, 此时它们自然连接可记为:

$$R \bowtie S$$

自然连接的含义可用下式表示:

$$R \bowtie S = \pi_{A_1, A_2, \dots, A_n, B_{j_1+1}, \dots, B_m} (\sigma_{A_{i_1}=B_{j_1} \wedge A_{i_2}=B_{j_2} \wedge \dots \wedge A_{i_j}=B_{j_l}} (R \times S))$$

例 5.4 设关系 R、S 如表 5.9(a)、(b) 所示, 则 $T = R \bowtie S$ 如表 5.9(c) 所示。

表 5.9 R、S 及 $T = R \bowtie S$

R				S		T				
A	B	C	D	D	E	A	B	C	D	E
1	2	3	4	5	1	2	4	2	6	4
1	5	8	3	6	4	2	4	2	6	8
2	4	2	6	7	3	1	1	4	7	3
1	1	4	7	6	8					

(a)

(b)

(c)



在以上运算中最常用的是投影运算、选择运算、自然连接运算、并运算及差运算。

4. 关系代数的应用实例

关系代数虽然形式简单,但它已经足以表达对表的查询、插入、删除及修改等要求。在所有这些操作中,查询是最复杂的操作。在20世纪70年代,关系数据库系统始终无法走向商品化,最主要的原因就是它的查询效率低下。关系数据库的查询语言一般是非过程语言,即仅仅说明要查询的要求,而不说明如何去进行查询。最终,通过查询优化技术解决了此问题,而对于查询语句(即代数表达式)本身的优化即代数优化是最基本的技术。下面通过一个例子来体会一下如何将关系代数应用于查询。

例 5.5 建立一个学生选课的关系数据库,它由下面三个关系模式组成:

$S(S\#, S_n, S_d, S_a)$;

$C(C\#, C_n, P\#)$;

$SC(S\#, C\#, G)$ 。

其中 $S\#, C\#, S_n, S_d, S_a, C_n, P\#, G$ 分别表示学号、课程号、学生姓名、学生系别、学生年龄、课程名、预修课程号、成绩,而 S, C, SC 则分别表示学生、课程、学生选课关系。

写出对关系模式 S, C 和 SC 中的下述查询表达式:

① 检索学生所有情况:

S

② 检索学生年龄大于等于20岁的学生姓名:

$\pi_{S_n}(\sigma_{S_a \geq 20}(S))$

③ 检索预修课号为 C_2 的课程课程号:

$\pi_{C\#}(\sigma_{P\# = C_2}(C))$

④ 检索课程号为 C ,且成绩为 A 的所有学生姓名:

$\pi_{S_n}(\sigma_{C\# = C \wedge G = A}(S \bowtie SC))$

注意:这是一个涉及两个关系的检索,此时需用连接运算。

⑤ 检索 s_1 所修读的所有课程名及其预修课号:

$\pi_{C_n, P\#}(\sigma_{S\# = s_1}(C \bowtie SC))$

⑥ 检索年龄为23岁的学生所修读的课程名:

$\pi_{C_n}(\sigma_{S_a = 23}(S \bowtie SC \bowtie C))$

注意:这是涉及三个关系的检索。

⑦ 检索至少修读 S_1 所修读的一门课的学生姓名。

这个例子比较复杂,需作一些分析。将问题分以下三步解决:

第1步:取得 S_1 修读的课程号,它可以表示为:

$R = \pi_{C\#}(\sigma_{S\# = S_1}(SC))$

第2步:取得至少修读为 S_1 修读的一门课的学号:

$W = \pi_{S\#}(SC \bowtie R)$

第3步:最后结果为:

$\pi_{S_n}(S \bowtie W)$

分别将 R、W 代入后即得检索要求之表达式:

$$\pi_{S_1}(S \bowtie (\pi_{S_2}(SC \bowtie (\pi_{C_1}(\sigma_{S_2=S_2}(SC))))))$$

对于一般较为复杂的查询,都是通过这样多步来解决的。注意到该过程中会产生一些中间的表,而查询优化中一般应尽可能使这些中间表比较小。

5.4 数据库设计与管理

数据库设计是数据库应用的核心。本节讨论数据库设计的任务特点、基本步骤和方法,重点介绍数据库的需求分析、概念设计及逻辑设计三个阶段,并用实际例子说明如何进行相关的设计。此外本节还简单讨论数据库管理的内容及 DBA 的工作。

5.4.1 数据库设计概述

在数据库应用系统中的一个核心问题就是设计一个能满足用户要求,性能良好的数据库,这就是数据库设计(database design)。

数据库设计的基本任务是根据用户对象的信息需求、处理需求和数据库的支持环境(包括硬件、操作系统与 DBMS)设计出数据模式。所谓信息需求主要是指用户对象的数据及其结构,它反映了数据库的静态要求;所谓处理需求则表示用户对象的行为和动作,它反映了数据库的动态要求。数据库设计中有一定的制约条件,它们是系统设计平台,包括系统软件、工具软件以及设备、网络等硬件。因此,数据库设计即是在一定平台制约下,根据信息需求与处理需求设计出性能良好的数据模式。

在数据库设计中有两种方法,一种是以信息需求为主,兼顾处理需求,称为面向数据的方法(data-oriented approach);另一种方法是以处理需求为主,兼顾信息需求,称为面向过程的方法(process-oriented approach)。这两种方法目前都有使用,在早期由于应用系统中处理多于数据,因此以面向过程的方法使用较多,而近期由于大型系统中数据结构复杂、数据量庞大,而相应处理流程趋于简单,因此用面向数据的方法较多。由于数据在系统中稳定性高,数据已成为系统的核心,因此面向数据的设计方法已成为主流方法。按照传统的系统设计方法,应用程序设计与数据库设计是分别进行的,两项设计完成之后再进行协调。目前,随着面向对象技术及统一建模语言(unified modeling language, UML)的广泛使用,采用 UML 进行系统分析和设计,把系统的应用程序设计和数据库设计统一起来,可有效地提高数据库设计的效率和质量,降低开发风险,提高软件部件的可重用性,降低开发成本。

数据库设计目前一般采用生命周期(life cycle)法,即将整个数据库应用系统的开发分解成目标独立的若干阶段。它们是:需求分析阶段、概念设计阶段、



图 5.17 数据库设计的四个阶段



逻辑设计阶段、物理设计阶段、编码阶段、测试阶段、运行阶段、进一步修改阶段。在数据库设计中采用上面几个阶段中的前四个阶段,并且重点以数据结构与模型的设计为主线,如图 5.17 所示。

5.4.2 数据库设计的需求分析

需求收集和分析是数据库设计的第一阶段,这一阶段收集到的基础数据和一组数据流图(data flow diagram, DFD)是下一步设计概念结构的基础。概念结构是整个组织中所有用户关心的信息结构,对整个数据库设计具有深刻影响。而要设计好概念结构,就必须在需求分析阶段用系统的观点来考虑问题、收集和分析数据及其处理。

需求分析阶段的任务是通过详细调查现实世界要处理的对象(组织、部门、企业等),充分了解原系统的工作概况,明确用户的各种需求,然后在此基础上确定新系统的功能。新系统必须充分考虑今后可能的扩充和改变,不能仅按当前应用需求来设计数据库。

调查的重点是“数据”和“处理”,通过调查要从中获得每个用户对数据库的如下要求:

① 信息要求。指用户需要从数据库中获得信息的内容与性质。由信息要求可以导出数据要求,即在数据库中需存储哪些数据。

② 处理要求。指用户要完成什么处理功能,对处理的响应时间有何要求,处理的方式是批处理还是联机处理。

③ 安全性和完整性的要求。为了很好地完成调查的任务,设计人员必须不断地与用户交流,与用户达成共识,以便逐步确定用户的实际需求,然后分析和表达这些需求。需求分析是整个设计活动的基础,也是最困难、最花时间的一步。需求分析人员既要懂得数据库技术,又要对应用环境的业务比较熟悉。

分析和表达用户的需求,经常采用的方法有结构化分析方法和面向对象的方法。结构化分析(structured analysis, 简称 SA 方法)方法用自顶向下、逐层分解的方式分析系统。用数据流图表达了数据和处理过程的关系,数据字典对系统中数据的详尽描述,是各类数据属性的清单。对数据库设计来讲,数据字典是进行详细的数据收集和数据分析所获得的主要结果。

数据字典是各类数据描述的集合,它通常包括 5 个部分,即数据项,是数据的最小单位;数据结构,是若干数据项有意义的集合;数据流,可以是数据项,也可以是数据结构,表示某一处理过程的输入或输出;数据存储,处理过程中存取的数据,常常是手工凭证、手工文档或计算机文件;处理过程。

数据字典是在需求分析阶段建立,在数据库设计过程中不断修改、充实、完善的。

在实际开展需求分析工作时有两点需要特别注意:

第一,在需求分析阶段一个重要而困难的任务是收集将来应用所涉及的数据。若设计人员仅仅按当前应用来设计数据库,新数据的加入不仅会影响数据库的概念结构,而且将影响逻辑结构和物理结构,因此设计人员应充分考虑到可能的扩充和改变,使设计易于更动。

第二,必须强调用户的参与,这是数据库应用系统设计的特点。数据库应用系统和广泛的用户有密切的联系,其设计和建立又可能对更多人的工作环境产生重要影响。因而,设计人员应该和用户充分合作进行设计,并对设计工作的最后结果承担共同的责任。

5.4.3 数据库概念设计

1. 数据库概念设计概述

数据库概念设计的目的是分析数据间内在语义关联,在此基础上建立一个数据的抽象模型。数据库概念设计的方法有以下两种:

(1) 集中式模式设计法

这是一种统一的模式设计方法,它根据需求由一个统一机构或人员设计一个综合的全局模式。这种方法设计简单方便,它强调统一与一致,适用于小型或并不复杂的单位或部门,而对大型的或语义关联复杂的单位则并不适合。

(2) 视图集成设计法

这种方法是将一个单位分解成若干个部分,先对每个部分作局部模式设计,建立各个部分的视图,然后以各视图为基础进行集成。在集成过程中可能会出现一些冲突,这是由于视图设计的分散性形成的不一致所造成的,因此需对视图作修正,最终形成全局模式。

视图集成设计法是一种由分散到集中的方法,它的设计过程复杂但它能较好地反映需求,适合于大型与复杂的单位,避免设计的粗糙与不周到,目前此种方法使用较多。

2. 数据库概念设计的过程

使用 E-R 模型与视图集成法进行设计时,需要按以下步骤进行:首先选择局部应用,再进行局部视图设计,最后对局部视图进行集成得到概念模式。

(1) 选择局部应用

根据系统的具体情况,在多层的数据流图中选择一个适当层次的数据流图,让这组图中每一部分对应一个局部应用,以这一层次的数据流图出发点,设计分 E-R 图。

(2) 视图设计

视图设计一般有三种设计次序,它们是:

① 自顶向下。这种方法是先从抽象级别高且普遍性强的对象开始逐步细化、具体化与特殊化,如学生这个视图可先从一般学生开始,再分成大学生、研究生等,进一步再由大学生细化为大学本科与专科,研究生细化为硕士生与博士生等,还可以再细化成学生姓名、年龄、专业等细节。

② 由底向上。这种设计方法是先从具体的对象开始,逐步抽象,普遍化与一般化,最后形成一个完整的视图设计。

③ 由内向外。这种设计方法是先从最基本与最明显的对象着手逐步扩充至非基本、不明显的其他对象,如学生视图可从最基本的学生开始逐步扩展至学生所读的课程、上课的教室与任课的教师等其他对象。

上面 3 种方法为视图设计提供了具体的操作方法,设计者可根据实际情况灵活掌握,可以单独使用也可混合使用。有某些共同特性和行为的对象可以抽象为一个实体。对象的组成成分可以抽象为实体的属性。

在进行设计时,实体与属性是相对而言的。同一事物,在一种应用环境中作为“属性”,在另一种应用环境中就必须作为“实体”。但是,在给定的应用环境中,属性必须是不可分的数据项,属性不能与其他实体发生联系,联系只发生在实体之间。

例 5.6 学籍管理局部应用中主要涉及的实体包括学生、宿舍、档案材料、班级、班主任。这



些实体之间的联系有:

① 一个宿舍可以住多个学生,一个学生只能住在一个宿舍中,因此宿舍与学生之间是 1:N 的联系。

② 一个班有若干名学生,一个学生只能属于一个班级,因此班级与学生之间也是 1:N 的联系。

③ 班主任与学生之间是 1:N 的联系。

④ 学生和他自己的档案材料之间是 1:1 的联系。

⑤ 班级与班主任之间都是 1:1 的联系。

于是,省略了实体的属性后学籍管理的 E-R 图如图 5.18 所示。

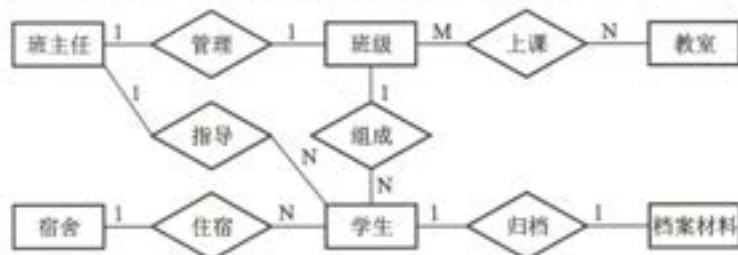


图 5.18 学籍管理局部 E-R 图

对应于各实体的属性分别为:

学生: |学号, 姓名, 出生日期, 所在系, 何时入学, 平均成绩|

档案材料: |档案号, ...|

班级: |班级号, 学生人数|

班主任: |职工号, 姓名, 性别, 是否为优秀班主任|

宿舍: |宿舍编号, 地址, 人数|

教室: |教室编号, 地址, 容量|

其中有下画线的属性为实体的码。

例 5.7 课程管理局部视图的设计: 在这一视图中共有五个实体, 分别是学生、课程、教室、教师及教科书, 描述这些实体的属性分别为:

学生: |学号, 姓名, 年龄, 性别, 入学时间|

课程: |课程号, 课程名, 学时数|

选修: |学号, 课程号, 成绩|

教科书: |书号, 书名, ISBN, 作者, 出版时间, 关键字|

教室: |教室编号, 地址, 容量|

同样, 省略了实体的属性后课程管理的 E-R 图如图 5.19 所示。

(3) 视图集成

视图集成的实质是将所有的局部视图统一与合并成一个完整的数据模式。在进行视图集成时, 最重要的工作便是解决局部设计中的冲突。在集成过程中由于每个局部视图在设计时的一致性因而会产生矛盾, 引起冲突, 常见冲突有下列几种:

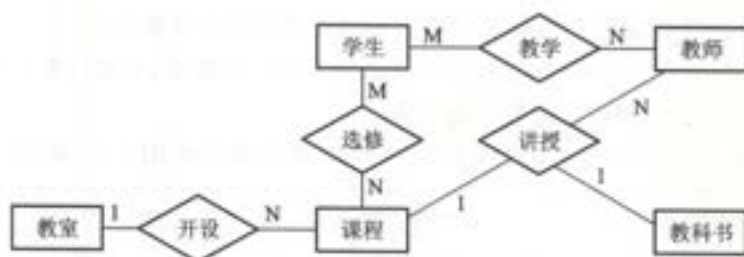


图 5.19 课程管理局部 E-R 图

① 命名冲突。命名冲突有同名异义和同义异名两种。如上面的实例中学生属性“何时入学”与“入学时间”属同义异名。

② 概念冲突。同一概念在一处为实体而在另一处为属性或联系。

③ 域冲突。相同的属性在不同视图中有不同的域，如学号在某视图中的域为字符串而在另一个视图中可为整数，有些属性采用不同度量单位也属域冲突。

④ 约束冲突。不同的视图可能有不同的约束。

视图经过合并生成的是初步 E-R 图，其中可能存在冗余的数据和冗余的实体间联系。冗余数据和冗余联系容易破坏数据库的完整性，给数据库维护增加困难。因此，对于视图集成后所形成的整体的数据库概念结构还必须进行进一步验证，确保它能够满足下列条件：

整体概念结构内部必须具有一致性，即不能存在互相矛盾的表达；

整体概念结构能准确地反映原来的每个视图结构，包括属性、实体及实体间的联系；

整体概念结构能满足需求分析阶段所确定的所有要求；

整体概念结构最终还应该提交给用户，征求用户和有关人员的意见，进行评审、修改和优化，然后把它确定下来，作为数据库的概念结构，作为进一步设计数据库的依据。

例 5.8 学籍管理局部视图与课程管理局部视图的集成。

根据上面所述的方法，集成过程可按下面步骤进行：

(1) 消除冲突

这两个子 E-R 图存在着多方面的冲突：

① 班主任也属于教师，学籍管理中的班主任实体与课程管理中的教师实体属于异名同义，可以统一称为教师。

② 将班主任改为教师后，教师与学生之间呈现两种不同类型的联系：指导联系和教学联系。由于指导联系实际上可以包含在教学联系之中，因此可以将这两种联系综合为教学联系。

③ 调整学生实体属性组成及次序，调整结果可为：

学生：|学号，姓名，出生日期，年龄，所在系，年级，平均成绩|

(2) 消除冗余

① 学生实体中的年龄可以由出生日期推算出来，属于冗余数据。

学生：|学号，姓名，出生日期，所在系，年级，平均成绩|

② 教室实体与班级实体之间的上课联系可以由教室与课程之间的开设联系、课程与学生之间的选修联系、学生与班级之间的组成联系三者推导出来，因此属于冗余联系。



③ 学生实体中的平均成绩可以从选修联系中的成绩属性中推算出来。

如果需要经常查询学生的平均成绩,可以考虑保留该冗余数据,提高效率。但是为了维护数据一致性,应采用一定的机制以保持数据的一致。

这样,集成这两个子 E-R 图后的学生管理子系统的 E-R 图如图 5.20 所示。

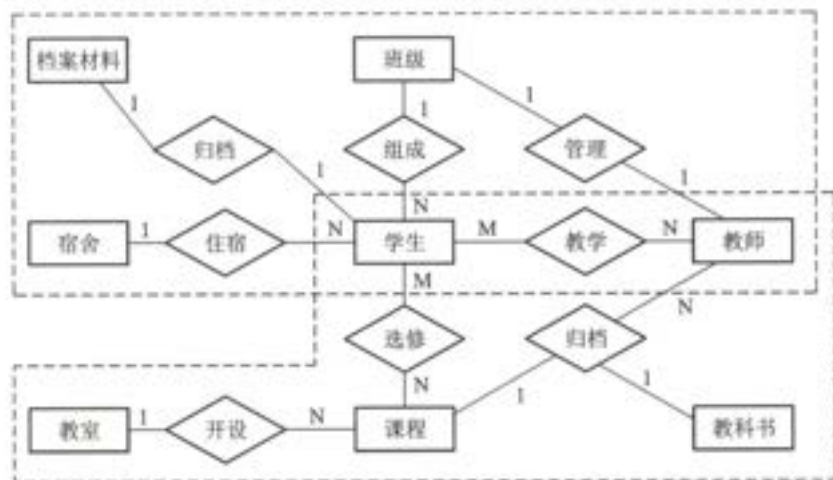


图 5.20 学生管理子系统的 E-R 图

学生管理子系统的基本 E-R 图还必须进一步和教师管理子系统以及后勤管理子系统等的 E-R 图合并,才能生成整个学校管理系统的基本 E-R 图。

5.4.4 数据库的逻辑设计

1. 从 E-R 图向关系模式转换

数据库的逻辑设计主要工作是将 E-R 图转换成指定 RDBMS 中的关系模式。首先,从 E-R 图到关系模式的转换是比较直接的,实体与联系都可以表示成关系,E-R 图中属性也可以转换成关系的属性。实体集也可以转换成关系。E-R 模型与关系间的转换如表 5.10 所示。

表 5.10 E-R 模型与关系间的比较表

E-R 模型	关系	E-R 模型	关系
属性	属性	实体集	关系
实体	元组	联系	关系

下面讨论由 E-R 图转换成关系模式时会遇到的一些转换问题。

(1) 命名与属性域的处理

关系模式中的命名可以用 E-R 图中原有命名,也可另行命名,但是应尽量避免重名,RDBMS 一般只支持有限种数据类型而 E-R 中的属性域则不受此限制,如出现有 RDBMS 不支持的数据类型时则要进行类型转换。

(2) 非原子属性处理

E-R 图中允许出现非原子属性,但在关系模式中一般不允许出现非原子属性,非原子属性主要有集合型和元组型。如出现此种情况时可以进行转换,其转换办法是集合属性纵向展开而元组属性则横向展开。

例 5.9 学生实体有学号、学生姓名及选读课程,其中前两个为原子属性而后一个为集合型非原子属性,因为一个学生可选读若干课程,设有学生 S1307,王承志,他修读 Database, Operating System 及 Computer Network 三门课,此时可将其纵向展开用关系形式如表 5.11 所示。

表 5.11 学生实体

学号	学生姓名	选读课程
S1307	王承志	Database
S1307	王承志	Operating System
S1307	王承志	Computer Network

(3) 联系的转换

在一般情况下联系可用关系表示,但是在有些情况下联系可归并到相关联的实体中。

2. 逻辑模式规范化及调整、实现

(1) 规范化

关系数据库设计的关键是关系数据库模式的设计,即确定构造几个关系模式及每一模式各自包含的属性,将相互关联的模式组成合适的关系模型。关系数据库的设计必须在关系数据库规范化理论的指导下进行。

设计不良的关系模式会有数据冗余、插入异常、删除异常及修改异常等问题。下面将通过一个例子展示设计不好的模式会出现的问题,以及如何通过分解的方法来进行规范化,设计出相对合理的关系模式。

例 5.10 对图 5.13 的 E-R 图,在实体 student 中再加入属性 De(系主任)。为这一扩展后的 E-R 图设计了关系模式 SC(S#,Sn,Sd,De,Sa,C#,Cn,P#,G)。从语义分析和关系表的属性可知表的关键字是复合关键字(S#,C#)。表 5.12 是这一关系的一个实例。

表 5.12 不当学生选课表关系 SC

S#	Sn	Sd	De	Sa	C#	Cn	P#	G
2001001	张浩然	EE	李槐	18	C001	数据结构	-	90
2001001	张浩然	EE	李槐	18	C010	操作系统	C001	92
2001002	李一明	EE	李槐	19	C001	数据结构	-	93
2001002	李一明	EE	李槐	19	C010	操作系统	C001	89
2001003	王伟	EE	李槐	18	C010	数据库	C001	89

显然,这样设计的关系中存在如下的问题:

① 数据冗余:表中每个学生相关的信息会多次出现,选几门课就会出现几次。而每门课的信息也会重复多次,有多少学生选就出现多少次;



② 插入异常:如果有学生当前没有选课,则该学生无法插入到表中。类似地,如果有课程没有学生选修,课程也不能插入到表中;

③ 删除异常:如果一门课只有一个学生选修,则删除学生会同时删掉课程。反之,删掉课程也会同时删除学生;

④ 修改异常:如果某学生改名,则该学生的所有记录都要逐一修改,一旦某一记录漏改,就会造成数据的不一致,比如对学生王伟和课程数据库的记录。

解决这一问题的方法是对关系通过分解进行规范化,其中进行分解的依据是关系属性之间的函数依赖。函数依赖就是一个属性集依赖于别的属性集,或一个属性集决定别的属性集。属性集 Y 依赖于属性集 X 记为 $X \rightarrow Y$,比如上面的关系中学生所在系依赖于学号,即 $S\# \rightarrow Sd$,课程名称依赖于课号,即 $C\# \rightarrow Cn$ 。

对于关系模式,若其中的每个属性都已不能再分为简单项,则它属于第一范式模式(1NF——first normal form),比如关系 SC 已经是第一范式了。如果某个关系模式 R 为第一范式,并且 R 中每一个非主属性完全函数依赖于 R 的某个候选键,则称其为第二范式模式(2NF——second normal form)。第二范式消除了非主属性对主键的部分依赖。对上面的模式 SC,主键为复合键($S\#, C\#$),但显然有 $S\# \rightarrow Sd, S\# \rightarrow Sa, S\# \rightarrow De$,以及 $C\# \rightarrow Cn, C\# \rightarrow P\#$ 等,存在非主属性对主属性的部分依赖。对上述模式进行如下的分解,就可以消除对非主属性的部分依赖:

$S1(S\#, Sn, Sd, De, Sa)$

$C(C\#, Cn, P\#)$

$SC1(S\#, C\#, G)$

此时就把原来的关系 SC 分解成了第二范式。

但分解后的第二范式仍然存在一些问题,比如系主任的名字在表中仍会多次重复。造成这一数据冗余的原因是属性 De 对主属性 $S\#$ 的传递依赖。在关系模式中,如果 $Y \rightarrow X, X \rightarrow A$,且 X 不决定 Y 和 A 不属于 X ,那么 $Y \rightarrow A$ 是传递依赖。对关系模式 $S1$,学生所在系依赖于学号($S\# \rightarrow Sd$),但系本身就确定了系主任($Sd \rightarrow De$),所以此时属性 De 传递依赖于主属性 $S\#$ 。

如果关系模式 R 是第二范式,并且每个非主属性都不传递依赖于 R 的候选键,则称 R 为第三范式模式(3NF——third normal form)。把传递依赖于主属性的属性放到另外一个关系中,消除传递依赖,如上面 $S1$ 中把属性 De 放到另外一个表中,得到下面的关系:

$S2(S\#, Sn, Sd, De, Sa)$

$D(Sd, De)$

$C(C\#, Cn, P\#)$

$SC1(S\#, C\#, G)$

该关系即是第三范式。在大部分应用中都需要将关系分解为 3NF,否则数据冗余太大。比 3NF 更高级的范式是 BCNF(Boyce-Codd normal form),它要求所有属性都不传递依赖于关系的任何候选键。在实际应用中,并不一定要求全部模式都达到 BCNF 不可,有时故意保留部分冗余可能更方便对数据查询,尤其对于那些更新频度不高、查询频度极高的数据关系更是如此。

关系模式进行规范化的目的是使关系结构更合理,消除存储异常,使数据冗余尽量小,便于插入、删除和更新等操作。关系模式进行规范化的原则是:遵从概念单一化“一事一地”原则,即一个关系模式描述一个实体或实体间的一种联系。规范化的实质就是概念的单一化。

(2) RDBMS

对逻辑模式进行调整以满足 RDBMS 的性能、存储空间等要求,同时对模式做适应 RDBMS 限制条件的修改,它们包括如下内容:

- ① 调整性能以减少连接运算;
- ② 调整关系大小,使每个关系数量保持在合理水平,从而可以提高存取效率;
- ③ 尽量采用快照(snapshot),因在应用中经常仅需某固定时刻的值,此时可用快照将某时刻值固定,并定期更换,此种方式可以显著提高查询速度。

3. 关系视图设计

逻辑设计的另一个重要内容是关系视图的设计,它又称为外模式设计。关系视图是在关系模式基础上所设计的直接面向操作用户的视图,它可以根据用户需求随时创建,一般 RDBMS 均提供关系视图的功能。

关系视图的作用大致有如下几点:

- ① 提供数据逻辑独立性:使应用程序不受逻辑模式变化的影响。数据的逻辑模式会随着应用的发展而不断变化,逻辑模式的变化必然会影响到应用程序的变化,这就会产生极为麻烦的维护工作。关系视图则起了逻辑模式与应用程序之间的隔离墙作用,有了关系视图后建立在其上的应用程序就不会随逻辑模式修改而产生变化,此时变动的仅是关系视图的定义。
- ② 能适应用户对数据的不同需求:每个数据库有一个非常庞大的结构,而每个数据库用户则希望只知道他们自己所关心的那部分结构,不必知道数据的全局结构以减轻用户在此方面的负担。此时,可用关系视图屏蔽用户所不需要的模式,而仅将用户感兴趣的部分呈现出来。
- ③ 有一定数据保密功能:关系视图为每个用户划定了访问数据的范围,从而在应用的各用户间起了一定的保密隔离作用。

5.4.5 数据库的物理设计

数据库物理设计的主要目标是对数据库内部物理结构做调整并选择合理的存取路径,以提高数据库访问速度及有效利用存储空间。在现代关系数据库中已大量屏蔽了内部物理结构,因此留给用户参与物理设计的余地并不多,一般的 RDBMS 中留给用户参与物理设计的内容大致有如下几种:索引设计、集簇设计和分区设计。

5.4.6 数据库管理

数据库是一种共享资源,它需要维护与管理,这种工作称为数据库管理(database administration),而实施此项管理的人则称为数据库管理员(DBA)。数据库管理一般包含如下一些内容:数据库的建立、数据库的调整、数据库的重组、数据库的安全性控制与完整性控制、数据库的故障恢复和数据库的监控。下面对这些管理内容作简单讨论。

1. 数据库的建立

数据库的建立包括两部分内容,数据模式的建立及数据加载。

- ① 数据模式建立。数据模式由 DBA 负责建立,DBA 利用 RDBMS 中的 DDL 语言定义数据库名,定义表及相应属性,定义主关键字、索引、集簇、完整性约束、用户访问权限,申请空间资源,定义分区等,此外还需定义视图。



② 数据加载。在数据模式定义后即可加载数据, DBA 可以编制加载程序将外界数据加载至数据模式内, 从而完成数据库的建立。

2. 数据库的调整

在数据库建立并经一段时间运行后往往会产生一些不适应的情况, 此时需要对其做调整, 数据库的调整一般由 DBA 完成, 调整包括下面一些内容:

- ① 调整关系模式与视图使之更能适应用户的需求;
- ② 调整索引与集簇使数据库性能与效率更佳;
- ③ 调整分区、数据库缓冲区大小以及并发度使数据库物理性能更好。

3. 数据库的重组

数据库在经过一定时间运行后, 其性能会逐步下降, 下降的原因主要是由于不断的修改、删除与插入所造成的。由于不断的删除而造成盘区内废块的增多而影响 I/O 速度, 由于不断的删除与插入而造成集簇的性能下降, 同时也造成了存储空间分配的零散化, 使得一个完整表的空间分散, 从而造成存取效率下降。基于这些原因需要对数据库进行重新整理, 重新调整存储空间, 此种工作叫数据库重组。一般数据库重组需花大量时间, 并做大量的数据搬迁工作。实际中, 往往是先做数据卸载, 然后再重新加载从而达到数据重组的目的。目前一般 RDBMS 都提供一定手段, 以实现数据重组功能。

4. 数据库安全性控制与完整性控制

数据库是一个单位的重要资源, 它的安全性是极端重要的, DBA 应采取措施保证数据不受非法盗用与破坏。此外, 为保证数据的正确性, 使录入库内的数据均能保持正确, 需要有数据库的完整性控制。

5. 数据库的故障校正

一旦数据库中的数据遭受破坏, 需要及时恢复, RDBMS 一般都提供此种功能, 并由 DBA 负责执行故障恢复功能。

6. 数据库监控

DBA 需随时观察数据库的动态变化, 并在发生错误、故障或产生不适应情况时随时采取措施, 如数据库死锁、对数据库的误操作等; 同时还需监视数据库的性能变化, 在必要时对数据库做调整。

习题 5

1. 数据库系统的核心是
A) 数据库 B) 数据库管理系统 C) 数据库管理员 D) 数据模型
2. 在关系数据库中, 用户所见的数据模式是数据库的
A) 概念模式 B) 外模式 C) 内模式 D) 物理模式
3. 下面属于数据库逻辑模型的是
A) 关系模型 B) 谓词模型 C) 物理模型 D) 实体-联系模型
4. 数据库系统中完成查询操作使用的语言是

A) 数据控制语言 B) 数据定义语言

C) 数据操纵语言

D) 数据巡查语言

5. 设有关系 R、S 和 T 如下:

R	
A	
m	
n	

S	
B	C
1	3

T		
A	B	C
m	1	3
n	1	3

则由关系 R 和 S 得到关系 T 的操作是

A) 交

B) 并

C) 笛卡儿积

D) 自然连接

6. 一个运动队有多个队员,一个队员仅属于一个运动队,一个队一般都有一个教练,则实体运动队和队员间的联系是

A) 多对多

B) 一对一

C) 多对一

D) 一对多

7. 大学生学籍管理系统中有关系模式 $S(S\#, S_n, S_g, S_d, S_a)$, 其中属性 $S\#, S_n, S_g, S_d, S_a$ 分别是学生学号、姓名、性别、系别和年龄, 关系的关键字是 $S\#$ 。检索全部大于 20 岁男生的姓名的表达式为

A) $\pi_{S_n}(\sigma_{S_g='男' \wedge S_a > 20}(S))$ B) $\sigma_{S_g='男'}(S)$ C) $\pi_{S_n}(\sigma_{S_g='男'}(S))$ D) $\pi_{S_n}(\sigma_{S_g='男' \wedge S_a > 20}(S))$

8. 设有表示客户、产品及购买的三张表, 其中客户(客户号, 姓名, 性别, 年龄, 地址), 产品(产品号, 产品名, 规格, 进价, 出厂时间), 购买(客户号, 产品号, 价格, 时间), 其中表客户和产品的关键字为客户号和产品号, 则关系表购买的关键字(键或码)为

A) 产品号

B) 客户号

C) 客户号, 产品号

D) 客户号, 产品号, 价格

9. 将 E-R 图转换为关系模式时, E-R 图中的实体一般转化为

A) 属性

B) 关系

C) 键

D) 域

10. 数据库设计的四个阶段是需求分析、概念设计、逻辑设计和

A) 编码设计

B) 测试阶段

C) 运行阶段

D) 物理设计

基本要求

1. 掌握计算机系统的基本概念,理解计算机硬件系统和计算机操作系统。
2. 掌握算法的基本概念。
3. 掌握基本数据结构及其操作。
4. 掌握基本排序和查找算法。
5. 掌握逐步求精的结构化程序设计方法。
6. 掌握软件工程的基本方法,具有初步应用相关技术进行软件开发的能力。
7. 掌握数据库的基本知识,了解关系数据库的设计。

考试内容

一、计算机系统

1. 掌握计算机系统的结构。
2. 掌握计算机硬件系统结构,包括 CPU 的功能和组成,存储器分层体系,总线和外部设备。
3. 掌握操作系统的基本组成部分,包括进程管理、内存管理、目录和文件系统、I/O 设备管理。

二、基本数据结构与算法

1. 算法的基本概念;算法复杂度的概念和意义(时间复杂度与空间复杂度)。
2. 数据结构的定义;数据的逻辑结构与存储结构;数据结构的图形表示;线性结构与非线性结构的概念。
3. 线性表的定义;线性表的顺序存储结构及其插入与删除运算。
4. 栈和队列的定义;栈和队列的顺序存储结构及其基本运算。
5. 线性单链表、双向链表与循环链表的结构及其基本运算。
6. 树的基本概念;二叉树的定义及其存储结构;二叉树的前序、中序和后序遍历。
7. 顺序查找与二分法查找算法;基本排序算法(交换类排序,选择类排序,插入类排序)。

三、程序设计基础

1. 程序设计方法与风格。
2. 结构化程序设计。
3. 面向对象的程序设计方法,对象、方法、属性及继承与多态性。

四、软件工程基础

1. 软件工程基本概念,软件生命周期概念,软件工具与软件开发环境。
2. 结构化分析方法,数据流图,数据字典,软件需求规格说明书。
3. 结构化设计方法,总体设计与详细设计。
4. 软件测试的方法,白盒测试与黑盒测试,测试用例设计,软件测试的实施,单元测试、集成测试和系统测试。
5. 程序的调试,静态调试与动态调试。

五、数据库设计基础

1. 数据库的基本概念:数据库,数据库管理系统,数据库系统。
2. 数据模型,实体联系模型及E-R图,从E-R图导出关系数据模型。
3. 关系代数运算,包括集合运算及选择、投影、连接运算,数据库规范化理论。
4. 数据库设计方法和步骤:需求分析、概念设计、逻辑设计和物理设计的相关策略。

考试方式

1. 公共基础知识不单独考试,与其他二级科目组合在一起,作为二级科目考核内容的一部分。
2. 上机考试,10道单项选择题,占10分。

- A) 基本路径测试 B) 等价类划分 C) 判定覆盖测试 D) 语句覆盖测试
9. 数据库管理系统是
- A) 操作系统的一部分 B) 系统软件
- C) 一种编译系统 D) 一种通信软件系统
10. 在 E-R 图中, 表示实体的图元是
- A) 矩形 B) 椭圆 C) 菱形 D) 圆
11. 有两个关系 R 和 T 如下:

R			T	
A	B	C	A	C
a	1	2	a	2
b	4	4	b	4
c	2	3	c	3
d	3	2	d	2

则由关系 R 得到关系 T 的操作是

- A) 选择 B) 交 C) 投影 D) 并
12. 对图书进行编目时, 图书有如下属性: ISBN 书号, 书名, 作者, 出版社, 出版日期。能作为关键字的是
- A) ISBN 书号 B) 书名 C) 作者 D) 出版社 E) 出版日期

参考答案

1. A 2. A 3. C 4. D 5. A 6. B 7. D 8. B 9. B 10. A 11. C 12. A

习题 1

1. C 2. A 3. D 4. A 5. C 6. A 7. B 8. A 9. A 10. A

习题 2

1. C 2. D 3. A 4. B 5. C 6. D 7. B 8. C 9. C 10. B
11. B 12. B 13. A 14. D 15. C 16. D 17. D

习题 3

1. B 2. A 3. D 4. B 5. A 6. C

习题 4

1. D 2. D 3. A 4. A 5. C 6. B 7. D 8. C 9. B 10. B
11. B 12. A

习题 5

1. B 2. B 3. A 4. C 5. C 6. D 7. A 8. C 9. B 10. D